

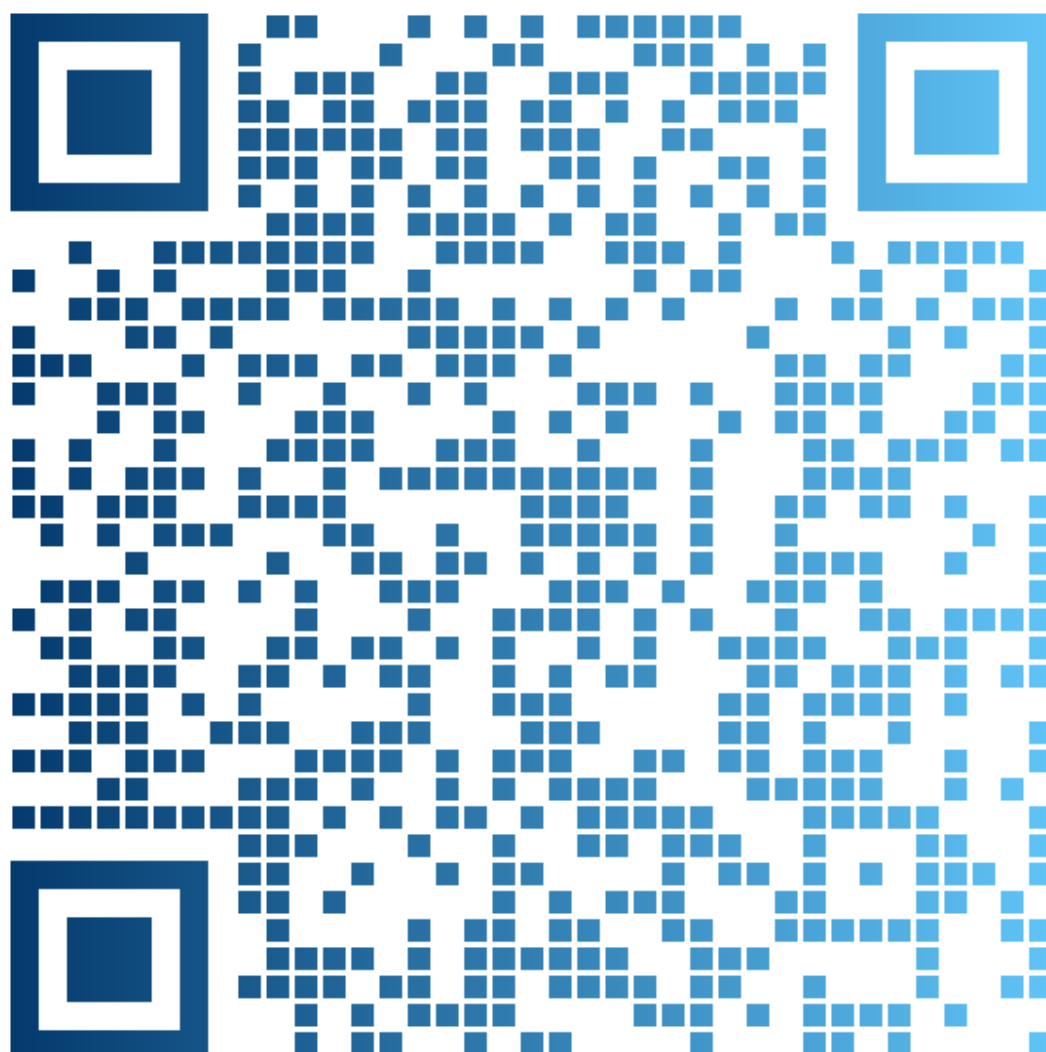


المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Analyse et visualisation de données

IAOUSSE M'barek

2026



Ce manuel a été élaboré pour servir de support de cours principal pour l'élément *Analyse et visualisation de données* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en Temps Aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](#).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). **Analyse et visualisation de données**. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent manuel est le support de cours officiel de l'élément **Analyse et visualisation des données**, dispensé dans le cadre de la formation **Développement d'Applications et Intelligence Artificielle** (DBT-DAIA) en temps aménagé.

La communauté étudiante visée par ce module provient de parcours techniques variés (DUT, BTS, DTS ou diplômes équivalents), ayant déjà acquis un bagage de base en programmation Python. La familiarité préalable avec la Data Science, l'analyse de données ou le Machine Learning n'est pas un prérequis : ce manuel se concentre sur ce qui précède et accompagne la modélisation (comprendre, préparer, explorer et représenter une donnée) sans aborder les algorithmes d'apprentissage automatique, qui font l'objet de l'élément **Machine Learning**.

Ce manuel vise à outiller les étudiants d'une **démarche méthodique** face à un jeu de données : savoir le questionner avant de le modéliser, savoir le nettoyer avant de l'analyser, et savoir le représenter avant de le communiquer. Il introduit progressivement les notions de statistique nécessaires, l'écosystème Python (**pandas**, **numpy**, **matplotlib**, **seaborn**), les principes d'un entrepôt de données, puis la construction d'une solution de Business Intelligence avec Power BI et Taipy.

Prérequis

Le contenu de ce manuel est organisé pour des personnes ayant déjà une base et une expérience avec :

- la programmation en langage Python (variables, boucles, fonctions, structures de données de base) ;
- la manipulation élémentaire de fichiers et de tableaux.

Aucune connaissance préalable en analyse de données, en statistique ou en data visualization n'est nécessaire : les indicateurs utiles sont introduits et illustrés progressivement.

Livres et ressources complémentaires

Ce manuel s'inspire des ouvrages de référence en analyse de données et en visualisation, notamment *Python for Data Analysis* (McKinney 2022), consacré à la manipulation de données avec **pandas**, ainsi que *Storytelling with Data* (Knaflitz 2015), référence sur la communication visuelle de résultats analytiques.

L'ensemble des exemples et exercices pratiques est développé en utilisant **Python** comme langage principal, avec **pandas** et **numpy** pour la manipulation des données, et **matplotlib/seaborn** pour la visualisation statique.

i Note

Les bibliothèques scientifiques Python évoluant rapidement, veillez à utiliser un environnement virtuel basé sur Python 3.10 ou supérieur pour garantir la compatibilité des extraits de code présentés dans ce manuel.

Table des matières

Préface	i
Introduction à l'analyse de données	1
1. Qu'est-ce que l'analyse de données ?	1
Définition et Positionnement	1
Les quatre types d'analyse de données	2
2. Cycle de vie et mise en pratique avec Python	3
Le cycle de vie d'un projet d'analyse de données	3
Écosystème d'outils Python pour l'analyse de données	5
3. Décrire une variable	8
3.1 Vocabulaire de base	8
3.2 Décrire une variable qualitative	9
3.3 Comparer deux périodes	10
3.4 Décrire une variable quantitative	10
Exercices	13
Collecte, Nettoyage et Préparation des Données	16
1. Où trouver les données ? Les stratégies de collecte	17
1.1. Les bases de données	17
1.2. Les fichiers plats et les principaux formats de données	17
1.3. Les API	18
1.4. Le <i>web scraping</i>	18
1.5. Données ouvertes et capteurs connectés	18
2. Représentativité et biais d'échantillonnage	19
3. Nettoyage des données	19
3.1. Les valeurs manquantes	20
3.2. Les valeurs aberrantes (<i>outliers</i>)	21
3.3. Les doublons et les incohérences	22
3.4. Vers une démarche de qualité systématique	22
4. Encodage des variables catégorielles	22
4.1. Encodage rapide avec pandas (DA)	23
4.2. Encodage robuste avec scikit-learn (ML)	23
4.3. Comparatif des méthodes d'encodage	24
5. Mise à l'échelle : standardisation et normalisation	25
5.1. Les deux transformations de référence	25
5.2. Quand utiliser l'une ou l'autre ?	25
5.3. Pourquoi certains algorithmes y sont-ils sensibles ? (ML)	26
6. Transformations complémentaires pour l'exploration (DA)	26
6.1. Discretisation (<i>binning</i>)	26
6.2. Quelques transformations usuelles supplémentaires	26
7. Construire un pipeline de prétraitement automatisé (ML)	27
7.1. ColumnTransformer et Pipeline de scikit-learn	27
7.2. Mettre à jour l'environnement du projet	28
8. Synthèse	29
Exercices	29
Collecte et représentativité	29
Valeurs manquantes	29
Valeurs aberrantes	30
Encodage	30
Mise à l'échelle	30

Machine Learning : fuite de données et pipeline (ML)	31
Analyse de Données : discrétisation (DA)	31
Exploration des données	32
1. Structurer l'exploration : univariée, bivariée, multivariée	32
1.1. Quel outil pour quelle paire de variables ?	33
2. Relation entre deux variables quantitatives : la corrélation	33
2.1. De la covariance au coefficient de corrélation	33
2.2. Ce que r ne voit jamais : la galerie des cas particuliers	35
2.3. Matrice de corrélation	35
2.4. Le quartet d'Anscombe	37
2.5. Pearson ou Spearman ?	37
3. Comparer une variable quantitative entre groupes	38
3.1. Statistiques par groupe	38
3.2. Décomposer la variance : l'idée de l'ANOVA	39
3.3. Interpréter le test et vérifier ses conditions	40
4. Relation entre deux variables qualitatives	42
4.1. Le tableau croisé, observé contre théorique	42
4.2. Le V de Cramér : une taille d'effet	43
5. Pièges de l'analyse bivariée et multivariée	44
5.1. Le paradoxe de Simpson	44
5.2. Corrélation n'est pas causalité	44
5.3. Un point invisible seul, flagrant à deux	45
5.4. Le piège des tests multiples	45
6. Distributions asymétriques : la transformation logarithmique	45
7. Résumer par groupe : groupby et tableaux croisés dynamiques	47
7.1. Le principe <i>split-apply-combine</i>	47
7.2. Le tableau croisé dynamique (pivot_table)	48
8. Explorer une série temporelle	48
9. Automatiser un premier diagnostic : le profilage de données	50
Exercices	50
Visualisation des données	53
1. Pourquoi visualiser des données ?	53
1.1. Ce que les statistiques seules ne montrent pas	53
1.2. Le rôle de la visualisation dans le cycle d'un projet	53
2. La grammaire des graphiques	54
2.1. Anatomie d'une figure matplotlib	55
2.2. Toutes les variables visuelles ne se valent pas	56
3. Choisir le bon graphique selon la question posée	56
4. Les distributions : histogramme, densité, boîte à moustaches	57
4.1. L'histogramme	57
4.2. Histogramme et diagramme en barres : deux graphiques que l'on confond trop souvent	57
4.3. Le choix du nombre de classes	58
4.4. Classes de largeurs inégales : effectif ou densité ?	59
4.5. L'estimation de densité par noyau (KDE)	60
4.6. La fonction de répartition empirique (ECDF)	60
4.7. La boîte à moustaches (<i>boxplot</i>)	62
4.8. Une même distribution, six représentations	62
4.9. Le diagramme quantile-quantile (QQ-plot)	63
5. Comparaisons : le diagramme en barres et ses variantes	63
5.1. Le diagramme en barres	63
5.2. Six variantes du diagramme en barres	64
5.3. Lollipop et <i>dot plot</i> : des barres allégées	65
6. Le diagramme circulaire (camembert) — et ses limites	65
6.1. Camembert contre barres triées : une comparaison directe	66
6.2. Quand un camembert reste défendable	66

7. Relations entre variables : nuage de points et <i>heatmap</i>	66
7.1. Le nuage de points (<i>scatter plot</i>)	66
7.2. Au-delà de deux dimensions : mapper des variables supplémentaires	67
7.3. Le problème de la surimpression (<i>overplotting</i>)	67
7.4. La <i>heatmap</i> (carte de chaleur)	68
8. Évolution : courbes et variantes temporelles	68
8.1. La courbe (<i>line chart</i>)	68
8.2. Visualiser une courbe décomposée	69
8.3. Aire empilée et <i>streamgraph</i>	70
8.4. Le <i>slopegraph</i> et le nuage de points connecté	70
9. Cartographie : représenter une dimension géographique	71
9.1. La carte choroplèthe	71
9.2. Le piège classique : la surface n'est pas l'importance	71
9.3. La carte de points géolocalisés	73
9.4. Le cartogramme	73
9.5. Un mot sur les projections	73
10. Graphiques avancés — à réserver aux cas qui le justifient	76
10.1. Petits multiples (facettes)	76
10.2. Radar et coordonnées parallèles	77
10.3. Représenter un réseau	78
10.4. Le diagramme de Gantt	79
10.5. Treemap et <i>waffle chart</i> : d'autres façons de montrer une composition	79
10.6. Éléments de tableau de bord : cartes indicateurs et jauges	80
11. Bibliothèques Python pour la visualisation	81
11.1. <i>matplotlib</i> : la bibliothèque de référence	81
11.2. <i>seaborn</i> : la visualisation statistique simplifiée	81
11.3. <i>geopandas</i> : pour la cartographie	81
11.4. Vers l'interactivité	81
12. Communiquer les résultats et rendre les graphiques accessibles	82
12.1. Partir du public et de la décision	82
12.2. Construire un message honnête et lisible	83
12.3. Accessibilité : ne pas transmettre l'information par la couleur seule	84
12.5. Vérification avant diffusion	85
Introduction à l'entrepôt de données	86
1. Pourquoi un entrepôt de données ?	86
1.1. Les systèmes opérationnels	86
1.2. La base analytique	86
1.3. Entrepôt, lac et datamart	87
2. Le modèle en étoile	88
2.1. La table de faits	88
2.2. Les tables de dimensions	88
2.3. Exemple de modèle de ventes	89
2.4. Modèle en étoile ou en flocon ?	90
2.5. Un mot sur l'historisation des dimensions	90
3. Les mesures et les indicateurs	91
4. Le flux ETL/ELT	92
4.1. Illustrer un contrôle de qualité au chargement	92
5. Gouvernance et sécurité, sans complexité inutile	93
5.1. Vers le cloud et le lakehouse	94
Exercices	94
Business Intelligence avec Power BI et Taipy	96
1. Comprendre la BI	96
1.1. Du besoin à l'indicateur	96
1.2. Le parcours d'une solution BI	97
1.3. Règles d'un tableau de bord débutant	97

2. Power BI : une démarche structurée	97
2.1. Étape 1 — Importer les sources	98
2.2. Étape 2 — Transformer avec Power Query	99
2.3. Étape 3 — Construire le modèle	99
2.4. Étape 4 — Créer des mesures en DAX	101
2.5. Étape 5 — Construire les visuels	102
2.6. Étape 6 — Vérifier et partager	103
3. Taipy : créer une application BI en Python	104
3.1. Quand choisir Taipy ?	104
3.2. Anatomie d'une application Taipy	104
3.3. Préparer les données et calculer les indicateurs	104
3.4. Construire une visualisation sophistiquée avec Python	105
3.5. Assembler l'application Taipy	106
3.6. Organiser une petite application	107
4. Comparaison synthétique	107
Exercices	108
Étude de cas : analyser les ventes d'une petite chaîne de magasins	110
1. Le besoin métier	110
1.1. Les parties prenantes	110
1.2. Les quatre questions posées	111
1.3. Périmètre et définitions	111
2. Le jeu de données	111
2.1. Première lecture	112
3. Préparer les données	112
3.1. Dictionnaire minimal	112
3.2. Transformation	112
4. Calculer les indicateurs	112
4.1. Montants ligne par ligne	112
4.2. Par mois	113
4.3. Par magasin et par catégorie	113
4.4. Moyenne et médiane des montants de lignes	114
5. Construire le modèle analytique	114
6. Concevoir la page Power BI	115
7. Construire une vue Taipy	116
7.1. Charger et préparer les données	116
7.2. Calculer les indicateurs	117
7.3. Construire une visualisation sophistiquée	117
7.4. Assembler l'application Taipy	118
8. Interpréter et communiquer	119
Exercices	119
Lab 0 : Environnement Python et statistique descriptive	121
1. Contexte et données	121
2. Préparer l'environnement	122
2.1. Créer le projet	122
2.2. Vérifier les bibliothèques	122
3. Télécharger et inspecter les données	123
3.1. Télécharger une copie locale	123
3.2. Les premiers réflexes	123
3.3. Distinguer les types de données	123
4. Fréquences et répartitions	124
4.1. Effectif et fréquence	124
4.2. Fréquences croisées	124
4.3. Fréquence d'une condition	124
5. Moyenne, médiane et quantiles	125
5.1. Tendance centrale	125
5.2. Quartiles et quantiles	125

5.3. Tableau descriptif	125
6. Variance et écart-type	126
6.1. Définitions utiles	126
6.2. Comparer deux groupes	126
7. Taux de croissance	127
8. Visualiser pour vérifier	127
8.1. Distribution des retards	127
8.2. Interpréter sans exagérer	128
9. Travail demandé	128
Partie A — Environnement et prise en main	128
Partie B — Fréquences et types	128
Partie C — Statistiques descriptives	128
Partie D — Évolution et visualisation	129
Partie E — Question d'analyse	129
Lab 1 : Collecte de données	130
1. Une collecte commence par une question	130
2. Les trois familles de collecte	131
2.1. Questionnaire	131
2.2. API	131
2.3. Fichier ou page Internet	131
3. Préparer le projet de collecte	132
4. Partie A — Concevoir un mini-questionnaire	132
4.1. Sujet	132
4.2. Export et lecture	132
5. Partie B — Interroger l'API NYC 311	133
5.1. Découvrir la ressource	133
5.2. Filtrer côté serveur	133
5.3. Pagination	134
6. Partie C — Une donnée publique téléchargée sur Internet	135
7. Contrôle de la collecte	135
8. Travail demandé	136
Partie A — Questionnaire	136
Partie B — API NYC 311	136
Partie C — Source Internet complémentaire	136
Partie D — Réflexion	136
Lab 2 : EDA et prétraitement	137
1. Le fichier de départ	137
2. Installer et charger proprement	138
3. Audit initial	138
3.1. Dimensions et types	138
3.2. Valeurs manquantes	139
3.3. Unicité et doublons	139
4. Convertir et valider	139
4.1. Dates	139
4.2. Catégories textuelles	140
4.3. Règles de validité	140
5. Créer les variables d'analyse	141
5.1. Délai de traitement	141
5.2. Variables temporelles	141
5.3. Catégories rares	141
6. EDA univariée	142
6.1. Variables catégorielles	142
6.2. Variables numériques	142
7. EDA bivariée et multivariée	142
7.1. Type de demande et délai	142
7.2. Quartier et canal	143

7.3. Volume dans le temps	143
8. Traiter les données avec une règle explicite	143
9. Données volumineuses	144
10. Travail demandé	145
Partie A — Audit	145
Partie B — Prétraitement	145
Partie C — Exploration	145
Partie D — Interprétation	145
Partie E — Préparer la suite	146
Lab 3 : Visualiser les données 311	147
1. Le fichier de départ	147
2. Installer et préparer	148
3. Distribution : à quoi ressemble le délai de traitement ?	149
4. Comparaison : quels types de demandes dominant ?	149
4.1. Contre-exemple volontaire : le camembert	149
5. Relation : le délai dépend-il du moment de la journée ?	150
6. Évolution : comment le volume de demandes varie-t-il dans le temps ?	151
7. Composition et multivarié : délai par arrondissement et par canal	151
8. Un premier graphique interactif	152
9. Construire une figure de communication complète	152
10. Travail demandé	153
Partie A — Choisir avant de coder	153
Partie B — Graphiques essentiels	153
Partie C — Critique et correction	153
Partie D — Interactivité	153
Partie E — Communication	153
Lab 4 : Entrepôt de données et dashboard avancé avec Power BI	154
1. Le jeu de données massif	154
2. Architecture cible	155
2.1. Les couches	155
2.2. Le grain	155
2.3. Schéma en étoile	155
3. Acquérir les données dans Power Query	156
3.1. Connexion Web	156
3.2. Pagination dans Power Query	156
3.3. Transformer la staging table	157
4. Créer les dimensions	158
4.1. Dimension Date	158
4.2. Dimensions par référence	158
4.3. Relations	158
5. Mesures DAX du modèle	159
5.1. Contrôles	159
6. Choisir le bon visuel	160
7. Construire un dashboard sophistiqué mais lisible	160
Page 1 — Pilotage	160
Page 2 — Diagnostic des délais	161
Page 3 — Canaux et géographie	161
Page 4 — Exploration	161
7.1. Interactions et navigation	161
8. Performance sur des données massives	161
9. Spécification du rapport attendu	162
10. Travail demandé	162
Lab 5 : Entrepôt de données et dashboard avancé avec Taipy	163
1. Power BI ou Taipy ?	163
1.1. Comparaison par besoin	163
1.2. Ce que l'on garde identique	164

2. Adapter une donnée massive à une application	164
2.1. Fichiers produits	164
3. Préparer les agrégats avec Python	165
3.1. Installer les dépendances	165
3.2. Contrôler les agrégats	167
4. Construire l'application Taipy	167
4.1. Charger les données	167
4.2. Préparer l'état de l'interface	168
4.3. Fonction de filtrage	168
4.4. Construire la page	169
4.5. Réagir aux filtres	169
4.6. Lancer l'application	169
5. Rendre l'application aussi riche que le rapport Power BI	170
Vue 1 — Pilotage	170
Vue 2 — Types de demandes	170
Vue 3 — Comparaison géographique	170
6. Vérifier la cohérence avec Power BI	171
7. Déploiement gratuit avec Hugging Face Spaces	171
7.1. Préparer les fichiers	171
7.2. Créer et lancer le Space	172
7.3. Autres options	172
8. Sécurité et maintenance	172
9. Travail demandé	173

References

174

Introduction à l'analyse de données

💡 Objectifs du chapitre

À la fin de ce chapitre introductif, l'étudiant sera capable de :

- **Définir** l'analyse de données et la **situer** par rapport à la Data Science, la Business Intelligence (BI) et le Machine Learning ;
- **Distinguer** les quatre types d'analyse : descriptive, diagnostique, prédictive et prescriptive ;
- **Identifier** les grandes étapes du cycle de vie d'un projet d'analyse de données ;
- **Identifier** les outils de base utilisés dans ce cours ;
- **Reconnaître** le type d'une variable et **calculer**, à la main comme avec **pandas**, les indicateurs de base (effectif, fréquence, variation relative) et les statistiques descriptives d'une variable numérique (tendance centrale, dispersion, forme) qui seront mobilisés dès le chapitre suivant.

1. Qu'est-ce que l'analyse de données ?

Définition et Positionnement

L'**analyse de données** (*Data Analysis*) désigne l'ensemble des démarches permettant d'inspecter, de nettoyer, de transformer et de modéliser statistiquement des données, dans le but d'en extraire des informations utiles, de soutenir une décision ou de valider une hypothèse.

💡 Analyse de données

Définition 0.0.1. L'analyse de données consiste à transformer des données brutes, souvent volumineuses et désorganisées, en **informations exploitables**, à travers une succession d'étapes de collecte, de nettoyage, d'exploration et de représentation.

Contrairement à ce que l'on pense parfois, l'essentiel du travail d'un analyste ne consiste pas à produire des graphiques, mais à **comprendre les données** : d'où elles viennent, ce qu'elles représentent réellement, quelles sont leurs limites et leurs biais. La visualisation intervient à deux moments : pendant l'exploration, pour repérer des tendances, des anomalies ou des relations, puis lors de la communication, pour présenter les résultats au public visé.

Il existe quatre domaines liés à l'analyse de données : Data Analysis, Data Science, Business Intelligence, et Machine Learning. Ces quatre termes sont souvent confondus. Ils partagent une matière première commune (la donnée) mais diffèrent par leurs objectifs et leurs méthodes.

TABLE 1 – Quatre disciplines complémentaires autour de la donnée

Domaine	Question typique	Résultat produit
Business Intelligence (BI)	« Que s'est-il passé, et où en sommes-nous ? »	Tableaux de bord, indicateurs (KPI) suivis dans le temps
Analyse de données (Data Analysis)	« Pourquoi cela s'est-il produit ? Que peut-on en conclure ? »	Rapports, visualisations, recommandations argumentées
Data Science	« Que va-t-il probablement se passer ? »	Modèles statistiques et modèles de Machine Learning

Domaine	Question typique	Résultat produit
Machine Learning	« Comment automatiser une prédiction ou une décision à grande échelle ? »	Un modèle entraîné, déployé en production

⚠ Périimètre de ce cours

Ce module se concentre sur les deux premières colonnes de la Table 1 : la **préparation**, l'**exploration** et la **visualisation** des données. Les algorithmes de Machine Learning (régression, classification, clustering...) ne sont **pas traités ici** : ils font l'objet du cours [Machine Learning](#). Une bonne partie du travail présenté dans ce cours reste néanmoins un prérequis indispensable à tout projet de Machine Learning.

En pratique, ces frontières sont poreuses : un même professionnel peut, selon les entreprises, être appelé *Data Analyst* et exercer des missions de *Data Scientist*, ou inversement. Ce qui compte est de comprendre la **nature de la question posée** plutôt que le titre du poste.

Les quatre types d'analyse de données

On distingue classiquement quatre niveaux d'analyse, de la simple observation du passé jusqu'à la recommandation d'action, chacun s'appuyant sur le précédent.

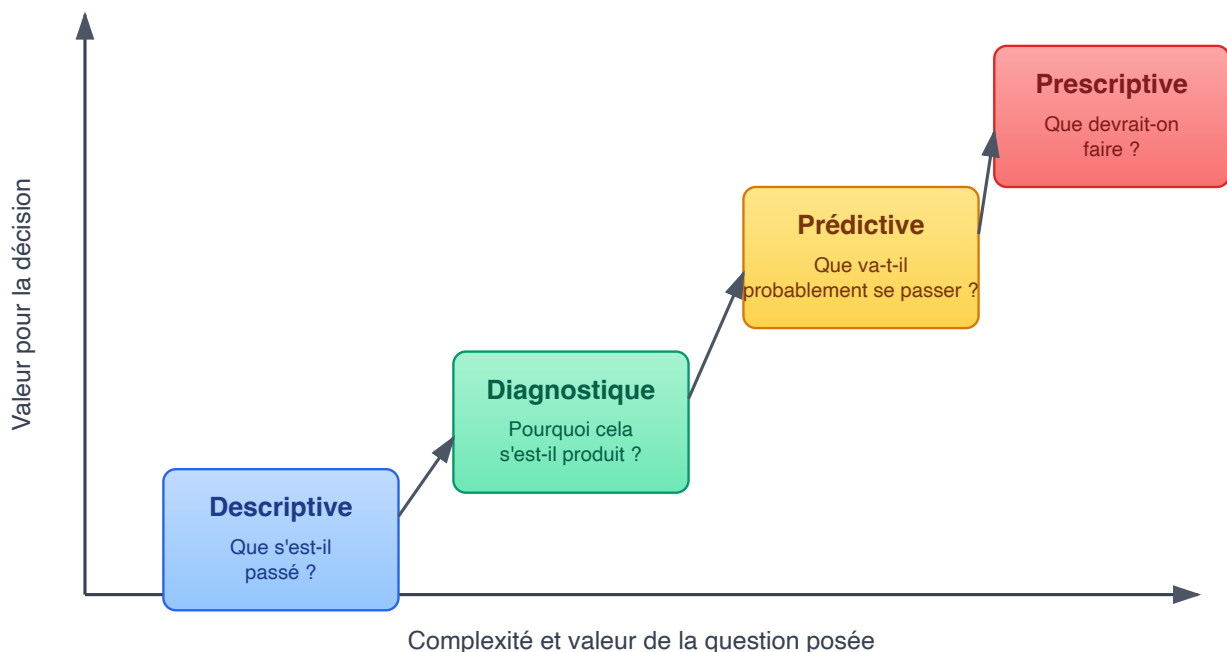


FIGURE 1 – Les quatre types d'analyse de données

TABLE 2 – Les quatre types d'analyse de données

Type d'analyse	Question posée	Exemple
Descriptive	Que s'est-il passé ?	Le chiffre d'affaires du mois dernier par région
Diagnostique	Pourquoi cela s'est-il passé ?	Pourquoi les ventes ont-elles chuté en mars dans le nord ?

Type d'analyse	Question posée	Exemple
Prédictive	Que va-t-il probablement se passer ?	Quel sera le chiffre d'affaires du mois prochain ?
Prescriptive	Que devrait-on faire ?	Quelle action marketing maximiserait les ventes du trimestre ?

💡 Exemple : une chaîne de supermarchés marocaine

- **Descriptive** : « Les ventes de la région de Casablanca ont baissé de 12 % en mars par rapport à février. »
- **Diagnostic** : en croisant les ventes avec le calendrier et la météo, on découvre que la baisse coïncide avec des travaux routiers ayant réduit l'affluence dans deux magasins.
- **Prédictive** (*hors périmètre de ce cours, traité en Machine Learning*) : un modèle estime les ventes attendues pour le mois suivant compte tenu de la reprise du trafic.
- **Prescriptive** : sur la base du diagnostic, l'équipe décide de renforcer temporairement les promotions dans les magasins concernés.

Ce cours se concentre principalement sur les analyses **descriptive** et **diagnostic** : elles reposent entièrement sur les techniques de préparation, d'exploration et de visualisation présentées dans les chapitres suivants, et ne nécessitent pas d'algorithme d'apprentissage automatique.

2. Cycle de vie et mise en pratique avec Python

Le cycle de vie d'un projet d'analyse de données

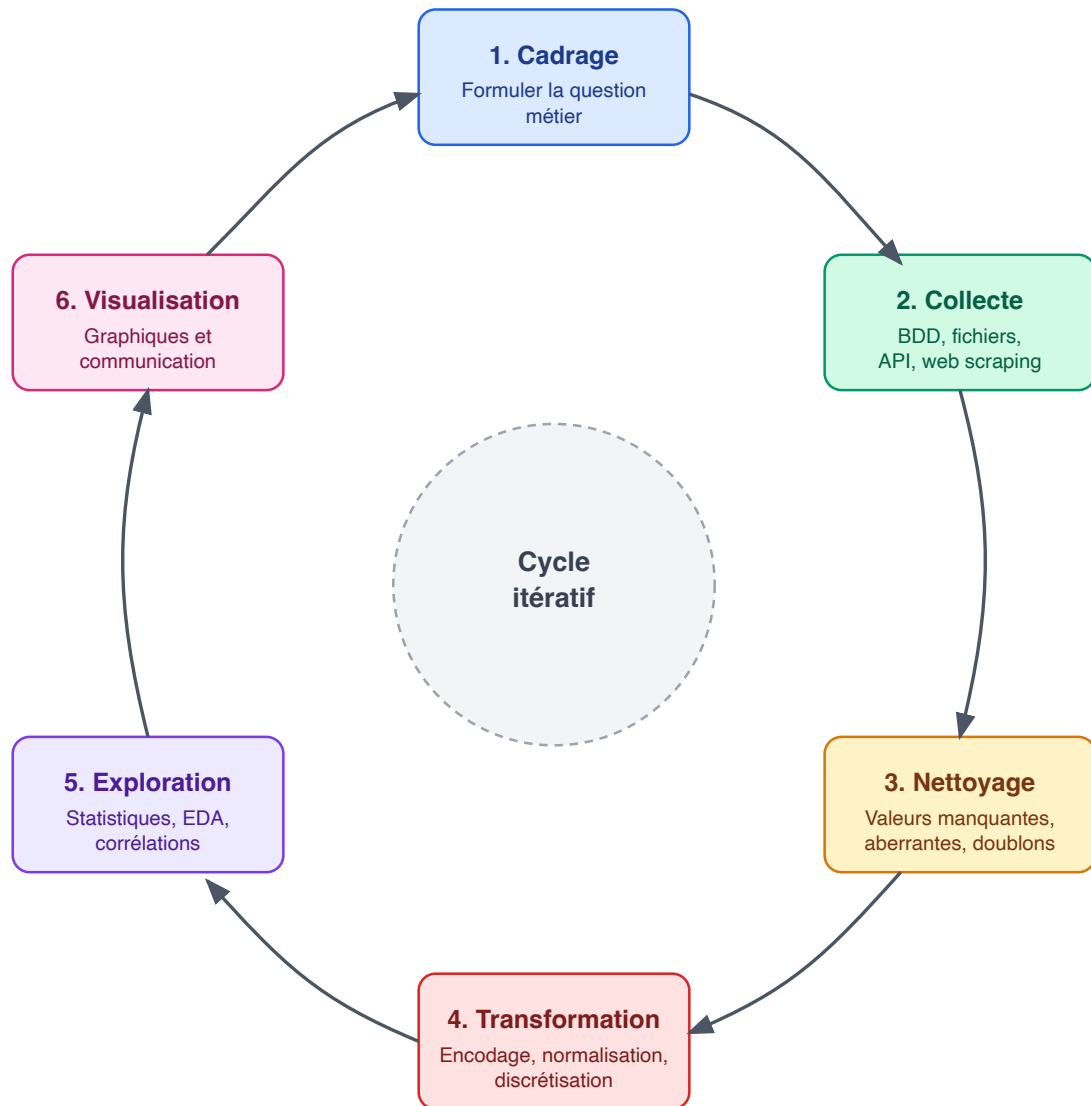
Un projet d'analyse de données suit une succession d'étapes, rarement linéaire, que l'on peut résumer ainsi :

1. **Cadrage** : formuler clairement la question métier à laquelle l'analyse doit répondre.
2. **Collecte** : rassembler les données nécessaires (bases de données, fichiers, API, web scraping...).
3. **Nettoyage** : traiter les valeurs manquantes, les valeurs aberrantes, les doublons et les incohérences. C'est l'étape la plus chronophage en pratique.
4. **Transformation** : encoder, normaliser, recalculer certaines variables pour les rendre exploitables.
5. **Exploration (EDA)** : décrire statistiquement les données, en dégager des tendances, des corrélations, des anomalies.
6. **Visualisation et communication** : représenter graphiquement les résultats et les traduire en recommandations compréhensibles par un public non technique.

i Repère

Ce découpage est proche du cadre **OSEMN** (*Obtain, Scrub, Explore, Model, iNterpret*), popularisé par la communauté Data Science. Nous l'adaptions ici en retirant l'étape de modélisation prédictive (« Model »), hors périmètre de ce cours, et en insistant davantage sur la communication des résultats.

Comme pour tout cycle de projet, ces étapes sont **itératives** : une exploration révélant un problème de qualité de données ramène généralement à l'étape de nettoyage, et une visualisation peu convaincante amène souvent à revisiter l'exploration.



Une exploration ou une visualisation décevante ramène souvent vers les étapes 2, 3 ou 4.

FIGURE 2 – Le cycle de vie d'un projet d'analyse de données

Écosystème d'outils Python pour l'analyse de données

Bibliothèques cœur

Comme pour le Machine Learning, Python s'impose comme langage principal pour l'analyse de données, grâce à la richesse de son écosystème scientifique et à la lisibilité de sa syntaxe.

TABLE 3 – Bibliothèques Python cœur de ce cours

Bibliothèque	Rôle
pandas	Manipulation de tableaux de données (DataFrame) : lecture, nettoyage, agrégation
numpy	Calcul numérique et vectoriel sous-jacent à pandas
matplotlib	Bibliothèque de visualisation de référence, hautement personnalisable
seaborn	Visualisation statistique construite sur matplotlib , orientée exploration rapide

Environnements de travail

Comme dans le reste de la formation, nous privilégions les **notebooks Jupyter** (JupyterLab, Google Colab) pour l'exploration interactive et la visualisation immédiate cellule par cellule. **Visual Studio Code** convient également aux étudiants qui souhaitent écrire des scripts Python ou y ouvrir des notebooks Jupyter.

Gestion de projet avec uv

Un projet d'Analyse de Données ne dépend pas uniquement du code que nous écrivons. Il repose également sur un **environnement logiciel** : une version de Python, un ensemble de bibliothèques, des versions compatibles de ces bibliothèques et différents outils nécessaires à l'exécution du projet.

La gestion de cet environnement est importante dès qu'un projet doit être **reproduit, partagé, maintenu ou déployé**. Deux utilisateurs disposant du même code peuvent, par exemple, rencontrer des problèmes différents si leurs versions de Python ou de certaines dépendances ne sont pas les mêmes. La gestion des dépendances constitue ainsi une composante essentielle de la reproductibilité et de la maintenance d'un projet.

Dans ce cours, nous présenterons d'abord l'approche classique de l'écosystème Python, puis nous adopterons **uv** comme outil de gestion de nos projets.

L'approche classique : Python, venv et pip

L'écosystème Python fournit déjà les outils nécessaires pour créer un environnement isolé et installer les bibliothèques d'un projet. Le module **venv** est l'outil standard de Python pour créer des environnements virtuels. Ceux-ci permettent d'isoler les dépendances d'un projet de celles installées dans les autres projets ou dans l'installation générale de Python.

L'installation des bibliothèques s'effectue généralement avec **pip**, l'installateur de paquets Python. Les bibliothèques sont notamment disponibles sur **PyPI**, le Python Package Index.

Une configuration classique peut donc être représentée de manière simplifiée par :

```
python -m venv .venv
```

puis, dans l'environnement virtuel :

```
python -m pip install numpy pandas scikit-learn
```

Les dépendances peuvent ensuite être consignées dans un fichier `requirements.txt`, par exemple :

```
numpy
pandas
scikit-learn
```

Cette approche est parfaitement valable et reste largement utilisée. Le [Python Packaging User Guide](#) présente d'ailleurs cette organisation ainsi que les pratiques modernes de gestion et de distribution des projets Python.

Elle présente cependant une certaine fragmentation : la création de l'environnement, la gestion des dépendances et leur installation sont assurées par des mécanismes distincts. Pour des projets plus structurés, il devient également utile de disposer d'une déclaration centralisée des dépendances et d'un mécanisme de verrouillage permettant de conserver une résolution précise de l'environnement.

Vers une gestion moderne du projet avec `pyproject.toml`

Les projets Python modernes utilisent notamment le fichier `pyproject.toml`. Ce fichier permet de centraliser différentes informations relatives au projet, notamment ses métadonnées et ses dépendances.

Il constitue aujourd'hui un élément central de l'écosystème de *packaging* Python et est utilisé par de nombreux outils modernes.

Dans ce cours, nous allons nous appuyer sur `uv`, développé par Astral, pour gérer nos projets Python.

Pourquoi `uv` ?

`uv` est un gestionnaire de projets et de paquets Python qui regroupe dans un même outil plusieurs opérations auparavant réalisées avec différents outils. Il permet notamment de gérer les projets Python, les environnements virtuels, les versions de Python, les dépendances et leur résolution, ainsi que l'exécution des commandes dans l'environnement du projet.

La documentation officielle de `uv` [présente](#) notamment les fonctions suivantes :

- création d'un projet avec `uv init` ;
- ajout et suppression de dépendances ;
- gestion de l'environnement virtuel du projet ;
- synchronisation des dépendances ;
- création et mise à jour du fichier de verrouillage ;
- exécution de commandes dans l'environnement du projet ;
- gestion de projets plus complexes au moyen de *workspaces*.

L'intérêt de `uv` dans notre cours est donc moins de remplacer une commande particulière que de fournir une **organisation cohérente de l'ensemble du projet Python**.

`pyproject.toml`, `uv.lock` et `.venv`

Avec `uv`, trois éléments sont particulièrement importants.

Le fichier `pyproject.toml` décrit le projet et ses besoins, notamment les dépendances et la version de Python requise.

Le fichier `uv.lock` enregistre la résolution précise des dépendances du projet. Contrairement au fichier `pyproject.toml`, qui exprime les besoins du projet de manière générale, le fichier de verrouillage conserve les versions effectivement résolues. Il peut ainsi être versionné avec le projet afin de faciliter la reconstruction d'un environnement cohérent sur différentes machines. La documentation officielle de `uv` [décrit précisément le rôle de `uv.lock`](#).

Enfin, `uv` utilise un environnement virtuel associé au projet, généralement situé dans `.venv`.

On peut donc représenter schématiquement l'organisation d'un projet ainsi :

```
projet/  
  .venv/  
  pyproject.toml  
  uv.lock  
  ...
```

Le principe peut être résumé ainsi :

pyproject.toml décrit les besoins du projet ; **uv.lock** conserve leur résolution précise ;
.venv contient l'environnement dans lequel le projet est exécuté.

La synchronisation entre ces éléments est prise en charge par **uv**. La documentation officielle explique notamment que les opérations telles que **uv sync** et **uv run** peuvent mettre à jour l'environnement à partir du fichier de verrouillage.

Pourquoi uv plutôt qu'Anaconda ou Conda ?

Il existe d'autres solutions parfaitement valables pour gérer les environnements Python. L'une des plus connues est l'écosystème **Conda**, qui permet notamment de créer, gérer, partager et supprimer des environnements contenant différentes versions de Python et de paquets.

Notre choix de **uv** dans ce cours ne signifie donc pas que **Conda** ou **Anaconda** sont inadaptés. Il s'agit d'un **choix pédagogique et technique** correspondant à l'organisation que nous souhaitons adopter.

Nous retenons **uv** principalement pour les raisons suivantes :

1. **Une approche directement centrée sur les projets Python.** **uv** s'intègre naturellement aux mécanismes modernes du *packaging* Python, notamment **pyproject.toml** et l'écosystème PyPI.
2. **Une gestion unifiée du projet.** Un même outil permet de gérer l'environnement, les dépendances, leur résolution, le verrouillage et l'exécution du projet.
3. **Une structure explicite et versionnable.** L'association de **pyproject.toml** et **uv.lock** fournit une description claire du projet et de son environnement de dépendances.
4. **Une bonne adéquation avec les projets du cours.** Les projets étudiés utiliseront principalement des bibliothèques de l'écosystème Python, telles que **numpy**, **pandas**, **matplotlib** et **scikit-learn**, ainsi que les outils Jupyter. **uv** fournit une organisation cohérente pour leur gestion dans un projet Python local.
5. **Une continuité entre expérimentation et développement.** Les étudiants pourront commencer par explorer les données et les modèles dans des notebooks, puis retrouver le même environnement de projet lorsqu'ils passeront à une organisation plus structurée du code Python.

Le choix de **uv** répond donc avant tout aux besoins de ce cours : disposer d'un environnement Python **isolé, explicite, reproductible et facile à partager**, tout en introduisant progressivement les pratiques modernes de gestion de projet.

Tendances récentes de l'écosystème

! Zoom sur les tendances actuelles

L'écosystème de l'analyse de données évolue rapidement. Sans remplacer les outils historiques présentés ci-dessus, plusieurs bibliothèques récentes gagnent en popularité et méritent d'être connues :

- **Polars** : une alternative à **pandas**, écrite en Rust, offrant des performances nettement supérieures sur de gros volumes de données grâce à l'exécution parallèle et à l'évaluation paresseuse (*lazy evaluation*).
- **DuckDB** : un moteur de base de données analytique embarqué, permettant d'exécuter des requêtes

SQL directement sur des fichiers CSV ou Parquet, sans serveur, avec des performances proches d'un entrepôt de données.

- **Outils de profilage automatique** (`ydata-profiling`, `Sweetviz`) : génèrent en une commande un rapport exploratoire complet d'un jeu de données (statistiques, distributions, corrélations, valeurs manquantes).¹
- **Assistants d'analyse par IA générative** : des outils permettant d'interroger un `DataFrame` en langage naturel commencent à s'intégrer aux environnements de data analysis, sans se substituer à une compréhension rigoureuse des données par l'analyste (l'humain !!).

Ces outils ne sont pas obligatoires dans ce cours, mais seront mentionnés ponctuellement à titre de complément, car ils représentent une évolution significative des pratiques professionnelles actuelles.

Les métiers de la donnée

TABLE 4 – Les principaux métiers autour de la donnée

Métier	Mission principale	Compétences clés
Data Analyst	Explorer, visualiser et interpréter les données pour éclairer une décision	SQL, <code>pandas</code> , visualisation, statistiques descriptives
BI Analyst / BI Developer	Construire et maintenir des tableaux de bord et indicateurs récurrents	SQL, outils de BI (Power BI, Tableau), modélisation de données
Data Scientist	Concevoir des modèles statistiques et prédictifs	Statistique inférentielle, Machine Learning, programmation
Data Engineer	Construire et fiabiliser les pipelines d'acquisition et de stockage des données	Bases de données, orchestration, ingénierie logicielle

Ce cours prépare avant tout au cœur du métier de **Data Analyst**, tout en posant des bases indispensables aux trois autres profils.

3. Décrire une variable

3.1 Vocabulaire de base

Avant de plonger dans la collecte et le nettoyage des données, nous allons aborder quelques notions du **vocabulaire statistique**. Cette section pose l'ensemble d'indicateurs que le reste du cours réutilisera : aussi bien pour le prétraitement et le nettoyage de données que pour l'exploration d'un jeu de données propre.

💡 Observation et variable

Une **observation** est une ligne du tableau de données (par exemple une commande, un client, une transaction). Une **variable** est une colonne (par exemple le montant d'une commande, la ville d'un client).

Avec `pandas`, on identifie rapidement le type déclaré de chaque colonne, à ne pas confondre avec le type *statistique* défini ci-dessous :

```
df.dtypes
df.info()
```

1. voir le chapitre sur l'exploration des données.

Les types de variables

Une variable peut être de trois natures, ce qui conditionne directement les indicateurs et les graphiques que l'on pourra lui appliquer par la suite :

TABLE 5 – Types et sous-types de variables

Type de variable	Sous-type	Définition	Exemple
Qualitative (catégorielle)	Nominale	Catégories sans ordre naturel	Ville, moyen de paiement, quartier
	Ordinale	Catégories avec un ordre naturel	Satisfaction (faible / moyenne / élevée)
Quantitative (numérique)	Discrète	Nombre entier, dénombrable	Nombre de produits, nombre d'enfants
	Continue	Nombre pouvant prendre toute valeur dans un intervalle	Prix, poids, durée
Temporelle	—	Date ou période, que l'on peut ordonner et découper	Date de commande, mois, trimestre

⚠ Pour quoi cette distinction est essentielle

Calculer une **moyenne** sur un code postal ou sur un identifiant client n'a aucun sens, même si ces valeurs sont numériques : ce sont en réalité des variables qualitatives. À l'inverse, une variable qualitative ordinale (comme un niveau de satisfaction) peut parfois être résumée par une médiane, mais jamais par une moyenne arithmétique classique. Cette distinction conditionne aussi bien le choix d'un **encodage** (ordinal ou One-Hot, chapitre suivant) que celui d'un indicateur ou d'un graphique — ce dernier lien sera repris au chapitre consacré à la visualisation.

3.2 Décrire une variable qualitative

💡 Effectif et fréquence

L'**effectif** d'une catégorie est le nombre d'observations qui appartiennent à cette catégorie. La **fréquence** (ou proportion) est la part que représente cet effectif dans l'ensemble des observations.

$$\text{fréquence} = \frac{\text{effectif de la catégorie}}{\text{effectif total}}$$

On l'exprime souvent en pourcentage en multipliant par 100.

Exemple — Effectif et fréquence

Sur 120 commandes, 30 proviennent de Casablanca :

$$\text{fréquence} = \frac{30}{120} = 0,25 = 25 \%$$

Interprétation : une commande sur quatre est passée à Casablanca.

Avec `pandas`, `value_counts()` calcule directement les effectifs (et, avec `normalize=True`, les fréquences) de chaque catégorie :

```
df["ville"].value_counts()           # effectifs par catégorie
df["ville"].value_counts(normalize=True) * 100 # fréquences en %
```

i Priorité pour débiter

Pour une variable qualitative, l'effectif et la fréquence de chaque catégorie constituent presque toujours le point de départ de toute description, avant tout graphique.

3.3 Comparer deux périodes

💡 Variation relative (ou taux de variation)

Pour comparer une même grandeur entre deux périodes (ou deux situations), on calcule la **variation relative** :

$$\text{variation en \%} = \frac{\text{valeur finale} - \text{valeur initiale}}{\text{valeur initiale}} \times 100.$$

- Un résultat positif indique une hausse.
- Un résultat négatif indique une baisse.

Exemple — Variation relative

Le chiffre d'affaires d'un produit passe de 200 MAD à 240 MAD :

$$\frac{240 - 200}{200} \times 100 = 20\%.$$

Interprétation : les ventes ont augmenté de 20 %.

Remarques

- Toujours préciser la période de référence (valeur initiale) et la période finale.
- Une variation de +100 % signifie un doublement ; une variation de -50 % signifie une division par deux.
- Une hausse de 20 % suivie d'une baisse de 20 % ne ramène **pas** à la valeur de départ : les deux variations ne s'appliquent pas à la même base.

Sur une série ordonnée dans le temps, **pandas** calcule automatiquement la variation relative d'une période à l'autre avec `pct_change()` :

```
df_mensuel["ventes"].pct_change() * 100    # variation en % par rapport au mois précédent
```

3.4 Décrire une variable quantitative

Lorsqu'on dispose d'une variable quantitative, on a besoin d'indicateurs qui résument sa **tendance centrale** (où se situent les valeurs) et sa **dispersion** (à quel point elles sont dispersées), puis, plus finement, sa **forme**.

Tendance centrale

TABLE 6 – Indicateurs de tendance centrale

Indicateur	Définition / formule	Sensibilité aux valeurs extrêmes
Moyenne ($\bar{x}$)	$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$	Élevée
Médiane (Me)	Valeur qui partage la série ordonnée en deux groupes de même effectif	Faible

Mode	Valeur (ou catégorie) la plus fréquente	Faible, mais peu informatif pour une variable continue
-------------	---	--

💡 Calcul de la médiane

On note $x_1, x_2, \dots, x_n$ les observations d'une variable quantitative et $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ la série ordonnée.

- Si n est **impair** : $Me = x_{(\frac{n+1}{2})}$
- Si n est **pair** : $Me = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$

```
df["prix"].mean()
df["prix"].median()
df["quartier"].mode()[0]
```

⚠️ Moyenne ou médiane ?

Sur une distribution fortement asymétrique — typiquement les salaires ou les prix de l'immobilier — la moyenne peut être fortement tirée vers le haut par quelques valeurs extrêmes. La **médiane** donne alors une image plus représentative de la « valeur typique ». C'est précisément pour cette raison qu'on préfère souvent la médiane à la moyenne pour imputer une valeur manquante (chapitre suivant, §3.1).

Dispersion

- l'**écart-type** (s) et la **variance** (s^2) mesurent l'écart moyen à la moyenne : $s = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$;
- l'**écart interquartile** ($IQR = Q_3 - Q_1$) mesure la dispersion de la moitié centrale des données, de façon robuste aux valeurs extrêmes ;
- l'**étendue** (maximum – minimum) donne une idée rapide, mais très sensible aux valeurs aberrantes.

💡 Calcul des quartiles (méthode de Tukey)

1. On calcule d'abord la **médiane** de toute la série.
2. On forme la **moitié inférieure** = toutes les observations **strictement inférieures** à la médiane.
3. On forme la **moitié supérieure** = toutes les observations **strictement supérieures** à la médiane.
4. Q_1 = médiane de la moitié inférieure.
5. Q_3 = médiane de la moitié supérieure.

```
df["prix"].std()
df["prix"].var()
df["prix"].quantile([0.25, 0.5, 0.75])
```

i Exemple détaillé — commandes

Montants de 9 commandes (en MAD) : 420, 95, 250, 110, 130, 160, 190, 80, 45

1. Série ordonnée

$$x_{(1)} = 45, x_{(2)} = 80, x_{(3)} = 95, x_{(4)} = 110, x_{(5)} = 130, x_{(6)} = 160, x_{(7)} = 190, x_{(8)} = 250, x_{(9)} = 420$$

2. Effectif : $n = 9$ · Minimum : 45 MAD · Maximum : 420 MAD

3. Moyenne

$$\bar{x} = \frac{45 + 80 + 95 + 110 + 130 + 160 + 190 + 250 + 420}{9} = \frac{1480}{9} \approx 164,44 \text{ MAD}$$

4. Médiane (n impair)

$$Me = x_{(\frac{n+1}{2})} = x_{(5)} = 130 \text{ MAD}$$

5. Quartiles

- Moitié inférieure (valeurs < 130) : 45, 80, 95, 110 $\rightarrow Q_1 = \text{médiane de ces 4 valeurs} = \frac{80 + 95}{2} = 87,5 \text{ MAD}$
- Moitié supérieure (valeurs > 130) : 160, 190, 250, 420 $\rightarrow Q_3 = \text{médiane de ces 4 valeurs} = \frac{190 + 250}{2} = 220 \text{ MAD}$

6. Écart interquartile

$$IQR = Q_3 - Q_1 = 220 - 87,5 = 132,5 \text{ MAD}$$

Interprétation : la moyenne (164 MAD) est tirée vers le haut par la commande de 420 MAD ; la médiane (130 MAD) est plus représentative du « client typique » ; 50 % des commandes se situent entre 87,5 et 220 MAD. Ce sont très exactement Q_1 et Q_3 qui serviront, au chapitre suivant, à détecter d'éventuelles valeurs aberrantes par la méthode de l'IQR.

Le **coefficient de variation**, défini par $s/\bar{x}$ lorsque la moyenne est positive et non nulle, permet parfois de comparer la dispersion relative de variables ayant des échelles différentes ; il est présenté ici comme un repère, non comme un indicateur obligatoire.

Forme de la distribution

Deux indicateurs complètent la description d'une distribution :

- l'**asymétrie** (*skewness*) : une valeur positive indique une distribution étalée vers la droite (queue de valeurs élevées), une valeur négative vers la gauche ;
- l'**aplatissement** (*kurtosis*) : indique si la distribution comporte davantage de valeurs extrêmes qu'une distribution normale.

```
df["prix"].skew()
df["prix"].kurt()
```

i Priorité pour débiter

Les indicateurs indispensables sont l'effectif, la moyenne, la médiane, le minimum, le maximum, les quartiles et l'écart-type ou l'IQR. L'asymétrie, l'aplatissement et le coefficient de variation sont des compléments : il suffit d'en comprendre l'idée et de savoir quand demander une explication supplémentaire.

Tout en une seule commande

La méthode `describe()` de **pandas** regroupe l'essentiel de ces indicateurs :

```
df.describe()
```

Elle fournit **count** (effectif), **mean** (moyenne), **50%** (médiane), **min/max** (extrêmes) et **25%/75%** (quartiles Q_1 et Q_3). Elle ne remplace pas l'interprétation : un indicateur doit toujours être accompagné de son **unité**, de sa **période** et de la **population** étudiée.

 Un indicateur n'est pas une conclusion

Une moyenne de 250 MAD ne dit pas si les commandes sont toutes proches de 250 MAD ou si quelques très grosses commandes compensent beaucoup de petites commandes. Pour comprendre la forme de la distribution, on complète toujours la moyenne par la médiane, les quartiles (ou l'IQR) et, idéalement, un graphique — histogramme ou boîte à moustaches, présentés au chapitre consacré à la visualisation.

 Bonnes pratiques

- Toujours indiquer l'unité (MAD, kg, jours...).
- Préciser la période et le périmètre (tous les clients, un segment, une région...).
- Présenter moyenne **et** médiane lorsque la distribution est asymétrique.
- Utiliser l'IQR plutôt que l'écart-type si l'on suspecte des valeurs extrêmes.

Exercices

Exercice 0.0.1 (Vrai ou Faux).

- a. La Business Intelligence répond principalement à la question « pourquoi ? ».
- b. Une analyse diagnostique s'appuie généralement sur une analyse descriptive préalable.
- c. Le Machine Learning est un prérequis indispensable pour produire une analyse descriptive.

Exercice 0.0.2 (Classer les questions). Pour chacune des questions suivantes, indiquez le type d'analyse correspondant (descriptive, diagnostique, prédictive ou prescriptive) :

- a. Combien de visiteurs le site web a-t-il reçus la semaine dernière ?
- b. Pourquoi le taux d'abandon de panier a-t-il augmenté en juillet ?
- c. Quelle campagne publicitaire faut-il privilégier pour la rentrée ?

Exercice 0.0.3 (Étude de cas courte). Une association marocaine de protection de l'environnement souhaite comprendre l'évolution du volume de déchets collectés dans trois villes sur les deux dernières années, puis identifier les facteurs expliquant les écarts entre villes.

- a. Quelles étapes du cycle de vie présenté en Figure 2 cette association devra-t-elle suivre ?
- b. À quel(s) type(s) d'analyse (Table 2) ce projet correspond-il ?

Exercice 0.0.4 (Choisir une analyse). Une responsable constate que les ventes sont plus faibles en août. Elle veut d'abord mesurer la baisse, puis comprendre si elle concerne seulement une région.

- a. Quel type d'analyse répond à la première question ?
- b. Quelle comparaison simple peut aider pour la seconde ?
- c. Pourquoi faut-il éviter de conclure à une cause avant d'avoir comparé les données ?

Exercice 0.0.5 (Reconnaître le type de variable). Classez les variables suivantes en qualitative (nominale ou ordinale), quantitative (discrète ou continue) ou temporelle : `quartier`, `nombre_de_produits`, `date_commande`, `satisfaction` codée « faible / moyenne / élevée ».

Exercice 0.0.6 (Moyenne, médiane et fréquence). Les montants de cinq commandes sont : 80, 100, 100, 120 et 600 MAD.

- a. Calculez la moyenne et la médiane.
- b. Quel indicateur décrit le mieux une commande « typique » ? Pourquoi ?
- c. Quel est l'effectif et la fréquence des commandes d'au moins 100 MAD ?

Exercice 0.0.7 (Lire une variation). Le nombre de visiteurs d'un site passe de 2 000 en avril à 2 400 en mai, puis à 2 280 en juin.

- Calculez la variation en pourcentage entre avril et mai.
- Calculez la variation en pourcentage entre mai et juin.
- La baisse de mai à juin annule-t-elle entièrement la hausse précédente ?

Exercice 0.0.8 (Identifier les indicateurs utiles). Pour chacune des questions, choisissez l'indicateur le plus directement utile : effectif, fréquence, moyenne, médiane, minimum/maximum ou IQR.

- Quelle part des clients utilise le paiement par carte ?
- Quel est le montant central des salaires ?
- Quelle est la largeur de la moitié centrale des durées de livraison ?
- Combien de commandes ont été enregistrées ?

Exercice 0.0.9 (Interpréter des statistiques descriptives). Pour une variable `revenu_mensuel` (en MAD), on obtient : moyenne = 8 500, médiane = 6 200, écart-type = 5 400, asymétrie (*skewness*) = 2,3.

- Que suggère l'écart important entre la moyenne et la médiane ?
- Que confirme la valeur positive et élevée de l'asymétrie ?

Exercice 0.0.10 (Lire une dispersion). Deux classes ont la même moyenne à un examen : 12/20. La classe A a un écart-type de 1,5 et la classe B un écart-type de 4.

- Quelle classe est la plus homogène ?
- Que signifie l'écart-type plus élevé de la classe B ?
- Peut-on dire que les étudiants de B ont de moins bonnes notes ?

Exercice 0.0.11 (Calculer des indicateurs simples). Les durées de livraison, en jours, sont : 2, 3, 3, 4, 8.

- Calculez la moyenne, la médiane et l'étendue.
- Quel indicateur est le moins influencé par la durée de 8 jours ?
- Quelle question supplémentaire poseriez-vous avant de conclure que le service est lent ?

Listing 0.1 Exemple de gestion d'un projet Python avec uv

```
# 1. Création d'un nouveau projet Python
# Cette commande crée la structure initiale du projet ainsi que
# le fichier pyproject.toml utilisé pour sa configuration.
uv init projet-analyse-donnees
cd projet-analyse-donnees

# 2. Ajout des dépendances du projet
# Les bibliothèques nécessaires au projet sont ajoutées au fichier
# pyproject.toml et leurs versions sont enregistrées dans uv.lock.
uv add pandas
uv add scikit-learn

# Ajout d'un outil utilisé uniquement pendant le développement.
# L'option --dev indique que Ruff est une dépendance de développement
# et n'est pas nécessaire à l'exécution du projet en production.
uv add ruff --dev

# Suppression d'une dépendance devenue inutile.
uv remove scikit-learn

# 3. Création et gestion de l'environnement virtuel
# uv peut créer automatiquement l'environnement virtuel lorsque cela
# est nécessaire. Cette commande permet de le créer explicitement.
uv venv
# cette n'est pas strictement nécessaire, car uv crée automatiquement l'environnement virtuel.

# Activation de l'environnement virtuel dans un terminal Bash.
source .venv/bin/activate

# 4. Synchronisation de l'environnement avec la configuration du projet
# Cette commande garantit que l'environnement correspond aux dépendances
# et aux versions enregistrées dans le fichier uv.lock.
uv sync

# 5. Création ou mise à jour du fichier de verrouillage
# Le fichier uv.lock enregistre les versions précises des dépendances
# afin de garantir la reproductibilité de l'environnement.
uv lock

# 6. Exécution de commandes dans l'environnement du projet
# uv run permet d'exécuter une commande avec les dépendances du projet
# sans avoir besoin d'activer manuellement l'environnement virtuel.
uv run python -c "import sys; print(sys.version)"
uv run mon_script.py

# Vérification de la qualité du code avec Ruff.
uv run ruff check .

# 7. Organisation d'un projet complexe avec un espace de travail
# Un workspace permet de regrouper plusieurs packages Python au sein
# d'un même projet et de gérer leurs dépendances de manière centralisée.
uv workspace init mon-workspace
```

Collecte, Nettoyage et Préparation des Données

! Un chapitre commun à deux cours

Ce chapitre est utilisé dans les deux éléments du module : « *Analyse de Données et Data Visualisation* » et « *Machine Learning* ».

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Identifier** les principales sources de collecte de données (bases de données, fichiers plats, API, *web scraping*, données ouvertes) et les principaux formats dans lesquels elles se présentent ;
- **Diagnostiquer** la représentativité d'un échantillon et les problèmes de qualité d'un jeu de données (valeurs manquantes, aberrantes, dupliquées, incohérentes) ;
- **Appliquer**, avec **pandas**, des stratégies adaptées de traitement des valeurs manquantes et aberrantes ;
- **Transformer** des variables catégorielles (encodage) et numériques (mise à l'échelle), en justifiant la méthode choisie ;
- **(DA)** Discretiser une variable numérique et en dériver de nouvelles variables utiles à l'exploration et à la visualisation ;
- **(ML)** Construire, avec **scikit-learn**, un pipeline de prétraitement complet (**ColumnTransformer** + **Pipeline**) exempt de fuite de données.

Si les données sont de mauvaise qualité, corrompues ou biaisées, ni les conclusions d'une analyse aussi soignée soit-elle, ni les prédictions d'un modèle aussi sophistiqué soit-il, ne pourront être fiables (*Garbage In, Garbage Out*).

— Principe fondamental de la Data Science

Ce chapitre couvre l'ensemble du travail qui précède l'analyse ou la modélisation à proprement parler : où trouver les données, comment s'assurer qu'elles sont représentatives, comment détecter et corriger leurs défauts, puis comment les transformer — qu'il s'agisse de les préparer pour l'exploration et la visualisation (cours d'Analyse de Données), ou pour les transmettre à un algorithme d'apprentissage (cours de Machine Learning). Dans les deux cours, ces étapes correspondent aux mêmes phases du cycle de vie d'un projet fondé sur la donnée, telles que présentées dans le chapitre d'introduction propre à chacun de vos deux cours.

i Une notation à connaître : le tableau de données, X et y

Tout au long de ce chapitre, une observation est une **ligne** d'un tableau et une variable est une **colonne**. En cours de Machine Learning, on distingue plus précisément les variables explicatives, regroupées dans une matrice notée X , de la variable à prédire, notée y — une notation que vous retrouverez dans toute la suite de ce cours. En Analyse de Données, cette distinction X/y n'est pas toujours nécessaire : on manipule plus simplement un unique tableau **pandas** (un **DataFrame**), dont certaines colonnes serviront à l'exploration et d'autres à la visualisation.

i Réflexe de départ : connaître son tableau avant d'y toucher

Avant toute modification, conservez une copie du fichier original et rédigez un petit **dictionnaire de données**. Pour chaque colonne, notez son nom, sa signification, son unité, son type attendu et les valeurs

possibles. Par exemple, **prix** peut être un nombre positif en MAD, tandis que **type_bien** peut prendre les valeurs « appartement », « maison » ou « terrain ». Ce dictionnaire évite de corriger à tort une valeur qui était en réalité parfaitement valide, et il documente le sens exact de chaque colonne pour quiconque reprendra votre travail par la suite.

1. Où trouver les données ? Les stratégies de collecte

Avant tout projet, il faut identifier la ou les sources permettant de constituer le jeu de données à traiter. Plusieurs grandes familles de sources coexistent dans la pratique professionnelle, qu'il s'agisse d'alimenter un tableau de bord ou d'entraîner un modèle.

1.1. Les bases de données

La source la plus courante en entreprise reste la base de données interne, qu'elle soit :

- **relationnelle (SQL)** : les données sont organisées en tables reliées par des clés (ex. PostgreSQL, MySQL, SQL Server) et interrogées via le langage **SQL** (*Structured Query Language*) ;
- **non-relationnelle (NoSQL)** : les données sont stockées sous une forme plus flexible — documents JSON (MongoDB), paires clé-valeur (Redis), ou graphes (Neo4j) — adaptée aux données peu structurées ou à très grande échelle.

pandas permet de charger directement le résultat d'une requête SQL dans un **DataFrame** :

```
import pandas as pd
import sqlite3

connexion = sqlite3.connect("ventes.db")
df = pd.read_sql("SELECT * FROM transactions WHERE annee = 2025", connexion)
```

1.2. Les fichiers plats et les principaux formats de données

En dehors des bases de données, les données sont fréquemment échangées sous forme de fichiers. Savoir reconnaître un format et choisir le bon outil de lecture est un prérequis à toute étape suivante :

TABLE 1 – Principaux formats de données rencontrés en pratique

Format	Caractéristiques	Lecture avec pandas
CSV	Texte tabulaire, très répandu, lisible par un humain	<code>pd.read_csv()</code>
JSON	Structure hiérarchique (objets imbriqués), courant pour les API	<code>pd.read_json()</code>
Excel (.xlsx)	Tableur, souvent utilisé en entreprise	<code>pd.read_excel()</code>
Parquet	Format binaire colonnaire, compressé, optimisé pour l'analytique	<code>pd.read_parquet()</code>

! Zoom sur le format Parquet

Contrairement au CSV, le format **Parquet** stocke les données par colonne plutôt que par ligne. Cela permet de ne lire que les colonnes réellement nécessaires à une analyse, de bénéficier d'une compression efficace et de conserver les types de données (un entier reste un entier, pas une chaîne de caractères à

réinterpréter). Il est aujourd'hui privilégié pour les jeux de données volumineux, et s'utilise nativement avec `pandas`, `Polars` ou `DuckDB`.

1.3. Les API

Une **API** (*Application Programming Interface*) est une interface mise à disposition par un fournisseur de données, qui permet d'obtenir des données structurées de façon normalisée — sans avoir à analyser du HTML.

On distingue notamment :

- les **API REST**, les plus répandues, où chaque ressource est accessible via une URL et une méthode HTTP (GET, POST...), avec une réponse généralement au format JSON ;
- les **API GraphQL**, plus récentes, qui permettent au client de préciser exactement les champs qu'il souhaite récupérer en une seule requête, réduisant ainsi le volume de données transférées inutilement.

```
import requests
import pandas as pd

reponse = requests.get("https://api.exemple.org/ventes?annee=2025")
reponse.raise_for_status()
df = pd.json_normalize(reponse.json()["resultats"])
```

`pd.json_normalize` est particulièrement utile lorsque la réponse JSON contient des objets imbriqués : elle « aplatit » automatiquement cette structure hiérarchique en un tableau exploitable.

1.4. Le web scraping

Lorsque les données ne sont disponibles que sur des pages web, sans base de données ni API accessibles, on a recours au **web scraping** : l'extraction automatisée d'informations à partir du code HTML de sites web. Les bibliothèques Python de référence sont :

- **BeautifulSoup** : idéale pour analyser (*parser*) le HTML d'une page unique et en extraire des éléments ciblés (balises, attributs) ;
- **Scrapy** : un *framework* complet, conçu pour des collectes à grande échelle sur de nombreuses pages, avec gestion native de la pagination, des files d'attente de requêtes et de l'exportation des résultats.

Éthique et légalité du web scraping

Le *web scraping* n'est pas toujours autorisé. Avant de collecter des données sur un site, il convient de vérifier :

- le fichier `robots.txt` du site, qui précise les zones autorisées ou interdites aux robots ;
- les conditions générales d'utilisation (CGU) du site, qui peuvent explicitement interdire la collecte automatisée ;
- le cadre réglementaire applicable à la protection des données personnelles (ex. loi marocaine 09-08, ou RGPD pour les données concernant des ressortissants européens), en particulier si des données à caractère personnel sont collectées.

1.5. Données ouvertes et capteurs connectés

Deux autres sources méritent d'être mentionnées :

- les **portails d'open data** (données publiques mises à disposition par des institutions : données démographiques, économiques ou climatiques) ;

- les **capteurs et objets connectés (IoT)**, qui génèrent des flux de données en continu (température, géolocalisation, fréquentation), souvent collectés via des files de messages (ex. MQTT, Kafka) avant d'être stockés pour l'analyse ou la modélisation.

TABLE 2 – Sources de données courantes

Source	Exemple d'outil ou de technologie
Bases de données	SQL (PostgreSQL, MySQL), NoSQL (MongoDB)
Fichiers plats	CSV, JSON, Excel, Parquet
API	REST, GraphQL
Web Scraping	BeautifulSoup, Scrapy
Données ouvertes / IoT	Portails <i>open data</i> , MQTT, Kafka

2. Représentativité et biais d'échantillonnage

Collecter des données ne suffit pas : encore faut-il qu'elles soient **représentatives** de la population sur laquelle les conclusions de l'analyse — ou les prédictions du modèle — seront ensuite utilisées. Un échantillon est représentatif lorsque sa distribution reflète fidèlement celle de la population cible.

! À retenir

Un échantillon non représentatif introduit un **biais** qui se propage à toutes les étapes suivantes, aussi rigoureuses soient-elles. Aucune technique d'analyse ou de modélisation, aussi avancée soit-elle, ne peut corriger un biais présent dès la collecte des données.

On distingue notamment trois types de biais fréquents en pratique :

- **Biais de sélection** : l'échantillon collecté ne couvre pas correctement certains sous-groupes de la population (ex. un sondage de satisfaction réalisé uniquement auprès des clients ayant un compte en ligne exclut de fait les clients moins digitalisés).
- **Biais de couverture temporelle** : les données ne couvrent qu'une période particulière, non représentative du comportement général (ex. collecter des données de vente uniquement pendant une période de soldes).
- **Biais de mesure** : l'instrument ou le protocole de collecte lui-même déforme la donnée (ex. un capteur mal calibré, un formulaire dont la formulation oriente la réponse).

i Exemple historique classique

Lors de l'élection présidentielle américaine de 1936, le magazine *Literary Digest* prédit, sur la base de plus de 2 millions de réponses à un sondage, la victoire du candidat Alf Landon. Le sondage fut envoyé par la poste à partir de listes d'abonnés au téléphone et de propriétaires de voitures — une population nettement plus aisée que la moyenne en pleine Grande Dépression. Malgré une taille d'échantillon considérable, le sondage souffrait d'un **biais de sélection** majeur, et la prédiction se révéla fausse : Franklin D. Roosevelt fut réélu avec une large majorité. Cet exemple illustre qu'un grand échantillon **non représentatif** reste moins fiable qu'un échantillon plus petit mais correctement constitué — un principe qui vaut aussi bien pour une étude statistique classique que pour l'entraînement d'un modèle de Machine Learning.

3. Nettoyage des données

Une fois collectées, les données contiennent presque toujours des imperfections : valeurs manquantes, doublons, incohérences et valeurs aberrantes sont la norme plutôt que l'exception. Un nettoyage rigoureux — c'est en général l'étape la plus chronophage d'un projet, souvent estimée (à titre indicatif) à 60-80 % du temps réel qui y est consacré — vérifie systématiquement au moins cinq dimensions de qualité :

1. **complétude** : les informations importantes sont-elles présentes ?
2. **validité** : les valeurs respectent-elles les règles attendues (un prix positif, une date valide) ?
3. **unicité** : une même observation n'est-elle pas enregistrée deux fois ?
4. **cohérence** : les colonnes et les unités sont-elles compatibles entre elles ?
5. **actualité** : les dates couvrent-elles la période utile à la question posée ?

💡 Documenter chaque correction

Il faut garder une trace de chaque correction apportée : colonne concernée, règle appliquée, nombre de lignes modifiées et justification. Cette documentation permet de revenir en arrière si nécessaire, et d'expliquer honnêtement les résultats obtenus en aval — qu'il s'agisse d'un graphique ou d'une prédiction.

3.1. Les valeurs manquantes

Une valeur manquante peut avoir des origines très différentes, ce qui conditionne directement la stratégie à adopter pour la traiter :

TABLE 3 – Typologie des valeurs manquantes

Type	Signification	Exemple
MCAR (<i>Missing Completely At Random</i>)	L'absence ne dépend d'aucune variable, ni observée ni cachée	Un capteur ou un bug technique tombe en panne aléatoirement
MAR (<i>Missing At Random</i>)	L'absence dépend d'une autre variable observée	Les hommes répondent moins souvent à une question sur le revenu, indépendamment du revenu lui-même
MNAR (<i>Missing Not At Random</i>)	L'absence dépend de la valeur manquante elle-même	Les personnes aux revenus les plus élevés refusent plus souvent de déclarer leur revenu

⚠ Le cas MNAR est le plus délicat

Le cas **MNAR** est le plus problématique : ignorer ou imputer naïvement ces valeurs manquantes peut introduire un biais systématique difficile à corriger, précisément parce que l'absence de la donnée est elle-même porteuse d'information. Ces trois étiquettes servent surtout à **réfléchir à la cause probable d'une absence** — il n'est pas nécessaire de réaliser un test statistique formel pour les distinguer dans ce cours, mais il faut absolument éviter d'imputer automatiquement une valeur manquante sans s'être interrogé sur ce qu'elle signifie.

Avec **pandas**, on détecte et on traite les valeurs manquantes ainsi :

```
# Détection
df.isna().sum()                # nombre de valeurs manquantes par colonne
df.isna().mean().round(2) * 100 # pourcentage de valeurs manquantes

# Suppression
df.dropna(subset=["prix"])      # supprime les lignes sans prix

# Imputation simple
df["surface"] = df["surface"].fillna(df["surface"].median())
df["quartier"] = df["quartier"].fillna("Non renseigné")
```

Stratégies de traitement des valeurs manquantes :

TABLE 4 – Stratégies d'imputation des valeurs manquantes

Stratégie	Principe	Limite
Suppression des lignes	Retirer toute observation comportant une valeur manquante	Perte d'information ; biaise l'échantillon si l'absence n'est pas MCAR
Imputation simple	Remplacer par la moyenne, la médiane (numérique) ou le mode (catégoriel)	Réduit artificiellement la variance de la variable
Valeur explicite	Remplacer par une catégorie dédiée, ex. « Non renseigné »	Adaptée aux variables catégorielles avec une proportion élevée de valeurs manquantes

💡 Comment choisir entre suppression et imputation ?

- Si la proportion de valeurs manquantes est **faible** (quelques pourcents) et que le mécanisme semble MCAR, la **suppression** des lignes concernées est souvent acceptable.
- Si la proportion est **élevée**, ou si l'absence porte sur une variable centrale de l'analyse, on préfère une **imputation**, en gardant à l'esprit qu'elle introduit un biais qu'il faut documenter.

⚠ Règle d'or (ML)

En Machine Learning, dès lors que les données seront séparées en jeux d'entraînement et de test (notion détaillée dans le chapitre suivant de votre cours), la stratégie d'imputation (moyenne, médiane...) doit être **calculée uniquement sur le jeu d'entraînement**, puis appliquée telle quelle aux jeux de validation et de test. Calculer cette statistique sur l'ensemble des données avant de les séparer constitue une **fuite de données** (*data leakage*) : le modèle bénéficierait indirectement d'une information provenant du jeu de test, ce qui fausserait l'évaluation finale de sa performance. Cette contrainte, propre à un contexte de modélisation prédictive, n'a pas d'équivalent en analyse de données exploratoire, où l'on travaille généralement sur l'ensemble du jeu de données disponible.

3.2. Les valeurs aberrantes (*outliers*)

Une valeur aberrante est une observation qui s'écarte fortement du comportement général des données — parfois une erreur de saisie, parfois un phénomène réel mais rare. Deux méthodes de détection sont couramment enseignées :

- **La méthode de l'écart interquartile (IQR)**, robuste aux distributions asymétriques : on considère comme aberrante toute valeur en dehors de l'intervalle

$$[Q_1 - 1,5 \times IQR, Q_3 + 1,5 \times IQR], \quad \text{avec } IQR = Q_3 - Q_1.$$

- **Le score-z**, qui suppose implicitement une distribution proche de la loi normale : toute valeur dont le score

$$z = \frac{x - \mu}{\sigma}$$

dépasse un seuil en valeur absolue (souvent $|z| > 3$) est considérée comme aberrante.

```
q1, q3 = df["prix"].quantile([0.25, 0.75])
iqr = q3 - q1
borne_basse, borne_haute = q1 - 1.5 * iqr, q3 + 1.5 * iqr

aberrants = df[(df["prix"] < borne_basse) | (df["prix"] > borne_haute)]
```

🔥 Une valeur aberrante n'est pas toujours une erreur

Avant de supprimer systématiquement les valeurs aberrantes détectées, il faut se demander si elles reflètent une **erreur de mesure** (à corriger ou supprimer) ou un **phénomène réel mais rare** (par exemple, un très gros client dans une base de transactions commerciales) — auquel cas leur suppression ferait perdre une information potentiellement précieuse, aussi bien pour une analyse exploratoire que pour un modèle.

ℹ Repérer visuellement les valeurs aberrantes (DA)

La **boîte à moustaches** (*boxplot*), présentée en détail dans le chapitre de votre cours consacré à la visualisation des données, reste l'outil graphique privilégié pour repérer d'un coup d'œil les valeurs aberrantes d'une variable numérique, sans avoir à calculer explicitement les bornes de l'IQR.

3.3. Les doublons et les incohérences

Un doublon (deux lignes strictement identiques, ou identiques sur les variables clés) doit généralement être supprimé, car il donne artificiellement plus de poids à une observation — un risque parfois qualifié de « double comptage », qu'il s'agisse de fausser une moyenne calculée pour un tableau de bord ou de biaiser les paramètres appris par un algorithme.

```
df.duplicated().sum()          # nombre de lignes strictement dupliquées
df = df.drop_duplicates()

df["ville"] = df["ville"].str.strip().str.title()      # espaces superflus, casse
df["date"] = pd.to_datetime(df["date"], errors="coerce") # type de données cohérent
```

Les incohérences les plus fréquentes concernent la **casse** et les **espaces superflus** dans les chaînes de caractères (« casablanca » vs « Casablanca »), les **types de données** mal interprétés (une date lue comme du texte), et les **unités hétérogènes** (des prix exprimés tantôt en MAD, tantôt en euros ; des distances tantôt en mètres, tantôt en kilomètres).

3.4. Vers une démarche de qualité systématique

Sur des projets récurrents — un tableau de bord alimenté quotidiennement, ou un modèle réentraîné chaque semaine sur de nouvelles données — il devient risqué de vérifier la qualité des données « à l'œil » à chaque exécution. Des bibliothèques comme **Great Expectations** permettent de définir des règles de validation explicites (« la colonne **prix** doit toujours être positive », « la colonne **date** ne doit jamais être manquante ») et de les exécuter automatiquement à chaque nouvelle collecte, afin de détecter une dérive de qualité avant qu'elle n'affecte l'analyse en aval ou, en Machine Learning, avant qu'elle ne dégrade silencieusement la performance d'un modèle en production.

4. Encodage des variables catégorielles

De nombreux outils, qu'ils soient statistiques, graphiques ou algorithmiques, manipulent plus facilement des valeurs **numériques** que du texte. C'est particulièrement vrai en Machine Learning, où la quasi-totalité des algorithmes reposent sur des opérations mathématiques — distances, produits scalaires, sommes pondérées — qui ne peuvent s'appliquer qu'à des nombres. L'**encodage** désigne l'ensemble des techniques permettant de transformer une variable catégorielle en une représentation numérique exploitable, **sans dénaturer l'information** qu'elle porte.

On distingue deux grandes stratégies, quelle que soit la bibliothèque utilisée pour les mettre en œuvre :

- l'**encodage ordinal**, adapté aux catégories qui possèdent un **ordre naturel** (ex. Faible < Moyen < Élevé) : chaque catégorie est simplement remplacée par un entier reflétant cet ordre ;
- l'**encodage One-Hot** (variables indicatrices), adapté aux catégories **sans ordre** (ex. les quartiers d'une ville) : chaque catégorie devient une nouvelle colonne binaire (0 ou 1).

⚠ Piège fréquent

Appliquer un encodage ordinal à une variable **nominale** (sans ordre naturel, ex. `quartier` : Centre, Nord, Sud) introduit un ordre artificiel que l'outil en aval interprétera à tort comme porteur de sens — un algorithme pourrait, par exemple, en déduire que `Sud` (codé 2) est « plus grand » que `Centre` (codé 0), ce qui n'a aucune signification réelle.

4.1. Encodage rapide avec pandas (DA)

Pour faciliter certains graphiques ou calculs de corrélation, un encodage simple, réalisé directement avec `pandas`, est généralement suffisant en Analyse de Données :

```
# Encodage ordinal
ordre = {"Faible": 0, "Moyen": 1, "Élevé": 2}
df["niveau_encode"] = df["niveau"].map(ordre)

# Encodage One-Hot
df_encode = pd.get_dummies(df, columns=["quartier"])
```

i Note

Cet encodage simple suffit pour préparer un graphique ou une matrice de corrélation. Un encodage plus robuste, fondé sur `scikit-learn` et pensé pour alimenter un algorithme d'apprentissage — avec gestion des catégories rares, des catégories inconnues au moment du déploiement, et intégration dans un pipeline reproductible — est approfondi dans le module de Machine Learning (§4.2 ci-dessous).

4.2. Encodage robuste avec scikit-learn (ML)

En Machine Learning, l'encodage doit être **réutilisable à l'identique** sur de nouvelles données (le jeu de test, puis les données rencontrées en production) : on ne se contente donc pas d'une transformation ponctuelle d'un `DataFrame`, mais d'un objet **ajusté** (`fit`) puis **appliqué** (`transform`), exactement comme les autres outils de prétraitement de ce chapitre.

Encodage ordinal, avec `OrdinalEncoder` : dans Scikit-Learn, on réserve `OrdinalEncoder` aux **variables explicatives** ordinales (dans X) ; `LabelEncoder`, plus rarement utile ici, est en réalité conçu pour encoder la **variable cible** y en classification, et non les colonnes de X — une confusion fréquente à éviter.

```
from sklearn.preprocessing import OrdinalEncoder

encodeur_ordinal = OrdinalEncoder(categories=[["Faible", "Moyen", "Élevé"]])
X["niveau_encode"] = encodeur_ordinal.fit_transform(X[["niveau"]])
```

Encodage One-Hot, avec `OneHotEncoder` : chaque catégorie devient une colonne binaire indépendante.

Par exemple, la variable `quartier` (Centre, Nord, Sud) devient :

TABLE 5 – Encodage One-Hot de la variable `quartier`

quartier_Centre	quartier_Nord	quartier_Sud
1	0	0
0	1	0
0	0	1

```
from sklearn.preprocessing import OneHotEncoder

encodeur_onehot = OneHotEncoder(drop="first", handle_unknown="ignore")
quartier_encode = encodeur_onehot.fit_transform(X[["quartier"]])
```

i Le piège de la variable factice (*Dummy Variable Trap*)

Lorsqu'une variable comporte k catégories, il suffit en réalité de $k - 1$ colonnes binaires pour représenter toute l'information : la dernière catégorie est déductible des autres (si `quartier_Centre` = 0 et `quartier_Nord` = 0, alors nécessairement `quartier_Sud` = 1). Conserver les k colonnes introduit une redondance parfaite entre variables (colinéarité), problématique pour certains algorithmes comme la régression linéaire. L'argument `drop='first'` de `OneHotEncoder` supprime automatiquement cette colonne redondante — une précaution utile pour un modèle linéaire avec constante (*intercept*), mais pas une règle universelle : pour un modèle fondé sur des arbres de décision, par exemple, supprimer une catégorie n'est généralement pas nécessaire, et conserver toutes les colonnes peut même faciliter l'interprétation. Le choix de la catégorie de référence supprimée doit donc rester **conscient**, et non automatique par réflexe.

`handle_unknown="ignore"` mérite une attention particulière : il indique à l'encodeur comment se comporter s'il rencontre, lors du déploiement, une catégorie **jamais vue** pendant l'entraînement (par exemple, un nouveau quartier apparu après la mise en production du modèle) — plutôt que de provoquer une erreur, il encode alors cette observation comme un vecteur de zéros.

4.3. Comparatif des méthodes d'encodage

TABLE 6 – Comparaison des principales méthodes d'encodage

Méthode	Adaptée à	Nombre de colonnes créées	Outil <code>pandas</code>	Outil <code>scikit-learn</code> (ML)
Encodage ordinal	Variables ordinales	1 (inchangé)	<code>.map()</code>	<code>OrdinalEncoder</code>
Encodage One-Hot	Variables nominales , peu de catégories	k (ou $k - 1$)	<code>pd.get_dummies()</code>	<code>OneHotEncoder</code>

💡 Au-delà de ce chapitre (ML)

Lorsqu'une variable catégorielle comporte des centaines, voire des milliers de catégories (ex. un identifiant de ville, un code produit), l'encodage One-Hot devient impraticable (explosion du nombre de colonnes). Des techniques plus avancées, comme le **Target Encoding** (remplacer chaque catégorie par la moyenne de la cible y associée) ou le **Frequency Encoding** (remplacer chaque catégorie par sa fréquence d'apparition), permettent de contourner ce problème — au prix d'une vigilance accrue vis-à-vis de la fuite de données, le Target Encoding utilisant directement la cible y pour construire l'encodage. Ces techniques dépassent le cadre de ce chapitre et seront abordées dans un cours de Machine Learning plus avancé.

5. Mise à l'échelle : standardisation et normalisation

Lorsque plusieurs variables numériques ont des échelles très différentes (ex. une surface en m^2 et un prix en millions de MAD), il est souvent utile de les ramener à une échelle comparable — que ce soit pour faciliter certaines visualisations combinées, ou parce qu'un algorithme y est directement sensible.

5.1. Les deux transformations de référence

La **standardisation** (*Z-score scaling*) centre chaque variable autour d'une moyenne de 0, avec un écart-type de 1 :

$$x' = \frac{x - \mu}{\sigma} \quad (0.1)$$

où μ est la moyenne et σ l'écart-type de la variable.

La **normalisation** (*Min-Max scaling*) ramène chaque variable dans un intervalle fixe, généralement $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (0.2)$$

```
df["prix_standard"] = (df["prix"] - df["prix"].mean()) / df["prix"].std()
df["prix_norm"] = (df["prix"] - df["prix"].min()) / (df["prix"].max() - df["prix"].min())
```

Lire un résultat de normalisation min-max

Une valeur normalisée de 0,5 ne signifie pas que l'observation correspond à la moyenne de la variable : elle signifie que l'observation se situe **exactement à mi-chemin** entre le minimum et le maximum observés. Ne confondez pas position relative dans l'intervalle et position relative à la moyenne.

5.2. Quand utiliser l'une ou l'autre ?

TABLE 7 – Choix entre standardisation et normalisation

Critère	Standardisation (Équation 0.1)	Normalisation (Équation 0.2)
Sensibilité aux valeurs aberrantes	Modérée	Élevée (les valeurs extrêmes déterminent $x_{\min}/x_{\max}$)
Hypothèse sur la distribution	Aucune (mais adaptée aux distributions proches de la normale)	Aucune
Intervalle de sortie	Non borné (généralement $[-3, 3]$ environ)	Borné, $[0, 1]$
Cas d'usage typique	Algorithmes basés sur le gradient, PCA, régression, comparaisons statistiques	Réseaux de neurones (fonctions d'activation bornées), traitement d'image (pixels)

La normalisation est très sensible aux valeurs extrêmes

Si une variable comporte quelques valeurs extrêmes mais légitimes (ex. `revenu_annuel`, avec l'essentiel des observations entre 30 000 et 200 000 MAD, mais quelques grandes fortunes dépassant 5 000 000

MAD), une normalisation min-max écrasera la quasi-totalité des observations « normales » dans un intervalle minuscule proche de 0, car x_{max} sera fixé par ces quelques valeurs extrêmes. Le **RobustScaler** de **scikit-learn**, fondé sur la **médiane** et l'**écart interquartile** plutôt que sur le minimum/maximum ou la moyenne/écart-type, est nettement plus robuste dans ce cas de figure.

5.3. Pourquoi certains algorithmes y sont-ils sensibles ? (ML)

De nombreux algorithmes de Machine Learning — en particulier ceux fondés sur une notion de **distance** entre observations (les k plus proches voisins, le clustering K-means) ou sur une **descente de gradient** (régression linéaire, réseaux de neurones) — sont directement sensibles à l'échelle des variables. Une variable exprimée en milliers de dirhams (ex. un prix) dominera artificiellement une variable exprimée en unités (ex. un nombre de pièces) dans le calcul d'une distance ou d'un gradient, même si cette dernière est tout aussi pertinente pour la tâche à résoudre. Mettre les variables à une échelle comparable neutralise cet artefact purement numérique.

⚠ Règle d'or (ML)

Comme pour l'imputation (§3.1), les paramètres de mise à l'échelle (μ , σ , x_{min} , x_{max}) doivent être **estimés uniquement sur le jeu d'entraînement** (méthode `.fit()`), puis appliqués tels quels aux jeux de validation et de test (méthode `.transform()`, sans nouveau `.fit()`) — toute autre pratique constitue une fuite de données. Ce n'est là qu'un cas particulier de la règle déjà énoncée pour l'imputation : toute statistique calculée à partir des données (moyenne, écart-type, minimum, maximum...) et réutilisée pour transformer de nouvelles observations doit être apprise **exclusivement** sur le jeu d'entraînement.

6. Transformations complémentaires pour l'exploration (DA)

Au-delà de l'encodage et de la mise à l'échelle, certaines transformations, bien que moins critiques pour un algorithme, facilitent considérablement l'exploration et la visualisation d'un jeu de données.

6.1. Discrétisation (*binning*)

La discrétisation consiste à transformer une variable numérique continue en catégories, souvent utile pour simplifier une visualisation (par exemple des tranches d'âge plutôt qu'un âge exact) :

```
df["tranche_age"] = pd.cut(
    df["age"],
    bins=[0, 18, 35, 60, 120],
    labels=["Mineur", "Jeune adulte", "Adulte", "Senior"],
)
```

Cette transformation simplifie la lecture d'un graphique et permet de comparer des groupes, mais elle fait perdre de la précision : deux personnes de 18 et 34 ans se retrouvent dans le même groupe malgré un écart d'âge important. Les bornes retenues doivent donc toujours être documentées et justifiées.

6.2. Quelques transformations usuelles supplémentaires

- **Extraction de composantes temporelles** : à partir d'une colonne `date`, on extrait fréquemment l'année, le mois ou le jour de la semaine (`df["date"].dt.month`), particulièrement utiles pour explorer des séries temporelles.
- **Agrégats dérivés** : création de nouvelles colonnes résumant une information (ex. le prix au mètre carré, `prix / surface`), qui facilitent grandement l'exploration et la comparaison entre observations.

i Le pendant en Machine Learning

Ces mêmes techniques — créer une nouvelle colonne à partir de colonnes existantes — portent, en Machine Learning, le nom de **feature engineering**. Le principe est identique ; seul l'objectif final diffère : ici, faciliter la lecture d'un graphique ; en Machine Learning, fournir à un algorithme une variable plus informative que les variables brutes dont elle est dérivée.

⚠ Ne pas transformer sans objectif

Une transformation n'est pas automatiquement une amélioration. Par exemple, remplacer toutes les valeurs manquantes par zéro change la signification de la variable : « absence d'information » devient « valeur nulle ». Avant chaque transformation, demandez-vous quelle question elle facilite et si son effet sera documenté.

7. Construire un pipeline de prétraitement automatisé (ML)

En pratique, un même jeu de données combine généralement des variables numériques et catégorielles, chacune nécessitant un traitement distinct. En Machine Learning, où ce prétraitement devra être **appliqué à l'identique** sur des données jamais vues (le jeu de test, puis les données rencontrées en production), il ne suffit plus d'appliquer des transformations ponctuelles à un `DataFrame` : il faut les **enchaîner et les automatiser** dans un objet réutilisable.

7.1. ColumnTransformer et Pipeline de scikit-learn

Scikit-Learn propose deux briques complémentaires pour construire un prétraitement reproductible et sans fuite de données :

- **Pipeline** : enchaîne plusieurs transformations (par ex. imputation puis mise à l'échelle) appliquées à **toutes** les colonnes d'un même sous-ensemble ;
- **ColumnTransformer** : applique des **Pipeline différents** selon le type de colonne (numérique ou catégorielle), puis **fusionne** les résultats en une seule matrice.

La Figure 1 illustre cette architecture : les colonnes numériques suivent une chaîne `SimpleImputer` → `StandardScaler`, tandis que les colonnes catégorielles suivent une chaîne `SimpleImputer` → `OneHotEncoder`, avant que les deux résultats ne soient recombinaés en une seule matrice, transmise au modèle.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

colonnes_numeriques = ["surface", "nb_pieces"]
colonnes_categorielles = ["quartier"]

pipeline_numerique = Pipeline([
    ("imputation", SimpleImputer(strategy="median")),
    ("mise_a_l_echelle", StandardScaler()),
])

pipeline_categorielle = Pipeline([
    ("imputation", SimpleImputer(strategy="most_frequent")),
    ("encodage", OneHotEncoder(drop="first", handle_unknown="ignore")),
])

pretraitement = ColumnTransformer([
    ("num", pipeline_numerique, colonnes_numeriques),
```

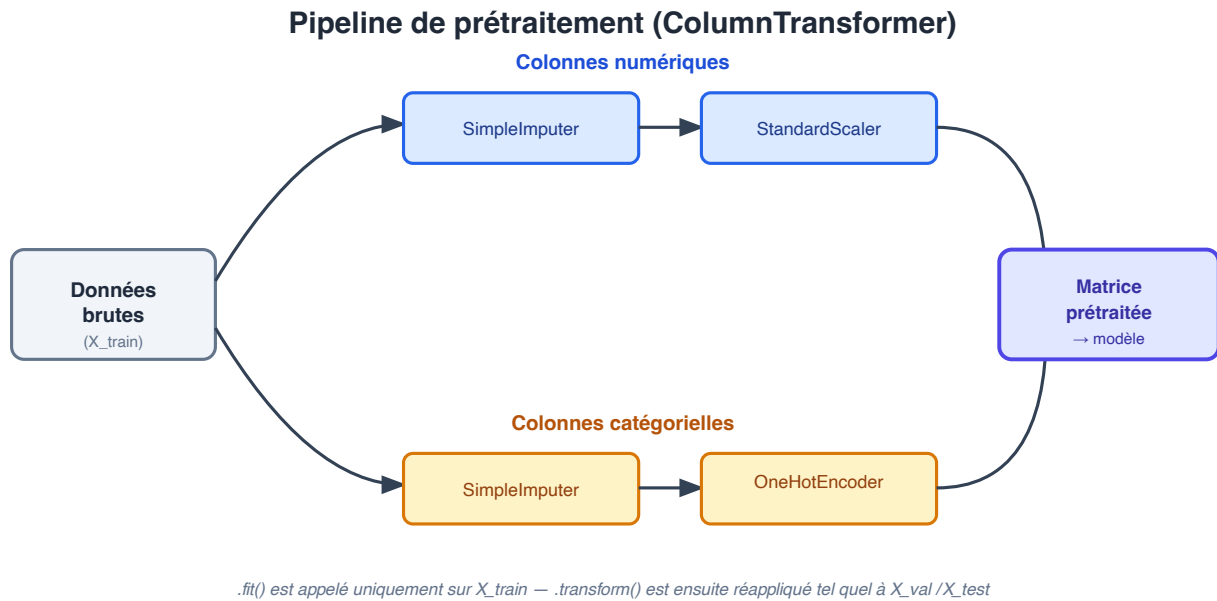


FIGURE 1 – Architecture d'un pipeline de prétraitement combinant colonnes numériques et catégorielles

```

("cat", pipeline_categorielle, colonnes_categorielles),
])

# Fit UNIQUEMENT sur les données d'entraînement (voir §3.1 et §5.3 de ce chapitre)
X_train_pret = pretraitement.fit_transform(X_train)
# Le même prétraitement, déjà appris, est simplement réappliqué :
X_test_pret = pretraitement.transform(X_test)

```

💡 Fonctionnalité récente : sortie en DataFrame

Depuis les versions récentes de Scikit-Learn, l'appel `pretraitement.set_output(transform="pandas")` permet d'obtenir directement un **DataFrame** pandas en sortie du pipeline (avec des noms de colonnes explicites comme `num_surface` ou `cat_quartier_Nord`), plutôt qu'un simple tableau `numpy` sans noms — un gain de lisibilité appréciable lors du diagnostic d'un pipeline. Cette fonctionnalité suppose une version suffisamment récente de `scikit-learn` et la présence de `pandas` ; elle ne garantit pas que **chaque** étape d'un pipeline retournera systématiquement un **DataFrame** sans condition, en particulier avec des transformateurs tiers.

7.2. Mettre à jour l'environnement du projet

En cohérence avec la gestion de projet de votre cours (gestionnaire d'environnement `uv`), les nouvelles dépendances nécessaires à ce chapitre sont ajoutées explicitement au projet :

```

uv add scikit-learn
uv lock # fige les versions exactes dans uv.lock, pour la reproductibilité

```

8. Synthèse

Exercices

i Consigne

Ces exercices sont **théoriques** : ils testent votre compréhension des concepts du chapitre, sans nécessiter d'écrire de code (sauf mention contraire). La mise en pratique sur des jeux de données réels fera l'objet du TP associé à ce chapitre, propre à chacun de vos deux cours. Les corrigés sont fournis juste après chaque énoncé, dans un bloc repliable.

Collecte et représentativité

Exercice 0.0.1 (Exercice 1 — Choisir une source de données). Pour chacun des besoins suivants, indiquez la stratégie de collecte la plus appropriée (base de données, fichier plat, *web scraping*, ou API) et justifiez en une phrase.

- Récupérer en temps réel le cours de change EUR/MAD.
- Constituer un historique des prix de l'immobilier à partir d'annonces publiées sur un site sans API publique.
- Extraire l'historique des commandes déjà stockées dans le système d'information interne de l'entreprise.

Exercice 0.0.2 (Exercice 2 — Identifier un biais). Une application mobile de course à pied souhaite tirer des conclusions sur le temps moyen que mettent ses utilisateurs sur un semi-marathon. Les données proviennent exclusivement des utilisateurs ayant activé le partage de leurs statistiques sur les réseaux sociaux.

- Quel type de biais cette collecte introduit-elle probablement ?
- Les conclusions tirées de cet échantillon seront-elles fiables pour un coureur occasionnel qui ne partage jamais ses performances ?

Valeurs manquantes

Exercice 0.0.3 (Exercice 3 — Nature d'une valeur manquante et stratégie). Un jeu de données de candidatures à un emploi comporte une colonne `salaire_souhaite` manquante pour 40 % des lignes. Une analyse rapide montre que les candidats les plus expérimentés (souvent plus exigeants sur le salaire) sont surreprésentés parmi les valeurs manquantes.

- À quel type de valeur manquante (Table 3) correspond cette situation ?
- Recommanderiez-vous une suppression ou une imputation ? Justifiez.

Exercice 0.0.4 (Exercice 4 — Nature d'une valeur manquante, second cas). Dans une enquête de satisfaction, on observe que les clients ayant donné une note très basse au service client laissent beaucoup plus souvent la question « Recommanderiez-vous ce service ? » sans réponse que les autres clients.

- À quelle catégorie (MCAR, MAR, MNAR) ce type d'absence appartient-il le plus probablement ?
- Pourquoi une imputation simple par le mode (la réponse la plus fréquente) serait-elle risquée dans ce cas précis ?

Exercice 0.0.5 (Exercice 5 — Choisir une stratégie d'imputation). Associez chaque situation à une stratégie raisonnable : supprimer quelques lignes, utiliser la médiane, créer la catégorie « Non renseigné », ou ne pas imputer sans investigation.

- 1 % des lignes n'ont pas de surface, et la surface est numérique.

- b. 35 % des lignes n'ont pas de quartier.
- c. Une date manquante empêche de savoir à quelle période appartient une vente.
- d. Une variable `remise` a 2 % de valeurs manquantes dans une distribution très asymétrique.

Valeurs aberrantes

Exercice 0.0.6 (Exercice 6 — Calculer des bornes IQR). Pour une variable `prix` avec $Q_1 = 200\,000$ et $Q_3 = 450\,000$ (en MAD), calculez les bornes de détection des valeurs aberrantes selon la méthode de l'IQR.

Exercice 0.0.7 (Exercice 7 — Diagnostic global de qualité). Un fichier de ventes contient 1 000 lignes. La colonne `date` comporte 20 valeurs manquantes, `prix` contient 5 valeurs négatives et 12 lignes sont des doublons exacts.

- a. Quel pourcentage de dates est manquant ?
- b. Quel problème doit être vérifié en priorité avant de calculer le chiffre d'affaires ?
- c. Les doublons doivent-ils être supprimés automatiquement ? Justifiez.

Exercice 0.0.8 (Exercice 8 — Unités et cohérence). Une colonne `distance` contient les valeurs 5, 12 et 8 pour trois trajets, mais le dictionnaire de données indique que certaines lignes sont en kilomètres et d'autres en mètres.

- a. Pourquoi est-il dangereux de calculer une moyenne directement ?
- b. Quelle action doit être réalisée avant l'analyse ?
- c. Est-ce un problème de valeur manquante, de validité ou de cohérence ?

Encodage

Exercice 0.0.9 (Exercice 9 — Choisir un type d'encodage). Pour chacune des variables suivantes, indiquez si un encodage **ordinal** ou **One-Hot** est le plus adapté, et justifiez.

- a. Le niveau d'étude (« Bac », « Bac+2 », « Bac+3 », « Bac+5 »).
- b. La couleur préférée d'un client (« Rouge », « Bleu », « Vert »).

Exercice 0.0.10 (Exercice 10 — Un cas moins évident). Pour chacune des variables suivantes, indiquez si un encodage ordinal ou un One-Hot Encoding est le plus adapté, et justifiez.

- a. Le niveau de scolarité d'un client : Primaire, Secondaire, Supérieur.
- b. La couleur préférée d'un client : Rouge, Bleu, Vert, Jaune.
- c. Le mois de la transaction (Janvier, ..., Décembre).

Mise à l'échelle

Exercice 0.0.11 (Exercice 11 — Calculer une normalisation min-max). Une variable `score` varie de 20 à 80. Calculez la valeur normalisée d'un score égal à 50 avec la formule de l'Équation 0.2. Que signifie le résultat obtenu ?

Exercice 0.0.12 (Exercice 12 — Standardisation ou normalisation face aux valeurs extrêmes). Un jeu de données contient une variable `revenu_annuel` dont la quasi-totalité des valeurs se situent entre 30 000 et 200 000 MAD, à l'exception de trois valeurs extrêmes dépassant 5 000 000 MAD (probablement des grandes fortunes, mais authentiques).

- a. Que se passerait-il si l'on appliquait une normalisation (*Min-Max scaling*) à cette variable en l'état ?
- b. Quelle alternative, parmi celles évoquées dans ce chapitre, serait plus robuste face à ces valeurs extrêmes ?

Machine Learning : fuite de données et pipeline (ML)

Exercice 0.0.13 (Exercice 13 — Repérer une fuite de données). Un étudiant applique la démarche suivante :

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)           # sur l'ensemble des données
X_train, X_test = train_test_split(X_scaled, test_size=0.2)
```

- Ce code contient-il une fuite de données ? Justifiez.
- Proposez, en français (sans code), la démarche correcte à suivre.

Exercice 0.0.14 (Exercice 14 — Rôle du `ColumnTransformer`). Un jeu de données comporte à la fois des variables numériques (`surface`, `age_bien`) et catégorielles (`quartier`, `type_bien`).

- Pourquoi ne peut-on pas appliquer un unique `Pipeline` (imputation puis `StandardScaler`) à l'ensemble des colonnes sans distinction ?
- Quel est le rôle précis du `ColumnTransformer` dans ce contexte ?

Exercice 0.0.15 (Exercice 15 — Doublons et double comptage dans un modèle). Une équipe fusionne deux exports de données provenant de deux systèmes différents de l'entreprise, contenant chacun une partie commune de clients. Après fusion, certains clients apparaissent deux fois avec des valeurs identiques.

- Quel est le risque si ces doublons ne sont pas détectés avant l'entraînement d'un modèle ?
- Ce risque affecte-t-il spécifiquement un algorithme de classification, ou tout algorithme entraîné sur ces données ?

Analyse de Données : discrétisation (DA)

Exercice 0.0.16 (Exercice 16 — Discrétisation et interprétation). On regroupe les âges selon les intervalles $[0, 18[$, $[18, 35[$, $[35, 60[$ et $[60, 120]$.

- Dans quelle classe place-t-on les âges 18, 34, 35 et 60 ?
- Quel avantage apporte cette transformation pour un tableau de bord ?
- Quel risque faut-il signaler ?

Exploration des données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Structurer** une exploration en trois temps — univarié, bivarié, multivarié — et choisir le bon outil selon les types de variables en présence ;
- **Mesurer** et **interpréter** une relation entre deux variables quantitatives (covariance, corrélation de Pearson et de Spearman) ;
- **Comparer** une variable quantitative entre plusieurs groupes (analyse de variance, η^2) et **tester** l'association entre deux variables qualitatives (χ^2 , V de Cramér) ;
- **Reconnaître** les pièges classiques de l'analyse bivariée : le quartet d'Anscombe, le paradoxe de Simpson, la confusion entre corrélation et causalité, les tests multiples ;
- **Transformer** une variable très asymétrique (transformation logarithmique) et **diagnostiquer** sa distribution ;
- **Résumer** un jeu de données par groupe avec `groupby` et des tableaux croisés dynamiques (`pivot_table`) ;
- **Explorer** une série temporelle : tendance, saisonnalité, variation ;
- **Situer** l'apport d'un outil de profilage automatique (`ydata-profiling`, `sweetviz`).

i Ce que ce chapitre suppose déjà acquis

Le chapitre d'introduction vous a donné le vocabulaire de base pour décrire **une variable prise isolément** : types de variables, effectif, fréquence, variation relative, moyenne, médiane, quartiles, écart-type. Le chapitre suivant vous a montré comment obtenir un jeu de données **propre**. Ce chapitre part de ces deux acquis pour aller plus loin : il s'intéresse aux **relations entre variables**, sur un jeu de données déjà nettoyé — exactement l'étape « Exploration » du cycle de vie présenté en introduction.

1. Structurer l'exploration : univariée, bivariée, multivariée

Une exploration désordonnée — un graphique après l'autre, sans plan — fait perdre du temps et laisse souvent passer l'essentiel. Il est plus efficace de progresser en entonnoir, comme le montre la Figure 1 :

1. **Premier regard** : dimensions du tableau, types de colonnes, valeurs manquantes — un rappel de l'audit déjà pratiqué au chapitre précédent.
2. **Analyse univariée** : que vaut chaque variable prise seule ? (chapitre d'introduction, §3).
3. **Analyse bivariée** : deux variables évoluent-elles ensemble ? C'est le cœur de ce chapitre.
4. **Analyse multivariée** : une troisième variable change-t-elle la lecture d'une relation à deux variables ?
5. **Hypothèses et suites** : que retenir, que vérifier plus formellement, que documenter ?

! Univarié, bivarié, multivarié : une question de nombre de variables

- **Univariée** : une seule variable à la fois (ex. la distribution des prix).
- **Bivariée** : deux variables croisées (ex. le prix *selon* la surface).
- **Multivariée** : trois variables ou plus étudiées conjointement (ex. le prix selon la surface, *en distinguant* les quartiers).

La démarche d'exploration en entonnoir

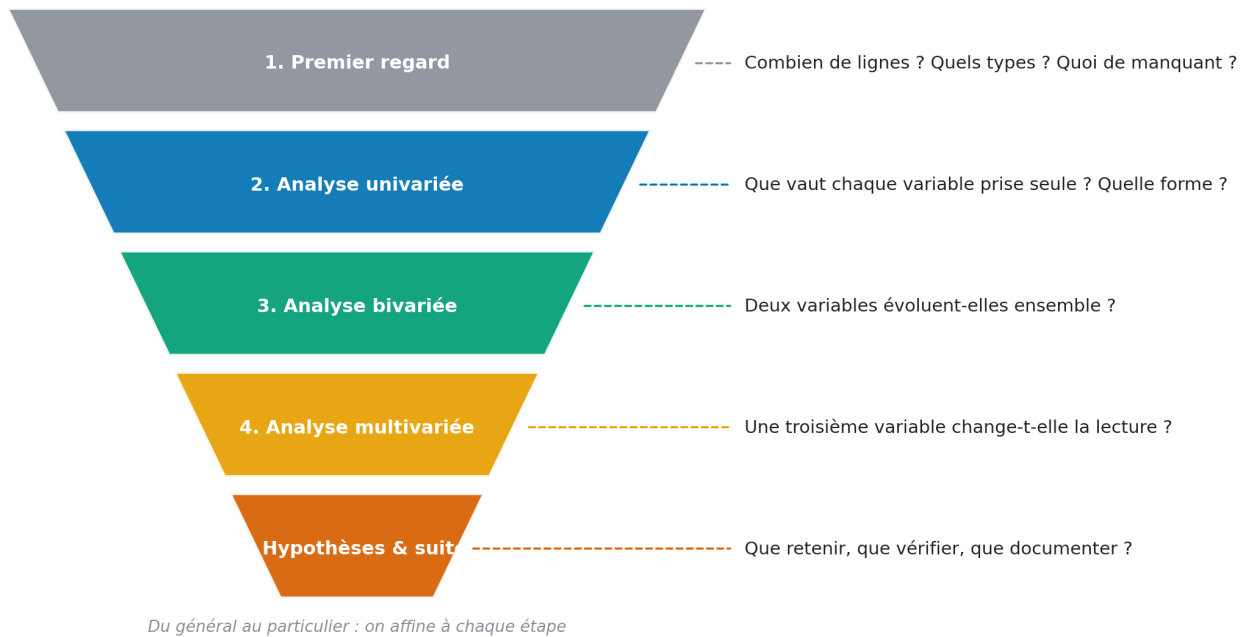


FIGURE 1 – La démarche d’exploration en entonnoir : du général au particulier

1.1. Quel outil pour quelle paire de variables ?

Le choix de l’outil statistique dépend directement des **types** des deux variables étudiées — la typologie posée en introduction (qualitative/quantitative) refait surface ici, appliquée cette fois à une **paire** de variables.

TABLE 1 – Choisir un outil d’analyse bivariable selon les types de variables

Paire de variables	Question typique	Outils présentés dans ce chapitre
Quantitative × Quantitative	Le prix augmente-t-il avec la surface ?	Nuage de points, covariance, corrélation de Pearson/Spearman (§2)
Quantitative × Qualitative	Le prix diffère-t-il selon le quartier ?	Statistiques par groupe, analyse de variance, η^2 (§3)
Qualitative × Qualitative	Le type de bien est-il lié à la présence d’un ascenseur ?	Tableau croisé, χ^2 , V de Cramér (§4)

2. Relation entre deux variables quantitatives : la corrélation

2.1. De la covariance au coefficient de corrélation

La **covariance** mesure si deux variables varient dans le même sens :

$$\text{Cov}(X, Y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

- Positive : quand x augmente, y a tendance à augmenter aussi.
- Négative : quand x augmente, y a tendance à diminuer.

Quel outil pour quelle paire de variables ?

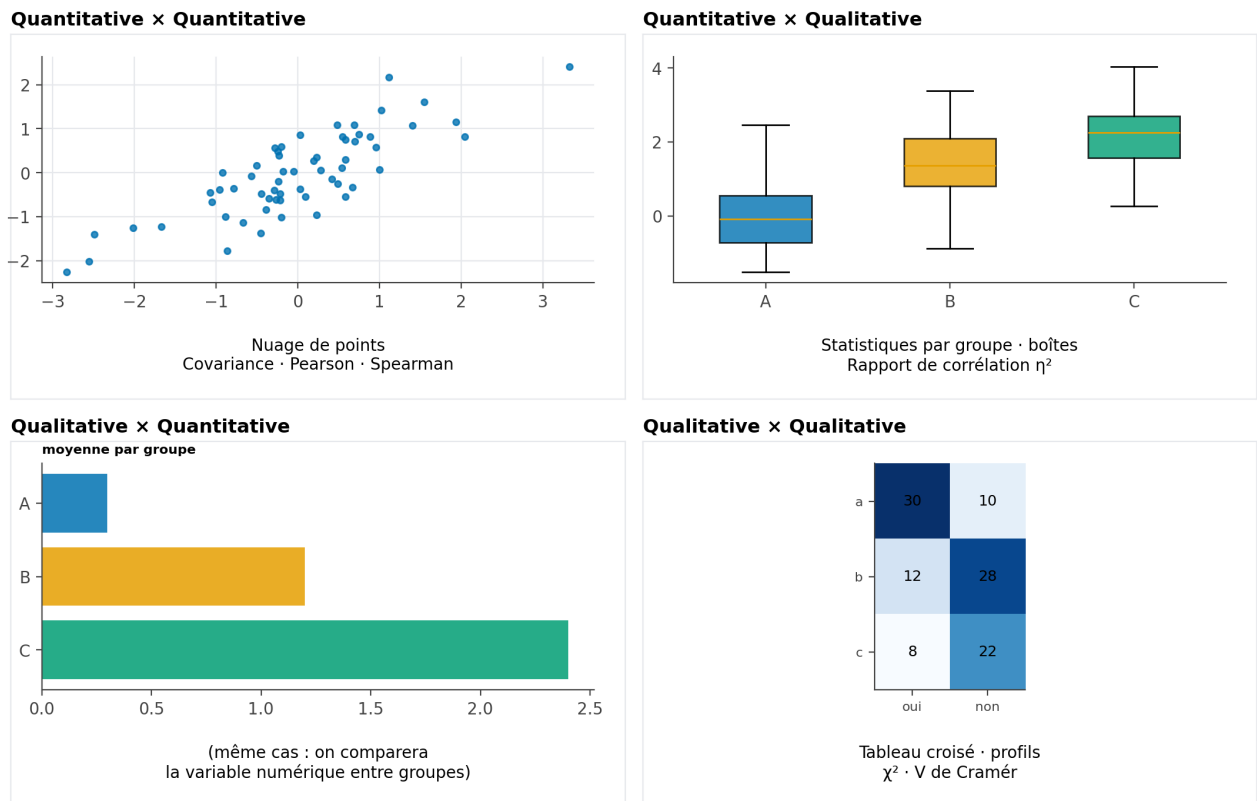


FIGURE 2 – Quel outil pour quelle paire de variables ?

— Proche de zéro : pas de tendance linéaire claire.

⚠ La covariance dépend des unités

La covariance entre une surface en m^2 et un prix en MAD n'est pas comparable à la covariance entre une surface en pi^2 et un prix en euros : elle change avec les unités choisies. C'est pour cela qu'on lui préfère presque toujours une version **normalisée**, sans unité : le coefficient de corrélation.

Le **coefficient de corrélation de Pearson** divise la covariance par le produit des écarts-types, ce qui le rend indépendant des unités et toujours compris entre -1 et 1 :

$$r = \frac{\text{Cov}(X, Y)}{s_X s_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (0.1)$$

TABLE 2 – Repères pour interpréter un coefficient de Pearson

Valeur de r	Interprétation
$r = 1$	Relation linéaire positive parfaite
$r \approx 0,7$ à $0,9$	Relation linéaire positive forte
$r \approx 0$	Pas de relation linéaire détectable
$r \approx -0,7$ à $-0,9$	Relation linéaire négative forte
$r = -1$	Relation linéaire négative parfaite

Valeur de r	Interprétation
---------------	----------------

```
r = df["surface"].corr(df["prix"]) # Pearson, par défaut
matrice = df[["surface", "prix", "nombre_pieces", "age_bien"]].corr()
```

2.2. Ce que r ne voit jamais : la galerie des cas particuliers

Ce que r mesure — et ce qu'il ne voit pas (cadres rouges : relations fortes, mais $r \approx 0$)

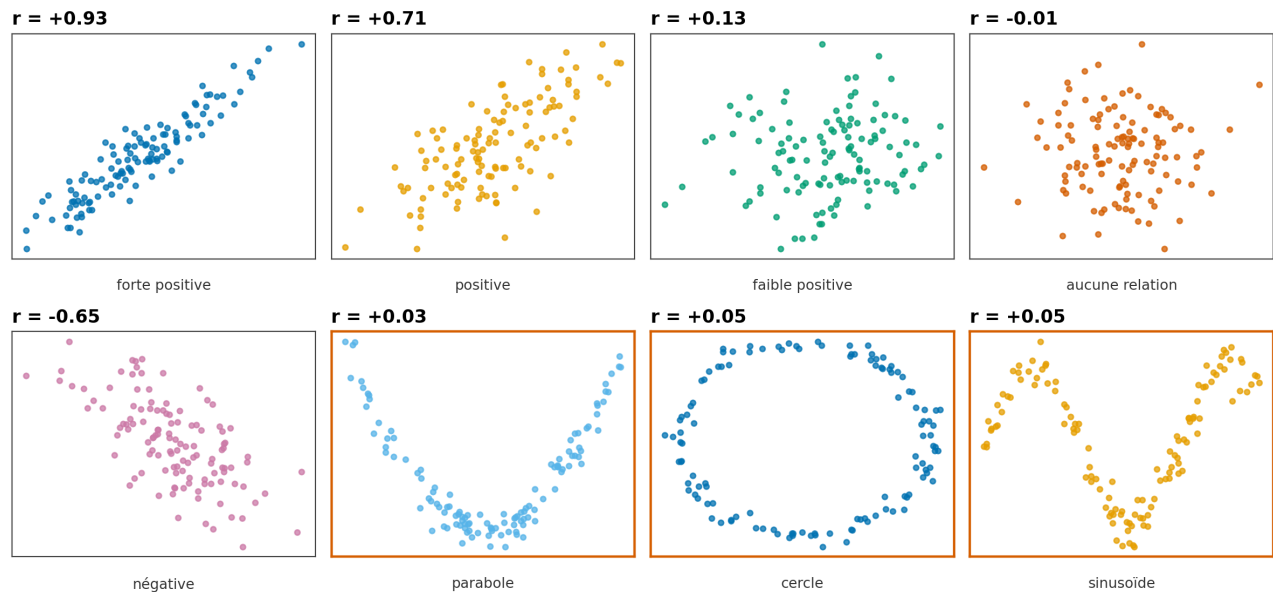


FIGURE 3 – r mesure une chose précise — et rate tout le reste

La Figure 3 affiche huit nuages de points très différents. Les trois premiers illustrent une échelle de relations linéaires (r de fort à nul) — c'est le cas pour lequel r a été conçu. Les trois panneaux encadrés en rouge sont plus instructifs : une **parabole**, un **cercle** et une **sinusoïde** peuvent chacun cacher une relation parfaitement déterministe — connaître x suffirait à connaître y exactement — tout en affichant un r proche de zéro.

! À retenir

Le coefficient de Pearson ne mesure qu'une chose : à quel point le nuage de points se rapproche d'une **droite**. Une relation forte mais non linéaire (courbe, cyclique, en U) peut très bien produire un r proche de zéro. **Toujours regarder le nuage de points avant de se fier au chiffre.**

2.3. Matrice de corrélation

Pour explorer simultanément les relations entre plusieurs variables numériques, on affiche la matrice de corrélation sous forme de *heatmap* :

```
import seaborn as sns
import matplotlib.pyplot as plt

matrice = df[["prix", "surface", "nombre_pieces", "age_bien", "etage"]].corr()
sns.heatmap(matrice, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1)
plt.title("Matrice de corrélation")
plt.show()
```

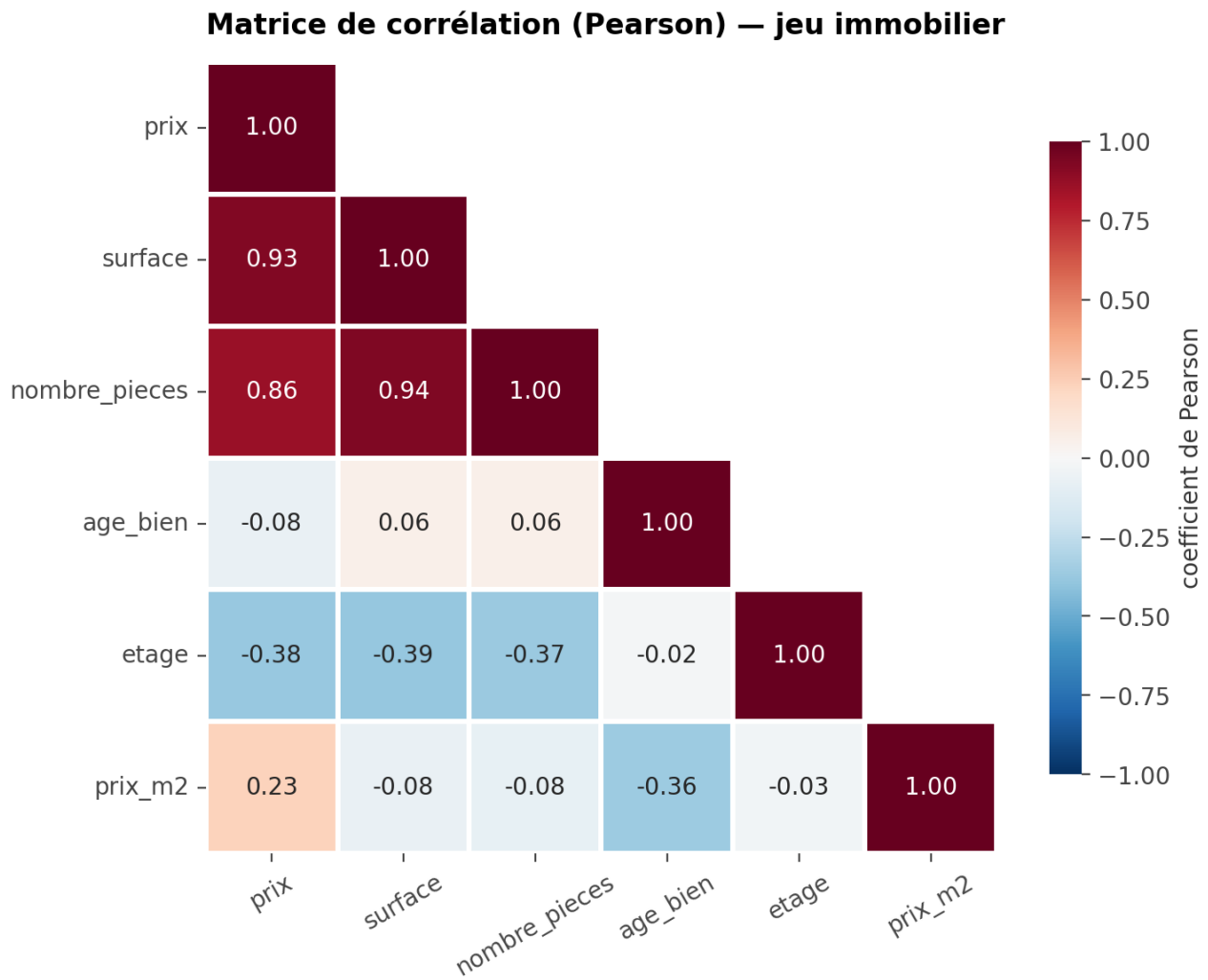


FIGURE 4 – Matrice de corrélation (Pearson) — jeu de données immobilier

La diagonale vaut toujours 1 (une variable est parfaitement corrélée à elle-même) ; la matrice est symétrique, ce qui justifie de n'afficher que la moitié inférieure comme sur la Figure 4.

2.4. Le quartet d'Anscombe

Quatre jeux de données peuvent avoir exactement la **même** moyenne, la **même** variance et le **même** r , tout en étant radicalement différents une fois représentés graphiquement.

Le quartet d'Anscombe : mêmes statistiques, quatre réalités

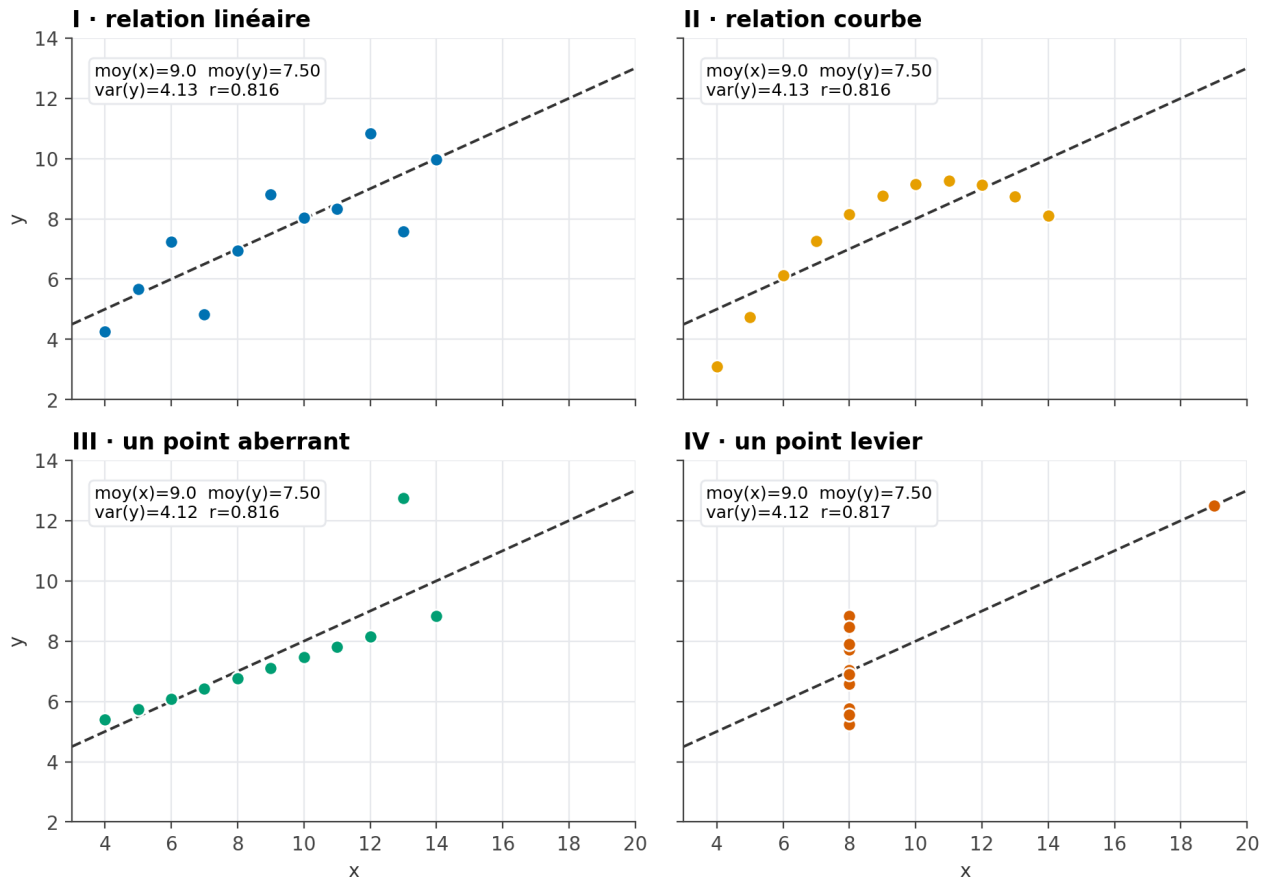


FIGURE 5 – Le quartet d'Anscombe (1973) : mêmes statistiques, quatre réalités

! La leçon du quartet d'Anscombe

Le panneau I montre une véritable relation linéaire ; le panneau II, une relation courbe que r interprète (à tort) comme « linéaire modérée » ; le panneau III, une relation linéaire quasi parfaite perturbée par un seul point ; le panneau IV, une absence totale de relation, sinon pour un unique point extrême qui, à lui seul, crée artificiellement toute la corrélation mesurée. **Aucune statistique résumée, aussi complète soit-elle, ne remplace un graphique.**

2.5. Pearson ou Spearman ?

Le coefficient de **Spearman** (ρ , ρ_{ho}) applique la formule de Pearson non pas aux valeurs brutes, mais à leurs **rangs**. Il mesure une relation **monotone** (qui va toujours dans le même sens), pas nécessairement linéaire, et se révèle bien plus robuste aux valeurs extrêmes.

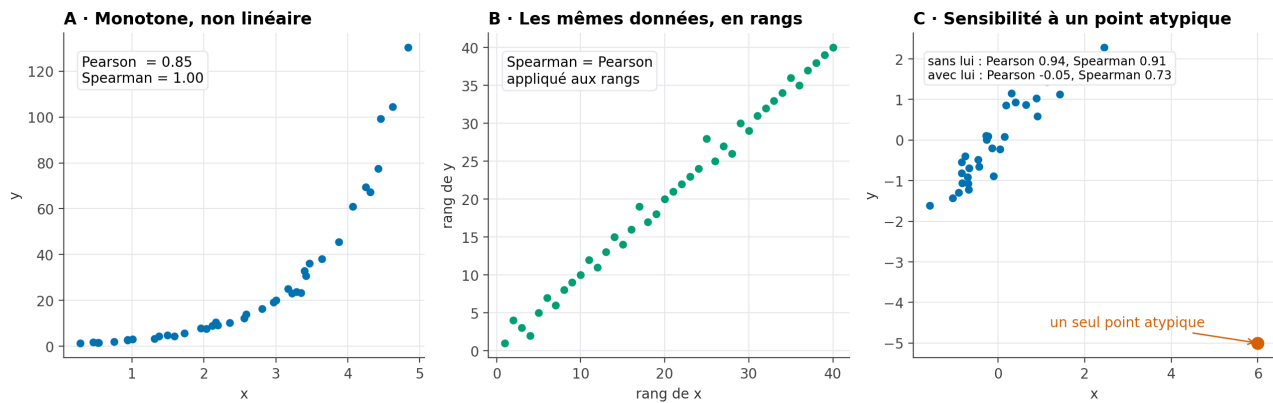


FIGURE 6 – Pearson vs Spearman : relation monotone non linéaire, principe des rangs, sensibilité aux valeurs atypiques

```
from scipy import stats

r_pearson, _ = stats.pearsonr(df["surface"], df["prix"])
r_spearman, _ = stats.spearmanr(df["surface"], df["prix"])
```

TABLE 3 – Quand préférer Spearman à Pearson

Critère	Pearson	Spearman
Type de relation détectée	Linéaire	Monotone (croissante ou décroissante, pas forcément en ligne droite)
Sensibilité aux valeurs extrêmes	Élevée	Faible (les rangs limitent l'effet d'un point isolé)
Nature des données	Quantitatives	Quantitatives ou ordinales

💡 Règle pratique

En cas de doute, ou en présence de valeurs extrêmes visibles sur le nuage de points, calculez les deux : un écart important entre r de Pearson et ρ de Spearman est lui-même une information — il signale une relation probablement non linéaire ou fortement influencée par quelques points.

3. Comparer une variable quantitative entre groupes

Lorsque la seconde variable est **qualitative** (ex. le quartier) plutôt que quantitative, la question change de nature : il ne s'agit plus de mesurer une corrélation, mais de comparer la distribution d'une variable numérique **entre plusieurs groupes**.

3.1. Statistiques par groupe

```
df.groupby("quartier")["prix_m2"].agg(["count", "mean", "median", "std"])
```

Un premier réflexe — déjà pratiqué en introduction — consiste à comparer moyennes et médianes par groupe. Mais une différence de moyennes observée sur l'échantillon reflète-t-elle une **vraie** différence entre groupes, ou

peut-elle s'expliquer par le simple hasard de l'échantillonnage ? L'**analyse de variance** formalise cette question.

3.2. Décomposer la variance : l'idée de l'ANOVA

L'idée de l'analyse de variance (ANOVA) est de décomposer la variabilité totale d'une variable en deux parts : celle qui s'explique par la différence **entre** les groupes, et celle qui reste **à l'intérieur** de chaque groupe.

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{SST \text{ (variabilité totale)}} = \underbrace{\sum_g n_g (\bar{y}_g - \bar{y})^2}_{SSB \text{ (inter-groupes)}} + \underbrace{\sum_g \sum_{i \in g} (y_i - \bar{y}_g)^2}_{SSW \text{ (intra-groupe)}}$$

où $\bar{y}$ est la moyenne générale, $\bar{y}_g$ la moyenne du groupe g , et n_g son effectif.

Le **rapport de corrélation** η^2 (*eta carré*) résume cette décomposition en un seul indicateur, entre 0 et 1 :

$$\eta^2 = \frac{SSB}{SST} \quad (0.2)$$

η^2 s'interprète comme la **proportion de la variance totale expliquée par l'appartenance au groupe** — un rôle proche de celui du R^2 que vous retrouverez dans un contexte de modélisation.

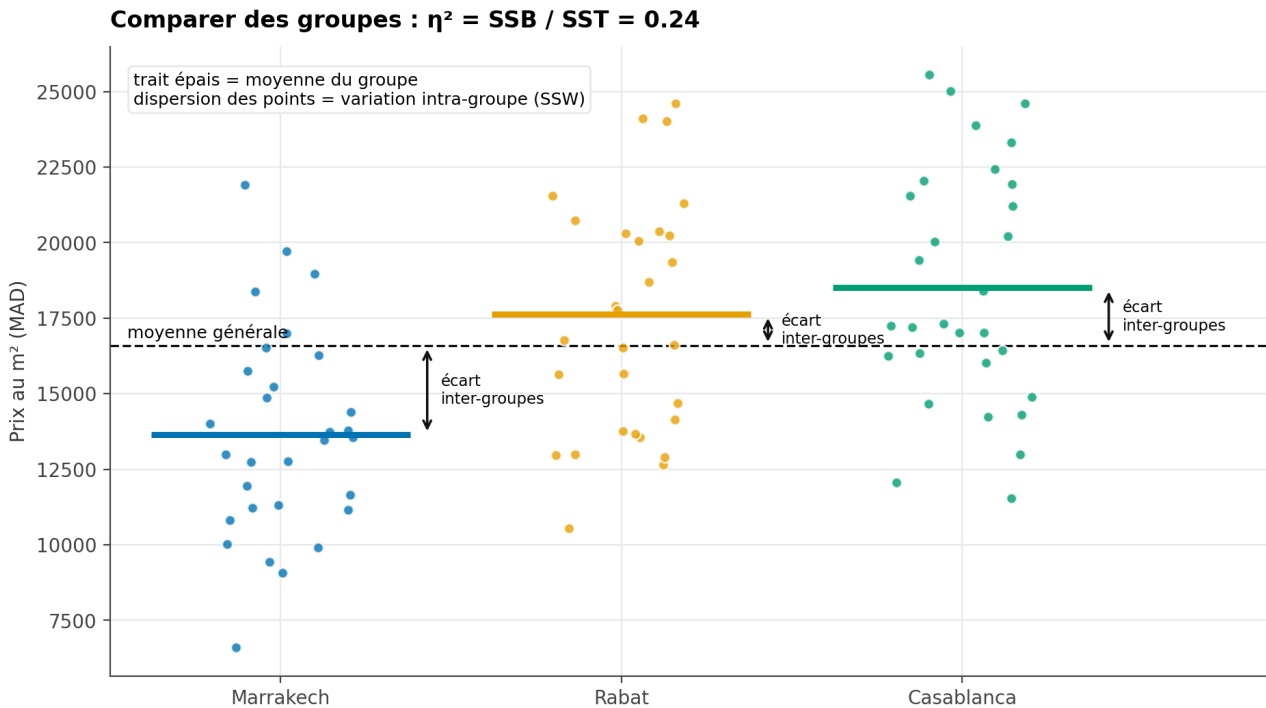


FIGURE 7 – Comparer des groupes : décomposition de la variance et η^2

Sur la Figure 7, le trait épais de chaque groupe représente sa moyenne ; la dispersion verticale des points autour de ce trait illustre la variabilité intra-groupe (SSW), tandis que l'écart entre chaque moyenne de groupe et la moyenne générale (ligne pointillée) illustre la variabilité inter-groupes (SSB).

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

modele = ols("prix_m2 ~ C(ville)", data=df).fit()
table_anova = sm.stats.anova_lm(modele, typ=2)
print(table_anova)

ssb = table_anova.loc["C(ville)", "sum_sq"]
sst = ssb + table_anova.loc["Residual", "sum_sq"]
eta2 = ssb / sst
```

i Repères pour interpréter η^2

Des seuils indicatifs, proposés par Cohen (1988) : $\eta^2 \approx 0,01$ = effet faible, $\approx 0,06$ = effet moyen, $\approx 0,14$ = effet fort. Ces repères doivent toujours être pondérés par le contexte métier — un effet « faible » au sens statistique peut rester important en pratique.

3.3. Interpréter le test et vérifier ses conditions

L'ANOVA à un facteur compare les moyennes de (k) groupes indépendants. Elle teste les hypothèses suivantes :

$$H_0 : \mu_1 = \mu_2 = \dots = \mu_k \quad \text{contre} \quad H_1 : \text{au moins deux moyennes de groupe sont différentes.}$$

L'hypothèse alternative ne signifie pas que toutes les moyennes sont différentes : le test indique seulement qu'au moins une différence existe, sans dire immédiatement entre quels groupes.

À partir des sommes de carrés déjà définies, la statistique de test est :

$$F_{\text{obs}} = \frac{SSB/(k-1)}{SSW/(N-k)},$$

où (SSB) mesure la variabilité entre les groupes, (SSW) la variabilité à l'intérieur des groupes, (k) est le nombre de groupes et (N) le nombre total d'observations utilisées. Sous (H_0), et si les conditions du modèle sont satisfaites, cette statistique suit une loi de Fisher à (k-1) et (N-k) degrés de liberté. Une valeur de (F_{obs}) élevée signifie que l'écart entre les moyennes de groupe est grand par rapport à la dispersion observée à l'intérieur des groupes.

Le tableau produit par `anova_lm` affiche la statistique (F) dans la colonne F et la p-valeur dans `PR(>F)`. La p-valeur est, **en supposant (H_0) vraie et les conditions du test satisfaites**, la probabilité d'obtenir une statistique (F) au moins aussi grande que celle observée. Elle n'est **pas** la probabilité que (H_0) soit vraie.

On fixe avant l'analyse un seuil (), souvent 0,05 pour un exercice introductif :

- si ($p < \alpha$), on rejette (H_0) : les données fournissent un indice statistique qu'au moins une moyenne diffère ;
- si ($p \geq \alpha$), on ne rejette pas (H_0) : les données ne suffisent pas à établir une différence de moyennes. Cela ne prouve pas que les moyennes sont identiques.

La p-valeur ne mesure ni l'importance pratique de la différence ni sa taille. Il faut la lire avec les moyennes par groupe, leurs effectifs, un intervalle d'incertitude si disponible et la taille d'effet (η^2) présentée plus haut. Un résultat statistiquement significatif peut correspondre à un effet très faible ; un résultat non significatif peut aussi manquer de puissance lorsque les groupes sont petits.

Conditions à examiner

1. **Indépendance des observations.** Chaque observation doit contribuer à un seul groupe et ne pas être une répétition dépendante d'une autre observation. Cette condition se justifie d'abord par la façon dont les données ont été collectées ; un graphique ou un test ne peut pas la garantir. Pour des mesures répétées sur les mêmes personnes, l'ANOVA à un facteur ordinaire n'est pas le bon modèle.
2. **Résidus approximativement normaux dans chaque groupe.** On examine les résidus du modèle, surtout lorsque les effectifs sont petits. Une légère déviation est souvent moins problématique avec des groupes assez grands et équilibrés, mais une forte asymétrie ou quelques valeurs extrêmes peuvent rendre la comparaison des moyennes fragile.
3. **Variances comparables entre groupes (homoscédasticité).** L'ANOVA classique suppose que la dispersion est similaire dans les groupes. Des tailles de groupe très inégales combinées à des variances très différentes sont particulièrement préoccupantes.
4. **Variable étudiée quantitative et groupes pertinents.** Les observations doivent mesurer la même grandeur et utiliser la même unité ; les catégories comparées doivent correspondre à la question posée.

Contrôles pratiques avec Python

Le graphique Q-Q des résidus sert à examiner leur forme. Le test de Levene évalue si les variances des groupes sont compatibles entre elles ; avec `center="median"`, il est moins sensible à une distribution non normale qu'une version centrée sur la moyenne.

```
import matplotlib.pyplot as plt
from scipy import stats

df_anova = df.dropna(subset=["prix_m2", "ville"])
groupes_par_ville = list(df_anova.groupby("ville", observed=True))
groupes = [
    groupe["prix_m2"].to_numpy()
    for _, groupe in groupes_par_ville
]
effectifs = df_anova.groupby("ville", observed=True)["prix_m2"].count()
print("Effectifs par ville :")
print(effectifs)

fig, axes = plt.subplots(
    1,
    len(groupes_par_ville),
    figsize=(5 * len(groupes_par_ville), 4),
    squeeze=False,
)
for (ville, groupe), ax in zip(groupes_par_ville, axes.ravel()):
    stats.probplot(modele.resid.loc[groupe.index], dist="norm", plot=ax)
    ax.set_title(f"Résidus - {ville}")
plt.tight_layout()
plt.show()

stat_levene, p_levene = stats.levene(*groupes, center="median")
print(f"Test de Levene : statistique={stat_levene:.3f}, p-valeur={p_levene:.4g}")
```

Le Q-Q plot s'interprète visuellement : des points globalement proches de la droite sont compatibles avec une normalité approximative ; des courbures marquées ou des points extrêmes appellent à la prudence. Pour Levene, une p-valeur inférieure à () signale des éléments contre l'égalité des variances. Une p-valeur supérieure ou égale à () ne prouve pas que les variances sont égales : on vérifie aussi leur taille et les effectifs par groupe.

Que faire si une condition est mal respectée ?

- Si les variances sont nettement différentes, utiliser l'ANOVA de Welch, qui ne suppose pas leur égalité :

```
from statsmodels.stats.oneway import anova_oneway

resultat_welch = anova_oneway(
    data=df_anova["prix_m2"],
    groups=df_anova["ville"],
    use_var="unequal",
)
print(resultat_welch)
```

- Si la variable est très asymétrique, contient des valeurs extrêmes influentes ou si les petits groupes rendent l'hypothèse de normalité peu crédible, examiner les données et envisager un test de Kruskal-Wallis :

```
stat_kruskal, p_kruskal = stats.kruskal(*groupes)
print(f"Kruskal-Wallis : statistique={stat_kruskal:.3f}, p-valeur={p_kruskal:.4g}")
```

Kruskal-Wallis est un test fondé sur les rangs : il ne teste pas directement l'égalité des moyennes et ne doit pas être décrit automatiquement comme un « test des médianes », sauf hypothèses supplémentaires sur la forme des distributions.

Enfin, si le test global indique une différence, il faut réaliser une comparaison post-hoc pour savoir quels groupes diffèrent, en corrigeant les comparaisons multiples. Pour une ANOVA classique lorsque l'hypothèse de variances égales est raisonnable, le test de Tukey compare les paires de groupes tout en contrôlant le risque d'erreur familial :

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

comparaisons = pairwise_tukeyhsd(
    endog=df_anova["prix_m2"],
    groups=df_anova["ville"],
    alpha=0.05,
)
print(comparaisons)
```

Après l'ANOVA de Welch, choisir une comparaison post-hoc compatible avec des variances inégales (par exemple Games-Howell). Après Kruskal-Wallis, utiliser une comparaison fondée sur les rangs et corriger les p-valeurs des comparaisons multiples. Il ne faut pas appliquer automatiquement Tukey après n'importe quel test global.

Une ANOVA ou un test post-hoc décrit une association entre groupe et mesure dans les données analysées ; il ne démontre pas, à lui seul, que l'appartenance au groupe cause la différence observée.

4. Relation entre deux variables qualitatives

4.1. Le tableau croisé, observé contre théorique

Le **tableau croisé** (déjà rencontré au chapitre précédent) croise les effectifs de deux variables qualitatives. Pour savoir si les deux variables sont réellement **liées**, on le compare à un tableau **théorique**, celui que l'on obtiendrait si les deux variables étaient parfaitement indépendantes.

L'effectif théorique d'une cellule se calcule ainsi :

Type de bien × ascenseur : $\chi^2 = 124.3$, ddl = 2, V de Cramér = 0.40

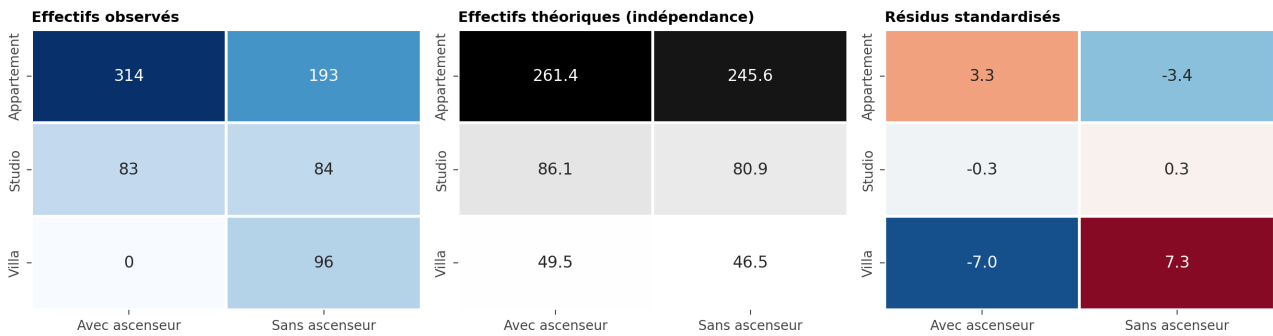


FIGURE 8 – Type de bien × ascenseur : effectifs observés, théoriques, et résidus

$$E_{ij} = \frac{(\text{total de la ligne } i) \times (\text{total de la colonne } j)}{\text{effectif total}}$$

Le test du χ^2 **d'indépendance** additionne, sur toutes les cellules, l'écart relatif entre observé et théorique :

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

Plus les effectifs observés s'écartent des effectifs théoriques, plus χ^2 est grand — signe d'une association entre les deux variables. Le troisième panneau de la Figure 8 affiche les **résidus standardisés** $\frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}}}$: ils indiquent, cellule par cellule, où se concentre l'écart à l'indépendance.

4.2. Le V de Cramér : une taille d'effet

Comme le χ^2 augmente mécaniquement avec la taille de l'échantillon, il ne mesure pas à lui seul la **force** de l'association. Le **V de Cramér** normalise le χ^2 entre 0 (aucune association) et 1 (association parfaite) :

$$V = \sqrt{\frac{\chi^2}{n \times (\min(r, c) - 1)}}$$

où n est l'effectif total, r le nombre de lignes et c le nombre de colonnes du tableau croisé.

```
from scipy.stats import chi2_contingency

tableau = pd.crosstab(df["type_bien"], df["ascenseur"])
chi2, p_valeur, ddl, theorique = chi2_contingency(tableau)

n = tableau.values.sum()
v_cramer = (chi2 / (n * (min(tableau.shape) - 1))) ** 0.5
```

i Repères pour interpréter le V de Cramér

$V \approx 0,1$ = association faible, $\approx 0,3$ = modérée, $\approx 0,5$ et plus = forte. Sur la Figure 8, un V proche de 0,4 confirme visuellement ce que les résidus suggéraient déjà : le type de bien et la présence d'un ascenseur ne sont pas indépendants.

5. Pièges de l'analyse bivariable et multivariable

Les outils précédents sont puissants, mais chacun peut induire en erreur s'il est utilisé sans esprit critique. Cette section rassemble quatre pièges classiques.

5.1. Le paradoxe de Simpson

Une relation peut s'**inverser** selon que l'on regarde les données globalement ou séparément par sous-groupe.

Le paradoxe de Simpson : la tendance globale peut inverser celle des sous-groupes

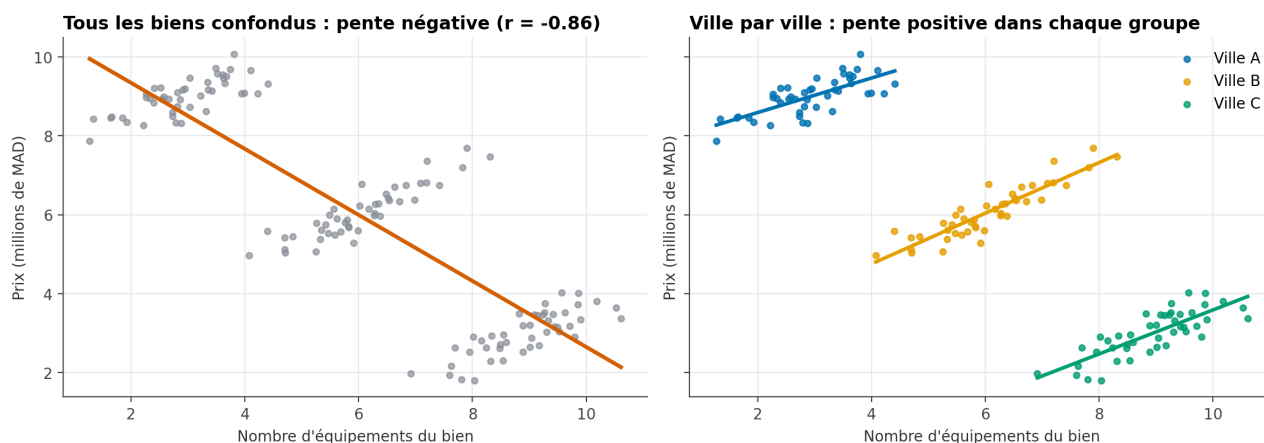


FIGURE 9 – Le paradoxe de Simpson : la tendance globale peut inverser celle des sous-groupes

Sur la Figure 9, la tendance globale (à gauche) suggère qu'un bien avec plus d'équipements se vend **moins cher** — une conclusion absurde en apparence. En réalité, chaque ville (à droite) affiche une relation **positive** entre équipements et prix ; c'est seulement parce que les villes les moins chères concentrent les biens les mieux équipés que la tendance globale s'inverse. La variable **ville** est ici une **variable confondante** : elle influence à la fois le nombre d'équipements et le prix.

! À retenir

Avant de conclure sur une relation entre deux variables, demandez-vous toujours s'il existe une **troisième variable**, non prise en compte, susceptible d'expliquer — ou d'inverser — la relation observée. C'est précisément le rôle de l'analyse **multivariable**.

5.2. Corrélation n'est pas causalité

L'exemple classique : les ventes de glaces et la fréquentation des piscines sont fortement corrélées, sans qu'aucune ne cause l'autre — les deux sont conjointement causées par la température (Figure 10). Établir un lien de causalité nécessite un dispositif que la seule observation ne fournit pas : une expérience randomisée (test A/B), ou des méthodes statistiques spécifiques dédiées à l'inférence causale, hors du cadre de ce cours.

Corrélation n'est pas causalité

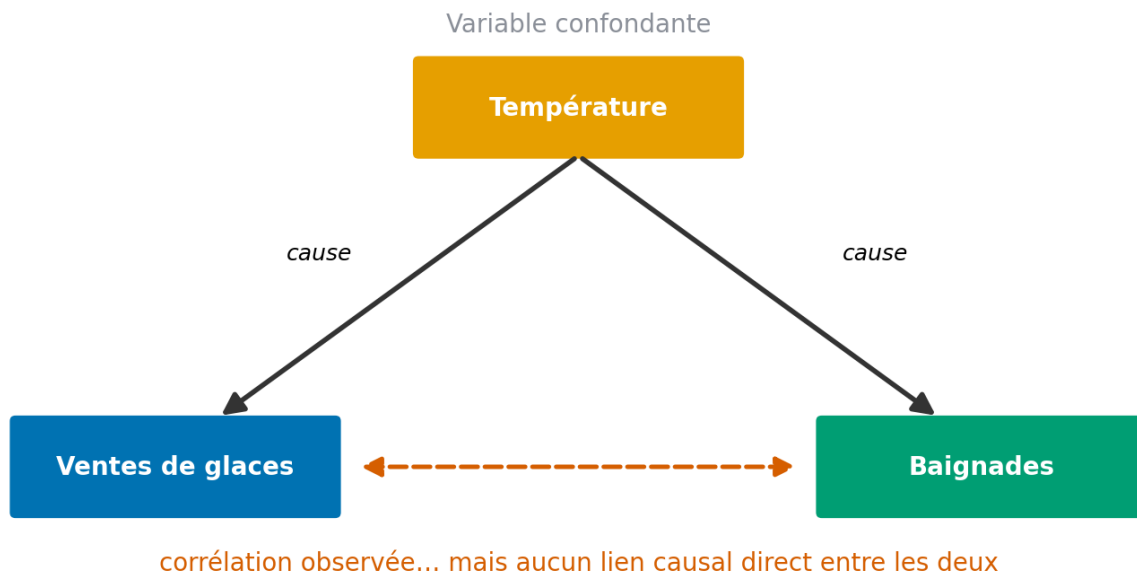


FIGURE 10 – Corrélation n'est pas causalité : le cas d'une variable confondante

5.3. Un point invisible seul, flagrant à deux

Une observation peut sembler parfaitement normale sur **chacune** des deux variables prises séparément, tout en étant nettement atypique une fois les deux considérées **ensemble**.

Sur la Figure 11, le point rouge se situe dans la plage normale de x et dans la plage normale de y (histogrammes du haut et de droite), mais s'écarte nettement du nuage principal, car la combinaison des deux valeurs rompt la relation habituelle entre x et y . Une détection d'outliers univariée (chapitre précédent, méthode de l'IQR) ne le repérerait jamais.

5.4. Le piège des tests multiples

Plus on teste de relations entre variables, plus on a de chances d'en trouver une « significative » par pur hasard. Avec un seuil habituel de 5 % de risque d'erreur par test, la probabilité d'obtenir **au moins une** fausse alerte parmi m relations testées vaut approximativement $1 - 0,95^m$ — déjà 64 % pour seulement 20 tests indépendants. Le panneau de droite de la Figure 12 le démontre sur des données **entièrement aléatoires** : parmi 20 variables de pur bruit, plusieurs paires affichent une corrélation dépassant le seuil de significativité, sans qu'aucune relation réelle n'existe.

⚠ À retenir

Explorer un grand nombre de relations avant de choisir de ne commenter que les plus « intéressantes » (*data dredging*, ou pêche aux corrélations) fabrique artificiellement des résultats. Une relation repérée par exploration doit toujours être formulée comme une **hypothèse à vérifier**, plutôt que comme une conclusion déjà établie.

6. Distributions asymétriques : la transformation logarithmique

De nombreuses variables économiques (prix, revenus, population) présentent une distribution fortement asymétrique à droite : la majorité des observations sont regroupées sur de faibles valeurs, avec une longue

Un point invisible variable par variable

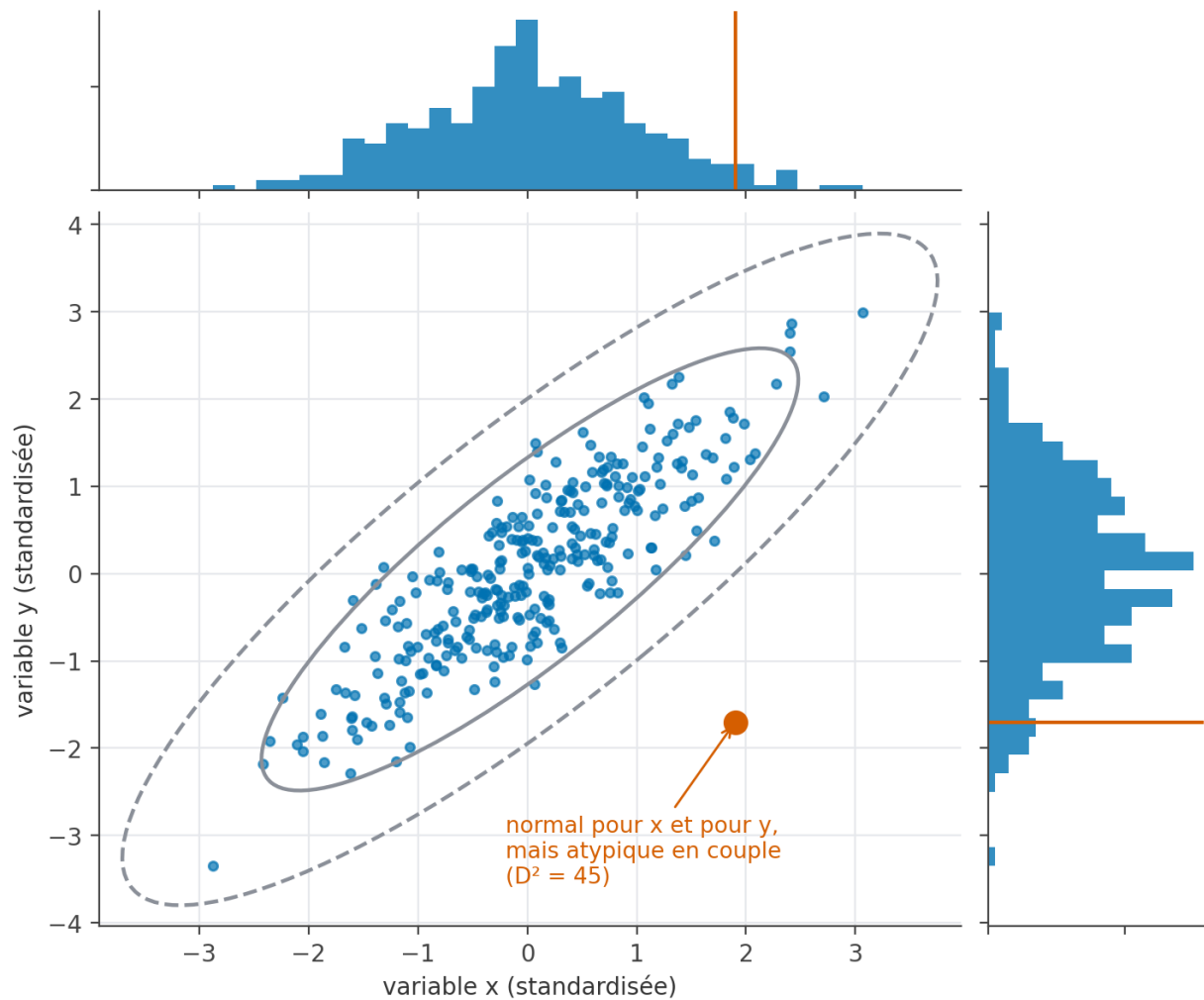


FIGURE 11 – Un point normal sur chaque axe, mais atypique en combinaison

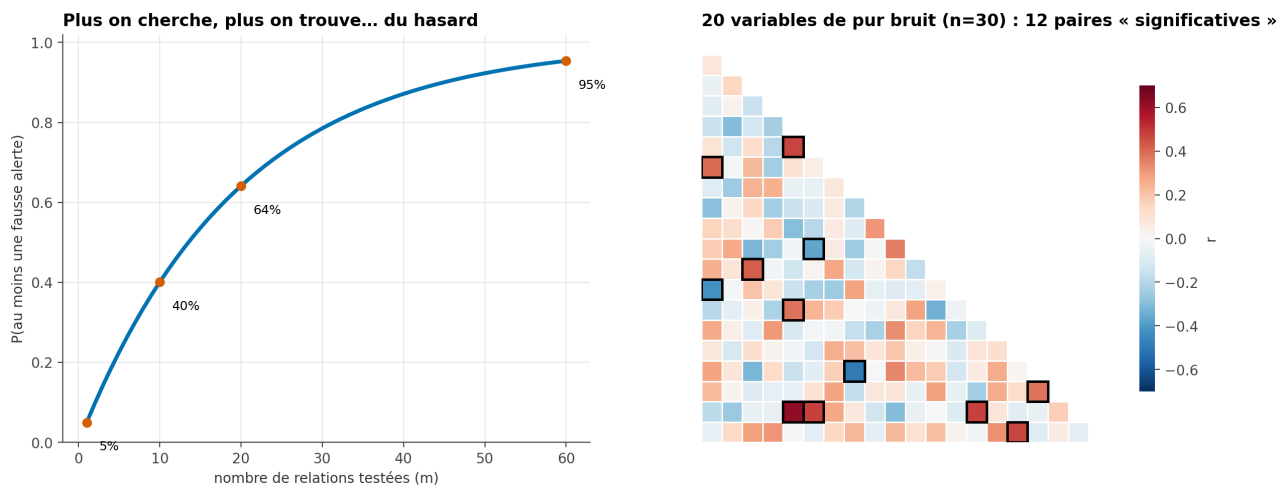


FIGURE 12 – Plus on cherche, plus on trouve — du hasard

traîne de valeurs élevées.

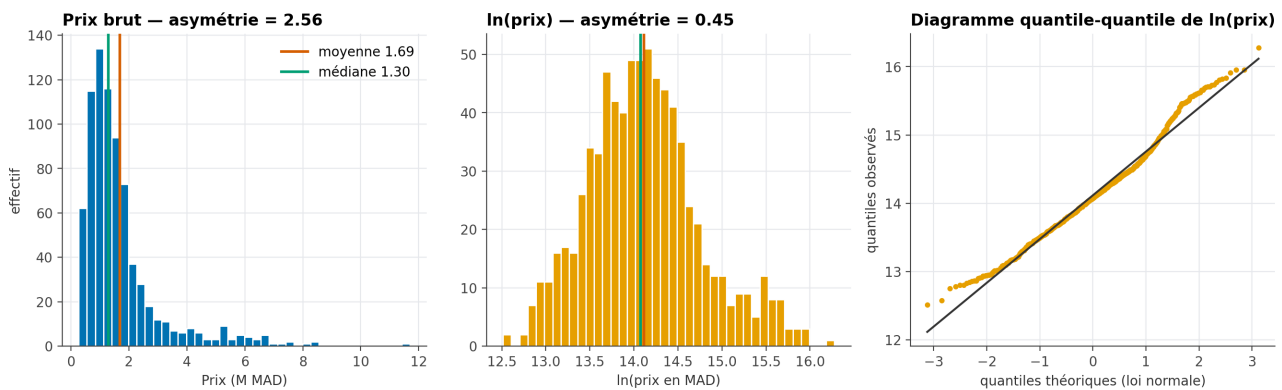


FIGURE 13 – Prix brut, $\log(\text{prix})$, et diagramme quantile-quantile

La **transformation logarithmique** $x \mapsto \ln(x)$ compresse les grandes valeurs et étale les petites, ce qui rapproche souvent la distribution d'une forme symétrique — visible sur la Figure 13, où l'asymétrie passe de 2,36 (prix brut) à une valeur beaucoup plus proche de zéro (log du prix).

```
df["log_prix"] = np.log(df["prix"])
```

Le troisième panneau de la Figure 13 est un **diagramme quantile-quantile** (QQ-plot) : il compare les quantiles observés de `log_prix` aux quantiles théoriques d'une loi normale. Plus les points suivent la droite rouge, plus la distribution se rapproche d'une loi normale.

💡 Quand transformer ?

La transformation logarithmique est utile pour mieux visualiser une distribution étalée, faciliter la lecture de certains graphiques (chapitre suivant), ou satisfaire une hypothèse de normalité requise par certaines méthodes statistiques. Elle ne s'applique qu'à des valeurs **strictement positives** ; en présence de zéros, on utilise parfois $\ln(x + 1)$.

7. Résumer par groupe : *groupby* et tableaux croisés dynamiques

7.1. Le principe *split-apply-combine*

`groupby` répond à un très grand nombre de questions d'analyse en trois étapes systématiques : **découper** le tableau selon une ou plusieurs variables, **appliquer** une fonction (moyenne, somme, effectif...) à chaque sous-tableau, puis **recombinaison** les résultats en un tableau de synthèse.

```
synthese = (
    df.groupby("quartier")
    .agg(
        effectif=("prix", "count"),
        prix_moyen=("prix", "mean"),
        prix_mediane=("prix", "median"),
        surface_moyenne=("surface", "mean"),
    )
    .sort_values("prix_moyen", ascending=False)
)
```

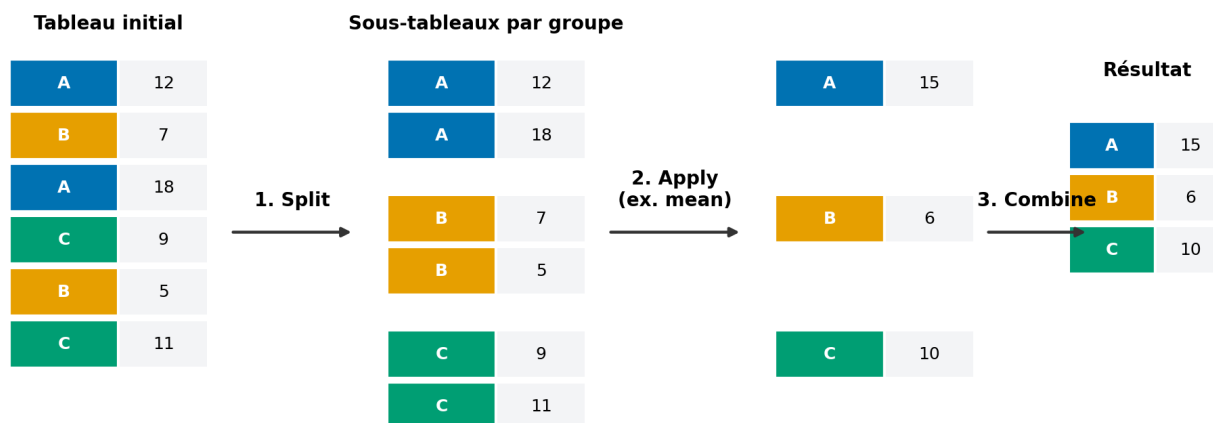
groupby : découper, appliquer, recombinaison

FIGURE 14 – groupby : découper, appliquer, recombinaison

💡 Toujours accompagner un agrégat de son effectif

Une moyenne calculée sur 3 observations n'a pas la même fiabilité qu'une moyenne calculée sur 3 000. La colonne **effectif** (via `count`) doit systématiquement accompagner toute statistique agrégée — c'est elle qui permet au lecteur de juger si une moyenne mérite d'être prise au sérieux.

7.2. Le tableau croisé dynamique (pivot_table)

Lorsque l'on souhaite croiser **deux** variables catégorielles pour résumer une troisième variable numérique, `pivot_table` construit directement le tableau à deux dimensions :

```
pivot = pd.pivot_table(
    df, values="prix_m2", index="quartier", columns="type_bien", aggfunc="median",
)
```

Représenté sous forme de *heatmap* (Figure 15 — la même géométrie qu'au chapitre suivant, consacré à la visualisation), un tel tableau croisé dynamique révèle en un coup d'œil les combinaisons quartier/type de bien les plus chères, sans avoir à lire un tableau de chiffres ligne par ligne.

8. Explorer une série temporelle

Lorsque les observations sont datées, l'exploration gagne à mobiliser des outils spécifiques à la dimension temporelle, au-delà du simple calcul de variation relative déjà vu en introduction.

```
mensuel = df.set_index("date_annonce").resample("MS").size() # effectif par mois

moyenne_mobile = mensuel.rolling(window=3, center=True).mean() # lisse le bruit
variation_annuelle = mensuel.pct_change(periods=12) * 100 # comparaison à 12 mois d'écart
profil_saisonnier = mensuel.groupby(mensuel.index.month).mean() # moyenne par mois calendaire
```

La **moyenne mobile** (panneau supérieur gauche de la Figure 16) lisse les fluctuations de court terme pour révéler la tendance de fond. La **décomposition saisonnière**, disponible dans `statsmodels`, va plus loin en séparant explicitement une série y_t en trois composantes :

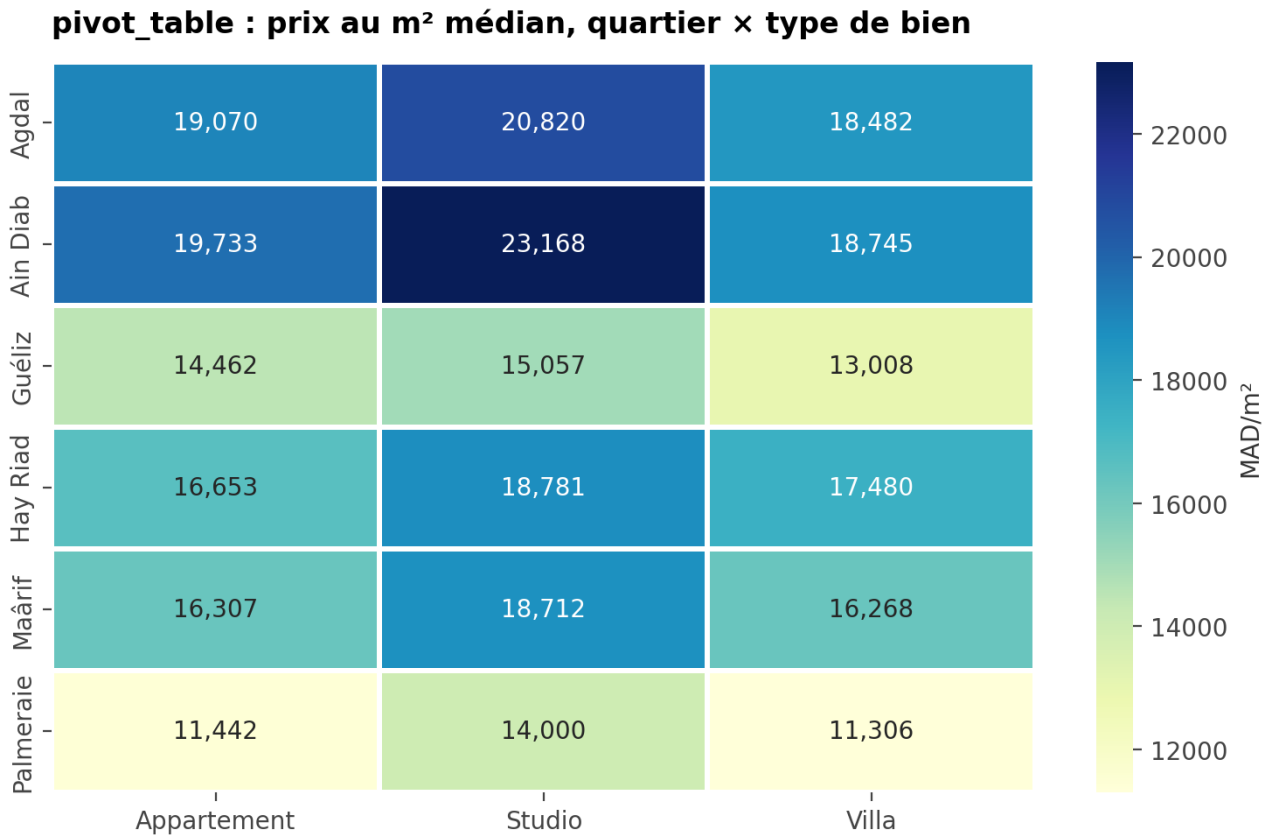
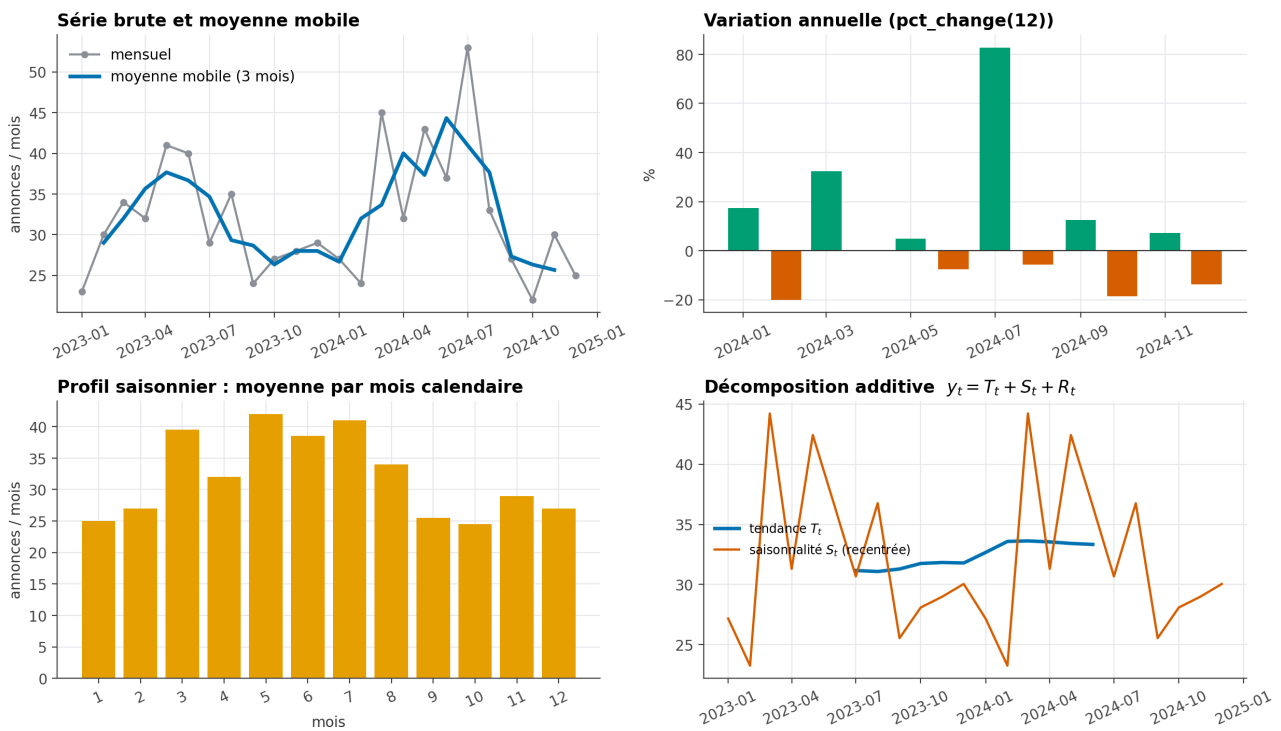
FIGURE 15 – pivot_table : prix au m² médian, quartier × type de bien

FIGURE 16 – Série brute, variation annuelle, profil saisonnier, décomposition

$$y_t = T_t + S_t + R_t$$

où T_t est la tendance de long terme, S_t la saisonnalité (un motif qui se répète, ici chaque année) et R_t le résidu, c'est-à-dire tout ce que la tendance et la saisonnalité n'expliquent pas.

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(mensuel, model="additive", period=12)
decomposition.trend
decomposition.seasonal
decomposition.resid
```

⚠ Un pic n'a pas de cause évidente

Un pic ou un creux ponctuel dans une série temporelle peut correspondre à un événement réel, à une erreur de collecte, ou à un changement dans le système qui produit la donnée. Comme au chapitre précédent (Figure 8 et §5), ne l'expliquez jamais sans une source complémentaire pour l'étayer.

9. Automatiser un premier diagnostic : le profilage de données

Toutes les statistiques univariées et bivariées présentées dans ce chapitre et le précédent peuvent être produites en une seule commande grâce à des bibliothèques de **profilage automatique** :

- **ydata-profiling** (anciennement **pandas-profiling**) génère un rapport HTML complet : distributions, valeurs manquantes, corrélations, alertes de qualité ;
- **sweetviz** produit un rapport similaire, avec un accent sur la comparaison entre deux jeux de données (par exemple, train et test).

```
from ydata_profiling import ProfileReport

rapport = ProfileReport(df, title="Rapport d'exploration - annonces immobilières")
rapport.to_file("rapport_exploration.html")
```

⚠ Un point de départ, jamais un point d'arrivée

Un rapport automatique est un excellent point de départ pour une première prise de contact avec un nouveau jeu de données : il détecte rapidement des problèmes évidents (fort taux de valeurs manquantes, corrélations élevées, colonnes constantes). Il ne remplace en rien le raisonnement mené dans ce chapitre : il ne connaît ni votre question métier, ni le sens réel de vos variables, ni les pièges comme le paradoxe de Simpson.

Exercices

Exercice 0.0.1 (Choisir le bon coefficient de corrélation). Pour chacune des situations suivantes, indiquez si vous calculeriez plutôt un coefficient de Pearson ou de Spearman.

- Le lien entre la taille d'un logement et son prix, sur un jeu de données sans valeur extrême.
- Le lien entre l'ancienneté d'un employé (en rangs de séniorité) et sa satisfaction (échelle de 1 à 5).
- Le lien entre le revenu d'un ménage et ses dépenses, sur un échantillon comportant deux ménages extrêmement riches.

Exercice 0.0.2 (Corrélation et causalité). Un journal affirme : « Les villes qui comptent le plus de pharmacies ont aussi le plus de cas de maladies — les pharmacies rendraient-elles malade ? »

- Cette conclusion est-elle valide ?
- Proposez une variable confondante plausible.

Exercice 0.0.3 (Construire un plan d'exploration). Vous disposez d'un jeu de données de commandes en ligne avec les colonnes : `montant` (quantitative), `region` (qualitative), `moyen_paiement` (qualitative), `date` (temporelle), `age_client` (quantitative).

- Proposez une question univariée, une question bivariée et une question multivariée que l'on pourrait explorer avec ce jeu de données.
- Pour votre question bivariée, précisez l'outil que vous utiliseriez (Table 1).

Exercice 0.0.4 (Interpréter un tableau agrégé). Le tableau suivant croise région et nombre moyen de commandes :

Région	Nombre de commandes	Montant moyen (MAD)
Nord	1 240	410
Sud	38	1 850
Est	690	395

- Pourquoi faut-il se méfier du montant moyen de la région Sud ?
- Quelle information supplémentaire demanderiez-vous avant toute conclusion ?

Exercice 0.0.5 (Lire une décomposition de variance). Pour une variable `note_satisfaction` comparée entre trois agences, on obtient $SST = 420$, $SSB = 92$.

- Calculez SSW .
- Calculez η^2 et interprétez-le à l'aide des repères du §3.2.

Exercice 0.0.6 (Association entre deux variables qualitatives). Un tableau croisé entre `canal_publicitaire` (3 catégories) et `a_achete` (oui/non) donne $\chi^2 = 18,4$ sur un échantillon de $n = 400$.

- Calculez le V de Cramér (le tableau a 3 lignes et 2 colonnes).
- L'association vous semble-t-elle faible, modérée ou forte (repères du §4.2) ?

Exercice 0.0.7 (Simpson en pratique). Une entreprise observe que le taux de satisfaction global de son service client a **baissé** cette année, alors que chacune de ses trois équipes affiche individuellement une **hausse** de satisfaction.

- Ce résultat est-il contradictoire ?
- Quelle variable faudrait-il examiner pour l'expliquer ?

Exercice 0.0.8 (Transformation logarithmique). Une variable `capital_social` d'entreprises varie de 10 000 MAD à 50 000 000 MAD, avec une asymétrie de 4,8.

- Recommanderiez-vous une transformation logarithmique ? Pourquoi ?
- Un histogramme de $\log(\text{capital_social})$ serait-il plus facile à lire qu'un histogramme de `capital_social` ? Justifiez à l'aide de la Figure 13.

Exercice 0.0.9 (`groupby` et `pivot_table`). Vous disposez d'un jeu de données `ventes` avec les colonnes `region`, `canal` (magasin/en ligne) et `montant`.

- Écrivez l'instruction `groupby` qui calcule le montant moyen et l'effectif par région.
- Écrivez l'instruction `pivot_table` qui croise région et canal pour afficher le montant total.

Exercice 0.0.10 (Lire une série temporelle décomposée). Une décomposition additive d'une série de ventes mensuelles donne, pour le mois de décembre : tendance $T = 1200$, saisonnalité $S = +350$, résidu $R = -40$.

- a. Calculez la valeur observée y pour ce mois.
- b. Que signifie une saisonnalité positive de 350 pour décembre ?

Visualisation des données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Expliquer** pourquoi la visualisation est indispensable, même après une exploration statistique complète ;
- **Décomposer** n'importe quel graphique selon la grammaire des graphiques, et **hiérarchiser** les variables visuelles (position, longueur, angle, aire, couleur) selon leur précision de lecture ;
- **Choisir** le graphique adapté à une question donnée (distribution, comparaison, relation, évolution, composition) et **distinguer sans confusion possible** un histogramme d'un diagramme en barres ;
- **Construire**, avec `matplotlib` et `seaborn`, l'ensemble des graphiques essentiels : histogramme, densité, boîte à moustaches et violon, barres (simples, groupées, empilées), camembert, nuage de points, courbe, *heatmap* ;
- **Représenter** des données géographiques : carte choroplèthe, carte de points, cartogramme, et connaître les pièges spécifiques à la cartographie ;
- **Mobiliser** des graphiques plus avancés lorsqu'ils sont réellement justifiés : petits multiples, coordonnées parallèles, réseaux, Gantt, tableaux de bord ;
- **Distinguer** une bibliothèque de visualisation statique d'une bibliothèque interactive, et savoir quand recourir à chacune ;
- **Identifier** les pièges les plus courants d'un graphique trompeur et **appliquer** les principes de base du *data storytelling* et de l'accessibilité.

Les chapitres précédents ont permis de préparer un jeu de données puis de l'explorer statistiquement. La **visualisation des données** (*data visualization*) en constitue le prolongement naturel : elle traduit ces résultats sous une forme que l'œil humain interprète beaucoup plus vite qu'un tableau de chiffres, et qu'un public non technique peut comprendre en quelques secondes.

1. Pourquoi visualiser des données ?

1.1. Ce que les statistiques seules ne montrent pas

Deux jeux de données peuvent partager exactement les mêmes statistiques descriptives — moyenne, variance, corrélation — tout en ayant des formes radicalement différentes. C'est ce qu'illustre le célèbre **quartet d'Anscombe** (1973), déjà présenté au chapitre précédent (Figure 5) : quatre jeux de données de onze points chacun, ayant tous la même moyenne, la même variance et le même coefficient de corrélation de Pearson ($r \approx 0,82$), mais dont les nuages de points révèlent des situations totalement distinctes.

! La leçon du quartet d'Anscombe

Aucune statistique résumée, aussi complète soit-elle, ne remplace un graphique. C'est pourquoi l'exploration statistique (chapitre précédent) et la visualisation (ce chapitre) doivent toujours être menées ensemble, et non l'une après l'autre.

1.2. Le rôle de la visualisation dans le cycle d'un projet

La visualisation intervient à deux moments distincts d'un projet d'analyse de données :

- pendant l'**exploration**, comme outil de travail personnel de l'analyste, pour repérer rapidement une distribution, une anomalie ou une relation ;
- au moment de la **communication**, comme outil destiné à un public (décideur, client, grand public), pour transmettre un message clair et argumenté.

Ces deux usages appellent des exigences différentes : un graphique d'exploration peut rester brut et rapide à produire, tandis qu'un graphique de communication doit être soigné, annoté et pensé pour son destinataire — nous y reviendrons en fin de chapitre (§9).

2. La grammaire des graphiques

Un graphique statistique, quel que soit son type, se construit à partir des mêmes briques élémentaires. Cette idée, formalisée par Leland Wilkinson sous le nom de **grammaire des graphiques** (*grammar of graphics*), popularisée ensuite par `ggplot2` en R, permet de décrire n'importe quel graphique comme la combinaison de quelques éléments simples.

Un graphique = une composition de couches indépendantes

... qui produit finalement :

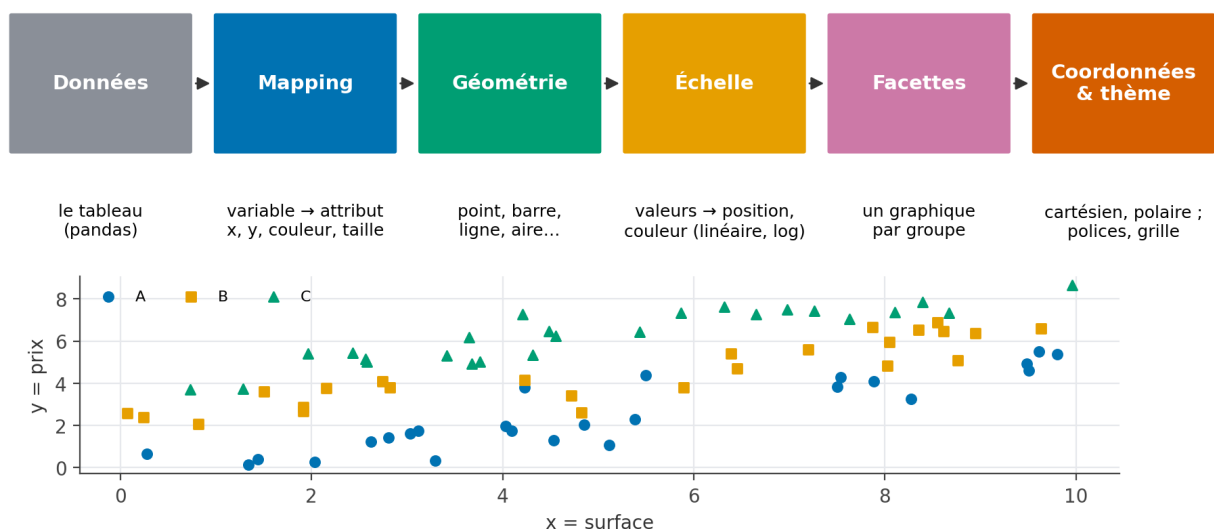


FIGURE 1 – Un graphique = une composition de couches indépendantes

TABLE 1 – Les composantes de la grammaire des graphiques

Élément	Rôle	Exemple
Données	Le jeu de données représenté	Le tableau des ventes mensuelles
Mapping (<i>aesthetics</i>)	L'association entre une variable et un attribut visuel	mois → axe des x, ventes → axe des y
Géométrie	La forme visuelle utilisée pour représenter chaque observation	Points, barres, lignes
Échelle	La façon dont les valeurs sont converties en position, taille ou couleur	Échelle linéaire, logarithmique, catégorielle
Facette	Le découpage du graphique en sous-graphiques répétés	Un graphique par région
Coordonnées & thème	Le système de repères et l'habillage visuel	Cartésien, polaire ; polices, grille, couleurs

i Repère

En Python, **seaborn** et la bibliothèque **plotnine** (portage de **ggplot2**) implémentent explicitement cette grammaire. **matplotlib**, plus bas niveau, ne l'impose pas mais permet de construire n'importe laquelle de ces briques manuellement.

Comprendre cette grammaire aide à répondre à la question centrale de ce chapitre : **quelle géométrie choisir pour quelle question ?**

2.1. Anatomie d'une figure matplotlib

Avant de construire le moindre graphique, il faut connaître le vocabulaire de **matplotlib**, car il conditionne toutes les commandes du chapitre.

Anatomie d'une figure matplotlib

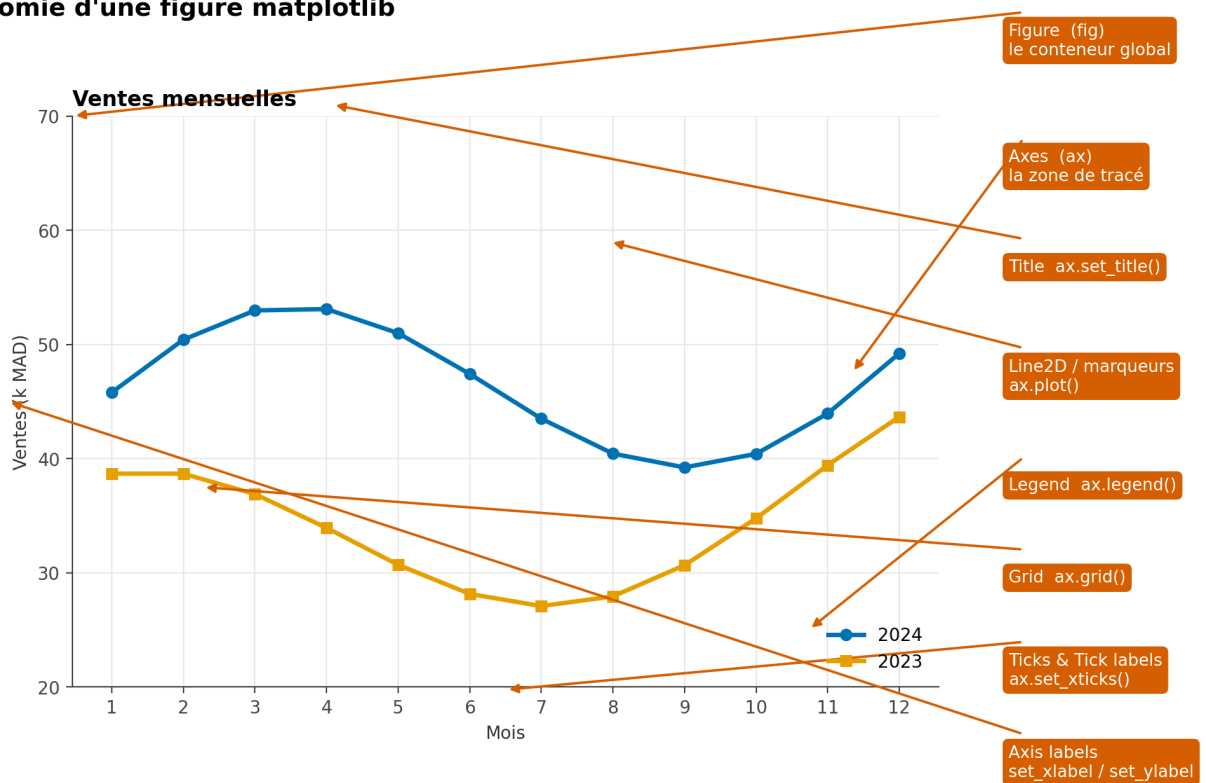


FIGURE 2 – Anatomie d'une figure matplotlib

```
fig, ax = plt.subplots(figsize=(8, 5)) # fig = le conteneur ; ax = la zone de tracé
ax.plot(x, y)
ax.set_title("Titre")
ax.set_xlabel("Axe des x")
ax.set_ylabel("Axe des y")
fig.savefig("figure.png", dpi=150, bbox_inches="tight")
```

! fig et ax : ne pas confondre

La **figure** (**fig**) est le conteneur global — elle peut contenir plusieurs graphiques. Les **axes** (**ax**) désignent chaque zone de tracé individuelle, avec son propre titre, ses propres axes x/y et sa propre légende. Cette distinction devient indispensable dès qu'on assemble plusieurs graphiques dans une même figure (`plt.subplots(nrows=2, ncols=2)`), comme nous le ferons à plusieurs reprises dans ce chapitre.

2.2. Toutes les variables visuelles ne se valent pas

Un même nombre peut être encodé visuellement de multiples façons — une position, une longueur, un angle, une aire, une couleur. Ces encodages ne sont pas équivalents : l’œil humain les compare avec des précisions très différentes, un résultat établi par les travaux de William Cleveland et Robert McGill (1984).

Toutes les variables visuelles ne se valent pas

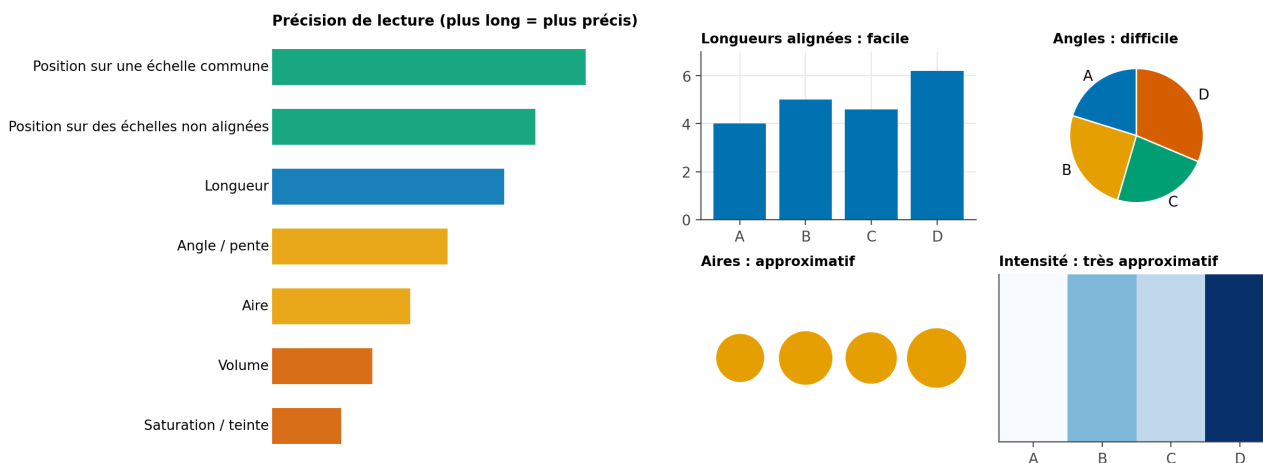


FIGURE 3 – Toutes les variables visuelles ne se valent pas

! Hiérarchie de précision (Cleveland & McGill)

Du plus précis au moins précis pour comparer des valeurs : **position sur une échelle commune** > position sur des échelles non alignées > **longueur** > angle/pente > **aire** > volume > **couleur** (teinte, saturation). Cette hiérarchie, illustrée par la Figure 3, explique à elle seule pourquoi le diagramme en barres (fondé sur la longueur) surpasse presque toujours le camembert (fondé sur l’angle) pour une tâche de comparaison — nous y reviendrons en détail au §4.3.

3. Choisir le bon graphique selon la question posée

Avant d’écrire la moindre ligne de code, il faut se demander quelle **question** le graphique doit permettre de répondre. Cinq grandes familles de questions couvrent l’essentiel des besoins d’un Data Analyst :

TABLE 2 – Cinq familles de questions et leurs graphiques adaptés

Type de question	Exemple de question	Graphiques adaptés
Distribution	Comment se répartissent les valeurs d’une variable ?	Histogramme, densité, boîte à moustaches, violon
Comparaison	Comment se comparent plusieurs catégories ?	Diagramme en barres, lollipop
Relation	Deux variables évoluent-elles ensemble ?	Nuage de points, <i>heatmap</i> de corrélation
Évolution	Comment une valeur change-t-elle dans le temps ?	Courbe, aire empilée
Composition	Quelle est la part de chaque catégorie dans un total ?	Barres empilées à 100 %, camembert (avec prudence), treemap

⚠ Un graphique ne doit répondre qu'à une seule question à la fois

Un graphique qui tente de répondre simultanément à une question de distribution, de comparaison et d'évolution devient généralement illisible. Mieux vaut produire plusieurs graphiques simples qu'un seul graphique surchargé.

4. Les distributions : histogramme, densité, boîte à moustaches

4.1. L'histogramme

L'**histogramme** répartit une variable numérique **continue** en intervalles contigus (*bins*) et affiche le nombre d'observations dans chacun. C'est le graphique de référence pour observer la **forme** d'une distribution (symétrique, asymétrique, bimodale).

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(data=df, x="prix", bins=30)
plt.title("Distribution des prix de vente")
plt.xlabel("Prix (MAD)")
plt.ylabel("Nombre de biens")
plt.show()
```

4.2. Histogramme et diagramme en barres : deux graphiques que l'on confond trop souvent

Un histogramme et un diagramme en barres se *ressemblent* — tous deux affichent des rectangles verticaux — mais ils répondent à des questions fondamentalement différentes et ne doivent **jamais** être confondus, ni dans leur lecture, ni dans leur construction.

Ne pas confondre : deux graphiques à barres, deux natures de variables

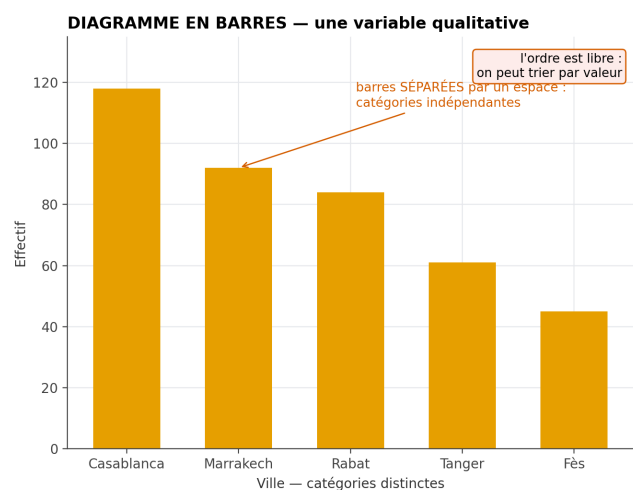
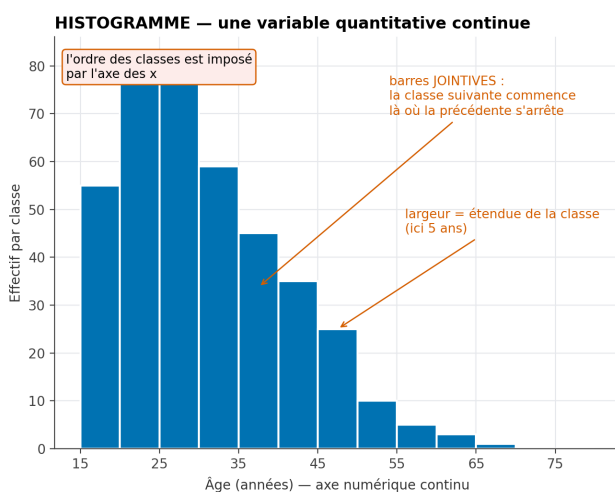


FIGURE 4 – Deux graphiques à barres, deux natures de variables

! La différence essentielle

TABLE 3 – Histogramme contre diagramme en barres

	Histogramme	Diagramme en barres
Variable représentée	Quantitative continue , découpée en classes	Qualitative , catégories déjà distinctes
Axe des x	Numérique et continu	Catégoriel, sans valeur intermédiaire
Espace entre les barres	Aucun — les classes sont contiguës	Un espace visible entre chaque catégorie
Ordre des barres	Imposé par l'axe numérique	Libre — on peut trier par valeur
Ce que mesure la hauteur	Effectif (ou densité) d'une classe de valeurs	Effectif (ou une autre mesure) d'une catégorie

Sur la Figure 4, l'absence d'espace entre les barres de l'histogramme (à gauche) signale visuellement que la classe « 35-40 ans » se termine exactement là où commence la classe « 40-45 ans » : il s'agit d'un découpage continu d'une seule variable. À l'inverse, l'espace entre chaque barre du diagramme (à droite) signale que « Casablanca » et « Marrakech » sont des catégories indépendantes, sans continuité entre elles — les réordonner ne change le sens d'aucune donnée.

```
# HISTOGRAMME : variable numérique continue, bins contigus
sns.histplot(data=df, x="age_bien", bins=15)

# DIAGRAMME EN BARRES : variable catégorielle, barres espacées
effectifs = df["ville"].value_counts()
sns.barplot(x=effectifs.index, y=effectifs.values)
```

4.3. Le choix du nombre de classes

Le nombre d'intervalles (bins) change radicalement la lecture d'un histogramme, pour un même échantillon.

Le même échantillon (n = 770), six histogrammes différents

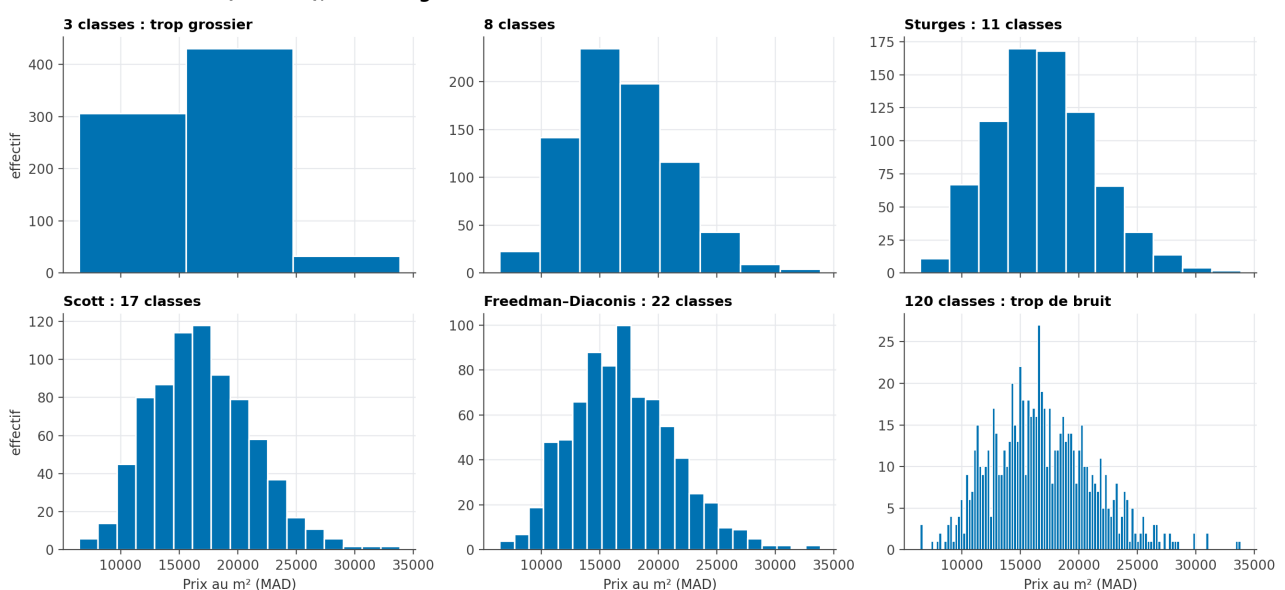


FIGURE 5 – Le même échantillon, six histogrammes différents

⚠ Trop peu, trop, ou juste assez de classes

Trop peu d'intervalles (en haut à gauche de la Figure 5) masquent la forme réelle de la distribution ; trop d'intervalles (en bas à droite) font apparaître du bruit statistique sans signification. Plusieurs règles empiriques proposent un nombre de classes raisonnable à partir de la taille de l'échantillon n :

- **règle de Sturges** : $k = \lceil \log_2(n) + 1 \rceil$, adaptée aux distributions proches de la loi normale ;
- **règle de Scott** : largeur de classe $h = 3,49 s n^{-1/3}$, où s est l'écart-type ;
- **règle de Freedman-Diaconis** : largeur de classe $h = 2 IQR n^{-1/3}$, plus robuste aux valeurs extrêmes car fondée sur l'IQR plutôt que sur l'écart-type.

```
import numpy as np
sns.histplot(data=df, x="prix", bins="sturges") # ou "scott", "fd" (Freedman-Diaconis)
```

Ces règles donnent un **point de départ raisonnable** ; il reste toujours utile de tester plusieurs valeurs de `bins` avant de retenir un graphique définitif.

4.4. Classes de largeurs inégales : effectif ou densité ?

Lorsque les classes d'un histogramme n'ont pas toutes la même largeur (par exemple des tranches de revenus définies par des seuils métier), afficher l'**effectif** en hauteur devient trompeur.

Histogramme à classes de largeurs inégales

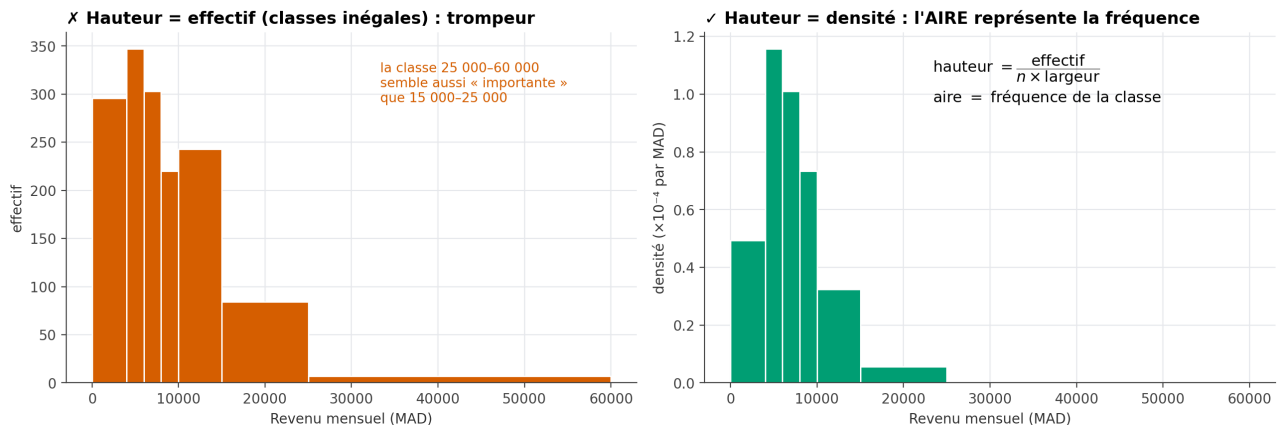


FIGURE 6 – Hauteur = effectif (trompeur) contre hauteur = densité (fidèle)

! La bonne pratique : afficher une densité

La hauteur d'une classe doit représenter sa **densité**, définie par

$$\text{densité} = \frac{\text{effectif de la classe}}{n \times \text{largeur de la classe}},$$

de sorte que ce soit l'**aire** de chaque rectangle — et non sa seule hauteur — qui représente fidèlement la fréquence de la classe (Figure 6, panneau de droite). `seaborn` applique cette correction automatiquement si l'argument `stat="density"` est précisé :

```
sns.histplot(data=df, x="revenu_mensuel", bins=[0, 4000, 6000, 8000, 10000, 15000, 25000, 60000], stat="density")
```

4.5. L'estimation de densité par noyau (KDE)

Plutôt que de découper une variable en classes discrètes, l'**estimation de densité par noyau** (*Kernel Density Estimation*, KDE) trace une courbe lisse en superposant une petite « cloche » (le noyau) sur chaque observation, puis en les additionnant.

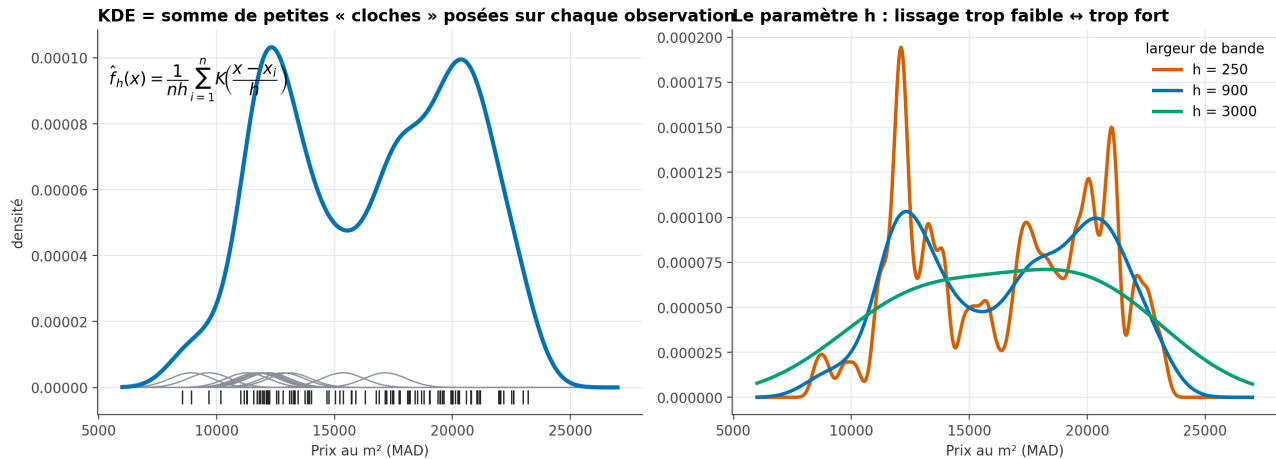


FIGURE 7 – KDE : somme de petites cloches, effet du paramètre de lissage h

La formule générale d'un estimateur à noyau gaussien s'écrit :

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

où K est une fonction noyau (souvent gaussienne) et h la **largeur de bande** (*bandwidth*), qui contrôle le lissage : trop petite, la courbe devient bruitée et reproduit chaque observation individuellement (panneau de droite de la Figure 7, courbe rouge) ; trop grande, elle masque une éventuelle bimodalité (courbe verte).

```
sns.kdeplot(data=df, x="prix_m2", fill=True)
sns.kdeplot(data=df, x="prix_m2", bw_adjust=0.3) # lissage plus fin
```

4.6. La fonction de répartition empirique (ECDF)

L'**ECDF** (*Empirical Cumulative Distribution Function*) affiche, pour chaque valeur x , la proportion d'observations inférieures ou égales à x . Contrairement à l'histogramme, elle ne nécessite **aucun choix de classes**, et permet de lire directement médiane et quartiles sur le graphique.

```
sns.ecdfplot(data=df, x="prix")
sns.ecdfplot(data=df, x="prix", hue="type_bien") # comparer plusieurs groupes sans choisir de classes
```

💡 Quand préférer l'ECDF ?

L'ECDF est particulièrement utile pour **comparer plusieurs groupes** (panneau droit de la Figure 8) : contrairement à des histogrammes superposés, les courbes ECDF restent lisibles même avec de nombreux groupes, et ne dépendent d'aucun choix de nombre de classes.

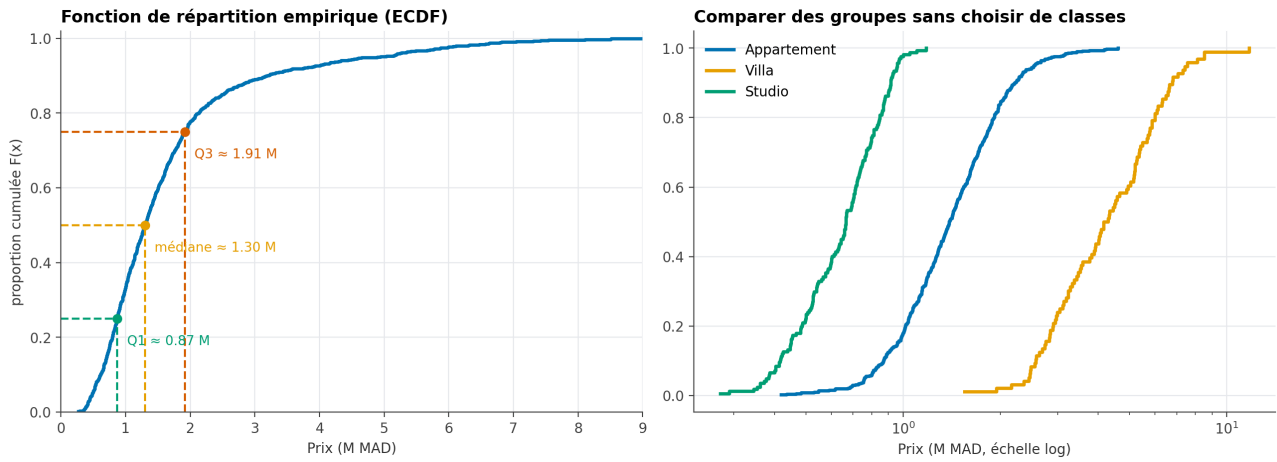


FIGURE 8 – Fonction de répartition empirique : lecture des quantiles, comparaison entre groupes

Anatomie d'une boîte à moustaches

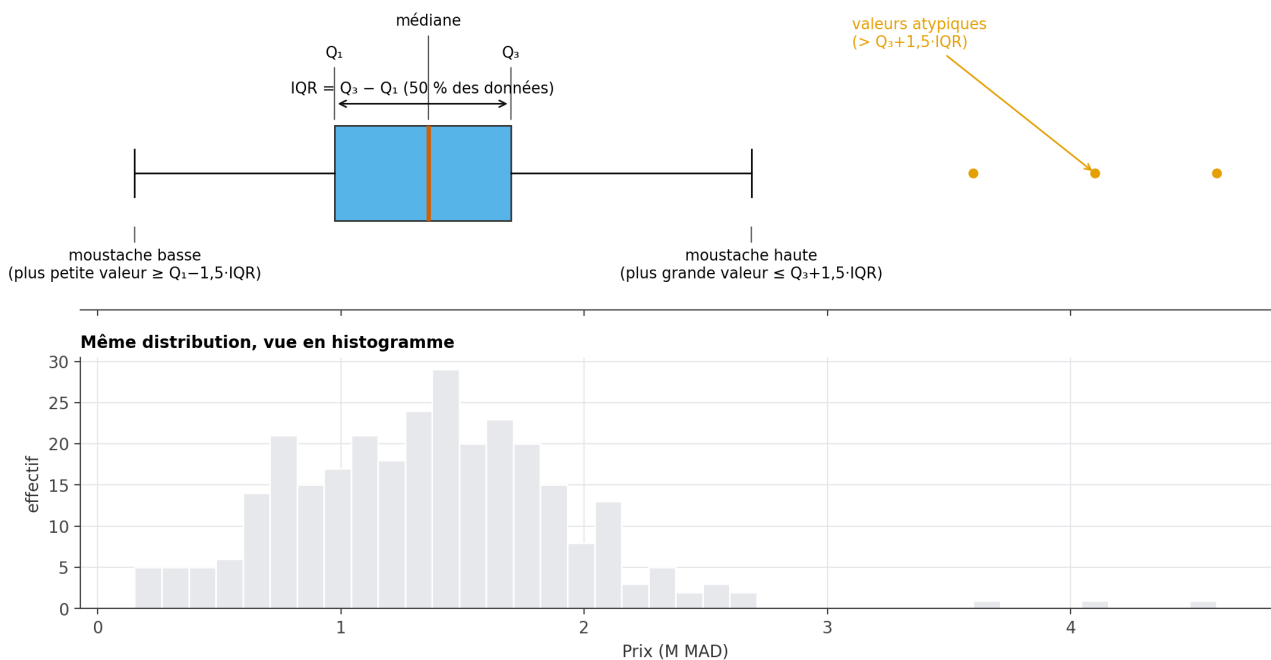


FIGURE 9 – Anatomie d'une boîte à moustaches

4.7. La boîte à moustaches (*boxplot*)

La **boîte à moustaches**, déjà mentionnée au chapitre 1 (chapitre commun) pour la détection des valeurs aberrantes, résume une distribution en cinq repères construits à partir des quartiles.

```
sns.boxplot(data=df, x="quartier", y="prix")
```

! Lire une boîte à moustaches

- la **boîte** s'étend de Q_1 à Q_3 (50 % des observations) ;
- le **trait central** marque la médiane ;
- les **moustaches** s'étendent jusqu'à la valeur la plus extrême encore **à l'intérieur** de $[Q_1 - 1,5 IQR, Q_3 + 1,5 IQR]$ (chapitre commun, §3.2) ;
- les **points isolés** au-delà des moustaches sont des valeurs potentiellement atypiques.

⚠ Ce qu'une boîte à moustaches ne montre jamais

Une boîte à moustaches ne révèle **pas** si une distribution est bimodale : deux groupes de valeurs distincts peuvent produire une boîte parfaitement banale, l'information étant résumée par des quantiles qui ne « voient » pas les creux entre les modes. C'est précisément ce que montre la section suivante.

4.8. Une même distribution, six représentations

Le choix entre histogramme, densité, boîte à moustaches ou violon n'est jamais neutre : chacun révèle — ou cache — des aspects différents de la même distribution.

Une même distribution bimodale vue par six représentations

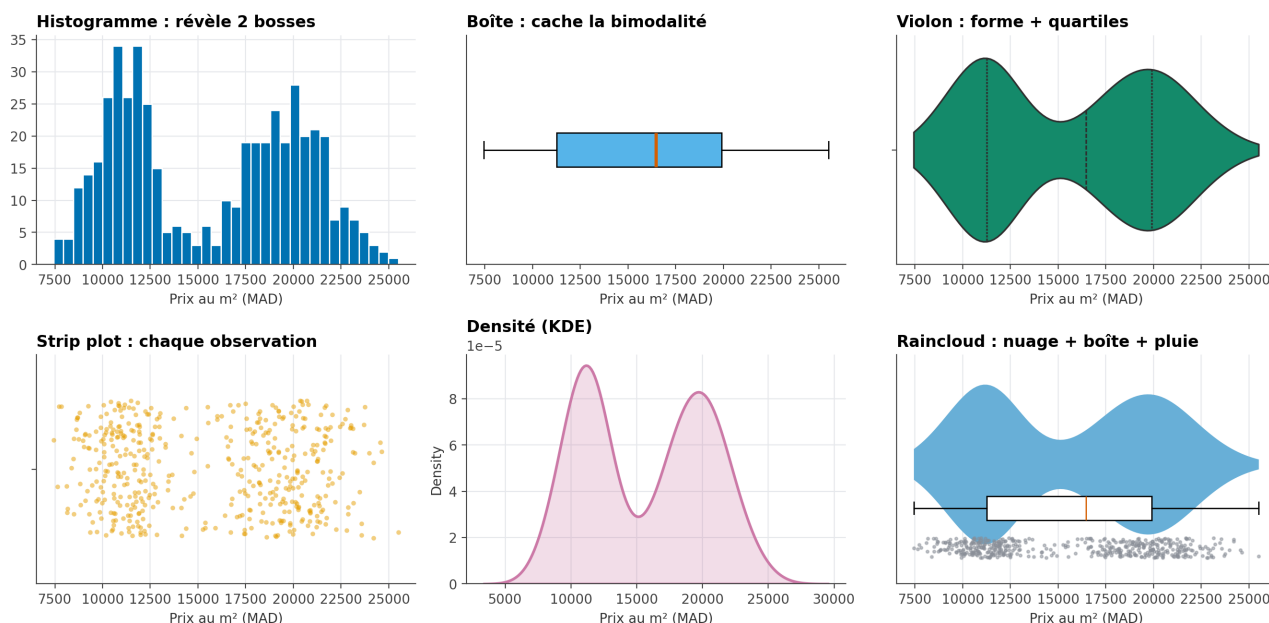


FIGURE 10 – Une distribution bimodale vue par six représentations

Sur la Figure 10, une distribution comportant deux « bosses » distinctes (par exemple, deux quartiers aux prix très différents mélangés dans une même variable) est clairement visible sur l'histogramme, le violon, le KDE et le *strip plot* (chaque observation individuelle) — mais **invisible** sur la simple boîte à moustaches, qui ne renvoie qu'une boîte centrale d'apparence banale.

```
sns.violinplot(data=df, x="prix_m2", inner="quartile") # combine boîte et densité
sns.stripplot(data=df, x="prix_m2", alpha=.5, jitter=.3) # chaque observation individuelle
```

Le dernier panneau, appelé *raincloud plot* (« nuage de pluie »), combine délibérément un demi-violon, une boîte à moustaches et un nuage de points individuels : il cumule les avantages des trois représentations, au prix d'une construction plus élaborée.

💡 Règle pratique

En exploration personnelle, préférez systématiquement un histogramme ou un KDE à une simple boîte à moustaches dès que vous soupçonnez une distribution complexe. Réservez la boîte à moustaches à la **comparaison rapide** de plusieurs groupes (§4.7), où sa compacité devient un avantage plutôt qu'une limite.

4.9. Le diagramme quantile-quantile (QQ-plot)

Pour vérifier si une variable suit approximativement une loi normale — une hypothèse requise par certaines méthodes statistiques —, le diagramme quantile-quantile compare ses quantiles observés aux quantiles théoriques d'une loi normale.

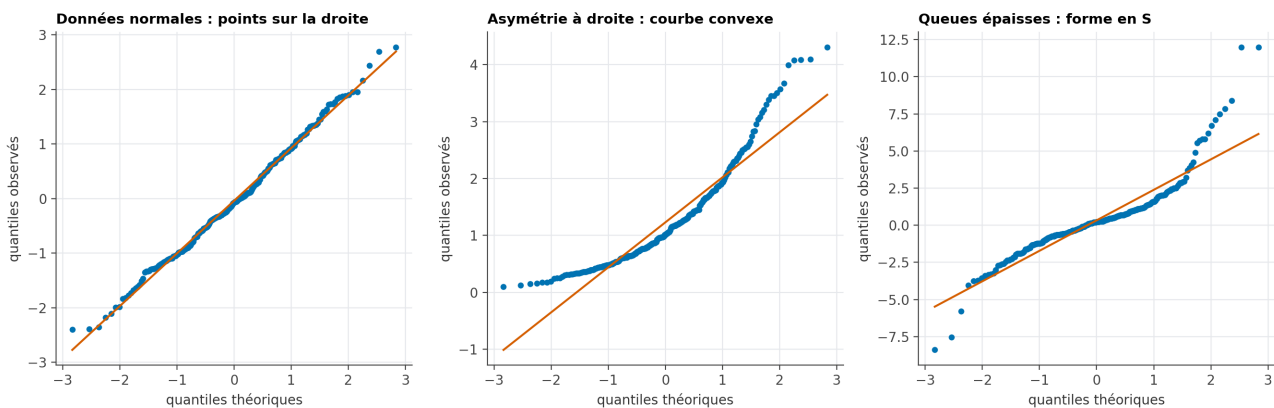


FIGURE 11 – Trois signatures caractéristiques sur un diagramme quantile-quantile

```
from scipy import stats
stats.probplot(df["log_prix"], dist="norm", plot=plt.gca())
```

Comme le montre la Figure 11, des points parfaitement alignés sur la droite rouge indiquent une distribution proche de la normale ; une courbe convexe signale une asymétrie ; une forme en S signale des « queues » plus épaisses que la loi normale (davantage de valeurs extrêmes).

5. Comparaisons : le diagramme en barres et ses variantes

5.1. Le diagramme en barres

Le **diagramme en barres** compare un indicateur (effectif, somme, moyenne...) entre plusieurs catégories. C'est le graphique le plus fiable pour une **comparaison**, car — la Figure 3 le rappelle — l'œil humain estime beaucoup mieux des longueurs alignées sur une base commune que des surfaces ou des angles.

```

ventes_par_region = df.groupby("region")["montant"].sum().sort_values(ascending=False)

sns.barplot(x=ventes_par_region.index, y=ventes_par_region.values)
plt.title("Chiffre d'affaires total par région - 2025")
plt.ylabel("Chiffre d'affaires (MAD)")
plt.xlabel("")
plt.show()

```

💡 Bonne pratique

Trier les barres par ordre décroissant de valeur (plutôt que par ordre alphabétique) facilite immédiatement la lecture des catégories les plus importantes.

5.2. Six variantes du diagramme en barres

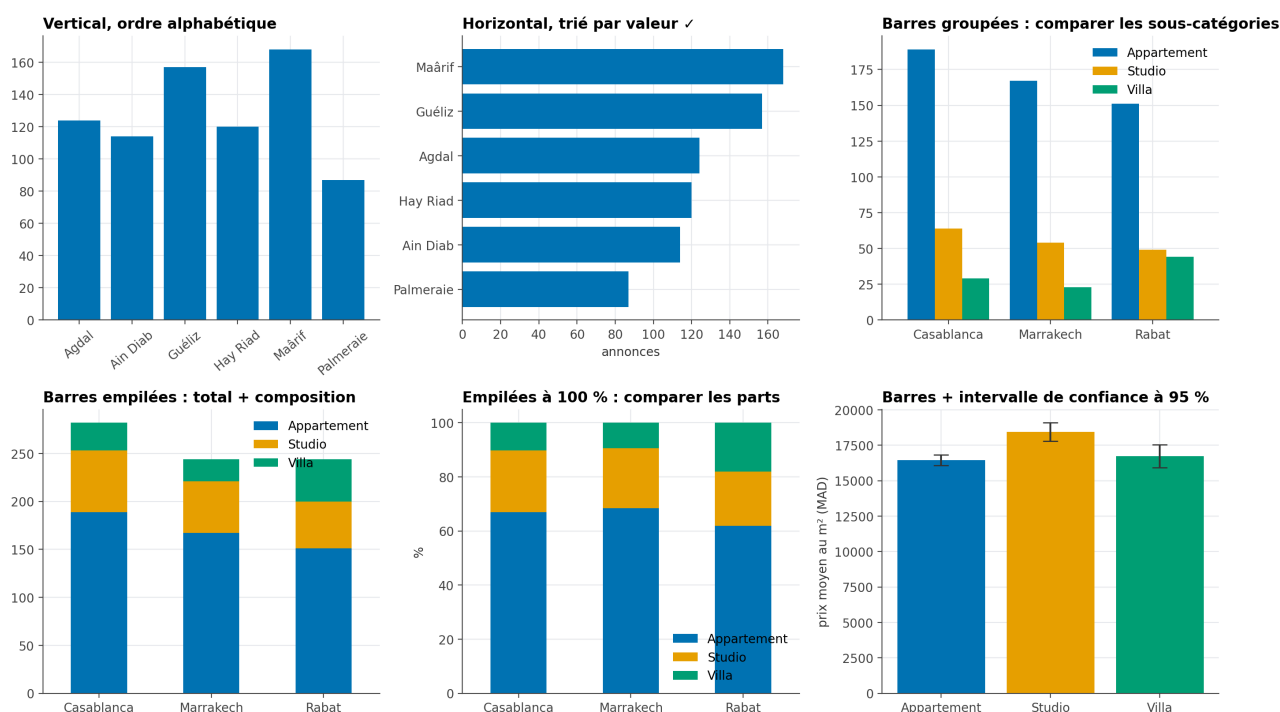


FIGURE 12 – Six variantes du diagramme en barres

TABLE 4 – Variantes du diagramme en barres

Variante	Usage	Fonction
Vertical	Peu de catégories, libellés courts	<code>sns.barplot(x=..., y=...)</code>
Horizontal, trié	Libellés longs ou nombreux — la variante à privilégier par défaut	<code>sns.barplot(y=..., x=...)</code>
Groupé	Comparer une sous-catégorie supplémentaire	<code>df.plot(kind="bar")</code> sur un tableau croisé
Empilé	Total et composition en une seule barre	<code>.plot(kind="bar", stacked=True)</code>
Empilé à 100 %	Comparer des parts plutôt que des valeurs absolues	Diviser chaque barre par son total avant de tracer
Avec intervalle de confiance	Signaler l'incertitude d'une moyenne estimée	<code>plt.bar(..., yerr=...)</code>

```

tableau = pd.crosstab(df["ville"], df["type_bien"])
tableau.plot(kind="bar", stacked=True) # empilé
(tableau.div(tableau.sum(axis=1), axis=0) * 100).plot(kind="bar", stacked=True) # empilé à 100 %

```

5.3. Lollipop et *dot plot* : des barres allégées

Lorsque les barres sont très longues et peu informatives (l'œil ne regarde en réalité que leur extrémité), deux variantes réduisent l'encre du graphique sans perdre en précision de lecture.

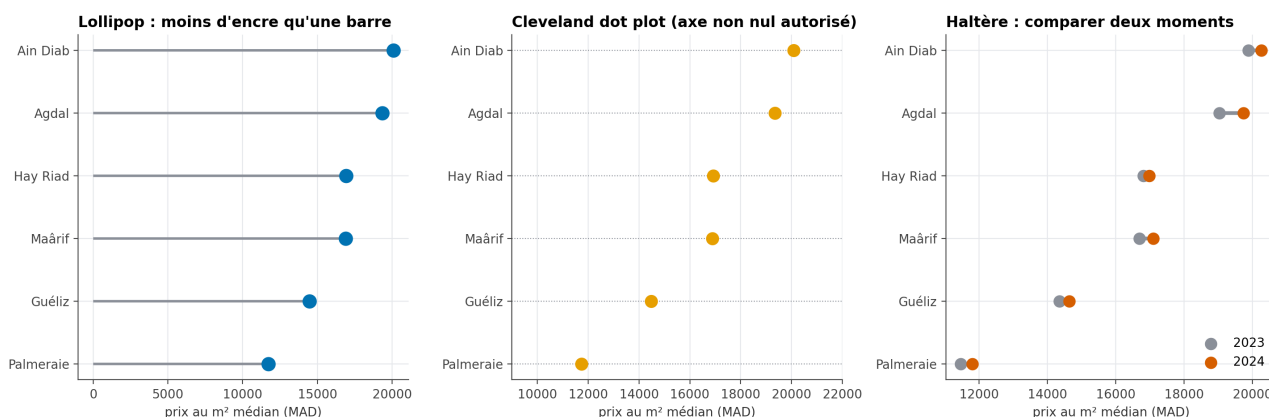


FIGURE 13 – Lollipop, dot plot de Cleveland et diagramme en haltère

```

r = df.groupby("quartier")["prix_m2"].median().sort_values()
plt.hlines(r.index, 0, r.values, color="gray")
plt.plot(r.values, r.index, "o")

```

Le **diagramme en haltère** (*dumbbell chart*, panneau de droite) est particulièrement adapté pour comparer une même catégorie à **deux moments** ou selon **deux conditions** — ici, l'évolution du prix médian par quartier entre deux années.

i Le *dot plot* de Cleveland autorise un axe non nul

Contrairement à un diagramme en barres classique (§6.2), un *dot plot* peut légitimement démarrer son axe ailleurs qu'à zéro : la position du point, et non la longueur d'une barre, porte l'information — aucune barre tronquée ne vient exagérer visuellement les écarts.

6. Le diagramme circulaire (camembert) — et ses limites

Le **camembert** (*pie chart*) représente la part de chaque catégorie dans un total, sous forme d'angles.

```

parts = df["categorie_produit"].value_counts()

plt.pie(parts.values, labels=parts.index, autopct="%1.0f%%")
plt.title("Répartition des ventes par catégorie de produit")
plt.show()

```

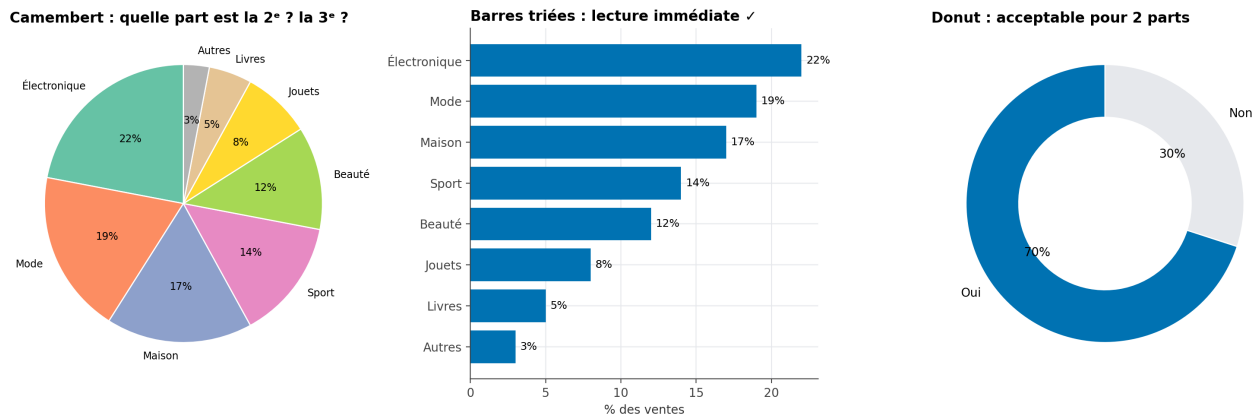


FIGURE 14 – Camembert, barres triées, et donut à deux catégories

6.1. Camembert contre barres triées : une comparaison directe

! Zoom sur les tendances : pourquoi éviter le camembert

Le camembert est l'un des graphiques les plus critiqués par la communauté de la visualisation de données, pour la raison énoncée dès le §2.2 : l'œil humain compare beaucoup moins bien des **angles** ou des **surfaces** que des **longueurs alignées**. Essayez, sur le panneau de gauche de la Figure 14, de répondre rapidement à la question « quelle est la 2^e catégorie la plus vendue, et de combien dépasse-t-elle la 3^e ? » — puis comparez avec la même question posée sur le panneau central.

Dès que l'on dépasse 4 ou 5 catégories, ou que deux parts sont proches en taille, un camembert devient difficile à lire correctement. Dans la quasi-totalité des cas, un **diagramme en barres trié** transmet la même information plus clairement.

6.2. Quand un camembert reste défendable

Le camembert reste acceptable dans un cas précis, illustré par le panneau de droite de la Figure 14 : une composition très simple, **deux à trois catégories**, avec des écarts marqués — par exemple pour illustrer une proportion unique (« 70 % oui / 30 % non »). Sa variante en **donut** (anneau) est parfois préférée pour cet usage, car le centre vide permet d'y afficher directement le chiffre clé.

```
plt.pie([70, 30], labels=["Oui", "Non"], autopct="%1.0f%%",
        wedgeprops=dict(width=0.4)) # width < 1 transforme le camembert en donut
```

7. Relations entre variables : nuage de points et *heatmap*

7.1. Le nuage de points (*scatter plot*)

Le **nuage de points** affiche chaque observation comme un point positionné selon deux variables numériques.

C'est le graphique de référence pour étudier une **relation** entre deux variables, en complément direct des coefficients de corrélation vus au chapitre précédent.

```
sns.scatterplot(data=df, x="surface", y="prix", hue="quartier")
plt.title("Prix en fonction de la surface, par quartier")
plt.xlabel("Surface (m²)")
plt.ylabel("Prix (MAD)")
plt.show()
```

7.2. Au-delà de deux dimensions : mapper des variables supplémentaires

Un nuage de points peut encoder simultanément jusqu'à quatre variables, en combinant position, couleur et taille — une application directe de la grammaire des graphiques (§2).

Le nuage de points peut coder jusqu'à 4 variables à la fois

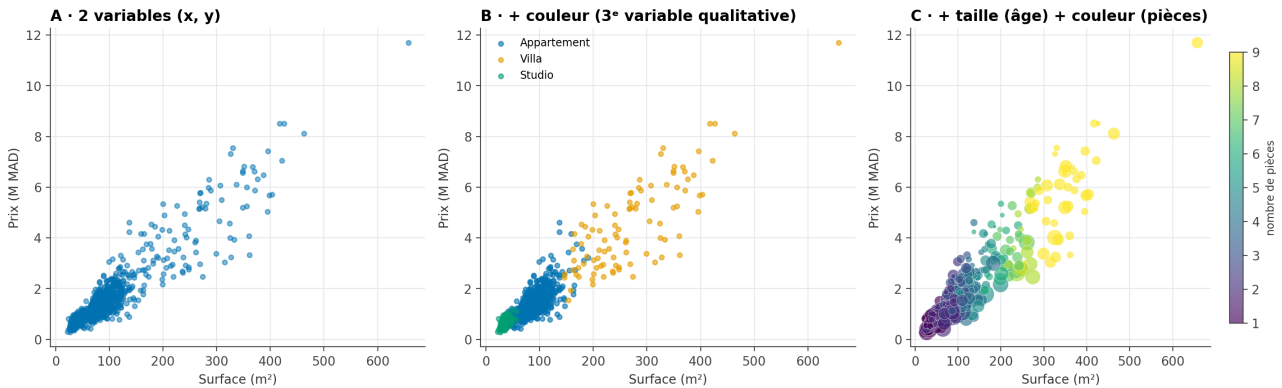


FIGURE 15 – Le nuage de points peut coder jusqu'à quatre variables à la fois

```
sns.scatterplot(
    data=df, x="surface", y="prix",
    hue="nombre_pieces",      # 3 variable : couleur
    size="age_bien",          # 4 variable : taille des points
)
```

i Le bubble chart

Une variante à la **taille des points** proportionnelle à une troisième variable numérique (plutôt qu'à une couleur catégorielle) est parfois appelée *bubble chart*.

7.3. Le problème de la surimpression (*overplotting*)

Avec plusieurs milliers de points, un nuage de points devient une « bouillie » de couleur où la densité réelle des observations disparaît sous les superpositions.

Le problème de la surimpression (*overplotting*) et ses remèdes

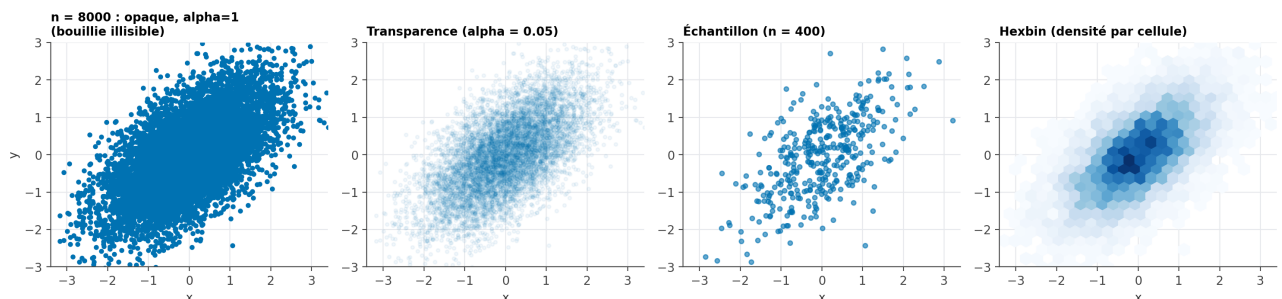


FIGURE 16 – Le problème de la surimpression et ses remèdes

TABLE 5 – Remèdes à la surimpression

Remède	Principe	Code
Transparence	Chaque point devient semi-transparent ; les zones denses apparaissent plus foncées	<code>alpha=0.05</code>
Échantillonnage	N'afficher qu'un sous-ensemble aléatoire des observations	<code>df.sample(n=500, random_state=42)</code>
<i>Hexbin</i>	Découper le plan en hexagones et colorer selon le nombre de points par cellule	<code>plt.hexbin(x, y, gridsize=30)</code>

7.4. La *heatmap* (carte de chaleur)

La **heatmap** représente une matrice de valeurs à l'aide d'une échelle de couleurs. Son usage le plus fréquent en analyse de données est la visualisation d'une **matrice de corrélation** (chapitre précédent, Figure 4) ou d'un **tableau croisé dynamique** (chapitre précédent, Figure 15) entre plusieurs variables.

```
matrice_correlation = df[["surface", "prix", "nombre_pieces", "age_bien"]].corr()

sns.heatmap(matrice_correlation, annot=True, cmap="coolwarm", center=0)
plt.title("Matrice de corrélation")
plt.show()
```

Une heatmap peut également représenter une activité dans le temps (par exemple le nombre de commandes par jour de la semaine et par heure), auquel cas on parle parfois de *calendar heatmap*.

💡 Récapitulatif des graphiques essentiels

TABLE 6 – Synthèse des graphiques essentiels

Graphique	Question	Type(s) de variable
Histogramme / KDE / ECDF	Distribution	1 quantitative
Barres / lollipop	Comparaison	1 qualitative + 1 quantitative
Camembert / donut	Composition (avec prudence)	1 qualitative
Nuage de points	Relation	2 quantitatives (+ 1-2 en couleur/taille)
Courbe	Évolution	1 temporelle + 1 quantitative
Boîte à moustaches / violon	Distribution par catégorie	1 qualitative + 1 quantitative
Heatmap	Relation entre plusieurs variables	Plusieurs quantitatives, ou 2 qualitatives (tableau croisé)

8. Évolution : courbes et variantes temporelles

8.1. La courbe (*line chart*)

La **courbe** relie des points ordonnés dans le temps. C'est le graphique de référence pour une **évolution**, une tendance ou une saisonnalité.

```

ventes_mensuelles = df.groupby("mois")["montant"].sum()

plt.plot(ventes_mensuelles.index, ventes_mensuelles.values, marker="o")
plt.title("Évolution mensuelle du chiffre d'affaires - 2025")
plt.xlabel("Mois")
plt.ylabel("Chiffre d'affaires (MAD)")
plt.show()

```

⚠ Ne jamais relier des catégories sans ordre naturel

Une courbe suppose un ordre implicite entre les points (le temps, le plus souvent). Relier par une ligne des catégories sans ordre — des villes, des produits — suggère une progression qui n'existe pas et induit le lecteur en erreur.

8.2. Visualiser une courbe décomposée

Le chapitre précédent (§8) a montré comment **calculer** la décomposition d'une série temporelle en tendance, saisonnalité et résidu. Voici comment la **lire** sous forme de graphique superposé :

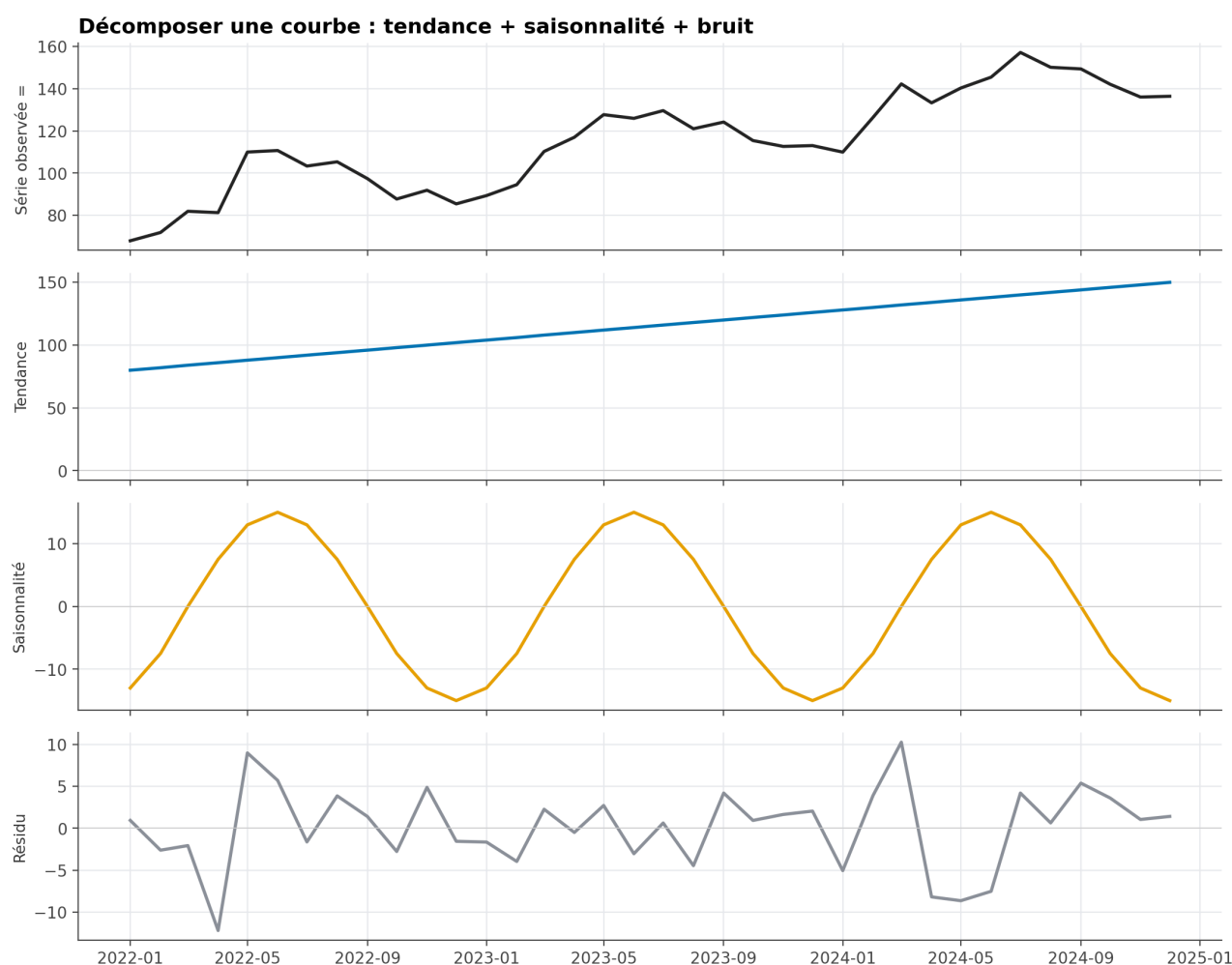


FIGURE 17 – Décomposer une courbe : tendance, saisonnalité, résidu

```
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
axes[0].plot(serie.index, serie.values); axes[0].set_ylabel("Observée")
axes[1].plot(decomposition.trend); axes[1].set_ylabel("Tendance")
axes[2].plot(decomposition.seasonal); axes[2].set_ylabel("Saisonnalité")
axes[3].plot(decomposition.resid); axes[3].set_ylabel("Résidu")
```

Empiler les quatre panneaux sur un même axe des x (temps partagé, `sharex=True`) permet de vérifier visuellement que la somme des trois composantes du bas reconstitue bien la série observée du haut.

8.3. Aire empilée et *streamgraph*

Lorsqu'il faut représenter à la fois un **volume total** dans le temps et sa **composition**, l'aire empilée prolonge naturellement la courbe simple.

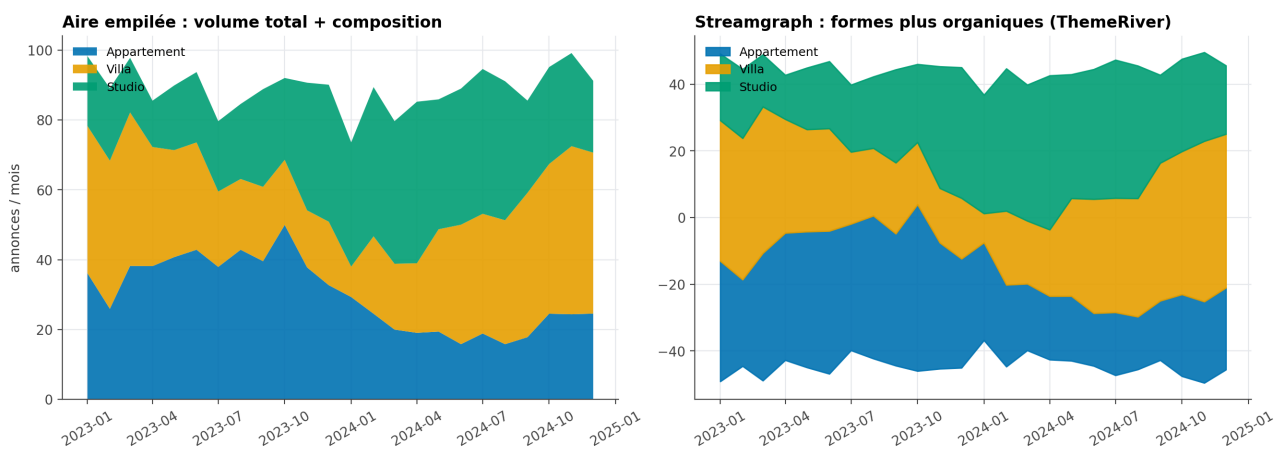


FIGURE 18 – Aire empilée et streamgraph

```
plt.stackplot(mois, tableau.T, labels=tableau.columns)
plt.legend(loc="upper left")
```

Le **streamgraph** (panneau de droite) centre les aires empilées autour d'une ligne médiane plutôt qu'une base à zéro, ce qui produit des formes plus organiques et évite qu'une catégorie placée tout en bas de la pile paraisse visuellement écrasée.

8.4. Le *slopegraph* et le nuage de points connecté

Pour comparer un petit nombre de séries à **deux instants précis** (plutôt qu'une évolution continue), le *slopegraph* offre une alternative très lisible à la courbe classique.

```
for ville in prix_2023.index:
    plt.plot([0, 1], [prix_2023[ville], prix_2024[ville]], marker="o")
plt.xticks([0, 1], ["2023", "2024"])
```

Le **nuage de points connecté** (*connected scatterplot*, panneau de droite) relie dans l'ordre chronologique des points positionnés selon deux variables numériques — ici, le volume mensuel d'annonces et le prix médian — révélant la **trajectoire** conjointe des deux variables plutôt que leur seule évolution séparée.

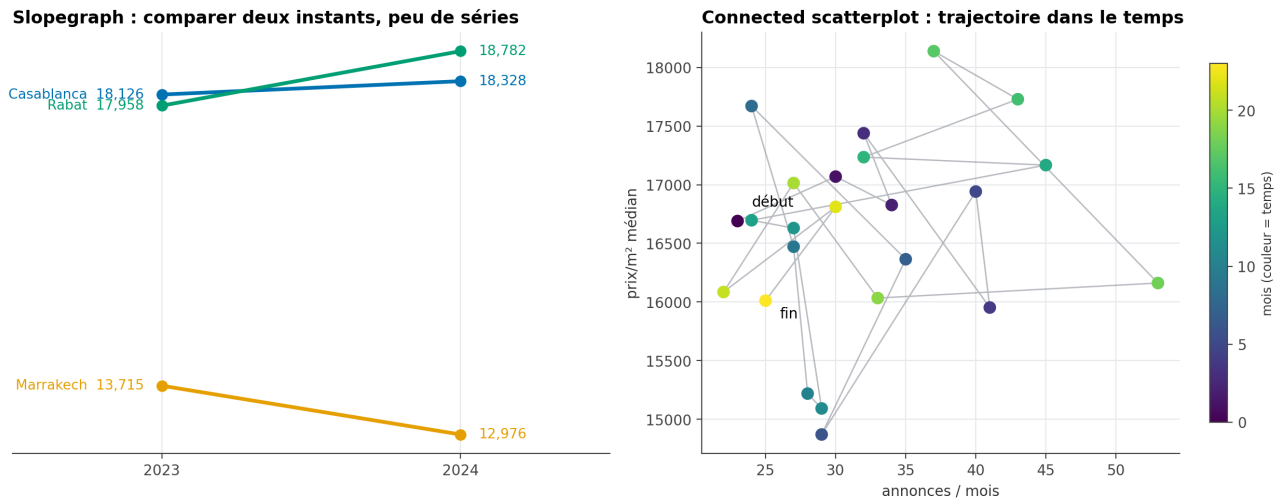


FIGURE 19 – Slopegraph et nuage de points connecté

9. Cartographie : représenter une dimension géographique

Lorsqu'une variable est associée à un territoire (un pays, une région, une ville), la représenter sur une carte exploite une capacité de lecture spatiale qu'aucun graphique classique ne peut offrir. Python manipule les données géographiques via **geopandas**, qui étend **pandas** avec une colonne spéciale **geometry** contenant les contours (points, lignes ou polygones).

```
import geopandas as gpd

regions = gpd.read_file("regions_maroc.geojson") # un GeoDataFrame, comme un DataFrame + geometry
regions.plot() # tracé minimal
```

9.1. La carte choroplèthe

La **carte choroplèthe** colore chaque zone géographique (région, pays, quartier) selon la valeur d'un indicateur — c'est l'équivalent cartographique d'une *heatmap*.

```
fig, ax = plt.subplots(figsize=(8, 8))
regions.plot(column="depense_moyenne", cmap="YlGnBu", linewidth=0.8,
             edgecolor="white", legend=True, ax=ax)
ax.set_axis_off()
ax.set_title("Dépense en ligne moyenne par région")
```

9.2. Le piège classique : la surface n'est pas l'importance

Une carte choroplèthe encode une valeur par une **couleur**, mais chaque zone occupe aussi une **surface** très inégale — un immense territoire faiblement peuplé attire visuellement l'œil bien plus qu'une petite région densément peuplée, même si cette dernière pèse davantage dans l'indicateur étudié.

! À retenir

Ce piège n'est qu'un cas particulier du principe posé au §2.2 (Figure 3) : la **surface** est un encodage visuel peu précis. Deux solutions courantes : représenter l'indicateur avec des **symboles proportionnels** plutôt qu'un remplissage de zone (panneau de droite de la Figure 21, §9.3), ou déformer délibérément la carte selon la valeur étudiée (le **cartogramme**, §9.4).

Choroplèthe : un indicateur, une couleur par région

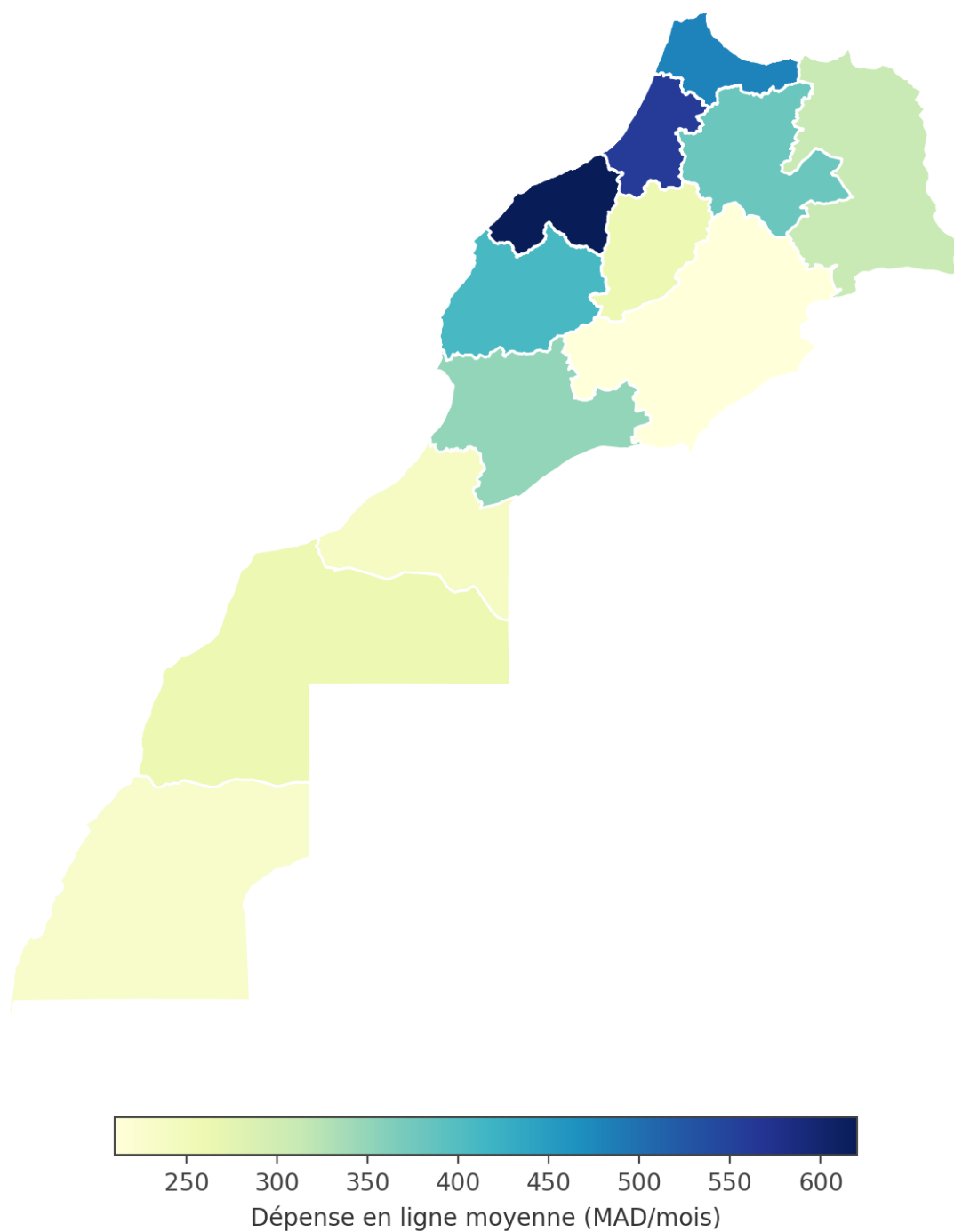


FIGURE 20 – Choroplèthe : un indicateur, une couleur par région

Le piège classique de la carte choroplèthe : surface $\neq$ importance

✗ Les grandes régions sahariennes dominent visuellement malgré une faible population

✓ Symboles proportionnels : la taille du cercle représente la population, pas la surface

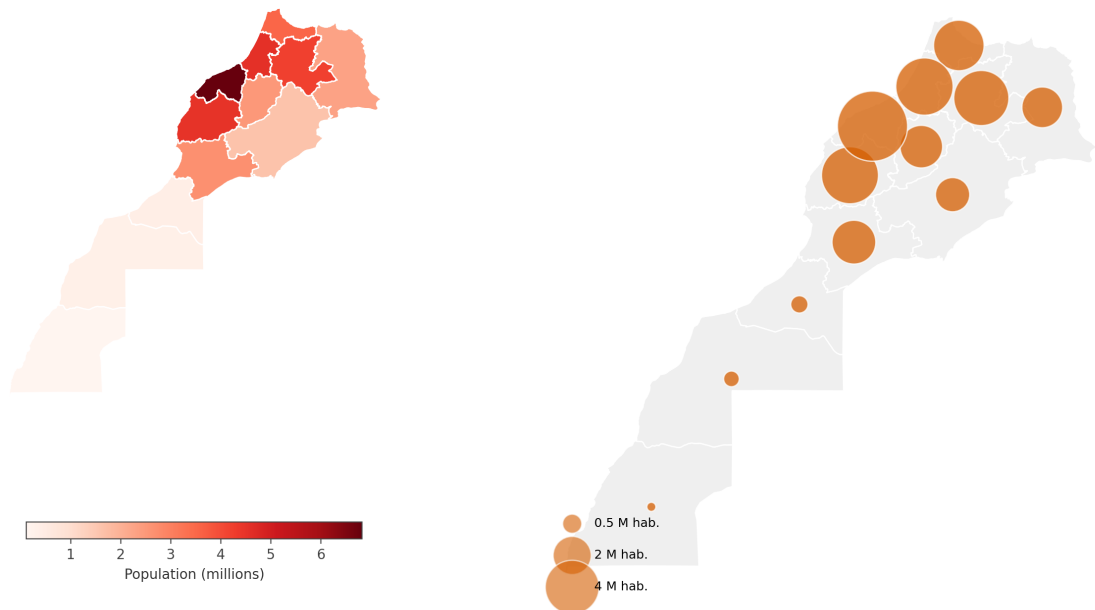


FIGURE 21 – Le piège classique de la carte choroplèthe

9.3. La carte de points géolocalisés

Lorsque les données correspondent à des points précis (villes, magasins, capteurs) plutôt qu'à des zones, la carte de points, éventuellement combinée à une taille et une couleur proportionnelles à un indicateur, évite justement le piège de la surface.

```
fig, ax = plt.subplots(figsize=(8, 8))
regions.plot(color="#F4F6F8", edgecolor="#CCCCCC", ax=ax) # fond de carte
ax.scatter(villes["lon"], villes["lat"], s=villes["population"] * 100,
           c=villes["population"], cmap="plasma", alpha=0.85)
```

9.4. Le cartogramme

Un **cartogramme** pousse la logique du symbole proportionnel plus loin : il déforme la carte elle-même — ou la remplace par des formes simples (cercles, carrés) — dont la **taille**, et non plus la position géographique exacte, représente la valeur étudiée.

💡 Quand utiliser un cartogramme ?

Le cartogramme est particulièrement efficace pour des indicateurs de **population** ou d'**activité économique**, où la forme géographique réelle importe moins que la comparaison des valeurs. Il perd en revanche la capacité à situer précisément un lieu sur une carte — un compromis à peser selon l'objectif du graphique.

9.5. Un mot sur les projections

La Terre est une sphère ; un écran est plat. Toute carte implique donc une **projection**, qui déforme inévitablement soit les surfaces, soit les angles, soit les distances.

Carte de points : villes principales (taille = population)

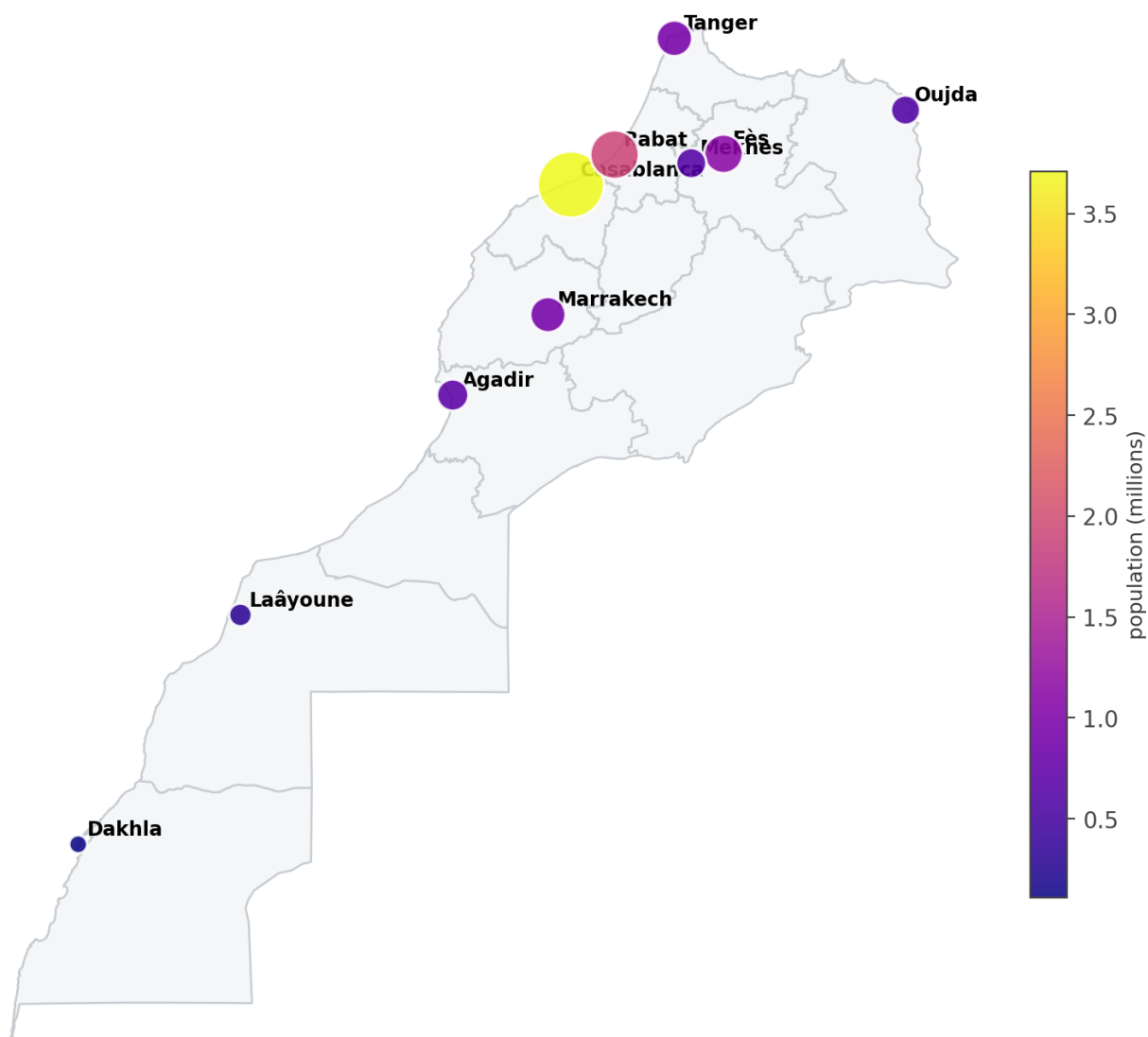
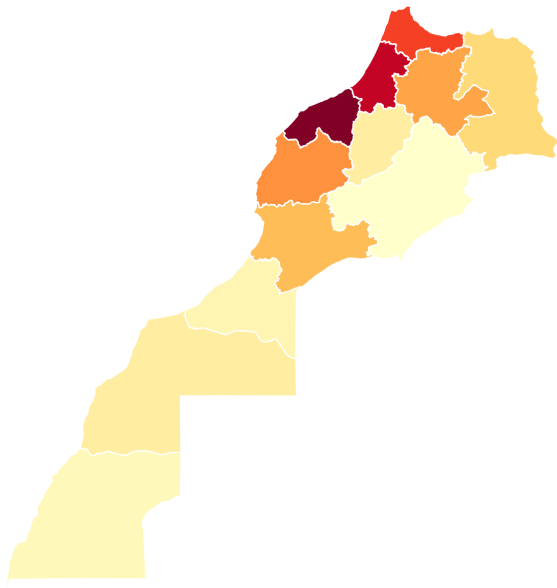


FIGURE 22 – Carte de points : villes principales, taille = population

Cartogramme : quand la valeur importe plus que la forme du territoire
Carte géographique classique



Cartogramme à bulles : la taille remplace la géographie



FIGURE 23 – Cartogramme : la taille remplace la géographie

Le choix d'une projection déforme différemment le territoire

Geographique (EPSG:4326)
lon/lat non projetées



Mercator (EPSG:3857)
usuelle sur le web, déforme les surfaces



Merchich / Lambert (EPSG:26191)
projection locale du Maroc



FIGURE 24 – Le choix d'une projection déforme différemment le territoire

```
regions.crs # système de coordonnées actuel (souvent EPSG:4326 - longitude/latitude)
regions_web = regions.to_crs("EPSG:3857") # projection Web Mercator, usuelle sur le web
```

i Repère

Le choix de la projection compte surtout pour des cartes à grande échelle (un continent, le monde) ou des calculs de surfaces/distances précis ; pour une carte régionale ou nationale comme celles de ce chapitre, l'effet reste modeste. Retenez simplement qu'une carte n'est jamais une représentation neutre du territoire, mais toujours le produit d'un choix de projection.

10. Graphiques avancés — à réserver aux cas qui le justifient

Les graphiques suivants sortent de la boîte à outils essentielle (§4 à §9). Ils apportent une réelle valeur ajoutée dans des situations précises, mais leur usage abusif nuit presque toujours à la lisibilité : à manier avec discernement.

10.1. Petits multiples (facettes)

Plutôt que de surcharger un seul graphique avec de nombreuses catégories, les **petits multiples** répètent le même graphique simple, une fois par catégorie — une application directe de la facette, cinquième composante de la grammaire des graphiques (§2).

Petits multiples (facettes) : un panneau par quartier

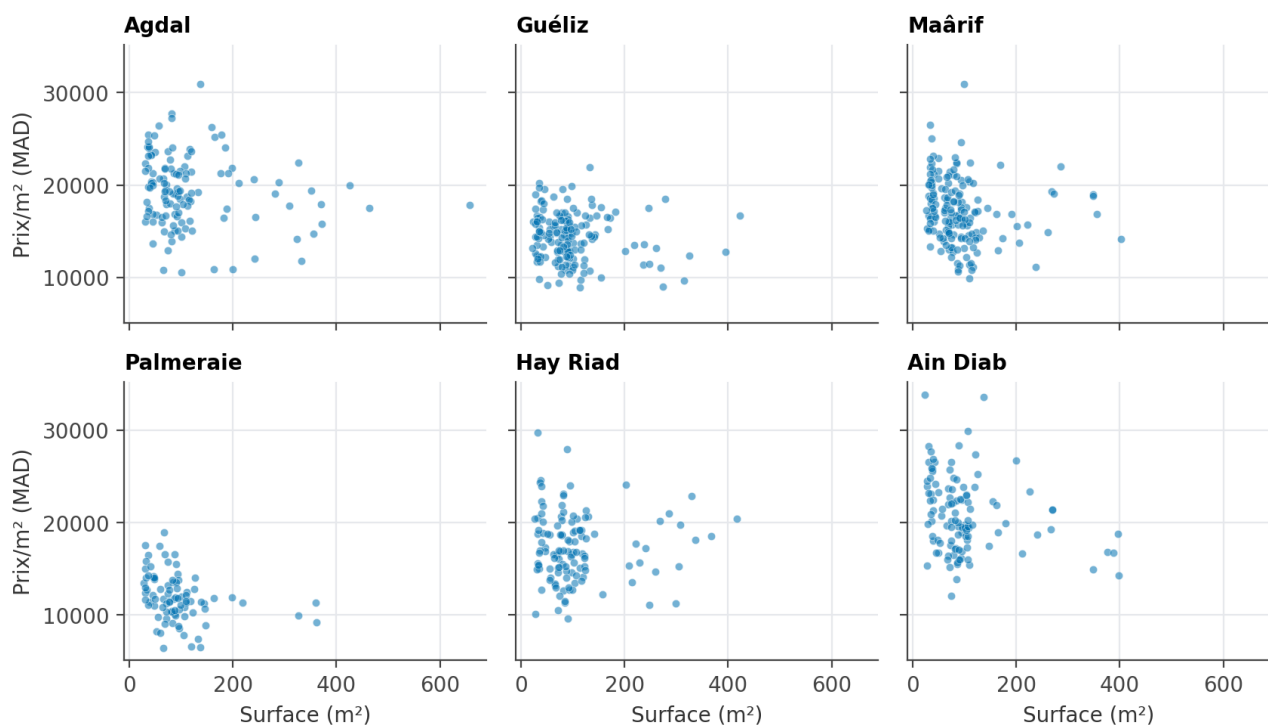


FIGURE 25 – Petits multiples : un panneau par quartier

```
g = sns.FacetGrid(df, col="quartier", col_wrap=3, height=2.5)
g.map_dataframe(sns.scatterplot, x="surface", y="prix_m2")
```

💡 Pourquoi ça fonctionne bien

Les petits multiples exploitent une capacité de comparaison très naturelle chez l'humain (repérer un panneau qui « sort du lot » parmi des panneaux similaires), tout en évitant la surimpression (§7.3) qu'un unique graphique combinant toutes les catégories en couleur produirait.

10.2. Radar et coordonnées parallèles

Pour comparer plusieurs **profils** décrits chacun par plusieurs variables numériques (par exemple, quatre quartiers décrits par leur prix, leur surface, leur âge et leur nombre de pièces médians), deux graphiques permettent une lecture simultanée de toutes les dimensions.

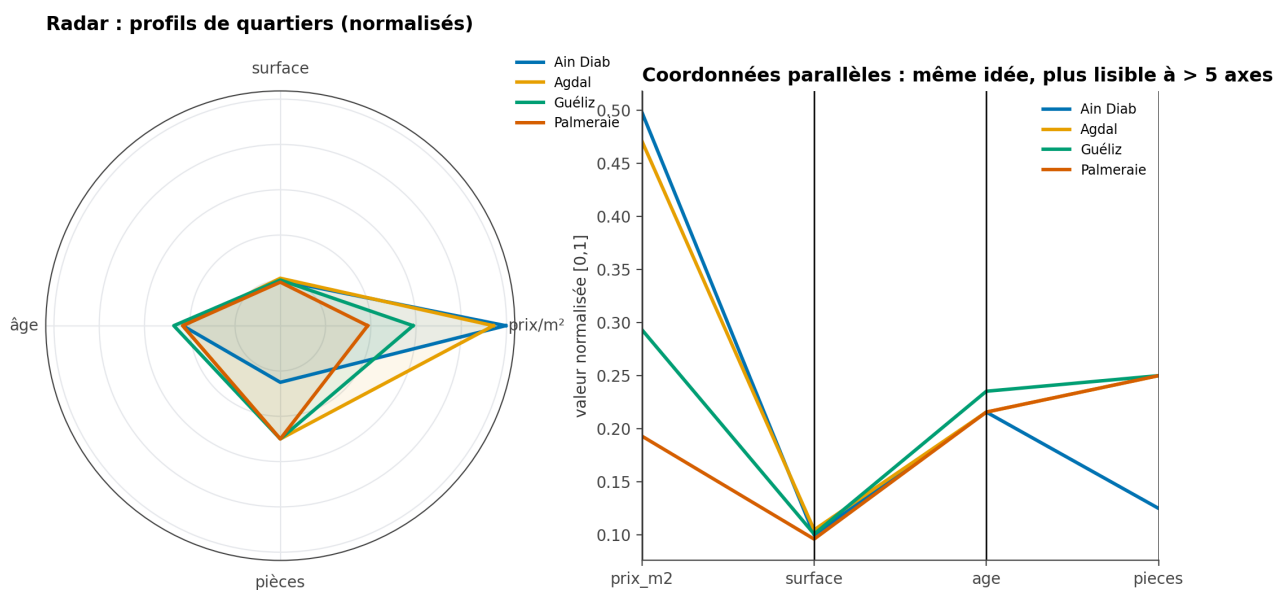


FIGURE 26 – Radar et coordonnées parallèles : deux façons de comparer des profils multivariés

```
# Radar (diagramme polaire)
ax = plt.subplot(projection="polar")
ax.plot(angles, valeurs_normalisees)
ax.fill(angles, valeurs_normalisees, alpha=0.1)

# Coordonnées parallèles
from pandas.plotting import parallel_coordinates
parallel_coordinates(tableau_normalise, class_column="quartier")
```

⚠️ Une lecture à manier avec prudence

Le radar encode les valeurs par une **position sur des axes radiaux**, ce qui reste, selon la hiérarchie du §2.2, moins précis qu'une position sur des axes alignés. Les **coordonnées parallèles**, illustrées à droite de la Figure 26, communiquent en général la même information plus fidèlement dès que le nombre de variables dépasse 4 ou 5 — et se lisent naturellement de gauche à droite, sans déformation angulaire. Les deux graphiques exigent, avant tout tracé, une **normalisation préalable** des variables (chapitre commun, §5), faute de quoi une variable à grande échelle écraserait visuellement les autres.

10.3. Représenter un réseau

Lorsque les données décrivent des **relations** entre entités (des agences qui traitent conjointement un même dossier, des pages web qui se citent, des comptes qui échangent), un graphe de réseau représente chaque entité comme un nœud et chaque relation comme une arête.

Graphe de coïncidence : agences citées ensemble dans une même demande

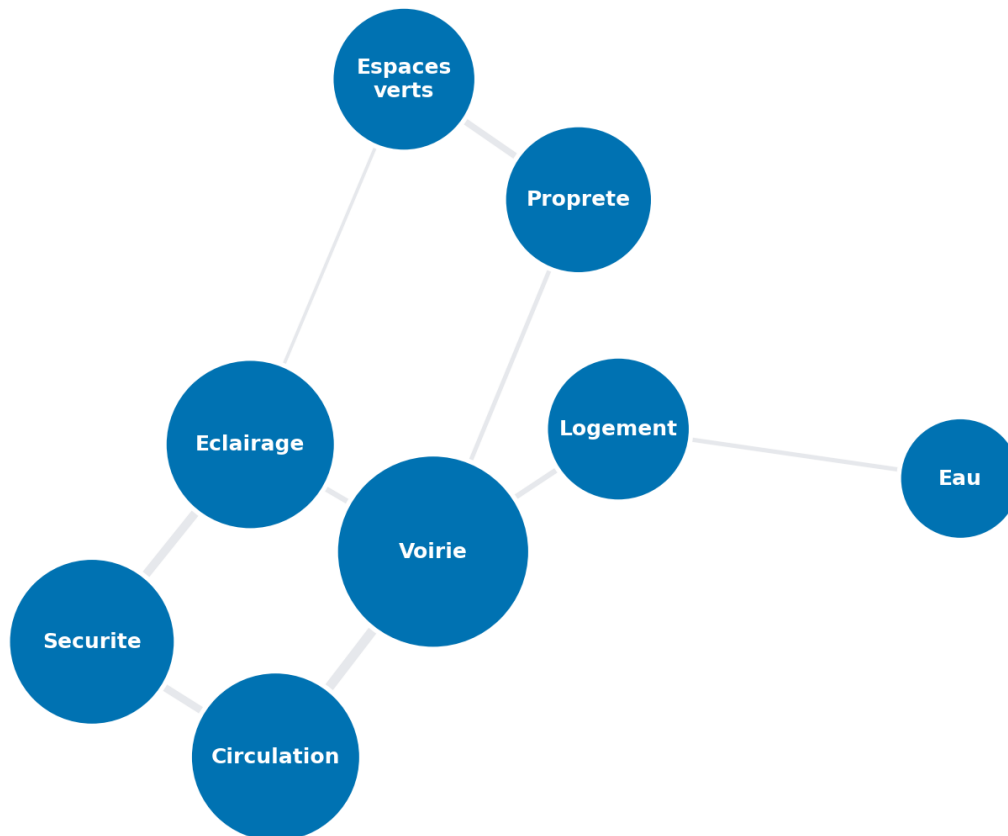


FIGURE 27 – Graphe de réseau : agences citées ensemble dans une même demande

```
import networkx as nx

G = nx.Graph()
G.add_edges_from(paires_agences, weight=...)
pos = nx.spring_layout(G) # positionne les nœuds pour minimiser les croisements d'arêtes
nx.draw(G, pos, with_labels=True, node_size=tailles, width=epaisseurs)
```

Sur la Figure 27, la **taille** de chaque nœud encode son degré d'implication global (le nombre de co-occurrences pondérées), et l'**épaisseur** de chaque arête encode la force du lien entre deux agences précises — deux mappings supplémentaires, dans le droit fil de la grammaire des graphiques.

10.4. Le diagramme de Gantt

Pour planifier ou présenter les étapes d'un projet dans le temps, le **diagramme de Gantt** représente chaque tâche comme une barre horizontale, positionnée selon sa date de début et sa durée.

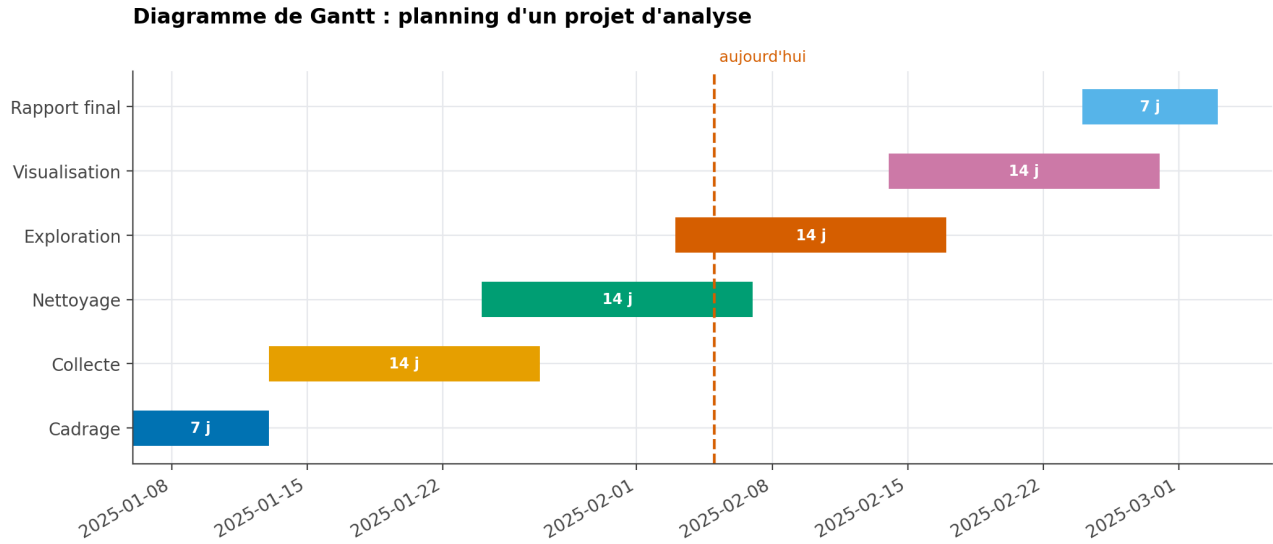


FIGURE 28 – Diagramme de Gantt : planning d'un projet d'analyse

```
for i, (tache, debut, fin) in enumerate(taches):
    plt.barh(tache, (fin - debut).days, left=debut)
plt.axvline(pd.Timestamp.today(), color="red", linestyle="--") # repère "aujourd'hui"
```

10.5. Treemap et waffle chart : d'autres façons de montrer une composition

Au-delà des barres empilées à 100 % (§5.2) et du camembert (§6), deux graphiques représentent une composition par des **surfaces** plutôt que par des angles ou des longueurs.

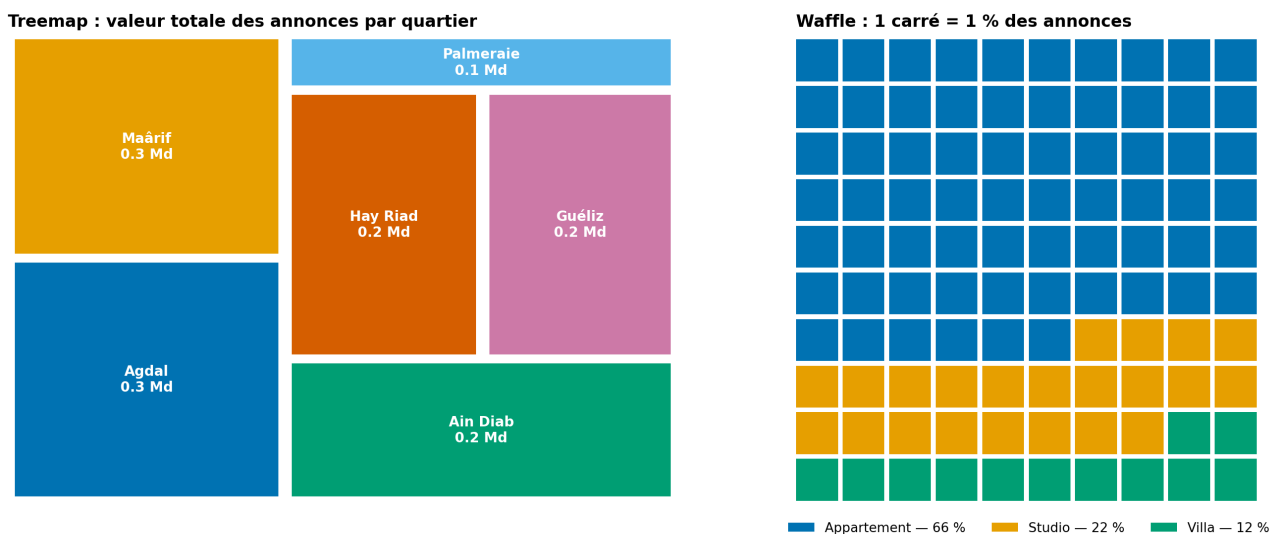


FIGURE 29 – Treemap et waffle chart

```
import squarify
squarify.plot(sizes=valeurs, label=categories, pad=True)
plt.axis("off")
```

Le **treemap** (à gauche) découpe un rectangle en zones proportionnelles à chaque catégorie — utile pour une composition à deux niveaux hiérarchiques (ex. quartier puis type de bien). Le **waffle chart** (à droite), une grille de petits carrés où chaque carré représente 1 % du total, offre une alternative au camembert qui reste fondée sur le comptage d'unités discrètes plutôt que sur un angle — plus proche, dans la hiérarchie du §2.2, d'un encodage par position que par angle.

10.6. Éléments de tableau de bord : cartes indicateurs et jauges

Un tableau de bord (approfondi dans un cours dédié à la Business Intelligence) combine souvent plusieurs graphiques de ce chapitre avec des éléments plus synthétiques, pensés pour une lecture en quelques secondes.

Cartes indicateurs (KPI cards)

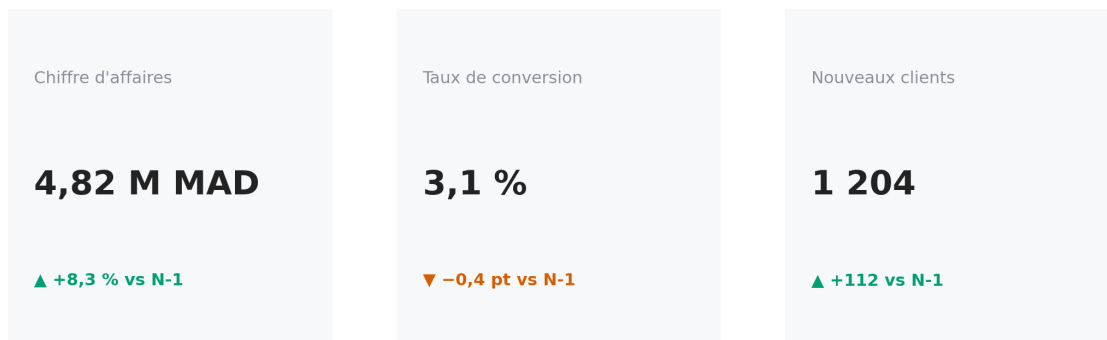


FIGURE 30 – Cartes indicateurs (KPI cards) et jauge

Jauge (gauge chart)

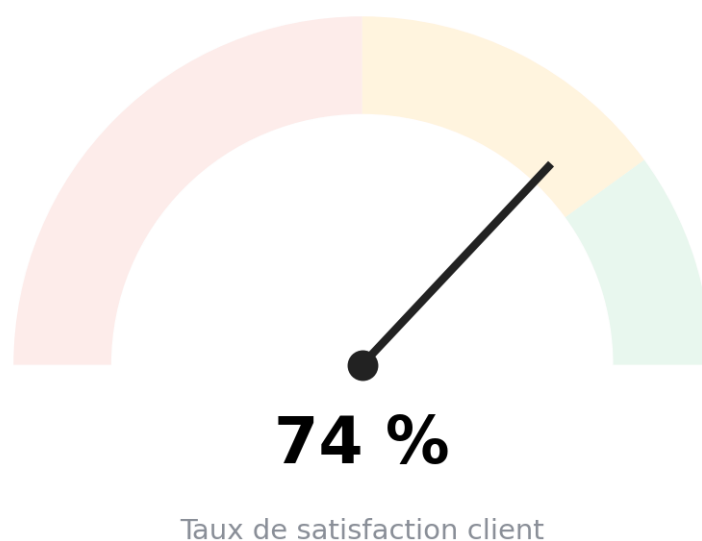


FIGURE 31 – Jauge (gauge chart)

i KPI card et jauge : deux briques complémentaires

La **carte indicateur** (*KPI card*) affiche une valeur unique, sa tendance et une flèche de variation — le résumé le plus condensé possible d'un indicateur. La **jauge** (Figure 31) positionne une valeur unique sur une échelle bornée, souvent divisée en zones de couleur (ex. rouge/orange/vert), pour signaler immédiatement si un indicateur se situe dans une zone satisfaisante. Ces deux éléments ne remplacent jamais un graphique détaillé : ils orientent le regard avant d'inviter, si nécessaire, à consulter un graphique plus complet.

11. Bibliothèques Python pour la visualisation

11.1. matplotlib : la bibliothèque de référence

`matplotlib` est la bibliothèque de visualisation historique de Python. Bas niveau et hautement personnalisable, elle repose sur les objets `fig/ax` présentés au §2.1 :

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(ventes_mensuelles.index, ventes_mensuelles.values)
ax.set_title("Évolution du chiffre d'affaires")
ax.set_xlabel("Mois")
ax.set_ylabel("Chiffre d'affaires (MAD)")
fig.savefig("evolution_ca.png", dpi=150, bbox_inches="tight")
```

Cette approche orientée objet (`fig, ax`) est particulièrement utile pour construire des figures composées de plusieurs sous-graphiques (`plt.subplots(nrows=2, ncols=2)`), comme la plupart des figures de ce chapitre.

11.2. seaborn : la visualisation statistique simplifiée

`seaborn` s'appuie sur `matplotlib` mais propose une interface de plus haut niveau, pensée directement pour des `DataFrame pandas` et pour les besoins statistiques courants (distributions, relations entre catégories, matrices de corrélation). C'est la bibliothèque à privilégier pour l'exploration rapide présentée dans ce chapitre, tandis que `matplotlib` reste utile pour un contrôle fin de la mise en forme finale.

11.3. geopandas : pour la cartographie

Comme introduit au §9, `geopandas` étend `pandas` d'une colonne `geometry` et s'appuie sur `shapely` (géométrie) et `pyproj` (projections). Il s'intègre naturellement à `matplotlib` (`.plot()`) pour produire des cartes statiques.

11.4. Vers l'interactivité

! Zoom sur les tendances : la visualisation interactive

`matplotlib` et `seaborn` produisent des images **statiques**. Pour un public qui consulte un graphique dans un navigateur, plusieurs bibliothèques Python permettent de produire des graphiques **interactifs** (zoom, infobulles au survol, filtres dynamiques) — résumés par la Figure 32 :

- **Plotly** : très populaire, produit des graphiques interactifs exportables en HTML autonome, avec une syntaxe proche de `seaborn` via son module `plotly.express` ;
- **Altair** : implémente directement la grammaire des graphiques (via la spécification `Vega-Lite`), avec une syntaxe déclarative concise ;
- **Bokeh** : orienté vers les tableaux de bord interactifs et les grands volumes de données.

Ce qu'ajoute un graphique interactif (Plotly / Altair)



FIGURE 32 – Ce qu’ajoute un graphique interactif

```
import plotly.express as px

fig = px.scatter(df, x="surface", y="prix", color="quartier",
                 hover_data=["type_bien"], title="Prix en fonction de la surface")
fig.show()
```

Ces bibliothèques ne remplacent pas `matplotlib/seaborn` pour l’exploration rapide ou pour un rapport imprimé, mais deviennent précieuses dès que le graphique est destiné à être consulté à l’écran par plusieurs personnes, ou intégré à un tableau de bord — sujet approfondi au chapitre consacré à la Business Intelligence avec Power BI et Taipy.

12. Communiquer les résultats et rendre les graphiques accessibles

Un graphique de communication n’est pas seulement un graphique techniquement correct. Il doit aider un public déterminé à comprendre un résultat sans lui demander de deviner la question, la période ou la signification des couleurs. Le *data storytelling* consiste à organiser les données et les annotations autour d’un message vérifiable, et non à décorer un graphique ou à lui faire dire davantage que les données ne permettent.

12.1. Partir du public et de la décision

Avant de choisir un graphique, répondre à quatre questions :

1. **À qui s’adresse-t-il ?** Un analyste peut lire une matrice détaillée ; un responsable peut avoir besoin de quelques indicateurs et d’une explication immédiate.

2. **Quelle question le lecteur doit-il pouvoir résoudre ?** Par exemple : « Quel type de demande est le plus fréquent en janvier ? »
3. **Quelle action ou conclusion le résultat peut-il éclairer ?** Préciser si le graphique décrit un constat, suggère une hypothèse ou soutient une décision.
4. **Quelles sont les limites des données ?** Indiquer la population, la période, la source, les exclusions et les valeurs manquantes qui affectent la lecture.

Un graphique ne doit pas accumuler plusieurs messages concurrents. Si la comparaison, l'évolution et la distribution sont toutes importantes, plusieurs graphiques simples avec des titres cohérents sont souvent plus efficaces qu'une figure surchargée.

12.2. Construire un message honnête et lisible

Un titre qui exprime le constat

Un titre-thème nomme le sujet (« Demandes 311 par type »). Un titre-message annonce le constat que les données permettent de vérifier (« Le bruit est le type de demande le plus fréquent en janvier »). Le titre-message doit rester proportionné aux preuves : remplacer « cause », « provoque » ou « explique » par « est associé à » lorsque l'analyse n'établit pas de causalité.

Un sous-titre ou une annotation peut préciser la période, la population ou le chiffre important. Éviter les titres qui présentent une interprétation comme un fait si l'échantillon est limité ou si les données ne couvrent qu'un sous-groupe.

Donner le contexte nécessaire

Un graphique de communication doit permettre de savoir :

- ce que représentent les axes, avec leur unité ;
- quelle période et quelle population sont incluses ;
- le dénominateur d'un taux ou d'un pourcentage ;
- la source et, si nécessaire, la date d'extraction ou de mise à jour ;
- si une échelle est logarithmique, si un axe est tronqué ou si des valeurs ont été regroupées ;
- combien d'observations sont concernées, surtout lorsque l'effectif est faible.

Les valeurs manquantes ou exclusions susceptibles de changer la conclusion doivent être signalées. Par exemple, une distribution de délais calculée uniquement sur les demandes clôturées ne décrit pas les demandes encore ouvertes. Une moyenne sans effectif peut masquer un groupe très petit et instable.

Choisir une forme adaptée et préserver l'échelle

Choisir le graphique selon la question, comme dans les sections précédentes : barres pour comparer des catégories, courbe pour une évolution ordonnée dans le temps, histogramme ou boîte à moustaches pour examiner une distribution, nuage de points pour une relation entre mesures. Trier les catégories lorsque cela facilite la comparaison.

Pour un diagramme en barres, faire partir l'axe des valeurs de zéro : sinon, la longueur des barres exagère les différences. Une autre représentation, comme un graphique de points, peut utiliser un axe resserré si ce choix est indiqué clairement et ne transforme pas visuellement une petite différence en écart spectaculaire. Ne jamais cacher une partie des observations sans le dire ; si l'affichage est limité à un quantile, l'indiquer dans le titre ou la légende et conserver les données complètes dans les calculs.

Utiliser couleur et annotations avec retenue

La couleur doit avoir une fonction : distinguer des catégories, signaler une série ou attirer l'attention sur un résultat. Garder une palette limitée et réutiliser les mêmes couleurs pour la même catégorie dans l'ensemble du rapport. Une annotation directe sur la barre ou le point important est souvent plus facile à lire qu'une légende éloignée. Toute annotation doit cependant rester vérifiable à partir des données.

```
import matplotlib.pyplot as plt

# Valeurs illustratives : les remplacer par les résultats réellement calculés.
categories = ["Autres", "Eau", "Bruit"]
valeurs = [200, 420, 820]

fig, ax = plt.subplots(figsize=(8, 4.5))
barres = ax.barh(categories, valeurs, color="#0072B2")
ax.bar_label(barres, padding=4, fmt="%0f")
ax.set_xlim(0, max(valeurs) * 1.18)
ax.set_xlabel("Nombre de demandes")
ax.set_ylabel("")
ax.set_title("En janvier 2025, le bruit est le premier motif de demande")
ax.grid(axis="x", color="#D9D9D9", linewidth=0.7)
ax.set_axisbelow(True)
fig.text(
    0.01,
    0.01,
    "Source : NYC 311 - janvier 2025. Valeurs illustratives à remplacer.",
    fontsize=8,
    color="#404040",
)
fig.tight_layout(rect=(0, 0.06, 1, 1))
plt.show()
```

Dans cet exemple, la comparaison ne dépend pas de couleurs différentes : chaque valeur est directement inscrite sur sa barre, l'axe commence à zéro et la période ainsi que la source sont précisées. Dans un rapport réel, remplacer les valeurs illustratives, vérifier le titre à partir des résultats et annoter les limites propres au jeu de données.

12.3. Accessibilité : ne pas transmettre l'information par la couleur seule

Certains lecteurs distinguent moins facilement certaines couleurs ; d'autres impriment le document en niveaux de gris ou consultent une figure avec un écran de qualité réduite. L'information essentielle doit donc rester compréhensible sans identifier une couleur particulière.

Bonnes pratiques :

- éviter de distinguer deux catégories uniquement par rouge et vert ; employer aussi des libellés, des motifs, des formes ou des traits différents ;
- choisir des palettes lisibles par les personnes daltoniennes, par exemple la palette `colorblind` de Seaborn, et vérifier le contraste du texte avec son fond ;
- pour une heatmap, choisir une palette séquentielle pour des valeurs allant de faible à forte, et une palette divergente avec un centre explicite pour des valeurs positives et négatives ; conserver une barre de couleur étiquetée ;
- ne pas coder simultanément de nombreuses catégories avec des couleurs proches ; regrouper ou séparer les catégories lorsque la légende devient difficile à suivre ;
- utiliser une taille de texte lisible, des marqueurs assez grands et des traits suffisamment épais ; éviter les libellés inclinés ou superposés quand une disposition horizontale ou une figure plus grande résout le problème ;

- ne pas utiliser une couleur seule pour indiquer une alerte : ajouter un mot, une forme ou un symbole ayant aussi un sens explicite.

Remplacer le chemin et les valeurs par ceux du graphique effectivement publié. Éviter de répéter mot pour mot la légende : décrire plutôt la comparaison, la tendance ou le point important. Pour le PDF imprimé, le texte alternatif n'étant pas toujours perceptible dans la page, accompagner le graphique d'une phrase de lecture ou d'un tableau lorsque l'information ne doit pas être perdue.

12.5. Vérification avant diffusion

Avant de publier une figure, vérifier les points suivants :

- ☐ Le graphique répond à une question définie et son titre annonce un constat fidèle.
- ☐ Les unités, la période, la population et le dénominateur sont compréhensibles.
- ☐ L'échelle ne grossit pas artificiellement l'effet présenté ; tout axe tronqué ou transformation est signalé.
- ☐ La source, les filtres importants et les limites pertinentes sont indiqués.
- ☐ Les couleurs ne sont pas le seul moyen de distinguer les groupes.
- ☐ Les textes, marqueurs, légendes et annotations restent lisibles dans la page HTML et après impression.
- ☐ Un lecteur qui ne voit pas l'image dispose d'une description textuelle et, si nécessaire, d'un tableau.
- ☐ Les chiffres et annotations ont été vérifiés à partir des données, et la conclusion ne prétend pas démontrer une causalité non étudiée.

Introduction à l'entrepôt de données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Expliquer** pourquoi une organisation sépare souvent les systèmes opérationnels des données destinées à l'analyse ;
- **Distinguer** une base transactionnelle, un entrepôt de données, un lac de données et un datamart ;
- **Lire** un modèle en étoile composé d'une table de faits et de tables de dimensions, et le distinguer d'un modèle en flocon ;
- **Définir** le grain d'une table de faits et choisir des mesures adaptées (additives, semi-additives, non additives) ;
- **Décrire** les grandes étapes d'un flux ETL/ELT, de la source au tableau de bord ;
- **Situer** les grands principes de gouvernance des données et les tendances actuelles des architectures analytiques (cloud, lakehouse).

Un tableau de bord fiable ne commence pas par un graphique. Il commence par des données organisées, documentées et mises à jour de façon régulière. L'**entrepôt de données** (*data warehouse*) fournit cette base pour l'analyse — c'est la fondation sur laquelle reposeront les rapports Power BI et les applications Taipy présentés au chapitre suivant.

1. Pourquoi un entrepôt de données ?

1.1. Les systèmes opérationnels

Une application de vente, une caisse ou un site web utilise une base **opérationnelle** (on parle aussi de système **OLTP**, *Online Transaction Processing*) pour enregistrer les événements du quotidien : créer une commande, modifier une adresse, enregistrer un paiement. Ces systèmes doivent répondre vite à de petites opérations, éviter les incohérences, et supportent souvent un grand nombre d'utilisateurs simultanés qui écrivent en permanence.

Une base opérationnelle est souvent organisée pour limiter la répétition des informations — une démarche appelée **normalisation**. Une commande peut être répartie entre `clients`, `commandes` et `lignes_commande`, reliées par des identifiants, plutôt que de répéter le nom et l'adresse du client sur chaque ligne de commande.

Cette organisation est adaptée à l'écriture rapide et fiable, mais une question analytique comme « Quel est le chiffre d'affaires mensuel par région et par catégorie depuis trois ans ? » peut nécessiter de nombreuses jointures entre plusieurs tables, sur plusieurs sources différentes. Exécuter ce type de requête directement sur la base de production peut ralentir l'application pour tous ses utilisateurs — un risque qu'aucune organisation ne peut se permettre sur son système de vente en direct.

1.2. La base analytique

Un **entrepôt de données** est un espace centralisé, organisé pour analyser l'historique de l'organisation. Il rassemble des données provenant de plusieurs sources, conserve leur contexte dans le temps et les transforme dans un format plus simple à interroger — quitte à répéter volontairement certaines informations (**dénormalisation**) pour accélérer la lecture.

TABLE 1 – OLTP et OLAP, deux logiques complémentaires

Système	Sigle	But principal	Exemple de question	Fréquence d'écriture
Base opérationnelle	OLTP	Enregistrer les événements courants	« Quel est le statut de la commande 1524 ? »	Continue, en temps réel
Entrepôt analytique	OLAP	Comparer, agréger et suivre l'historique	« Comment les ventes ont-elles évolué par région ? »	Périodique (souvent quotidienne ou horaire)

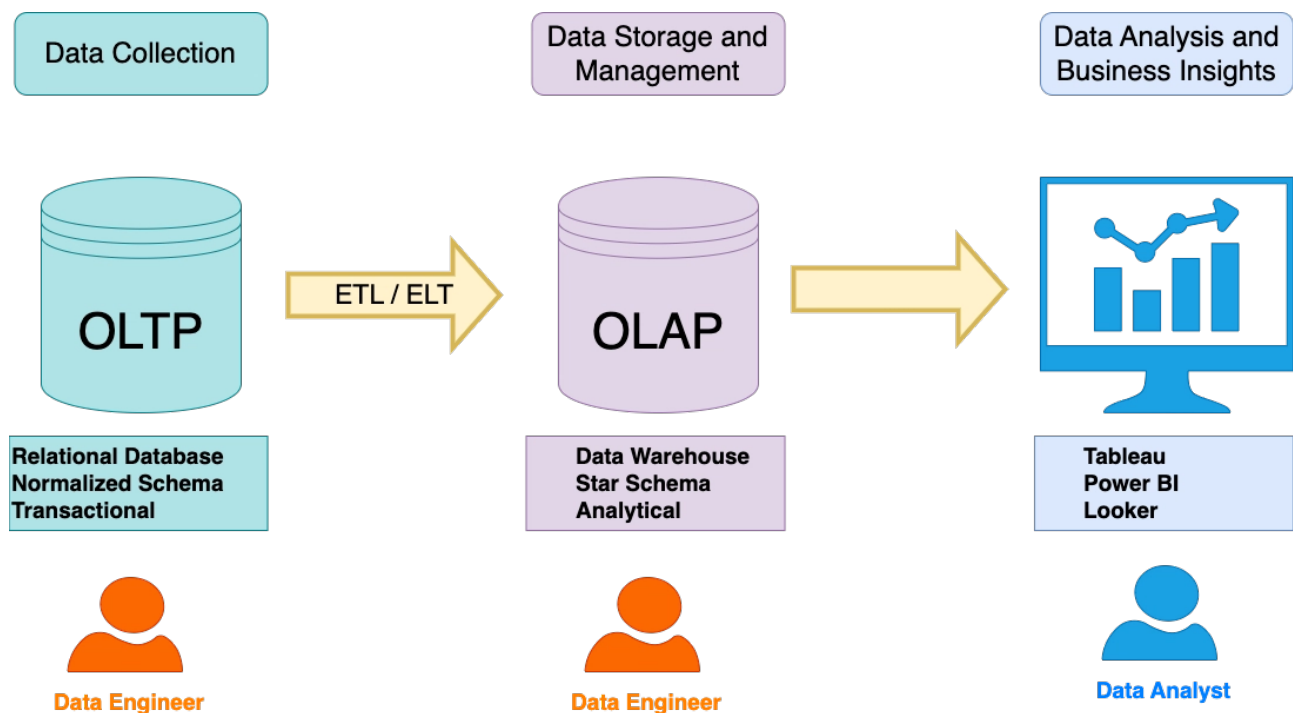


FIGURE 1 – Deux systèmes complémentaires : la base opérationnelle (OLTP) alimente, après transformation, l'entrepôt analytique (OLAP)

Les deux systèmes sont complémentaires, jamais concurrents. L'entrepôt n'a pas pour objectif de remplacer l'application métier : il en est le prolongement analytique, alimenté à intervalles réguliers plutôt qu'en continu.

1.3. Entrepôt, lac et datamart

Trois notions sont souvent confondues, alors qu'elles répondent à des besoins différents :

- Un **data warehouse** (entrepôt de données) contient des données **structurées**, préparées et gouvernées pour l'analyse. Les tables suivent un schéma défini à l'avance (*schema-on-write*) : on sait, avant même d'écrire une ligne, à quoi ressemblera la table.
- Un **data lake** (lac de données) peut stocker des données brutes de formats variés (fichiers CSV, JSON, images, journaux d'événements) avant leur usage, sans schéma imposé à l'écriture (*schema-on-read* : le schéma n'est défini qu'au moment de la lecture). Il offre une grande flexibilité, mais demande des règles claires — un catalogue de données, des conventions de nommage — pour ne pas devenir un « marécage de données » (*data swamp*) où plus personne ne sait ce que contient chaque fichier.
- Un **datamart** est une partie de l'entrepôt orientée vers un besoin ou un service précis, par exemple un datamart des ventes ou un datamart des ressources humaines. Il est généralement plus petit, plus rapide à interroger et plus simple à comprendre pour l'équipe qui l'utilise au quotidien.

TABLE 2 – Trois niveaux de granularité pour organiser les données de l'entreprise

Composant	Type de données	Schéma	Utilisateur typique
Data lake	Brutes, tous formats	<i>Schema-on-read</i>	Data engineer, data scientist
Data warehouse	Structurées, nettoyées	<i>Schema-on-write</i>	Data analyst, BI developer
Datamart	Structurées, ciblées sur un domaine	<i>Schema-on-write</i>	Équipe métier (ventes, RH, finance)

 Un entrepôt ne corrige pas une mauvaise question

Centraliser beaucoup de données ne suffit pas. Il faut définir le vocabulaire métier, les unités, la fréquence de mise à jour et les indicateurs attendus. Sinon, on obtient plusieurs chiffres différents pour un même « chiffre d'affaires » — un problème de gouvernance, pas un problème technique, que nous retrouverons en section 5.

2. Le modèle en étoile

Le modèle en étoile (*star schema*) est une organisation pédagogique et professionnelle très fréquente pour les données analytiques. Il place une **table de faits** au centre et des **tables de dimensions** autour, reliées directement à la table de faits.

2.1. La table de faits

Une table de faits contient les événements mesurables : ventes, livraisons, appels ou visites. Elle comprend généralement :

- des **clés étrangères** vers les dimensions (`client_id`, `produit_id`, `date_id`, `magasin_id`);
- des **mesures numériques** (`quantite`, `montant`, `remise`);
- une ligne par événement, selon un niveau de détail défini.

 Le grain d'une table de faits

Le **grain** est la phrase qui précise ce que représente exactement une ligne de la table de faits. Par exemple : « une ligne représente un produit d'une commande, dans un magasin, à une date donnée ». Le grain doit être défini **avant** de choisir les mesures et les dimensions : toutes les mesures de la table doivent être cohérentes avec ce niveau de détail.

Un grain mal défini — ou changé en cours de route — est l'une des erreurs les plus fréquentes et les plus coûteuses dans la conception d'un entrepôt : elle oblige souvent à reconstruire les mesures et les rapports qui en dépendent.

2.2. Les tables de dimensions

Une dimension décrit le **contexte** d'un fait — le qui, le quoi, le où et le quand :

- `dim_date` : jour, mois, trimestre, année, jour ouvré ou non ;
- `dim_produit` : produit, catégorie, sous-catégorie, marque ;
- `dim_client` : ville, région, segment, ancienneté ;
- `dim_magasin` : magasin, ville, zone commerciale.

Les dimensions contiennent les attributs utilisés pour filtrer, regrouper et expliquer les mesures. Elles sont souvent volontairement dénormalisées (une seule table plate plutôt que plusieurs tables reliées) pour rendre l'exploration plus simple pour un analyste ou un outil de BI.

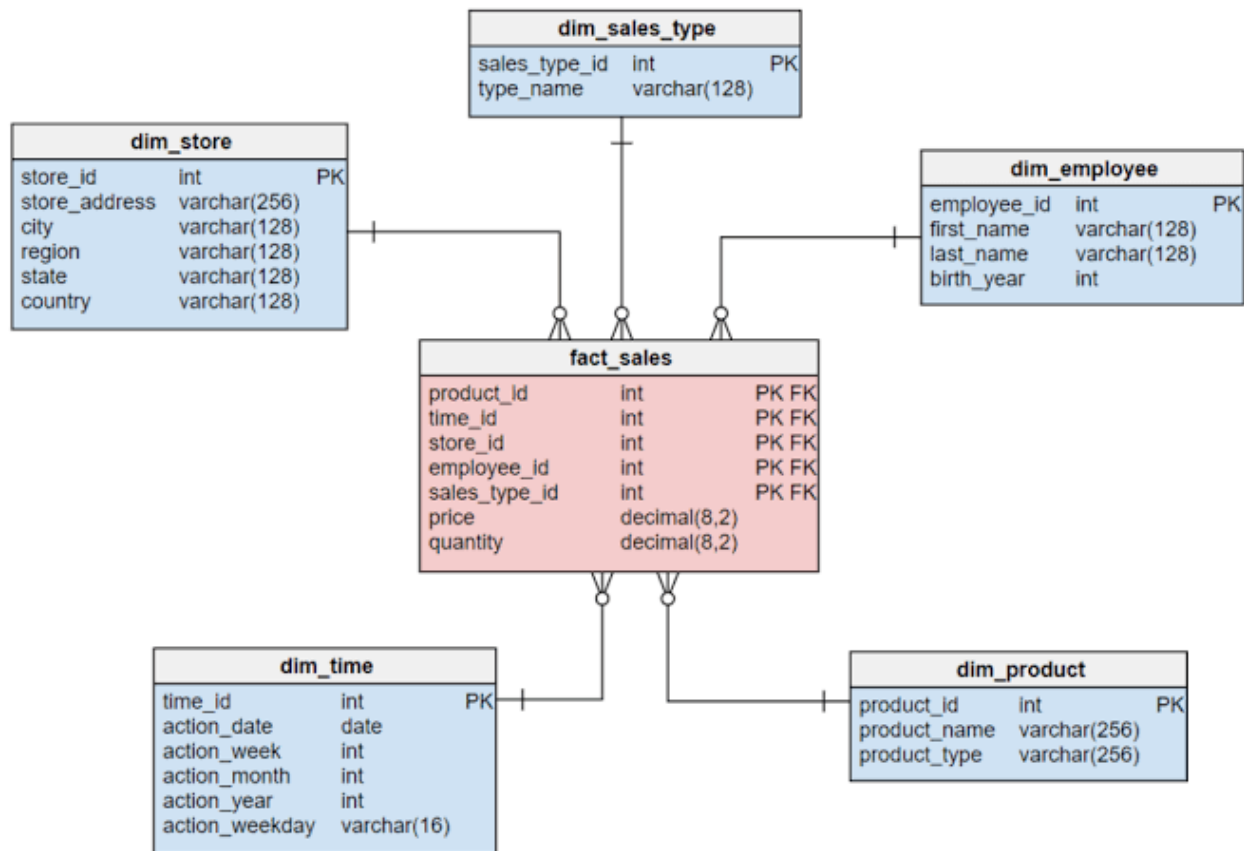


FIGURE 2 – Le modèle en étoile : une table de faits centrale reliée directement à ses tables de dimensions

2.3. Exemple de modèle de ventes

```

fact_ventes
-----
date_id
produit_id
client_id
magasin_id
quantite
prix_unitaire
remise
montant_net

dim_produit
-----
produit_id
libelle
categorie
marque

dim_date

```

```
-----
date_id
date_complete
mois
trimestre
annee
```

Le montant net peut être calculé par :

$$\text{montant_net} = \text{quantité} \times \text{prix unitaire} - \text{remise}.$$

Un modèle en étoile ne dicte pas un seul outil : il peut être utilisé dans une base SQL, un fichier analytique (Parquet, vu au chapitre 1) ou directement dans le modèle sémantique d'un outil comme Power BI, présenté au chapitre suivant.

i Le rôle de la dimension Date

Même si la source contient déjà une date, une dimension Date facilite les regroupements par mois, trimestre, semaine ou jour ouvré, et permet de définir clairement les périodes comparées (par exemple « le même mois l'an dernier »). Elle est presque toujours présente dans un modèle en étoile, même minimal.

2.4. Modèle en étoile ou en flocon ?

Lorsqu'une dimension est elle-même normalisée en plusieurs tables reliées entre elles (par exemple `dim_produit` reliée à une table séparée `dim_categorie`, elle-même reliée à `dim_departement`), on parle de **modèle en flocon** (*snowflake schema*) plutôt que de modèle en étoile.

TABLE 3 – Modèle en étoile et modèle en flocon

Critère	Modèle en étoile	Modèle en flocon
Nombre de tables de dimensions	Plus petit, dimensions dénormalisées	Plus grand, dimensions normalisées
Simplicité pour l'analyste	Plus simple à comprendre et à interroger	Plus de jointures à connaître
Espace de stockage	Légèrement plus élevé (répétition de données)	Réduit (moins de répétition)
Usage recommandé	Par défaut pour la plupart des projets de BI	Dimensions très volumineuses ou très hiérarchisées

i Priorité pour débiter

Pour la grande majorité des projets, notamment ceux présentés dans ce cours, le modèle en étoile — plus simple à lire et à interroger — est le point de départ recommandé. Le modèle en flocon reste une option pour des dimensions très volumineuses (des millions de produits, par exemple).

2.5. Un mot sur l'historisation des dimensions

Que se passe-t-il lorsqu'un client déménage, ou qu'un produit change de catégorie ? Ce problème classique, appelé **dimension à évolution lente** (*Slowly Changing Dimension*, ou SCD), admet plusieurs réponses :

- **Type 1** : on écrase l'ancienne valeur par la nouvelle — on perd l'historique, mais le modèle reste simple.

- **Type 2** : on ajoute une nouvelle ligne dans la dimension, avec des dates de validité — on conserve l'historique complet, au prix d'un modèle un peu plus complexe.

Ce sujet est approfondi dans des cours dédiés à la modélisation ; il est mentionné ici pour que le concept de dimension ne soit pas perçu comme figé dans le temps.

3. Les mesures et les indicateurs

Une **mesure additive** peut être additionnée sur toutes les dimensions, comme le nombre d'articles ou le montant des ventes. Une mesure **semi-additive**, comme un stock de fin de journée, peut être additionnée par produit mais pas dans le temps (additionner le stock du lundi et du mardi n'a pas de sens ; on préfère en général le dernier stock connu, ou une moyenne). Une mesure **non additive**, comme un taux ou une moyenne, doit être recalculée à partir de ses éléments plutôt que sommée directement.

TABLE 4 – Trois familles de mesures

Type de mesure	Peut-on l'additionner...	... dans le temps ?	... entre produits ?	Exemple
Additive	Oui	Oui	Oui	Montant des ventes, quantité vendue
Semi-additive	Parfois	Non	Oui	Stock de fin de journée
Non additive	Non	Non	Non	Taux de conversion, marge en %

⚠ Exemple : ne jamais faire la moyenne d'une moyenne

Si une région a 10 commandes et une autre 2 commandes, la moyenne globale ne doit pas être calculée comme la moyenne des deux moyennes régionales si les effectifs sont différents. Il faut utiliser :

$$\text{moyenne globale} = \frac{\text{total des montants}}{\text{nombre total de commandes}}.$$

Cette règle, déjà rencontrée au chapitre 2, redevient centrale dans un entrepôt de données : c'est précisément ce type d'erreur que les mesures DAX de Power BI, vues au chapitre suivant, permettent d'éviter — à condition d'être écrites correctement.

Les noms des indicateurs doivent préciser leur définition. « Ventes » peut signifier chiffre d'affaires brut, chiffre d'affaires net, commandes payées ou unités vendues. Dans un **dictionnaire métier** (ou *data dictionary*), on documente pour chaque indicateur sa formule, son unité, sa période de référence et ses filtres :

TABLE 5 – Extrait d'un dictionnaire métier

Indicateur	Formule	Unité	Période	Filtre appliqué
Chiffre d'affaires net	$\sum \text{montant_net}$	MAD	Mensuel	Statut = « Payée »
Panier moyen	CA net / commandes distinctes	MAD	Mensuel	Statut = « Payée »
Taux de livraison à temps	Livraisons à temps / livraisons terminées	%	Hebdomadaire	Toutes livraisons closes

4. Le flux ETL/ELT

Les données passent généralement par un flux en plusieurs étapes avant d'atteindre l'entrepôt :

1. **Extract (extraire)** : lire les sources opérationnelles, fichiers ou API — les mêmes sources de collecte que celles vues au chapitre 1 ;
2. **Transform (transformer)** : convertir les dates, harmoniser les unités, dédupliquer, contrôler les valeurs et calculer les champs dérivés — les techniques de nettoyage et de transformation du chapitre 1 ;
3. **Load (charger)** : déposer les données préparées dans les tables de l'entrepôt.

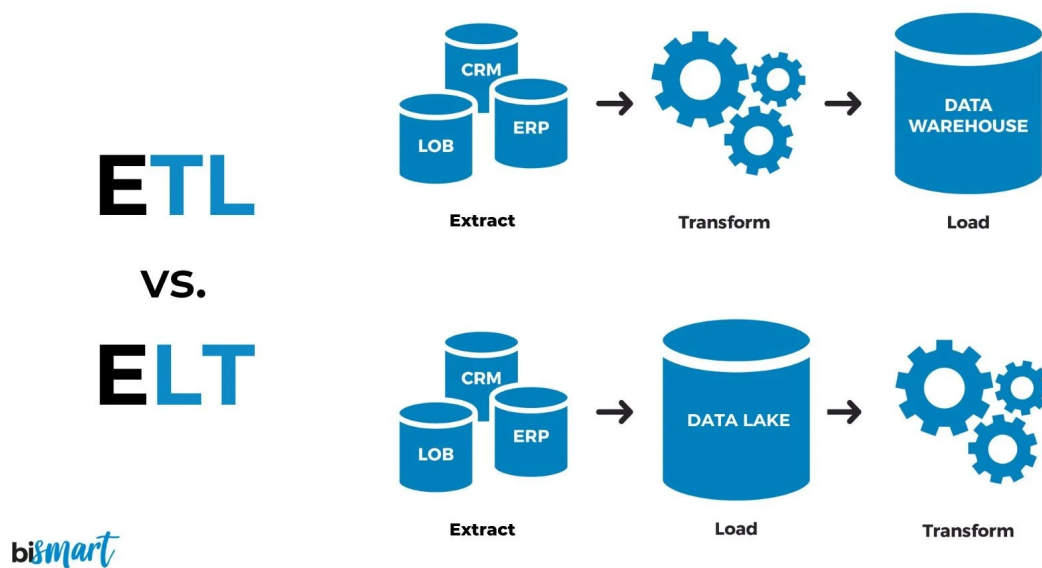


FIGURE 3 – Le flux ETL : extraction, transformation puis chargement dans l'entrepôt

Dans une approche **ELT** (*Extract, Load, Transform*), on inverse les deux dernières étapes : on charge d'abord les données brutes dans une plateforme suffisamment puissante pour les traiter (un entrepôt cloud, par exemple), puis on réalise les transformations à l'intérieur même de cette plateforme, en SQL. Les deux approches poursuivent le même but — rendre les données fiables et exploitables — mais diffèrent par l'endroit où s'exécute la transformation.

TABLE 6 – ETL et ELT

Approche	Où la transformation a-t-elle lieu ?	Avantage principal
ETL	Sur un serveur de traitement dédié, avant le chargement	Contrôle fin, adapté aux volumes modérés
ELT	Dans l'entrepôt cible lui-même, après le chargement	Exploite la puissance de calcul du cloud, adapté aux gros volumes

4.1. Illustrer un contrôle de qualité au chargement

Même sans plateforme d'orchestration, un petit script Python illustre la logique d'un contrôle de qualité effectué à l'étape de transformation, dans l'esprit de **Great Expectations** déjà mentionné au chapitre 1 :

```
import pandas as pd

brut = pd.read_csv("ventes_source.csv")
lignes_avant = len(brut)

# Règles de qualité simples
brut = brut.dropna(subset=["date_vente", "montant"])
brut = brut[brut["montant"] >= 0]
brut = brut.drop_duplicates(subset=["commande_id", "produit_id"])

lignes_apres = len(brut)
rejets = lignes_avant - lignes_apres

print(f"Lignes lues : {lignes_avant}")
print(f"Lignes conservées : {lignes_apres}")
print(f"Lignes rejetées : {rejets}")
```

Un chargement quotidien doit pouvoir répondre à quatre questions simples : quelles sources ont été lues ? combien de lignes ont été reçues ? combien ont été rejetées ? quand le chargement s'est-il terminé ?

! Zoom sur les tendances : l'orchestration automatisée

Sur un projet réel, ce type de flux ne s'exécute pas manuellement chaque jour : un **orchestrateur** planifie et surveille son exécution. Les outils les plus utilisés actuellement sont **Apache Airflow**, **Dagster** et **Prefect**, qui permettent de définir un flux comme une succession d'étapes (un *graphe orienté acyclique*, ou DAG), de relancer automatiquement une étape échouée, et d'alerter une équipe en cas d'anomalie. Ce sujet est approfondi dans les modules d'ingénierie des données ; il est mentionné ici car il complète directement le rôle du Data Analyst.

! Qualité et traçabilité

Conservez les données sources ou un journal de chargement, le nombre de lignes avant et après transformation, les erreurs détectées et la date de mise à jour. Ces informations permettent de comprendre pourquoi un indicateur a changé d'un jour à l'autre — une question que tout analyste finit par recevoir.

5. Gouvernance et sécurité, sans complexité inutile

La **gouvernance des données** désigne les règles qui rendent les données compréhensibles, accessibles aux bonnes personnes et utilisées de manière responsable. Pour commencer, un projet peut mettre en place :

- un **dictionnaire des tables et des indicateurs**, comme celui de la Table 5 ;
- un **responsable** (*data owner*) pour chaque donnée importante ;
- des **règles de qualité** (unicité d'un identifiant, prix positif, date obligatoire), dans l'esprit du script de la section 4.1 ;
- des **droits d'accès** adaptés, notamment pour les données personnelles, en cohérence avec le cadre réglementaire déjà évoqué au chapitre 1 (RGPD, loi marocaine 09-08) ;
- une **fréquence de mise à jour** annoncée et respectée.

Un tableau de bord ne devrait pas exposer inutilement des noms, numéros de téléphone ou adresses. On peut agréger les résultats, masquer les colonnes sensibles et limiter les accès selon le rôle de l'utilisateur — un mécanisme appelé **sécurité au niveau des lignes** (*Row-Level Security*, RLS), que nous retrouverons concrètement dans Power BI au chapitre suivant.

5.1. Vers le cloud et le lakehouse

! Zoom sur les tendances : les entrepôts analytiques dans le cloud

Les entrepôts de données historiques étaient hébergés sur des serveurs internes à l'organisation. Aujourd'hui, la majorité des nouveaux projets s'appuient sur des plateformes analytiques **cloud**, qui séparent le stockage et le calcul et permettent de payer à l'usage : **Snowflake**, **Google BigQuery**, **Amazon Redshift** ou **Microsoft Fabric** (qui intègre directement Power BI). Une tendance plus récente, le **lakehouse** (popularisé notamment par Databricks), cherche à combiner la flexibilité d'un data lake avec les garanties de qualité et de structure d'un data warehouse, en s'appuyant sur des formats de table ouverts (comme Delta Lake ou Apache Iceberg). Ces plateformes ne changent pas les principes présentés dans ce chapitre — grain, dimensions, mesures, gouvernance — mais modifient l'endroit et la façon dont ils sont mis en œuvre.

Exercices

i Consigne

Les exercices vérifient la compréhension et peuvent être résolus sans ordinateur. Les calculs sont volontairement courts.

Exercice 0.0.1 (OLTP ou OLAP ?). Indiquez si la question relève plutôt d'un système opérationnel ou analytique :

- Modifier l'adresse d'un client.
- Comparer les ventes des quatre derniers trimestres par région.
- Vérifier si une commande est payée.
- Calculer la médiane des délais de livraison de l'année.

Exercice 0.0.2 (Identifier le grain). Une table **fact_ventes** contient une ligne par produit présent dans une commande. Formulez le grain avec une phrase et expliquez pourquoi une commande de trois produits peut produire trois lignes.

Exercice 0.0.3 (Faits ou dimensions). Classez les éléments suivants : **montant_net**, **categorie**, **annee**, **quantite**, **ville_client**, **prix_unitaire**.

Exercice 0.0.4 (Calculer une mesure). Une ligne indique 3 articles à 120 MAD chacun et une remise de 40 MAD. Calculez le montant net. Une deuxième ligne indique 2 articles à 80 MAD sans remise. Quel est le montant total des deux lignes ?

Exercice 0.0.5 (Mesure additive ou non ?). Indiquez si l'on peut additionner directement la mesure sur les périodes suivantes :

- Le montant de ventes.
- Le stock disponible à la fin de chaque jour.
- Un taux de conversion en pourcentage.

Exercice 0.0.6 (Comprendre ETL). Remettez dans l'ordre : charger les données préparées, extraire un fichier CSV, convertir les dates et contrôler les prix négatifs.

Exercice 0.0.7 (Moyenne globale). La région A compte 10 commandes de 100 MAD en moyenne. La région B compte 2 commandes de 300 MAD en moyenne.

- a. Quel est le total estimé de chaque région ?
- b. Calculez la moyenne globale.
- c. Pourquoi la moyenne des deux moyennes, 200 MAD, est-elle incorrecte ?

Exercice 0.0.8 (Qualité et sécurité). Proposez une règle de qualité et une mesure de protection pour chacune des situations :

- a. La colonne `prix` contient parfois des valeurs négatives.
- b. Un tableau de bord doit être partagé avec des responsables régionaux.

Exercice 0.0.9 (Étoile ou flocon ?). Une équipe hésite entre deux organisations pour la dimension `dim_produit` : une seule table dénormalisée, ou trois tables reliées (`dim_produit`, `dim_categorie`, `dim_departement`).

- a. Quel modèle correspond à chaque option (Table 3) ?
- b. Dans quel cas le modèle en flocon devient-il particulièrement pertinent ?

Exercice 0.0.10 (ETL ou ELT ?). Une organisation dispose d'une plateforme cloud très puissante pour exécuter des requêtes SQL sur de gros volumes, et souhaite limiter le nombre de serveurs de traitement intermédiaires à maintenir.

- a. Quelle approche (ETL ou ELT, Table 6) est la plus cohérente avec ce contexte ?
- b. Quel principe reste identique, quelle que soit l'approche choisie ?

Business Intelligence avec Power BI et Taipy

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Définir** la Business Intelligence et distinguer un indicateur, un rapport et un tableau de bord ;
- **Construire** mentalement le parcours d'une donnée depuis la source jusqu'à une décision ;
- **Importer et préparer** des données dans Power BI avec Power Query ;
- **Comprendre** un modèle en étoile dans Power BI et créer des mesures en DAX, y compris des comparaisons dans le temps ;
- **Choisir** entre Power BI et une application Python avec Taipy selon le besoin ;
- **Construire** une application Taipy fonctionnelle intégrant une visualisation Python soignée ;
- **Expliquer** les principes d'un tableau de bord lisible : objectif, public, filtres, contexte et action.

La **Business Intelligence (BI)** transforme des données préparées — celles de l'entrepôt du chapitre précédent — en informations régulièrement consultées pour piloter une activité. Elle ne consiste pas à accumuler des graphiques : un bon tableau de bord aide une personne précise à répondre à des questions précises et à décider.

1. Comprendre la BI

1.1. Du besoin à l'indicateur

Un **indicateur** est une mesure définie par une formule, une période et une population. Un **KPI** (*Key Performance Indicator*) est un indicateur considéré comme essentiel pour suivre un objectif.

Exemple pour un site de livraison :

TABLE 1 – De l'objectif métier au KPI

Élément	Exemple
Objectif	Livrer les commandes à temps
KPI	Taux de livraison à temps
Formule	livraisons à temps / livraisons terminées
Période	semaine ou mois
Cible	au moins 95 %
Action possible	examiner les villes sous la cible

Un **rapport** peut contenir plusieurs pages et des analyses détaillées, destinées à être explorées. Un **tableau de bord** est généralement plus synthétique : il met en évidence les quelques indicateurs et alertes nécessaires au suivi régulier d'une activité, souvent en une seule page consultée chaque jour ou chaque semaine.

TABLE 2 – Trois livrables BI, trois usages différents

Livrable	Fréquence de consultation	Profondeur	Exemple
Tableau de bord	Quotidienne ou hebdomadaire	Quelques KPI clés	Suivi des ventes du jour
Rapport	Ponctuelle ou mensuelle	Plusieurs pages, détail	Bilan trimestriel des ventes
Analyse ad hoc	Une seule fois	Ciblée sur une question précise	Pourquoi les ventes ont-elles chuté en mars ?

1.2. Le parcours d'une solution BI

```

Sources → préparation → modèle sémantique → mesures → visuels
                        ↓
                    rapport / tableau de bord
                        ↓
                    décision et action

```

Ce parcours reprend directement les briques des deux chapitres précédents : les **sources** (chapitre 1), le **modèle sémantique** qui s'appuie sur le modèle en étoile (chapitre 4), et les **visuels** qui suivent les principes de choix de graphique du chapitre 3.

Le modèle **sémantique** donne un sens commun aux tables, aux relations et aux mesures. Deux personnes doivent obtenir le même chiffre lorsqu'elles utilisent le même KPI — c'est tout l'enjeu du dictionnaire métier présenté au chapitre précédent.

1.3. Règles d'un tableau de bord débutant

Un tableau de bord lisible :

1. commence par le public et la décision attendue ;
2. affiche une période et une unité ;
3. présente quelques KPI importants avant les détails ;
4. utilise une couleur pour signaler une information, pas pour décorer chaque objet — voir les principes d'accessibilité du chapitre 3 ;
5. permet d'explorer sans cacher la définition des indicateurs ;
6. indique la date de dernière actualisation.

2. Power BI : une démarche structurée

Power BI est une plateforme de Business Intelligence qui permet de connecter des sources, de les transformer, de créer un modèle de données, de calculer des mesures et de construire des rapports interactifs. Le parcours présenté ici commence dans **Power BI Desktop** (application gratuite pour Windows) ; le partage dépend ensuite de l'environnement et des droits de l'organisation, via le **Power BI Service**.

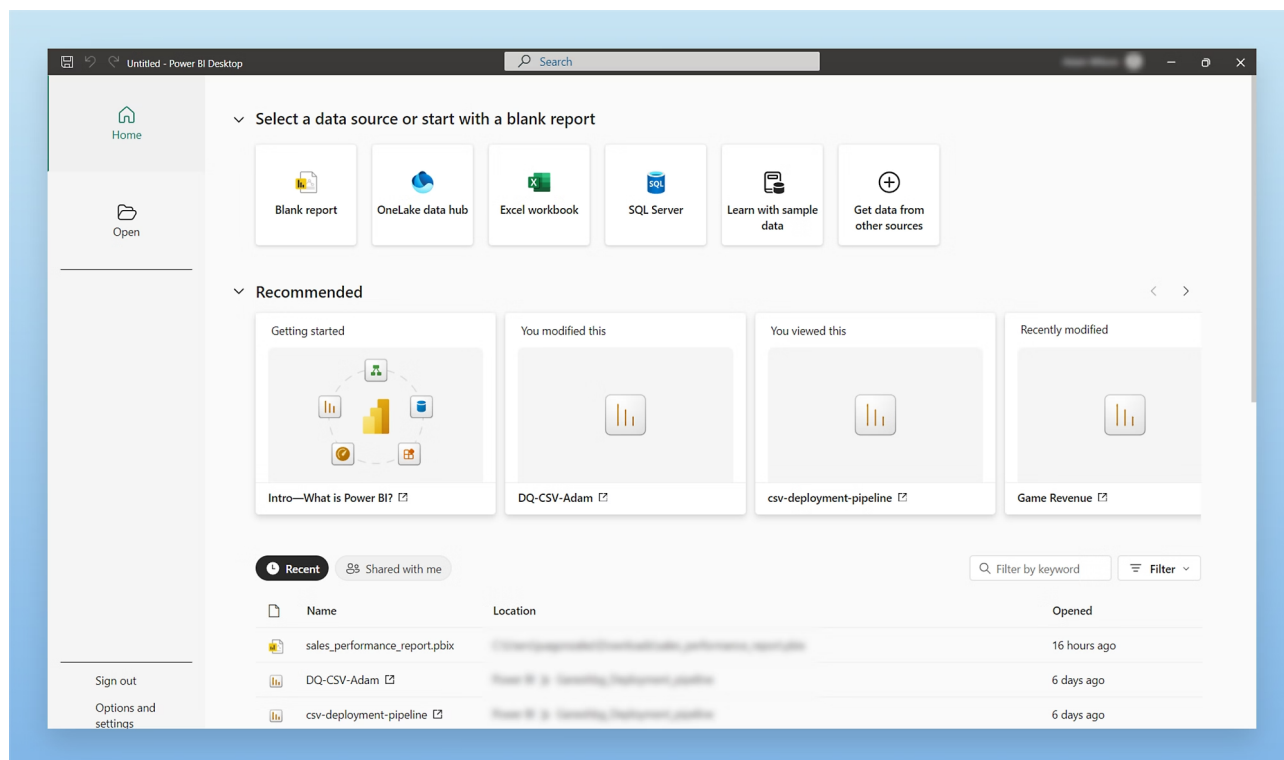


FIGURE 1 – Écran « Obtenir les données » de Power BI Desktop, avec la liste des connecteurs disponibles (fichiers, bases de données, services en ligne)

2.1. Étape 1 — Importer les sources

Le bouton **Obtenir les données** (*Get Data*) de Power BI Desktop donne accès à une longue liste de connecteurs : fichiers (CSV, Excel, Parquet, JSON), bases de données (SQL Server, PostgreSQL, MySQL), services cloud, API web, et bien d'autres — les mêmes familles de sources que celles présentées au chapitre 1.

À l'import, il faut vérifier :

- les noms de colonnes et leurs types ;
- le séparateur et l'encodage d'un CSV ;
- le type date, nombre ou texte ;
- le nombre de lignes et la période couverte ;
- la présence de doublons et de valeurs manquantes.

Importer n'est pas encore analyser : il faut comprendre ce qui a été chargé, exactement comme lors de la première lecture d'un jeu de données présentée au chapitre 1.

! Zoom sur les tendances : mode Import ou DirectQuery ?

Power BI propose deux modes de connexion. En mode **Import** (le plus courant, et celui utilisé dans ce chapitre), les données sont copiées et stockées dans le fichier Power BI, ce qui offre les meilleures performances mais nécessite une actualisation régulière. En mode **DirectQuery**, Power BI interroge la source à chaque affichage, sans copier les données : les résultats sont toujours à jour, au prix de performances qui dépendent directement de la source interrogée. Un mode hybride, appelé **composite**, permet de mélanger les deux selon les tables.

2.2. Étape 2 — Transformer avec Power Query

Power Query sert à préparer les données avant leur utilisation dans le modèle — c'est l'équivalent visuel des étapes de nettoyage et de transformation vues au chapitre 1 (valeurs manquantes, doublons, encodage, types). On peut supprimer des colonnes inutiles, renommer un champ, convertir un type, filtrer des lignes, remplacer une valeur, fusionner des tables ou ajouter une colonne calculée.

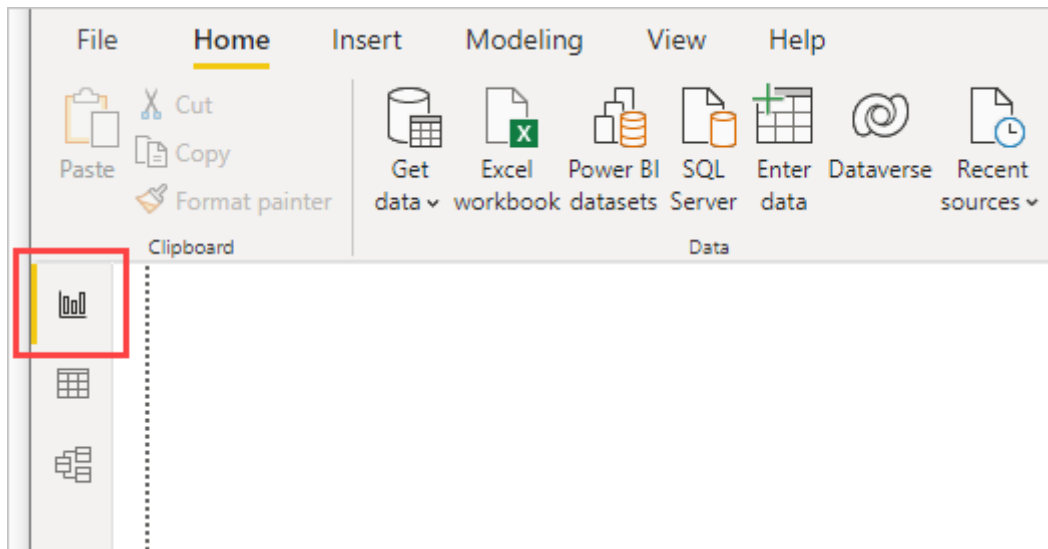


FIGURE 2 – L'éditeur Power Query, avec la liste des étapes de transformation appliquées dans le panneau de droite

Une transformation doit rester explicable et reproductible. Power Query conserve les étapes appliquées dans un panneau dédié (« Étapes appliquées ») ; il vaut mieux créer plusieurs étapes nommées et simples qu'une modification manuelle impossible à refaire. En coulisses, chaque étape est écrite dans le langage **M**, que l'on peut lire et modifier directement pour les transformations les plus avancées.

Exemples de contrôles à effectuer avant de quitter Power Query :

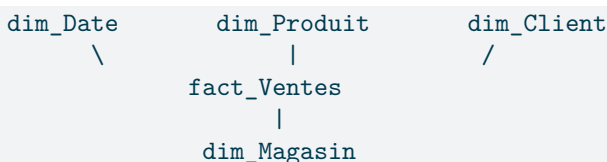
- **Montant** est numérique et positif ;
- **DateCommande** est bien de type date ;
- **ProduitID** et **DateID** sont présents ;
- la catégorie vide est traitée explicitement ;
- le nombre de lignes final est comparé au nombre attendu.

💡 Bonne pratique

Nommez vos étapes Power Query de façon explicite (« Filtrer lignes annulées » plutôt que « Étape 7 »). Un collègue — ou vous-même dans six mois — doit pouvoir comprendre le raisonnement sans exécuter le fichier.

2.3. Étape 3 — Construire le modèle

Pour un modèle de ventes, on privilégie un schéma en étoile — exactement celui présenté au chapitre 4 :



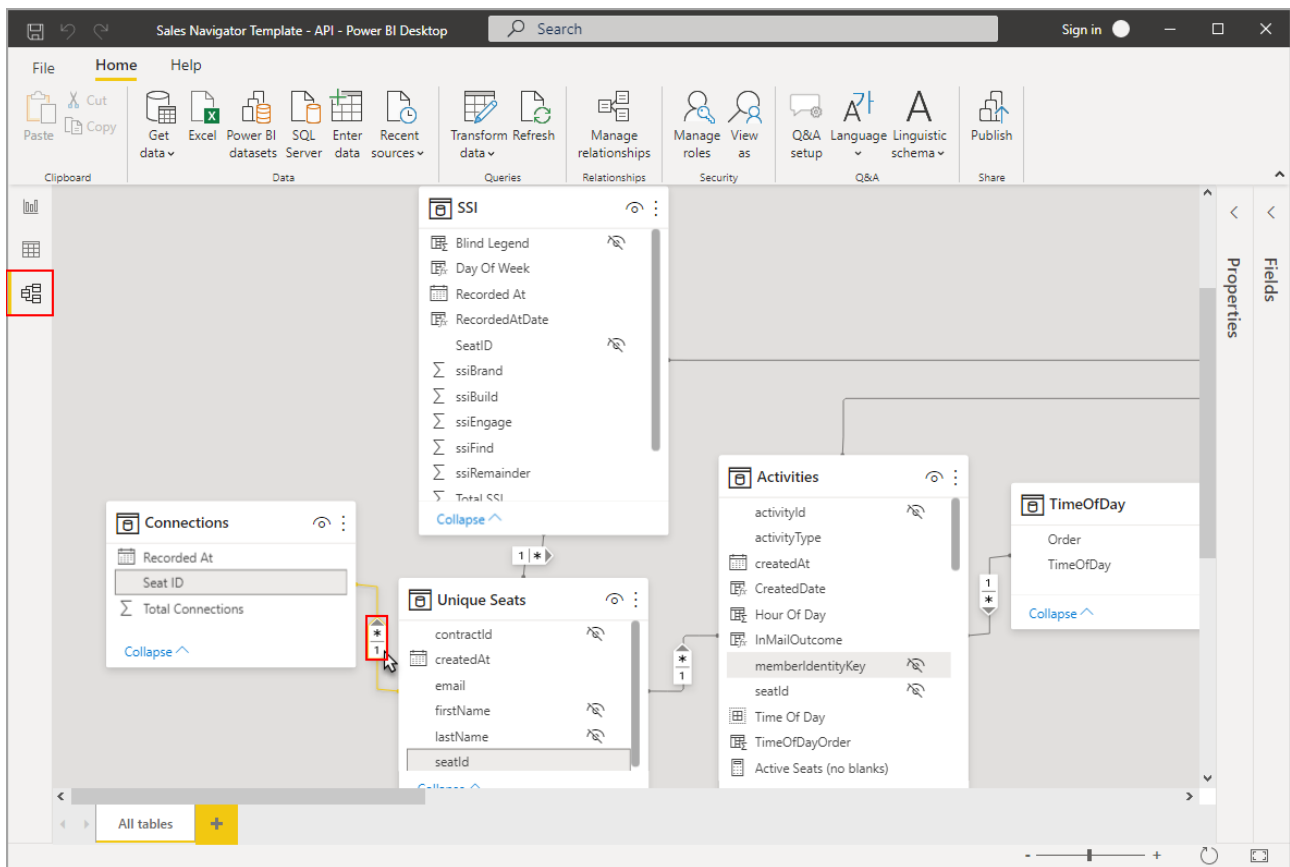


FIGURE 3 – Vue « Modèle » de Power BI Desktop, montrant les relations entre la table de faits et les tables de dimensions

La table de faits contient les lignes de vente et les dimensions permettent de filtrer par date, produit, client ou magasin. Les relations doivent être vérifiées : une dimension possède généralement une ligne par identifiant (**cardinalité un**), et plusieurs lignes de faits peuvent lui être liées (**cardinalité plusieurs**) — on parle de relation « **un-à-plusieurs** » (1 :). *Le sens du filtre* (cross-filter direction*) détermine si un filtre appliqué sur la dimension se répercute sur la table de faits (le cas normal), ou aussi dans l'autre sens.

⚠ Le piège des relations ambiguës

Si une dimension contient plusieurs lignes pour le même identifiant, ou si deux tables sont reliées de manière inattendue, les totaux peuvent être multipliés. Vérifiez toujours le grain, les clés et un total connu avant de publier un rapport — la même précaution que pour l'exercice de vérification du chapitre 4.

2.4. Étape 4 — Créer des mesures en DAX

DAX (*Data Analysis Expressions*) est le langage de formules utilisé dans les modèles Power BI. Une **mesure** est recalculée dynamiquement selon le **contexte de filtre** du visuel, de la page ou de l'utilisateur — c'est ce qui la distingue fondamentalement d'une **colonne calculée**, qui est évaluée une seule fois, ligne par ligne, au chargement des données.

```
ChiffreAffaires = SUM(fact_Ventes[MontantNet])

NombreCommandes = DISTINCTCOUNT(fact_Ventes[CommandeID])

PanierMoyen = DIVIDE([ChiffreAffaires], [NombreCommandes])
```

DIVIDE évite une erreur lorsque le dénominateur est nul — une bonne pratique systématique, préférable à l'opérateur / brut.

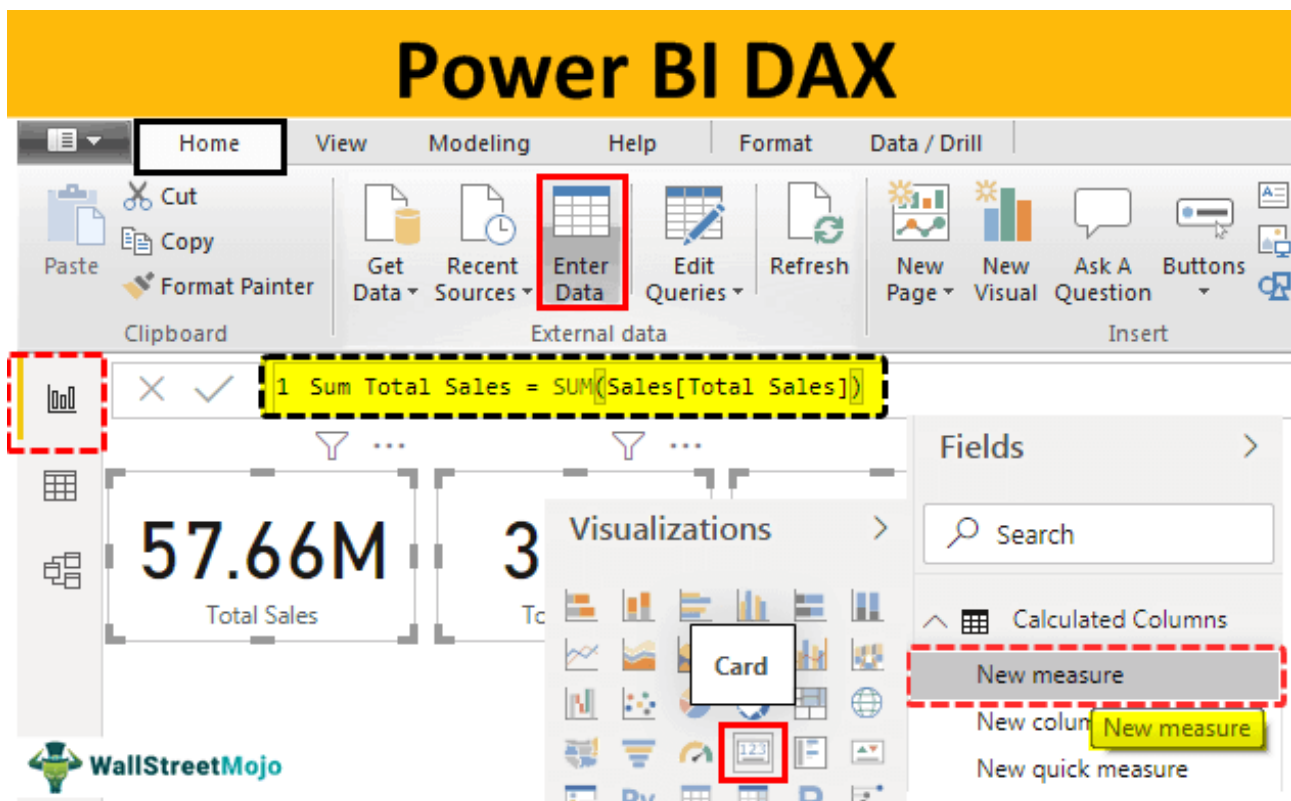


FIGURE 4 – Barre de formule DAX de Power BI Desktop lors de la création d'une nouvelle mesure

💡 Mesure ou colonne calculée ?

TABLE 3 – Différence entre mesure et colonne calculée

	Mesure	Colonne calculée
Quand est-elle calculée ?	À chaque interaction, selon le contexte de filtre	Une seule fois, au chargement des données
Où apparaît-elle ?	Dans les visuels (cartes, graphiques)	Comme une colonne normale de la table
Exemple	<code>ChiffreAffaires = SUM(...)</code>	<code>MargeUnitaire = fact_Ventes[Prix] - fact_Ventes[Cout]</code>

Aller plus loin : CALCULATE et l'intelligence temporelle

La fonction `CALCULATE` modifie le contexte de filtre d'une mesure, ce qui permet des comparaisons dans le temps très fréquentes dans un tableau de bord :

```
VentesAnneeDerniere =
CALCULATE(
    [ChiffreAffaires],
    SAMEPERIODLASTYEAR(dim_Date[DateComplete])
)

EvolutionAnnuelle =
DIVIDE([ChiffreAffaires] - [VentesAnneeDerniere], [VentesAnneeDerniere])
```

Ce type de comparaison — utile pour répondre à « comment se comportent nos ventes par rapport à l'an dernier ? » — nécessite une **dimension Date complète et marquée comme table de dates** dans le modèle, condition indispensable au bon fonctionnement des fonctions d'intelligence temporelle.

TABLE 4 – Quelques fonctions DAX fréquemment utilisées

Fonction DAX	Rôle
<code>SUM</code> , <code>AVERAGE</code> , <code>COUNT</code> <code>DISTINCTCOUNT</code>	Agrégations de base Compte les valeurs distinctes (ex. commandes uniques)
<code>DIVIDE</code>	Division sécurisée (gère le zéro)
<code>CALCULATE</code>	Modifie le contexte de filtre d'une mesure
<code>SAMEPERIODLASTYEAR</code> , <code>DATEADD</code>	Comparaisons dans le temps
<code>FILTER</code>	Filtre une table selon une condition

Pour débiter, il faut surtout retenir la distinction entre mesure et colonne calculée, et le rôle protecteur de `DIVIDE`. Dans un premier tableau de bord, trois ou quatre mesures bien définies valent mieux que de nombreuses formules difficiles à expliquer.

2.5. Étape 5 — Construire les visuels

Pour une page de suivi des ventes, une composition simple peut suivre les principes du chapitre 3 :

- une ligne de **cartes KPI** : chiffre d'affaires, commandes, panier moyen, taux de livraison à temps ;
- une **courbe** du chiffre d'affaires par mois (évolution, chapitre 3 §4.5) ;

- un **graphique en barres** par région ou catégorie (comparaison, §4.2) ;
- un **tableau ou une matrice** détaillé et filtrable ;
- des **segments (slicers)** pour l'année, la région et la catégorie.

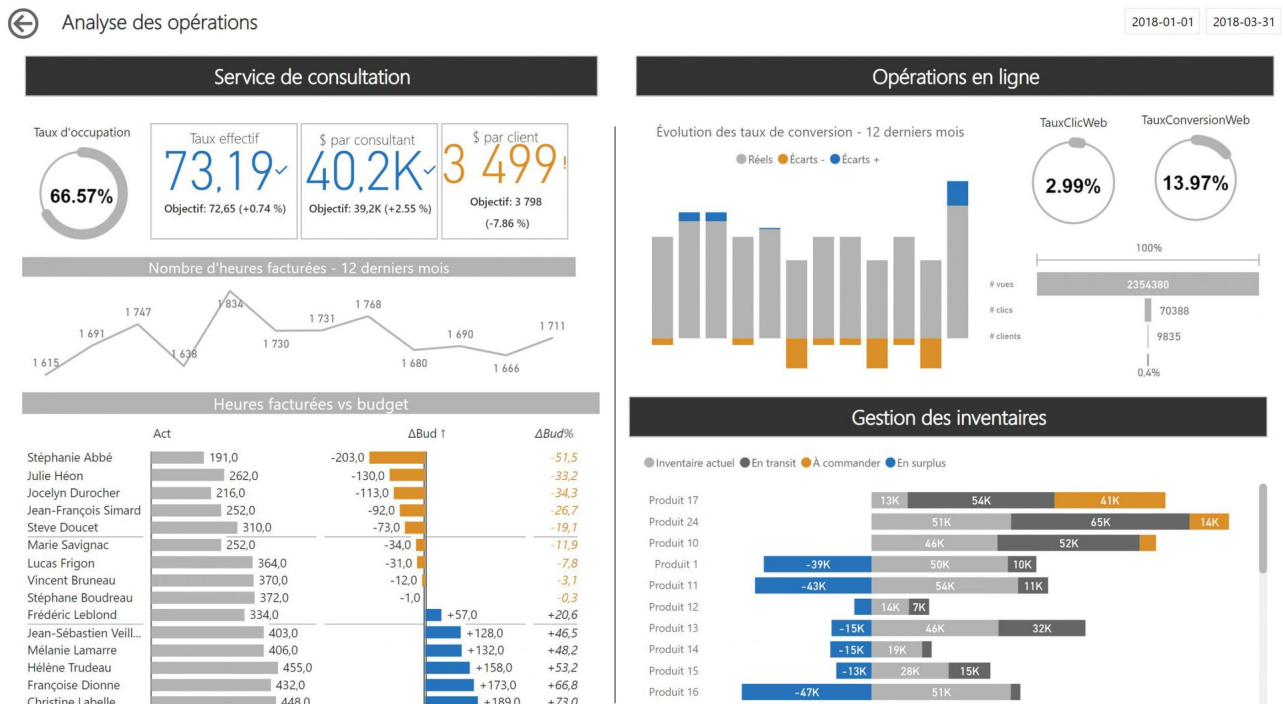


FIGURE 5 – Page de rapport Power BI combinant cartes KPI, courbe d'évolution, barres par catégorie et segments de filtre

Chaque visuel doit répondre à une question. Un graphique avec trop de séries ou trop de couleurs devient difficile à lire — le même principe de sobriété que celui du *data storytelling* présenté au chapitre 3.

! Zoom sur les tendances : l'IA dans Power BI

Les versions récentes de Power BI intègrent des fonctionnalités d'intelligence artificielle : le module **Q&A** permet d'interroger les données en langage naturel (« quel est le chiffre d'affaires de mars à Casablanca ? »), les **visuels de décomposition** (*decomposition tree*) explorent automatiquement quelles catégories expliquent le plus une variation, et **Copilot pour Power BI** peut générer une première ébauche de rapport ou résumer les tendances observées. Ces outils accélèrent l'exploration, mais ne dispensent jamais de vérifier la définition exacte des mesures affichées.

2.6. Étape 6 — Vérifier et partager

Avant tout partage, vérifiez les totaux sur un petit exemple connu, la date d'actualisation, les filtres, les titres et les droits d'accès.

Le partage s'effectue en publiant le rapport dans un **espace de travail (workspace)** du **Power BI Service**, où il peut être organisé en **application**, actualisé automatiquement selon une planification, et consulté depuis un navigateur ou l'application mobile. Pour restreindre l'accès aux données selon le profil de l'utilisateur (par exemple, un directeur régional ne voit que sa région), Power BI propose la **sécurité au niveau des lignes** (*Row-Level Security, RLS*), déjà annoncée comme principe de gouvernance au chapitre 4.

Un rapport partagé avec des données personnelles doit respecter les règles de l'organisation et le principe du moindre accès.

3. Taipy : créer une application BI en Python

Taipy permet de construire des applications Python interactives avec des contrôles, des graphiques, des tableaux et des filtres. Il est particulièrement intéressant quand l'équipe veut intégrer l'analyse dans une application métier ou conserver une logique Python personnalisée, en s'appuyant sur les bibliothèques de visualisation déjà vues au chapitre 3.

3.1. Quand choisir Taipy ?

Taipy peut être pertinent lorsque :

- les traitements sont déjà écrits en Python ;
- l'application doit combiner un tableau de bord avec une logique métier ;
- l'on veut contrôler l'interface et le déploiement comme un projet logiciel ;
- l'utilisateur doit modifier un paramètre et observer immédiatement un résultat.

Power BI peut être plus adapté lorsqu'une organisation veut une solution de BI prête à l'emploi, un catalogue de rapports, des connexions gérées et des usages majoritairement orientés vers l'exploration métier.

Ce ne sont pas des outils absolument opposés. Une équipe peut préparer les données avec Python, les exposer dans Power BI, ou intégrer un indicateur Power BI dans un parcours applicatif selon son architecture.

3.2. Anatomie d'une application Taipy

Taipy s'organise autour de deux briques complémentaires :

TABLE 5 – Les deux briques de Taipy

Brique	Rôle
Taipy GUI	Construit l'interface : pages, contrôles (sélecteurs, tableaux, graphiques), mise en page
Taipy Core	Orchestre les traitements : données, tâches, scénarios reproductibles — utile pour des pipelines plus complexes

Pour un tableau de bord simple comme celui de ce chapitre, **Taipy GUI** suffit largement ; **Taipy Core** devient utile lorsque l'application doit orchestrer plusieurs traitements enchaînés (par exemple, un pipeline de préparation suivi d'un calcul de prévision).

3.3. Préparer les données et calculer les indicateurs

Contrairement au reste de ce chapitre, les cellules de code qui suivent sont **réellement exécutées** lors de la génération de ce livre : les données, les indicateurs et le graphique que vous voyez ci-dessous sont calculés en direct par Python, exactement comme le ferait l'application Taipy finale.

```
import pandas as pd

ventes = pd.DataFrame({
    "mois": ["Janvier", "Janvier", "Janvier",
            "Février", "Février", "Février",
            "Mars", "Mars", "Mars"],
    "magasin": ["Casablanca", "Rabat", "Tanger"] * 3,
    "chiffre_affaires": [15200, 9800, 7100,
                        16100, 10450, 7600,
```

```

        17800, 9950, 8200],
    })

indicateurs = {
    "chiffre_affaires_total": ventes["chiffre_affaires"].sum(),
    "meilleur_mois": ventes.groupby("mois")["chiffre_affaires"].sum().idxmax(),
    "magasin_principal": ventes.groupby("magasin")["chiffre_affaires"].sum().idxmax(),
}

indicateurs

```

```

{'chiffre_affaires_total': np.int64(102200),
 'meilleur_mois': 'Mars',
 'magasin_principal': 'Casablanca'}

```

Cette table `ventes`, volontairement compacte, joue ici le rôle de la table de faits déjà préparée par l'entrepôt de données du chapitre 4. Dans une application réelle, elle proviendrait d'un fichier, d'une base de données ou d'une API, préparée avec les techniques du chapitre 1.

3.4. Construire une visualisation sophistiquée avec Python

L'application Taipy affichera le graphique produit ici, construit en appliquant directement les bonnes pratiques du chapitre 3 : deux graphiques complémentaires (évolution et comparaison), une palette accessible, des titres-messages et une source.

```

import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")

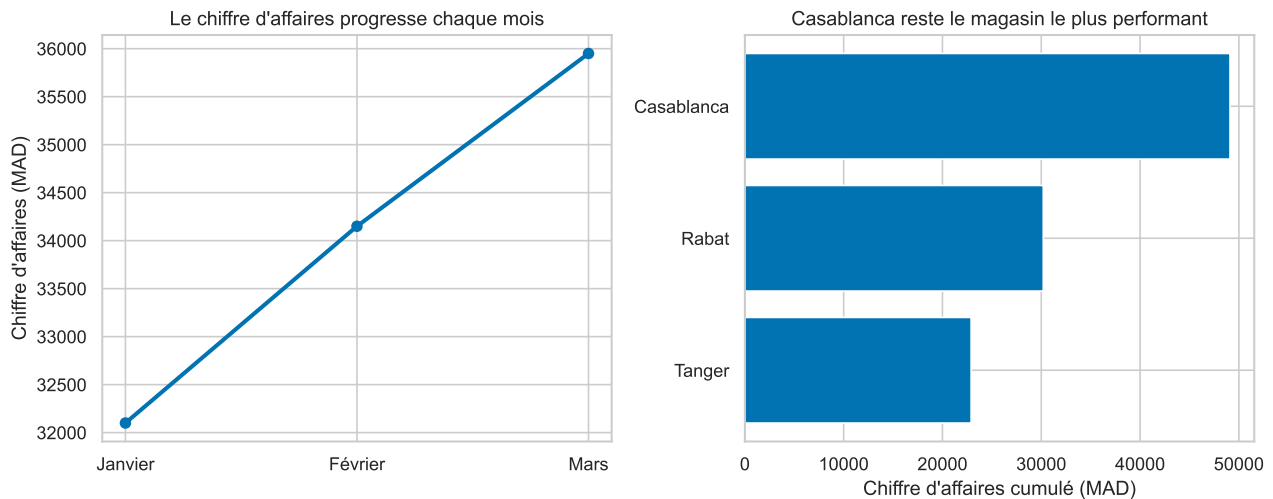
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

# Panneau 1 : évolution du chiffre d'affaires total par mois
evolution = ventes.groupby("mois", sort=False)["chiffre_affaires"].sum()
axes[0].plot(evolution.index, evolution.values, marker="o", linewidth=2.5)
axes[0].set_title("Le chiffre d'affaires progresse chaque mois")
axes[0].set_ylabel("Chiffre d'affaires (MAD)")
axes[0].set_xlabel("")

# Panneau 2 : comparaison par magasin, triée
par_magasin = ventes.groupby("magasin")["chiffre_affaires"].sum().sort_values()
axes[1].barh(par_magasin.index, par_magasin.values)
axes[1].set_title("Casablanca reste le magasin le plus performant")
axes[1].set_xlabel("Chiffre d'affaires cumulé (MAD)")

fig.text(0.01, -0.02, "Source : extrait pédagogique - janvier à mars", fontsize=8, color="gray")
plt.tight_layout()
plt.show()

```



Source : extrait pédagogique — janvier à mars

i Pourquoi cette cellule s'exécute-t-elle alors que le reste de ce chapitre ne s'exécute pas ?

Les extraits Power BI et DAX de ce chapitre décrivent une interface graphique et un langage propres à Power BI, qu'un livre Quarto ne peut pas exécuter. À l'inverse, la préparation des données et la construction du graphique ci-dessus sont écrites en Python : elles s'exécutent réellement au moment de la génération du livre (`execute: freeze: auto` dans `_quarto.yml`), exactement comme elles s'exécuteraient à l'intérieur d'une application Taipy.

3.5. Assembler l'application Taipy

Le code suivant montre comment ce même graphique et ces mêmes indicateurs seraient intégrés dans une page Taipy interactive, avec un sélecteur de magasin. **Ce bloc n'est pas exécuté dans le livre :** `Gui(page).run()` démarre un serveur web local qui reste actif en attendant les interactions de l'utilisateur, ce qui bloquerait indéfiniment la génération de ce livre. Pour le voir fonctionner, copiez ce code dans un fichier `app.py` et exécutez-le vous-même avec `uv run app.py`.

```
import pandas as pd
import matplotlib.pyplot as plt
from taipy.gui import Gui

ventes = pd.DataFrame({
    "mois": ["Janvier", "Février", "Mars"] * 3,
    "magasin": ["Casablanca"] * 3 + ["Rabat"] * 3 + ["Tanger"] * 3,
    "chiffre_affaires": [15200, 16100, 17800, 9800, 10450, 9950, 7100, 7600, 8200],
})

magasin_selectionne = "Tous"
liste_magasins = "Tous;Casablanca;Rabat;Tanger"

def filtrer(magasin):
    if magasin == "Tous":
        return ventes
    return ventes[ventes["magasin"] == magasin]

def construire_figure(state):
    df_filtre = filtrer(state.magasin_selectionne)
```

```

evolution = df_filtre.groupby("mois", sort=False)["chiffre_affaires"].sum()
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(evolution.index, evolution.values, marker="o", linewidth=2.5)
ax.set_title(f"Chiffre d'affaires - {state.magasin_selectionne}")
ax.set_ylabel("MAD")
state.figure = fig

page = """
# Suivi des ventes - Atlas Marchés

Magasin : <|{magasin_selectionne}|selector|lov={liste_magasins}|on_change=construire_figure|>

<|{figure}|chart|>

<|{ventes}|table|>
"""

Gui(page=page).run()

```

Dans cet exemple, `ventes` contient les données affichées, `page` décrit l'interface en langage Taipy Markdown, et le sélecteur déclenche `construire_figure` à chaque changement de magasin. Dans un vrai projet, on ajoute des indicateurs calculés (chiffre d'affaires, panier moyen) affichés sous forme de cartes, en réutilisant directement les fonctions de préparation vues en section 3.3. Les noms exacts de certains contrôles peuvent évoluer selon la version de Taipy : il faut vérifier la documentation de la version utilisée.

3.6. Organiser une petite application

Même une application pédagogique gagne à séparer les responsabilités, à l'image de la séparation ETL vue au chapitre 4 :

```

projet/
  data/                # données sources ou exemple
  preparation.py        # lecture et nettoyage (chapitre 1)
  indicateurs.py        # calculs métier (chapitre 2 et 4)
  visualisations.py     # graphiques matplotlib/seaborn (chapitre 3)
  app.py               # interface Taipy
  README.md            # installation et utilisation

```

Cette séparation permet de tester les indicateurs indépendamment de l'interface. L'application ne doit pas recalculer silencieusement un indicateur avec une définition différente de celle du rapport Power BI — le dictionnaire métier du chapitre 4 doit rester la référence commune aux deux outils.

4. Comparaison synthétique

TABLE 6 – Power BI et Taipy, deux approches complémentaires de la BI

Critère	Power BI	Taipy
Profil	Plateforme BI avec interface graphique	Bibliothèque/applications Python
Préparation	Power Query et connexions	Python, pandas ou autres outils
Mesures	DAX et modèle sémantique	Fonctions et code Python

Critère	Power BI	Taipy
Visualisation	Visuels intégrés (cartes, courbes, barres...)	<code>matplotlib/seaborn/Plotly</code> , comme au chapitre 3
Usage principal	Rapports, exploration et partage BI	Application interactive intégrée à un projet
Licence et coût	Gratuit pour Desktop ; licence pour le partage en Service	Open source (bibliothèque Python)
Courbe d'apprentissage	Interface visuelle, langage DAX à apprendre	Nécessite de savoir programmer en Python
Point d'attention	Relations, contexte des mesures, droits	Organisation du code, déploiement, versions

Le choix doit partir du besoin, pas de la préférence pour un outil. Dans les deux cas, les mêmes bases restent indispensables : question métier, qualité des données (chapitre 1), modèle clair (chapitre 4), indicateurs documentés et visualisation lisible (chapitre 3).

Exercices

i Consigne

Ces exercices sont conceptuels ou demandent seulement un calcul court. Aucun logiciel n'est nécessaire, sauf mention contraire.

Exercice 0.0.1 (Indicateur ou action ?). Pour l'objectif « réduire les retards de livraison », proposez un KPI, sa formule et une action possible si sa valeur diminue.

Exercice 0.0.2 (Lire un parcours BI). Associez chaque étape à son rôle : préparer, modéliser, visualiser, décider.

- Relier les ventes à la dimension Date.
- Convertir une colonne texte en date.
- Afficher les ventes par mois.
- Renforcer un transporteur après avoir constaté des retards.

Exercice 0.0.3 (Calculer un KPI). Sur 250 livraisons terminées, 225 ont été livrées à temps. Calculez le taux de livraison à temps et indiquez s'il atteint une cible de 95 %.

Exercice 0.0.4 (Power Query ou DAX ?). Indiquez l'outil le plus adapté :

- Corriger le type d'une colonne `Date`.
- Calculer le chiffre d'affaires selon les filtres du rapport.
- Supprimer une colonne inutilisée avant le modèle.
- Comparer le chiffre d'affaires à celui de l'année précédente.

Exercice 0.0.5 (Vérifier une relation). Une dimension `Produit` contient deux lignes pour le même `ProduitID`. Quel risque cela crée-t-il lorsqu'on relie cette dimension à une table de ventes ? Quelle vérification simple proposeriez-vous ?

Exercice 0.0.6 (Choisir un outil). Une équipe possède déjà plusieurs fonctions Python et veut intégrer un indicateur interactif dans une application interne. Une autre équipe veut surtout créer rapidement des rapports métiers partagés. Quel outil recommander à chacune, et pourquoi ?

Exercice 0.0.7 (Lire une mesure). Le chiffre d'affaires est de 48 000 MAD pour 300 commandes.

- a. Calculez le panier moyen.
- b. Pourquoi faut-il utiliser le nombre de commandes et non le nombre de lignes de produits si le KPI est défini par commande ?

Exercice 0.0.8 (Critiquer une page). Une page affiche 25 graphiques sans unité, sans date de mise à jour et avec une couleur différente pour chaque barre. Citez trois améliorations prioritaires et justifiez-les.

Exercice 0.0.9 (Mesure ou colonne calculée ?). Pour chacun des besoins suivants, indiquez s'il s'agit plutôt d'une mesure ou d'une colonne calculée (Table 3) :

- a. Calculer, pour chaque ligne de vente, la marge unitaire (**Prix - Coût**).
- b. Calculer le chiffre d'affaires total affiché sur une carte, qui doit changer selon le mois sélectionné.

Exercice 0.0.10 (Choisir la visualisation Taipy). En reprenant les données de la section 3.3 (**ventes**, avec les colonnes **mois**, **magasin**, **chiffre_affaires**), quel type de graphique du chapitre 3 utiliseriez-vous pour répondre à chacune des questions suivantes, et pourquoi ?

- a. Comment le chiffre d'affaires évolue-t-il entre janvier et mars ?
- b. Quel magasin contribue le plus au chiffre d'affaires total ?

Étude de cas : analyser les ventes d'une petite chaîne de magasins

💡 Objectifs du chapitre

À la fin de ce cas pratique, l'étudiant sera capable de :

- **Cadrer** une question d'analyse à partir d'un besoin métier et des parties prenantes concernées ;
- **Repérer** les problèmes de qualité dans un petit jeu de données ;
- **Calculer et interpréter** des indicateurs simples : chiffre d'affaires, quantité, panier moyen, médiane et taux ;
- **Choisir** un modèle en étoile adapté à une analyse de ventes et en dériver des mesures DAX ;
- **Proposer** une page Power BI et construire une véritable application Taipy cohérentes avec les questions ;
- **Rédiger** une conclusion prudente, fondée sur les données et accompagnée de limites.

Ce chapitre suit un projet de bout en bout, en mobilisant tous les chapitres précédents : collecte et préparation (chapitre 1), exploration (chapitre 2), visualisation (chapitre 3), modélisation en entrepôt (chapitre 4) et restitution en BI (chapitre 5). L'entreprise fictive **Atlas Marchés** possède trois magasins à Casablanca, Rabat et Tanger. La direction souhaite comprendre les ventes du premier trimestre 2026 et décider où concentrer ses efforts commerciaux. Les données sont volontairement petites afin que les calculs puissent être vérifiés à la main — et, dans ce chapitre, également par une cellule Python réellement exécutée.

1. Le besoin métier

1.1. Les parties prenantes

Trois profils attendent des réponses différentes de cette analyse :

TABLE 1 – Parties prenantes de l'analyse Atlas Marchés

Rôle	Question principale	Niveau de détail attendu
Directrice générale	Où concentrer les efforts commerciaux du prochain trimestre ?	Synthétique, quelques KPI
Responsable de magasin	Comment mon magasin se compare-t-il aux deux autres ?	Intermédiaire, comparaisons
Analyste de gestion	D'où provient exactement le chiffre d'affaires ?	Détaillé, par produit et catégorie

Cette diversité de publics justifie, comme annoncé au chapitre 5 (§1.3), une page de synthèse et une page de détail plutôt qu'une seule page surchargée.

1.2. Les quatre questions posées

La direction pose quatre questions :

1. Quel est le chiffre d'affaires net par mois et par magasin ?
2. Quels produits ou catégories contribuent le plus aux ventes ?
3. Le panier moyen est-il différent selon les magasins ?
4. Quelles informations faudrait-il surveiller dans un tableau de bord mensuel ?

La question n'est pas « faire un graphique ». Le livrable attendu est une analyse descriptive, quelques explications prudentes et une proposition de suivi — exactement la distinction entre analyse descriptive et diagnostique posée dès l'introduction de ce cours.

1.3. Périmètre et définitions

Le périmètre est limité aux ventes **payées** enregistrées entre le 1er janvier et le 31 mars 2026. Une ligne du fichier correspond à une ligne de produit dans une commande. Le montant net est défini ainsi :

$$\text{montant net} = \text{quantité} \times \text{prix unitaire} - \text{remise}.$$

Le **panier moyen** est :

$$\text{panier moyen} = \frac{\text{chiffre d'affaires net}}{\text{nombre de commandes distinctes}}.$$

Ces définitions sont annoncées avant les calculs ; elles évitent de changer de formule au milieu de l'analyse, et constitueront le point de départ du dictionnaire métier de la section 5.

2. Le jeu de données

Voici un extrait pédagogique. Dans un vrai projet, le fichier complet pourrait contenir beaucoup plus de lignes, collectées comme décrit au chapitre 1 (export de caisse, base de données de vente, ou API).

Ligne	Commande	Date	Magasin	Catégorie	Produit	Qté	Prix unitaire	Remise	Statut
1	C001	05/01/2026	Casablanca	Épicerie	Huile	2	75	0	Payée
2	C001	05/01/2026	Casablanca	Épicerie	Riz	3	20	0	Payée
3	C002	08/01/2026	Rabat	Boissons	Jus	4	18	5	Payée
4	C003	15/01/2026	Tanger	Hygiène	Savon	5	12	0	Payée
5	C004	03/02/2026	Casablanca	Épicerie	Huile	1	75	0	Payée
6	C005	10/02/2026	Rabat	Épicerie	Riz	5	20	10	Payée
7	C006	18/02/2026	Tanger	Boissons	Jus	2	18	0	Annulée
8	C007	22/02/2026	Casablanca	Hygiène	Savon	6	12	0	Payée
9	C008	06/03/2026	Rabat	Boissons	Jus	6	18	0	Payée
10	C009	12/03/2026	Tanger	Épicerie	Riz	2	20	0	Payée
11	C010	20/03/2026	Casablanca	Boissons	Jus	3	18	5	Payée
12	C010	20/03/2026	Casablanca	Hygiène	Savon	2	12	0	Payée

2.1. Première lecture

Le grain est **une ligne de produit dans une commande**, au sens défini au chapitre 4. La commande C001 apparaît deux fois car elle contient deux produits. Pour compter les commandes, il faut compter les identifiants distincts, et non les lignes.

La ligne C006 est annulée. Si le chiffre d'affaires concerne les ventes payées, elle doit être exclue, mais cette décision doit être documentée. Elle ne doit pas être supprimée du fichier original : on conserve le statut pour pouvoir analyser les annulations — un exemple concret de la traçabilité demandée au chapitre 4 (§4.1).

3. Préparer les données

3.1. Dictionnaire minimal

Colonne	Type attendu	Règle
Commande	Texte	Identifiant non vide
Date	Date	Date comprise dans la période
Magasin	Catégorie	Casablanca, Rabat ou Tanger
Qté	Entier	Strictement positive pour une vente
Prix unitaire	Décimal	Positif, en MAD
Remise	Décimal	Supérieure ou égale à 0, en MAD
Statut	Catégorie	Payée ou Annulée

Il faut également vérifier que la remise ne dépasse pas le montant brut de la ligne, que les dates ne sont pas inversées et que les identifiants ne comportent pas d'espaces cachés — les mêmes contrôles de qualité que ceux présentés au chapitre 1.

3.2. Transformation

Pour chaque ligne, on calcule :

```
montant_brut = quantité × prix_unitaire
montant_net = montant_brut - remise
mois = mois(date)
```

Puis on filtre **Statut = Payée** pour les indicateurs de chiffre d'affaires. On conserve une copie de l'ensemble initial et on compte les lignes exclues.

Pour l'extrait, 11 lignes sur 12 sont payées. La ligne annulée représente 1 ligne sur 12, soit environ 8,3 % des lignes ; ce pourcentage ne doit pas être interprété comme un taux de commandes annulées, car une commande peut comporter plusieurs lignes.

4. Calculer les indicateurs

4.1. Montants ligne par ligne

Sur les lignes payées :

Commande	Produit	Calcul	Montant net
C001	Huile	$2 \times 75 - 0$	150
C001	Riz	$3 \times 20 - 0$	60

Commande	Produit	Calcul	Montant net
C002	Jus	$4 \times 18 - 5$	67
C003	Savon	$5 \times 12 - 0$	60
C004	Huile	$1 \times 75 - 0$	75
C005	Riz	$5 \times 20 - 10$	90
C007	Savon	$6 \times 12 - 0$	72
C008	Jus	$6 \times 18 - 0$	108
C009	Riz	$2 \times 20 - 0$	40
C010	Jus	$3 \times 18 - 5$	49
C010	Savon	$2 \times 12 - 0$	24

Le chiffre d'affaires net de l'extrait est donc :

$$150 + 60 + 67 + 60 + 75 + 90 + 72 + 108 + 40 + 49 + 24 = \mathbf{795 \text{ MAD.}}$$

4.2. Par mois

Mois	Chiffre d'affaires net	Commandes distinctes	Panier moyen
Janvier	337	3	112,33
Février	237	3	79,00
Mars	221	3	73,67
Total	795	9	88,33

Le panier moyen total est $795/9 = 88,33$ MAD. Il ne faut pas faire la moyenne simple des trois paniers mensuels sans vérifier les effectifs ; ici ils sont égaux, mais ce n'est pas toujours le cas.

4.3. Par magasin et par catégorie

En regroupant les lignes payées :

Magasin	Chiffre d'affaires net	Commandes distinctes	Panier moyen
Casablanca	430	4	107,50
Rabat	265	3	88,33
Tanger	100	2	50,00

Catégorie	Chiffre d'affaires net
Épicerie	415
Boissons	224
Hygiène	156

Casablanca contribue le plus au chiffre d'affaires et possède le panier moyen le plus élevé dans cet extrait. Cela ne suffit pas à conclure que le magasin est le plus performant : il faudrait connaître le nombre de visiteurs, les coûts, la taille des magasins et une période plus longue.

4.4. Moyenne et médiane des montants de lignes

Les montants des lignes payées sont :

60, 60, 67, 72, 75, 90, 108, 24, 40, 49, 150

Triés :

24, 40, 49, 60, 60, 67, 72, 75, 90, 108, 150.

La médiane est la sixième valeur, soit **67 MAD**. La moyenne est $795/11 \approx \mathbf{72,27}$ MAD. La moyenne un peu supérieure à la médiane suggère que quelques lignes élevées, notamment 150 MAD, tirent la moyenne vers le haut — un rappel direct du chapitre 2 (§4.1).

i Vérification

Tous les indicateurs de cette section sont recalculés, à partir des mêmes 12 lignes, par une véritable cellule Python exécutée en section 7 : les deux méthodes doivent converger vers les mêmes chiffres.

5. Construire le modèle analytique

À partir de ce fichier, un modèle en étoile simple pourrait être :

```
dim_date -----\  
dim_produit ----- fact_ventes  
dim_magasin -----/
```

`fact_ventes` contient `Commande`, `DateID`, `ProduitID`, `MagasinID`, `Quantite`, `PrixUnitaire`, `Remise`, `MontantNet` et `Statut`. Les dimensions décrivent les dates, produits et magasins — exactement la structure présentée au chapitre 4.

Les mesures attendues, avant même d'être écrites dans un outil précis, sont :

```
ChiffreAffaires = somme de MontantNet pour les lignes payées  
Commandes = nombre d'identifiants Commande distincts pour les lignes payées  
PanierMoyen = ChiffreAffaires / Commandes  
Quantités = somme de Quantite pour les lignes payées
```

Traduites en DAX pour Power BI (chapitre 5, §2.4), ces mesures s'écrivent :

```
ChiffreAffaires =  
CALCULATE(  
    SUM(fact_Ventes[MontantNet]),  
    fact_Ventes[Statut] = "Payée"  
)  
  
Commandes =  
CALCULATE(  
    DISTINCTCOUNT(fact_Ventes[Commande]),  
    fact_Ventes[Statut] = "Payée"  
)
```

```

PanierMoyen = DIVIDE([ChiffreAffaires], [Commandes])

Quantites =
CALCULATE(
    SUM(fact_Ventes[Quantite]),
    fact_Ventes[Statut] = "Payée"
)

```

Le filtre `fact_Ventes[Statut] = "Payée"` est appliqué explicitement dans chaque mesure via `CALCULATE`, plutôt que d'être filtré une fois pour toutes en amont : cela permet de garder, dans le même modèle, la possibilité d'analyser aussi les commandes annulées si besoin, sans dupliquer la table.

Le tableau de bord ne doit pas calculer le panier moyen comme la moyenne des montants de lignes : une commande peut contenir plusieurs lignes. Le grain explique cette différence.

6. Concevoir la page Power BI

Une page « Vue générale » peut contenir, en suivant les principes de composition du chapitre 5 (§2.5) :

1. quatre cartes : chiffre d'affaires net, commandes, panier moyen et quantités vendues ;
2. une courbe du chiffre d'affaires par mois ;
3. des barres du chiffre d'affaires par magasin ;
4. des barres du chiffre d'affaires par catégorie ;
5. des segments pour le mois, le magasin et la catégorie ;
6. une note indiquant « données payées, période janvier–mars 2026, dernière actualisation ».

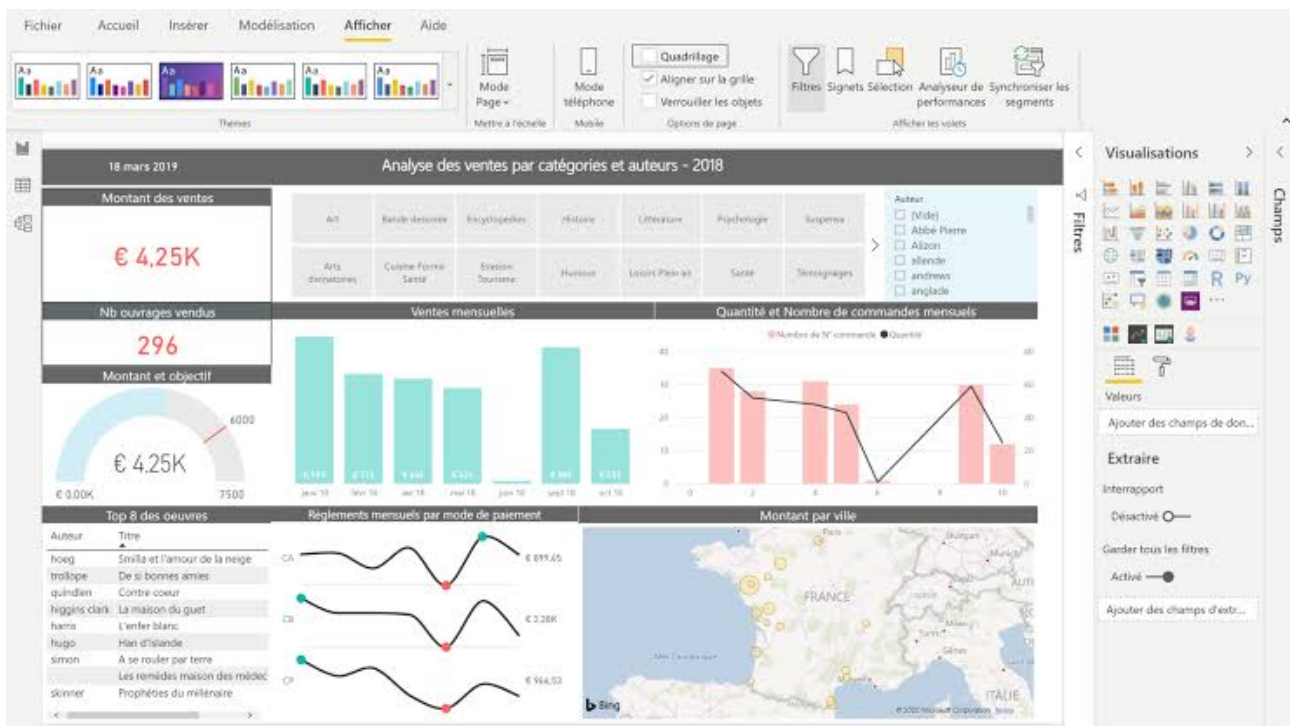


FIGURE 1 – Maquette de la page « Vue générale » du tableau de bord Atlas Marchés dans Power BI Desktop, construite à partir des mesures ChiffreAffaires, Commandes, PanierMoyen et Quantites

Une deuxième page « Diagnostic » peut détailler les produits, les annulations et les différences entre magasins :

- un tableau détaillé par produit, avec quantité et chiffre d'affaires ;
- un graphique en barres du chiffre d'affaires par catégorie et par magasin (comparaison croisée) ;
- un indicateur du nombre et du montant des lignes annulées, pour suivre ce phénomène séparément du chiffre d'affaires payé.

La première page répond à « où en sommes-nous ? » (analyse descriptive), la seconde aide à explorer « où regarder ensuite ? » (analyse diagnostique) — la distinction posée dès l'introduction de ce cours.

Avant le partage, on compare les cartes aux totaux calculés manuellement : **795 MAD, 9 commandes distinctes** et **88,33 MAD** de panier moyen.

7. Construire une vue Taipy

La même analyse peut être intégrée à une application Python. Contrairement aux sections précédentes, qui décrivent une interface Power BI que ce livre ne peut pas exécuter, le code Python qui suit est **réellement exécuté** lors de la génération de ce chapitre : les indicateurs et le graphique affichés ci-dessous sont calculés en direct, à partir des 12 lignes du jeu de données de la section 2.

7.1. Charger et préparer les données

```
import pandas as pd

ventes = pd.DataFrame({
    "commande": ["C001", "C001", "C002", "C003", "C004", "C005",
                "C006", "C007", "C008", "C009", "C010", "C010"],
    "date":      ["2026-01-05", "2026-01-05", "2026-01-08", "2026-01-15",
                "2026-02-03", "2026-02-10", "2026-02-18", "2026-02-22",
                "2026-03-06", "2026-03-12", "2026-03-20", "2026-03-20"],
    "magasin":   ["Casablanca", "Casablanca", "Rabat", "Tanger",
                "Casablanca", "Rabat", "Tanger", "Casablanca",
                "Rabat", "Tanger", "Casablanca", "Casablanca"],
    "categorie": ["Épicerie", "Épicerie", "Boissons", "Hygiène",
                "Épicerie", "Épicerie", "Boissons", "Hygiène",
                "Boissons", "Épicerie", "Boissons", "Hygiène"],
    "produit":   ["Huile", "Riz", "Jus", "Savon", "Huile", "Riz",
                "Jus", "Savon", "Jus", "Riz", "Jus", "Savon"],
    "qte":       [2, 3, 4, 5, 1, 5, 2, 6, 6, 2, 3, 2],
    "prix_unitaire": [75, 20, 18, 12, 75, 20, 18, 12, 18, 20, 18, 12],
    "remise":     [0, 0, 5, 0, 0, 10, 0, 0, 0, 0, 5, 0],
    "statut":     ["Payée", "Payée", "Payée", "Payée", "Payée", "Payée",
                "Annulée", "Payée", "Payée", "Payée", "Payée", "Payée"],
})

ventes["date"] = pd.to_datetime(ventes["date"])
ventes["montant_net"] = ventes["qte"] * ventes["prix_unitaire"] - ventes["remise"]
ventes["mois"] = ventes["date"].dt.strftime("%B")

payees = ventes[ventes["statut"] == "Payée"]
print(payees.head().to_string(index=False))
```

commande	date	magasin	categorie	produit	qte	prix_unitaire	remise	statut	montant_net	mo
C001	2026-01-05	Casablanca	Épicerie	Huile	2	75	0	Payée	150	Janua
C001	2026-01-05	Casablanca	Épicerie	Riz	3	20	0	Payée	60	Janua
C002	2026-01-08	Rabat	Boissons	Jus	4	18	5	Payée	67	Janua

C003	2026-01-15	Tanger	Hygiène	Savon	5	12	0	Payée	60	Janua
C004	2026-02-03	Casablanca	Épicerie	Huile	1	75	0	Payée	75	Februa

7.2. Calculer les indicateurs

```
def indicateurs(df):
    payees = df[df["statut"] == "Payée"]
    chiffre_affaires = payees["montant_net"].sum()
    commandes = payees["commande"].nunique()
    panier_moyen = chiffre_affaires / commandes if commandes else 0
    quantites = payees["qte"].sum()
    return {
        "chiffre_affaires": chiffre_affaires,
        "commandes": commandes,
        "panier_moyen": round(panier_moyen, 2),
        "quantites": quantites,
    }

resultats = indicateurs(ventes)
resultats
```

```
{'chiffre_affaires': np.int64(795),
 'commandes': 9,
 'panier_moyen': np.float64(88.33),
 'quantites': np.int64(39)}
```

Ces résultats doivent — et retrouvent effectivement — les valeurs calculées à la main en section 4 : un chiffre d'affaires net de 795 MAD, pour 9 commandes distinctes, et un panier moyen de 88,33 MAD.

7.3. Construire une visualisation sophistiquée

En reprenant les bonnes pratiques du chapitre 3 (titre-message, palette accessible, deux graphiques complémentaires plutôt qu'un seul graphique surchargé) :

```
import matplotlib.pyplot as plt
import seaborn as sns

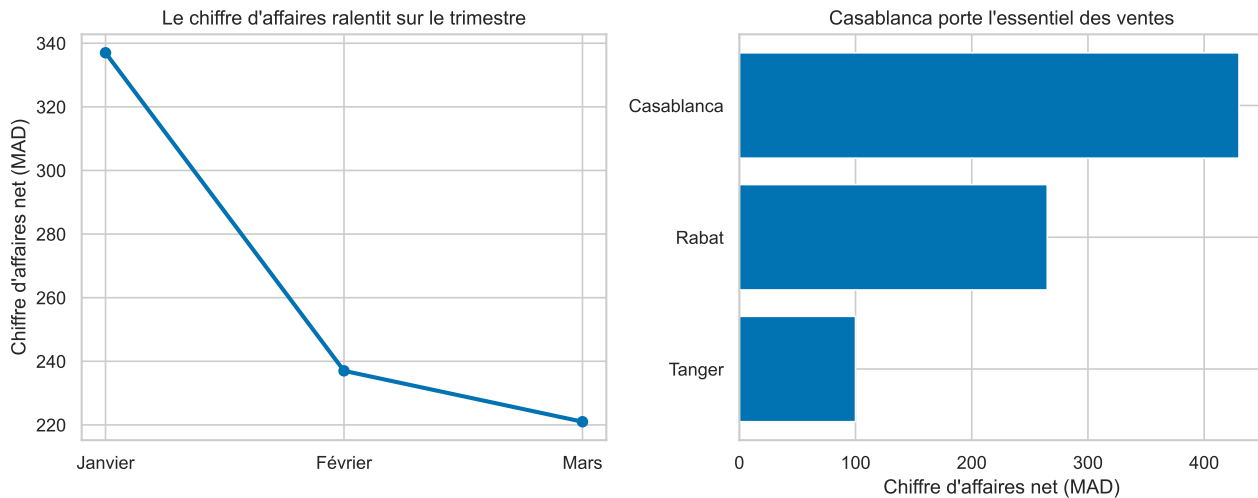
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

# Panneau 1 : chiffre d'affaires net par mois
par_mois = payees.groupby("mois", sort=False)["montant_net"].sum()
par_mois = par_mois.reindex(["January", "February", "March"])
axes[0].plot(["Janvier", "Février", "Mars"], par_mois.values, marker="o", linewidth=2.5)
axes[0].set_title("Le chiffre d'affaires ralentit sur le trimestre")
axes[0].set_ylabel("Chiffre d'affaires net (MAD)")

# Panneau 2 : chiffre d'affaires net par magasin, trié
par_magasin = payees.groupby("magasin")["montant_net"].sum().sort_values()
axes[1].barh(par_magasin.index, par_magasin.values)
axes[1].set_title("Casablanca porte l'essentiel des ventes")
axes[1].set_xlabel("Chiffre d'affaires net (MAD)")
```

```
fig.text(0.01, -0.02, "Source : Atlas Marchés - ventes payées, janvier à mars 2026", fontsize=8, color=
plt.tight_layout()
plt.show()
```



Source : Atlas Marchés — ventes payées, janvier à mars 2026

Ce graphique répond directement à la première question de la direction (§1.2) : il montre, en un coup d'œil, que le chiffre d'affaires ralentit d'un mois à l'autre et que Casablanca domine largement les deux autres magasins.

7.4. Assembler l'application Taipy

Comme au chapitre 5, l'assemblage final dans une interface Taipy interactive n'est **pas exécuté** dans ce livre : `Gui(page).run()` démarrerait un serveur bloquant. Le code suivant montre comment les résultats des sections 7.1 à 7.3 s'intègrent dans une page filtrable par magasin.

```
import pandas as pd
import matplotlib.pyplot as plt
from taipy.gui import Gui

# ventes, indicateurs() et le graphique reprennent le code des sections 7.1 à 7.3

magasin_selectionne = "Tous"
liste_magasins = "Tous;Casablanca;Rabat;Tanger"

def filtrer_et_recalculer(state):
    if state.magasin_selectionne == "Tous":
        sous_ensemble = ventes
    else:
        sous_ensemble = ventes[ventes["magasin"] == state.magasin_selectionne]

    resultats = indicateurs(sous_ensemble)
    state.chiffre_affaires = resultats["chiffre_affaires"]
    state.commandes = resultats["commandes"]
    state.panier_moyen = resultats["panier_moyen"]

page = """
# Atlas Marchés - Suivi des ventes T1 2026
```

```
Magasin : <|{magasin_selectionne}|selector|lov={liste_magasins}|on_change=filtrer_et_recalculer|>

<|layout|columns=1 1 1|
<|{chiffre_affaires}|text|format=%d MAD|class_name=kpi-card|>
<|{commandes}|text|format=%d commandes|class_name=kpi-card|>
<|{panier_moyen}|text|format=%.2f MAD|class_name=kpi-card|>
|>

<|{ventes}|table|>
"""

Gui(page=page).run()
```

Cette page reprend fidèlement le dictionnaire métier de la section 1.3 : les mêmes formules, qu'il s'agisse de la version Power BI (DAX, section 5) ou de la version Taipy (Python, ci-dessus), doivent produire les mêmes résultats. C'est précisément ce que la cellule exécutée en section 7.2 permet de vérifier.

8. Interpréter et communiquer

Une conclusion raisonnable pour l'extrait serait :

Sur les ventes payées de l'extrait, le chiffre d'affaires net atteint 795 MAD pour 9 commandes, avec un panier moyen de 88,33 MAD. Casablanca représente 430 MAD et le panier moyen y est supérieur à celui de Rabat et Tanger. Les lignes de vente ont une médiane de 67 MAD et une moyenne de 72,27 MAD, ce qui signale une légère influence de quelques montants élevés. Ces résultats sont exploratoires : l'échantillon est très petit, ne couvre que trois mois et ne permet pas d'expliquer les causes ni de mesurer la rentabilité.

La conclusion sépare les **faits calculés**, les **interprétations prudentes** et les **limites**. Elle ne transforme pas une différence observée en causalité — le même principe de prudence que celui rappelé au chapitre 2 à propos de la corrélation.

i Aller plus loin

Sur un jeu de données réel, cette même démarche se prolongerait naturellement : automatiser le chargement et le contrôle de qualité avec un flux ETL/ELT planifié (chapitre 4, §4), historiser les ventes pour comparer un trimestre à l'année précédente avec `CALCULATE` et `SAMEPERIODLASTYEAR` (chapitre 5, §2.4), et enrichir le modèle avec de nouvelles dimensions — un profil client, une équipe commerciale — pour affiner le diagnostic au-delà du simple magasin.

Exercices

i Consigne

Répondez sans ordinateur pour les exercices 1 à 8 ; les données de l'énoncé suffisent et les calculs sont courts. Les exercices 9 et 10 portent sur les choix d'outillage et peuvent s'appuyer sur le code des sections 5 à 7.

Exercice 0.0.1 (Définir le grain). Quelle phrase décrit le grain du fichier ? Pourquoi la commande C001 occupe-t-elle deux lignes ?

Étude de cas : analyser les ventes d'une petite chaîne de magasins

Exercice 0.0.2 (Exclure une annulation). Pourquoi la ligne C006 n'est-elle pas incluse dans le chiffre d'affaires payé ? Peut-on la supprimer du fichier original ?

Exercice 0.0.3 (Calculer une ligne). Calculez le montant net de 4 jus à 18 MAD avec une remise de 5 MAD, puis celui de 5 savons à 12 MAD sans remise.

Exercice 0.0.4 (Vérifier le total). Le chiffre d'affaires net est de 795 MAD et le magasin de Casablanca réalise 430 MAD.

- a. Quelle part du chiffre d'affaires total représente Casablanca ?
- b. Quel montant reste pour Rabat et Tanger ensemble ?

Exercice 0.0.5 (Commandes et lignes). Il y a 11 lignes payées mais seulement 9 commandes distinctes.

- a. Pourquoi ces deux nombres sont-ils différents ?
- b. Quel nombre utiliser pour calculer le panier moyen ?

Exercice 0.0.6 (Comparer moyenne et médiane). La moyenne des montants de lignes est 72,27 MAD et la médiane est 67 MAD. Que suggère cette différence ? Quelle valeur de la liste contribue particulièrement à tirer la moyenne vers le haut ?

Exercice 0.0.7 (Choisir les indicateurs du tableau de bord). Parmi les éléments suivants, lesquels doivent apparaître en priorité sur la page générale : chiffre d'affaires net, identifiant de chaque client, commandes distinctes, panier moyen, une liste de 25 couleurs, date d'actualisation ? Justifiez les choix.

Exercice 0.0.8 (Limites et décisions). La direction demande : « Casablanca est-elle le meilleur magasin ? »

- a. Donnez une conclusion que les données permettent de soutenir.
- b. Citez deux informations manquantes avant de conclure sur la performance globale.

Exercice 0.0.9 (Power BI ou Taipy pour Atlas Marchés ?). La direction d'Atlas Marchés veut un tableau de bord consultable par les trois responsables de magasin, actualisé chaque semaine, sans écrire de code. L'équipe technique veut, séparément, intégrer les mêmes indicateurs dans l'application de caisse déjà développée en Python.

En vous appuyant sur la Table 6 du chapitre 5, quel outil recommanderiez-vous à chacun des deux besoins, et pourquoi ?

Exercice 0.0.10 (Vérifier la cohérence entre les deux outils). La mesure DAX `ChiffreAffaires` de la section 5 et la fonction Python `indicateurs()` de la section 7.2 doivent produire le même résultat pour la même question.

- a. Quelle valeur commune les deux doivent-elles retrouver pour le chiffre d'affaires net total ?
- b. Que faudrait-il vérifier si, sur un vrai projet, les deux outils affichaient des chiffres différents ?

Lab 0 : Environnement Python et statistique descriptive

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook** `.ipynb` pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 0 et téléchargez-le.
- **Fichier** `.qmd` pour travailler avec Quarto : [téléchargez le fichier source du Lab 0](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le `.qmd`, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Créer** un projet Python reproductible avec `uv` et JupyterLab ;
- **Charger** un jeu de données réel et volumineux avec `pandas` ;
- **Identifier** les types de variables et les premières erreurs de qualité ;
- **Calculer** des effectifs, fréquences, taux de croissance, moyenne, médiane, quartiles, quantiles, variance et écart-type ;
- **Interpréter** les indicateurs avec leur unité, leur période et leur population ;
- **Produire** quelques graphiques simples pour vérifier les résultats numériques.

i Positionnement

Ce Lab ne demande ni Machine Learning ni formule compliquée. Le but est de prendre de bons réflexes avec Python et `pandas` : observer les données avant de les modifier, calculer des indicateurs adaptés à la question et expliquer ce qu'ils signifient.

1. Contexte et données

Nous utiliserons le jeu de données **NYC Flights 2013**, qui contient environ 336 000 vols au départ de New York en 2013 et plusieurs types de variables :

- numériques : retards, durée de vol, distance ;
- catégorielles : compagnie, aéroport de départ, destination ;
- temporelles ou dérivées du temps : mois, jour, heure ;
- identifiants : numéro de vol et numéro d'avion ;
- valeurs manquantes : certaines heures d'arrivée ou durées ne sont pas renseignées.

Le jeu de données est distribué dans le dépôt public `Rdatasets`, à partir du paquet `nycflights13` :

<https://raw.githubusercontent.com/Rdatasets/nycflights13/master/csv/flights.csv>

La source originale est décrite dans le projet `nycflights13` : <https://github.com/tidyverse/nycflights13>.

⚠ Données réelles, résultats à vérifier

Le fichier est public et adapté à l'apprentissage, mais il ne représente pas tous les vols américains ni tous les aéroports. Les conclusions du Lab portent uniquement sur les vols présents dans ce fichier et sur l'année 2013.

2. Préparer l'environnement

2.1. Créer le projet

Dans un terminal :

```
mkdir cours-analyse-donnees
cd cours-analyse-donnees
uv init --python 3.12
uv add pandas numpy matplotlib seaborn jupyter ipykernel
uv lock
```

La structure attendue est :

```
cours-analyse-donnees/
  pyproject.toml
  uv.lock
  data/
    raw/
    processed/
  notebooks/
  outputs/
```

```
mkdir -p data/raw data/processed notebooks outputs
uv run jupyter lab
```

Sous Windows, la commande de création des dossiers peut être exécutée séparément dans l'explorateur ou avec PowerShell. L'essentiel est de conserver la même organisation.

2.2. Vérifier les bibliothèques

Créez une première cellule dans un notebook :

```
import sys
import numpy as np
import pandas as pd
import matplotlib
import seaborn as sns

print("Python      :", sys.version.split()[0])
print("NumPy      :", np.__version__)
print("pandas      :", pd.__version__)
print("Matplotlib   :", matplotlib.__version__)
print("Seaborn      :", sns.__version__)
```

Si une bibliothèque manque, installez-la avec `uv add nom-de-la-bibliotheque`, puis redémarrez le noyau du notebook.

3. Télécharger et inspecter les données

3.1. Télécharger une copie locale

Ne faites pas dépendre tout le Lab d'une nouvelle requête Internet à chaque exécution. Téléchargez une copie brute et conservez-la.

```
from pathlib import Path
import pandas as pd

URL = (
    "https://raw.githubusercontent.com/Rdatasets/nycflights13/"
    "master/csv/flights.csv"
)

RAW_DIR = Path("../data/raw") # à adapter si le notebook est ailleurs
RAW_DIR.mkdir(parents=True, exist_ok=True)

flights = pd.read_csv(URL)
flights.to_csv(RAW_DIR / "flights_2013.csv", index=False)
```

Si le téléchargement a déjà été effectué :

```
flights = pd.read_csv("../data/raw/flights_2013.csv")
```

Vérifiez le chemin à partir du dossier depuis lequel Jupyter exécute le notebook :

```
from pathlib import Path
print(Path.cwd())
```

3.2. Les premiers réflexes

```
flights.head()
flights.tail()
flights.shape
flights.columns.tolist()
flights.info()
```

`shape` renvoie le nombre de lignes et de colonnes. `info()` indique les types détectés et le nombre de valeurs non manquantes. Ne commencez pas par modifier les données : commencez par les décrire.

3.3. Distinguer les types de données

```
flights.dtypes

numeriques = flights.select_dtypes(include="number").columns.tolist()
textuelles = flights.select_dtypes(include="object").columns.tolist()

print("Variables numériques :", numeriques)
print("Variables textuelles :", textuelles)
```

Dans une analyse, un numéro de vol est numériquement stocké mais n'est pas forcément une grandeur à additionner. Le type Python et le rôle statistique d'une variable ne sont donc pas exactement la même chose.

4. Fréquences et répartitions

4.1. Effectif et fréquence

Commençons par les aéroports de départ :

```
effectifs_origine = flights["origin"].value_counts()
frequences_origine = flights["origin"].value_counts(normalize=True) * 100

repartition_origine = pd.DataFrame({
    "effectif": effectifs_origine,
    "frequence_pct": frequences_origine.round(2),
})
repartition_origine
```

L'**effectif** est un nombre de lignes. La **fréquence** est une proportion, souvent présentée en pourcentage :

$$\text{fréquence en \%} = \frac{\text{effectif de la catégorie}}{\text{effectif total}} \times 100.$$

La somme des fréquences est proche de 100 %. De petits écarts peuvent venir de l'arrondi.

4.2. Fréquences croisées

Pour comparer l'origine et la compagnie :

```
table_compagnie_origine = pd.crosstab(
    flights["carrier"],
    flights["origin"],
    normalize="index",
) * 100

table_compagnie_origine.round(1)
```

`normalize="index"` calcule la composition de chaque compagnie. Si l'on souhaite plutôt connaître la composition de chaque aéroport, on utilise `normalize="columns"`.

4.3. Fréquence d'une condition

La proportion de vols retardés au départ de plus de 15 minutes se calcule ainsi :

```
retards = flights["dep_delay"].dropna()
taux_retard_15 = (retards > 15).mean() * 100
print(f"{taux_retard_15:.2f} %")
```

Dans pandas, une comparaison produit des valeurs **True** et **False**, assimilées à 1 et 0 dans une moyenne. Cette technique est utile, mais il faut annoncer le dénominateur : ici, les vols dont le retard au départ est connu.

5. Moyenne, médiane et quantiles

5.1. Tendance centrale

Pour les retards au départ :

```
dep_delay = flights["dep_delay"].dropna()

moyenne = dep_delay.mean()
mediane = dep_delay.median()
mode = dep_delay.mode()

print("Moyenne :", moyenne)
print("Médiane :", mediane)
print("Mode(s) :", mode.head())
```

- La **moyenne** est la somme divisée par le nombre de valeurs.
- La **médiane** est la valeur qui sépare les données triées en deux groupes.
- Le **mode** est la valeur la plus fréquente ; il est parfois peu intéressant pour une mesure continue comme un retard.

Une moyenne supérieure à la médiane indique souvent une queue de valeurs élevées, mais il faut regarder la distribution avant d'affirmer qu'il existe des valeurs aberrantes.

5.2. Quartiles et quantiles

```
quartiles = dep_delay.quantile([0.25, 0.50, 0.75])
deciles = dep_delay.quantile([0.10, 0.90])

print(quartiles)
print(deciles)
```

Le quantile p est une valeur en dessous de laquelle se trouvent environ $p \times 100$ % des observations :

- Q_1 ou quantile 0,25 : environ 25 % des valeurs sont inférieures ou égales ;
- médiane ou quantile 0,50 : environ 50 % ;
- Q_3 ou quantile 0,75 : environ 75 %.

L'écart interquartile est :

$$IQR = Q_3 - Q_1.$$

Il décrit la largeur de la moitié centrale des données et résiste mieux aux valeurs extrêmes que l'étendue.

5.3. Tableau descriptif

```
resume = flights[[
    "dep_delay",
    "arr_delay",
    "air_time",
    "distance",
]].describe().T

resume
```

La colonne 50% de `describe()` correspond à la médiane. Le tableau doit être lu avec l'unité : minutes pour les retards et la durée, miles pour la distance.

6. Variance et écart-type

6.1. Définitions utiles

La variance mesure la dispersion autour de la moyenne. L'écart-type est sa racine carrée :

$$\text{Var}(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \sigma = \sqrt{\text{Var}(X)}.$$

Dans un échantillon, on utilise souvent $n - 1$ au dénominateur pour estimer la variance d'une population plus large. `pandas` utilise par défaut `ddof=1` :

```
variance_echantillon = dep_delay.var()
ecart_type_echantillon = dep_delay.std()

variance_population = dep_delay.var(ddof=0)
ecart_type_population = dep_delay.std(ddof=0)
```

Pour ce Lab, reprenez surtout deux règles :

1. l'écart-type est exprimé dans la même unité que la variable ;
2. il est sensible aux grandes valeurs, donc on le complète souvent par la médiane et l'IQR.

Un écart-type de 20 minutes ne signifie pas que tous les retards valent moyenne $\pm$ 20. Il indique une dispersion globale, pas une limite stricte.

6.2. Comparer deux groupes

```
resume_origine = (
    flights.groupby("origin")["dep_delay"]
    .agg(
        effectif="count",
        moyenne="mean",
        mediane="median",
        q1=lambda s: s.quantile(0.25),
        q3=lambda s: s.quantile(0.75),
        ecart_type="std",
    )
    .round(2)
)

resume_origine
```

Ne comparez jamais deux moyennes sans regarder les effectifs et la médiane. Un groupe très petit peut présenter une moyenne instable.

7. Taux de croissance

Pour suivre une évolution mensuelle, on calcule d'abord une mesure par mois :

```
volumes_mensuels = flights.groupby("month").size()

taux_croissance = volumes_mensuels.pct_change() * 100

evolution_mensuelle = pd.DataFrame({
    "vols": volumes_mensuels,
    "taux_croissance_pct": taux_croissance.round(2),
})

evolution_mensuelle
```

Le taux de croissance entre une période initiale et une période finale est :

$$\text{croissance en \%} = \frac{\text{valeur finale} - \text{valeur initiale}}{\text{valeur initiale}} \times 100.$$

Le premier mois n'a pas de période précédente dans le tableau : son taux est donc **NaN**. Une baisse est un taux négatif. Une croissance de 10 % suivie d'une baisse de 10 % ne ramène pas exactement à la valeur de départ, car les dénominateurs sont différents.

8. Visualiser pour vérifier

8.1. Distribution des retards

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(dep_delay.clip(-30, 180), bins=40, ax=axes[0])
axes[0].set_title("Distribution des retards au départ")
axes[0].set_xlabel("Retard (minutes)")

sns.boxplot(x=dep_delay, ax=axes[1])
axes[1].set_title("Boîte à moustaches des retards")
axes[1].set_xlabel("Retard (minutes)")

fig.tight_layout()
fig.savefig("../outputs/lab0_distribution_retards.png", dpi=150)
```

Le `clip` limite uniquement l'affichage aux valeurs comprises entre -30 et 180 minutes ; il ne modifie pas la série utilisée pour les calculs précédents. Cette distinction doit être écrite dans le notebook.

8.2. Interpréter sans exagérer

Une bonne interprétation contient :

1. l'indicateur et sa valeur ;
2. l'unité et la population ;
3. une comparaison éventuelle ;
4. une limite ou une question à approfondir.

Exemple de forme attendue :

Pour les vols dont le retard au départ est connu, la médiane est de ____ minutes et la moyenne de ____ minutes. La moyenne est plus élevée, ce qui suggère quelques retards importants. Il faut vérifier la distribution par compagnie et par mois avant d'attribuer cette différence à une cause.

9. Travail demandé

! Consigne générale

Travaillez dans la copie du support téléchargée. Chaque réponse doit être accompagnée du code qui permet de la retrouver. Les résultats numériques peuvent varier si vous modifiez le filtre ou si le fichier téléchargé n'est pas exactement la même version : indiquez toujours votre source et votre filtre.

Partie A — Environnement et prise en main

1. Affichez les versions de Python, NumPy, pandas, Matplotlib et Seaborn.
2. Montrez l'organisation du projet et expliquez le rôle de `pyproject.toml`, `uv.lock`, `data/raw/` et `outputs/`.
3. Téléchargez le fichier et indiquez son nombre de lignes et de colonnes.
4. Listez les colonnes et classez-les en variables numériques, catégorielles, temporelles ou identifiants.

Partie B — Fréquences et types

5. Calculez l'effectif et la fréquence des vols par aéroport de départ.
6. Calculez la fréquence des vols par compagnie, puis affichez uniquement les cinq compagnies les plus fréquentes.
7. Construisez un tableau croisé compagnie/aéroport. Comparez les fréquences par ligne et par colonne.
8. Calculez le taux de vols dont `dep_delay > 15`. Indiquez précisément le dénominateur utilisé.

Partie C — Statistiques descriptives

9. Pour `dep_delay`, calculez la moyenne, la médiane et le mode. Comparez moyenne et médiane en une phrase.
10. Calculez Q_1 , la médiane, Q_3 , le 10e et le 90e quantile.
11. Calculez l'IQR, la variance et l'écart-type avec `ddof=1`. Donnez les unités de chaque résultat.
12. Recalculez variance et écart-type avec `ddof=0`. Expliquez brièvement pourquoi les résultats sont différents.
13. Répétez les indicateurs par aéroport de départ et comparez les trois groupes sans désigner un « meilleur » aéroport sans justification.

Partie D — Évolution et visualisation

14. Calculez le nombre de vols par mois et le taux de croissance mensuel.
15. Produisez un graphique de l'évolution mensuelle du nombre de vols.
16. Produisez un histogramme et une boîte à moustaches des retards.
17. Écrivez un paragraphe de conclusion avec trois chiffres, deux interprétations et une limite.

Partie E — Question d'analyse

18. La question « quelle compagnie est la plus ponctuelle ? » est-elle suffisamment précise ? Reformulez-la en indiquant la mesure, le seuil, la période et le dénominateur.
19. Proposez deux indicateurs complémentaires qui éviteraient de se limiter à la moyenne.
20. Expliquez pourquoi une corrélation éventuelle entre distance et retard ne prouverait pas qu'une variable cause l'autre.

Lab 1 : Collecte de données

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook .ipynb** pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 1 et téléchargez-le.
- **Fichier .qmd** pour travailler avec Quarto : [téléchargez le fichier source du Lab 1](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le .qmd, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Formuler** une question de collecte avant de chercher des données ;
- **Concevoir** un questionnaire court, compréhensible et respectueux des personnes ;
- **Interroger** une API REST avec **requests** et traiter une réponse JSON ;
- **Télécharger** un fichier public et conserver sa source ;
- **Documenter** la provenance, la période, les limites et les transformations d'un jeu de données ;
- **Construire** un petit catalogue de données réutilisable dans les Labs suivants.

i Fil rouge des Labs 1 à 5

Nous utiliserons progressivement les données ouvertes **NYC 311 Service Requests** pour construire une solution de Business Intelligence. Ce service contient des demandes adressées à la ville de New York : type de problème, agence, quartier, dates de création et de clôture, canal de transmission et localisation. Le volume est important et permet de comparer une exploration pédagogique à une solution destinée à des données massives.

1. Une collecte commence par une question

Une collecte ne consiste pas à télécharger tout ce qui est disponible. Avant de choisir une source, écrivez :

1. la question à laquelle vous voulez répondre ;
2. la population étudiée ;
3. la période ;
4. les variables nécessaires ;
5. les contraintes de qualité, de droit et de confidentialité.

Exemple de question pour le fil rouge :

Comment le volume et le délai de traitement des demandes 311 évoluent-ils selon le type de problème, le quartier et le canal de transmission ?

Cette question nécessite notamment une date de création, une date de clôture, un type de demande, un quartier, un statut et un canal. Une variable comme le nom d'un agent n'est pas nécessaire.

2. Les trois familles de collecte

2.1. Questionnaire

Un questionnaire est utile lorsque l'information n'existe pas encore dans une base ou lorsque l'on cherche une perception, une intention ou une expérience vécue. Il faut distinguer :

- une question **fermée**, facile à compter et à comparer ;
- une question **ouverte**, plus riche mais plus difficile à analyser ;
- une échelle ordinale, par exemple de 1 à 5, dont les catégories doivent être expliquées.

Un questionnaire court doit comporter un titre, une introduction, la durée estimée, l'usage des réponses et une possibilité de ne pas répondre à une question non obligatoire.

Protection des personnes

Pour ce Lab, ne collectez pas de nom, numéro de téléphone, adresse précise, mot de passe, information médicale ou autre donnée sensible. Utilisez des réponses anonymes et demandez un consentement clair. Dans un projet réel, la collecte doit respecter les règles de l'établissement et la réglementation applicable.

2.2. API

Une API est une interface qui renvoie des données structurées. Une requête REST utilise souvent :

- une URL de ressource ;
- des paramètres de filtrage ;
- une limite et parfois un décalage de pagination ;
- une réponse JSON ;
- un code HTTP à vérifier.

Une réponse 200 signifie que la requête a été traitée ; cela ne garantit pas que les données sont complètes ou correctes. Il faut lire les métadonnées, les noms de champs et la période.

2.3. Fichier ou page Internet

Un fichier CSV, JSON, Excel ou Parquet peut être téléchargé depuis un portail open data. Une page HTML peut parfois être lue, mais le scraping doit respecter les conditions d'utilisation du site, **robots.txt**, la fréquence raisonnable des requêtes et la protection des données.

La source doit être enregistrée avec :

```
nom de la source
URL
date et heure de téléchargement
période couverte
filtres utilisés
nombre de lignes téléchargées
colonnes conservées
limites connues
```

3. Préparer le projet de collecte

```
uv add pandas requests beautifulsoup4 pyarrow
mkdir -p data/raw data/metadata notebooks outputs
```

Une structure minimale :

```
data/
  raw/           # données telles que reçues
  metadata/      # sources, dictionnaires, journaux
  processed/     # optionnel : données transformées plus tard
```

Conservez les données brutes sans les modifier. Si un fichier est trop volumineux pour Git, conservez au minimum le script de téléchargement, l'URL, les paramètres et un échantillon documenté.

4. Partie A — Concevoir un mini-questionnaire

4.1. Sujet

Le questionnaire porte sur l'utilisation des outils numériques par les étudiants pour apprendre l'analyse de données. Il ne doit pas recueillir d'identité.

Proposition de questions :

1. Combien d'heures par semaine consacrez-vous au travail personnel ?
2. Quel environnement utilisez-vous principalement ? JupyterLab, VS Code, Google Colab, autre.
3. À quel niveau évaluez-vous votre aisance avec Python ? *débutant, intermédiaire, à l'aise*.
4. Quelle difficulté rencontrez-vous le plus souvent ? question à choix unique.
5. Quelle amélioration proposeriez-vous ? question ouverte facultative.

Ne mettez pas en relation les réponses avec un nom. Si la classe est très petite, évitez de publier des croisements qui permettraient de reconnaître une personne.

4.2. Export et lecture

Après export en CSV, placez le fichier dans `data/raw/questionnaire_outils.csv` :

```
import pandas as pd

questionnaire = pd.read_csv("../data/raw/questionnaire_outils.csv")
questionnaire.head()
questionnaire.info()
questionnaire["environnement"].value_counts(dropna=False)
```

Vérifiez les valeurs autorisées, les réponses manquantes et la présence éventuelle de colonnes ajoutées automatiquement par l'outil de formulaire.

5. Partie B — Interroger l'API NYC 311

5.1. Découvrir la ressource

Le portail public de New York expose les demandes 311 via l'API Socrata. L'endpoint utilisé dans ce Lab est :

<https://data.cityofnewyork.us/resource/erm2-nwe9.json>

Une première requête limitée permet de comprendre la structure :

```
import requests

endpoint = "https://data.cityofnewyork.us/resource/erm2-nwe9.json"

params = {
    "$limit": 5,
    "$order": "created_date DESC",
}

response = requests.get(endpoint, params=params, timeout=30)
response.raise_for_status()
sample = response.json()
sample[:1]
```

Transformez ensuite la réponse en tableau :

```
import pandas as pd

requests_sample = pd.json_normalize(sample)
requests_sample.columns.tolist()
requests_sample.to_csv("../data/raw/nyc311_sample.csv", index=False)
```

5.2. Filtrer côté serveur

Il vaut mieux demander au serveur uniquement les lignes et colonnes nécessaires :

```
params = {
    "$select": (
        "unique_key,created_date,closed_date,agency,"
        "complaint_type,borough,open_data_channel_type,status"
    ),
    "$where": (
        "created_date between '2024-01-01T00:00:00' "
        "and '2024-01-31T23:59:59'"
    ),
    "$order": "created_date, unique_key",
    "$limit": 50000,
}

response = requests.get(endpoint, params=params, timeout=60)
response.raise_for_status()
nyc311_janvier = pd.DataFrame(response.json())
```

Le serveur peut renvoyer moins de lignes que prévu si le filtre est mal écrit ou si la limite est atteinte. Comparez le nombre obtenu avec une requête de comptage :

```
count_params = {
    "$select": "count(*) as nombre",
    "$where": (
        "created_date between '2024-01-01T00:00:00' "
        "and '2024-01-31T23:59:59'"
    ),
}

count_response = requests.get(endpoint, params=count_params, timeout=60)
count_response.raise_for_status()
count_response.json()
```

5.3. Pagination

Pour télécharger davantage de lignes, utilisez `$limit` et `$offset`. Ne lancez pas une collecte massive sans définir une période et sans mesurer le volume.

```
def collect_page(endpoint, where, limit=50_000, offset=0):
    params = {
        "$select": (
            "unique_key,created_date,closed_date,agency,"
            "complaint_type,borough,open_data_channel_type,status"
        ),
        "$where": where,
        "$order": "created_date, unique_key",
        "$limit": limit,
        "$offset": offset,
    }
    response = requests.get(endpoint, params=params, timeout=120)
    response.raise_for_status()
    return response.json()

where = (
    "created_date between '2024-01-01T00:00:00' "
    "and '2024-01-31T23:59:59'"
)

nombre_attendu = int(count_response.json()[0]["nombre"])
pages = []
for offset in range(0, nombre_attendu, 50_000):
    page = collect_page(endpoint, where, offset=offset)
    if not page:
        break
    pages.extend(page)

nyc311 = pd.DataFrame(pages)
nyc311.to_csv("../data/raw/nyc311_janvier.csv", index=False)
print(f"Lignes collectées : {len(nyc311):,} / {nombre_attendu:,} attendues")
```

L'ordre stable par date et identifiant évite de décaler les pages lorsque plusieurs demandes ont la même date. Si le nombre collecté diffère du comptage, notez l'écart et vérifiez si la source a changé pendant l'extraction.

Pour une collecte destinée à un usage régulier, sauvegardez aussi les paramètres et le nombre de lignes dans un fichier de métadonnées :

```
import json
from datetime import datetime, timezone

metadata = {
    "source": "NYC 311 Service Requests",
    "url": endpoint,
    "downloaded_at_utc": datetime.now(timezone.utc).isoformat(),
    "where": where,
    "rows": len(nyc311),
    "columns": nyc311.columns.tolist(),
}

with open("../data/metadata/nyc311_janvier.json", "w", encoding="utf-8") as f:
    json.dump(metadata, f, indent=2, ensure_ascii=False)
```

6. Partie C — Une donnée publique téléchargée sur Internet

Choisissez une source publique complémentaire, par exemple une météo, une population ou une donnée économique. La source doit être :

- accessible sans identifiant personnel ;
- accompagnée d'une documentation ;
- téléchargeable sous forme de CSV, JSON ou Parquet ;
- suffisamment claire pour préciser la période et les unités.

Téléchargez une copie et écrivez une fiche :

```
# Fiche source
- Nom :
- URL :
- Date de téléchargement :
- Période :
- Nombre de lignes :
- Variables principales :
- Unités :
- Valeurs manquantes :
- Licence ou conditions d'utilisation :
- Limites :
```

La fiche est aussi importante que le fichier : sans elle, une autre personne ne sait pas comment les données ont été obtenues.

7. Contrôle de la collecte

Avant de passer à l'analyse, répondez à ces questions :

- La source est-elle bien celle annoncée ?
- Le nombre de lignes est-il plausible ?
- La période couvre-t-elle la question ?
- Les colonnes ont-elles les types attendus ?
- Y a-t-il des identifiants dupliqués ?
- Les dates sont-elles dans le même fuseau ou format ?
- Les valeurs manquantes sont-elles documentées ?
- La collecte peut-elle être rejouée demain ?

Une collecte de données est réussie lorsque quelqu'un d'autre peut comprendre et reproduire le chemin entre la source et le fichier brut.

8. Travail demandé

Partie A — Questionnaire

1. Rédigez une question de recherche et quatre à six questions de questionnaire.
2. Justifiez le type de chaque question : fermée, ordinale, numérique ou ouverte.
3. Expliquez quelles informations vous avez volontairement exclues pour protéger les répondants.
4. Exportez les réponses anonymisées et donnez le nombre de lignes, de colonnes et de réponses manquantes.

Partie B — API NYC 311

5. Téléchargez 5 lignes et décrivez au moins huit colonnes disponibles.
6. Téléchargez les demandes d'un mois précis en conservant au moins huit colonnes utiles au fil rouge.
7. Comparez le nombre retourné avec une requête `count(*)`.
8. Ajoutez une pagination et sauvegardez les paramètres de collecte.
9. Calculez le nombre de demandes par `borough` et par `complaint_type`.
10. Écrivez deux limites de cette collecte : période, pagination, qualité ou interprétation. Notez aussi la date de collecte, les paramètres de requête utilisés et le nombre de lignes effectivement obtenu.

Partie C — Source Internet complémentaire

11. Choisissez un fichier ou une API complémentaire.
12. Rédigez sa fiche source et indiquez sa licence ou ses conditions d'utilisation.
13. Comparez les types de variables de cette source avec ceux de NYC 311.
14. Expliquez si une jointure entre les deux sources est réellement possible et quelle clé commune serait nécessaire.

Partie D — Réflexion

15. Pourquoi est-il préférable de filtrer côté serveur lorsque la donnée est massive ?
16. Quelle différence faites-vous entre la donnée brute et la donnée traitée ?
17. Qu'est-ce qu'une collecte complète pour une question donnée ?
18. Donnez un exemple de biais possible dans un questionnaire et un exemple de biais possible dans une API.

Lab 2 : EDA et prétraitement

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook** `.ipynb` pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 2 et téléchargez-le.
- **Fichier** `.qmd` pour travailler avec Quarto : [téléchargez le fichier source du Lab 2](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le `.qmd`, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Auditer** un jeu de données avec une démarche structurée ;
- **Mesurer** les valeurs manquantes, doublons, types incorrects et catégories incohérentes ;
- **Transformer** les dates et créer des variables utiles à la question métier ;
- **Explorer** une variable seule, deux variables et plusieurs variables ;
- **Décider** quoi corriger, conserver, exclure ou documenter ;
- **Produire** une table propre et un rapport EDA reproductible.

! Idée centrale

Dans de nombreux projets, la plus grande partie du temps n'est pas consacrée aux graphiques finaux. Elle est consacrée à comprendre les sources, vérifier les hypothèses, corriger les formats, traiter les valeurs manquantes, contrôler les résultats et recommencer. Ce Lab transforme les données collectées au Lab 1 en données analysables, sans masquer les problèmes.

1. Le fichier de départ

Utilisez `data/raw/nyc311_janvier.csv` obtenu au Lab 1. Si vous n'avez pas réussi la collecte, utilisez un export documenté du même endpoint et indiquez la période.

Les colonnes minimales attendues sont :

Colonne	Rôle
<code>unique_key</code>	Identifiant de la demande
<code>created_date</code>	Date de création
<code>closed_date</code>	Date de clôture éventuelle
<code>agency</code>	Agence responsable
<code>complaint_type</code>	Type de problème déclaré
<code>borough</code>	Arrondissement
<code>open_data_channel_type</code>	Canal de transmission
<code>status</code>	Statut de la demande

La question d'analyse est :

Quels types de demandes sont les plus fréquents et quels facteurs semblent associés à un délai de traitement plus long ?

Le mot **associé** est volontaire : l'EDA peut repérer des différences et des relations, mais elle ne prouve pas une cause.

2. Installer et charger proprement

```
uv add pandas numpy matplotlib seaborn pyarrow
```

Dans le notebook :

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

RAW_PATH = Path("../data/raw/nyc311_janvier.csv")
PROCESSED_PATH = Path("../data/processed/nyc311_janvier_clean.csv")

raw = pd.read_csv(RAW_PATH)
df = raw.copy()

print(df.shape)
df.head()
```

Gardez deux objets conceptuellement séparés :

- `raw` : copie de la donnée telle qu'elle a été reçue ;
- `df` : copie de travail, transformée par des étapes documentées.

3. Audit initial

3.1. Dimensions et types

```
df.shape
df.columns.tolist()
df.info()
df.describe(include="all").T
```

Demandez-vous pour chaque colonne :

- Est-elle nécessaire à la question ?
- Est-elle une mesure, une catégorie, une date ou un identifiant ?
- Son type pandas correspond-il à son rôle ?
- Son nom et son unité sont-ils explicites ?

3.2. Valeurs manquantes

```
missing = (
    df.isna()
    .agg(["sum", "mean"])
    .T
    .rename(columns={"sum": "nombre", "mean": "proportion"})
)
missing["pourcentage"] = missing["proportion"] * 100
missing.sort_values("pourcentage", ascending=False)
```

Une valeur manquante dans `closed_date` peut signifier qu'une demande est encore ouverte. Elle n'a donc pas la même signification qu'une date de création manquante. Ne remplacez pas toutes les absences par zéro.

3.3. Unicité et doublons

```
df["unique_key"].nunique()
len(df)
df["unique_key"].duplicated().sum()
df.duplicated().sum()
```

Un doublon exact et deux lignes portant le même identifiant ne sont pas nécessairement le même problème. Examinez les lignes concernées avant de supprimer :

```
doublons_id = df[df["unique_key"].duplicated(keep=False)]
doublons_id.sort_values("unique_key").head(20)
```

4. Convertir et valider

4.1. Dates

Pour NYC 311, les dates fournies sans décalage explicite correspondent à l'heure locale de New York. Il ne faut donc pas leur ajouter `utc=True` au moment de les lire : cela les interpréterait à tort comme des heures UTC. Le code ci-dessous localise les dates naïves à New York et convertit en heure de New York les dates qui contiennent déjà un fuseau. Les rares heures ambiguës ou inexistantes lors du changement d'heure sont marquées manquantes et doivent être comptabilisées dans l'audit.

```
NYC_TZ = "America/New_York"

for column in ("created_date", "closed_date"):
    dates = pd.to_datetime(df[column], errors="coerce")
    if dates.dt.tz is None:
        dates = dates.dt.tz_localize(
            NYC_TZ,
            ambiguous="NaT",
            nonexistent="NaT",
        )
    else:
        dates = dates.dt.tz_convert(NYC_TZ)
    df[column] = dates
```

`errors="coerce"` transforme les dates illisibles en `NaT`, ce qui permet de les compter. Il faut ensuite les examiner, pas les oublier.

```
print(df["created_date"].isna().sum())
print(df["closed_date"].isna().sum())
print(df["created_date"].min(), df["created_date"].max())
```

4.2. Catégories textuelles

Les catégories peuvent contenir des espaces ou des valeurs vides :

```
categorical_columns = [
    "agency",
    "complaint_type",
    "borough",
    "open_data_channel_type",
    "status",
]

for column in categorical_columns:
    df[column] = df[column].astype("string").str.strip()
    df[column] = df[column].replace({"": pd.NA, "nan": pd.NA})
```

Ne remplacez pas une catégorie par une autre sans conserver la règle. Si deux libellés désignent le même concept, écrivez une table de correspondance :

```
mapping_borough = {
    "BRONX": "Bronx",
    "BROOKLYN": "Brooklyn",
    "MANHATTAN": "Manhattan",
    "QUEENS": "Queens",
    "STATEN ISLAND": "Staten Island",
}

df["borough_normalized"] = df["borough"].map(mapping_borough)
```

Les valeurs non reconnues deviennent `NaN` dans la nouvelle colonne : elles doivent être inspectées avant décision.

4.3. Règles de validité

Construisez des indicateurs de contrôle plutôt que de supprimer directement :

```
df["date_invalide"] = df["created_date"].isna()
df["cloture_avant_creation"] = (
    df["closed_date"].notna()
    & df["created_date"].notna()
    & (df["closed_date"] < df["created_date"])
)

df["statut_incoherent"] = (
    df["closed_date"].isna()
    & df["status"].eq("Closed")
)
```

```
df[[
    "date_invalide",
    "cloture_avant_creation",
    "statut_incoherent",
]].sum()
```

Une règle de qualité produit d'abord un signal. La correction dépend ensuite de la cause et de la question.

5. Créer les variables d'analyse

5.1. Délai de traitement

```
df["processing_hours"] = (
    df["closed_date"] - df["created_date"]
).dt.total_seconds() / 3600
```

Les demandes encore ouvertes ont un délai manquant. Les délais négatifs sont impossibles dans cette définition et doivent être contrôlés :

```
df["processing_hours"].describe()
df[df["processing_hours"] < 0][
    ["unique_key", "created_date", "closed_date", "processing_hours"]
]
```

Ne supprimez pas les grandes valeurs uniquement parce qu'elles sont grandes : une demande réellement restée ouverte longtemps peut être précisément un résultat important.

5.2. Variables temporelles

```
df["created_day"] = df["created_date"].dt.day
df["created_weekday"] = df["created_date"].dt.day_name()
df["created_hour"] = df["created_date"].dt.hour
df["created_week"] = df["created_date"].dt.isocalendar().week.astype("int")
```

Pour comparer les jours, tenez compte du volume : un lundi avec beaucoup de demandes peut avoir plus de demandes longues simplement parce qu'il y a plus de dossiers.

5.3. Catégories rares

```
top_types = df["complaint_type"].value_counts().head(15)
top_types
```

Pour un graphique lisible, vous pouvez afficher les quinze catégories les plus fréquentes et regrouper les autres sous « Autres ». Cette opération est une transformation d'affichage : ne détruisez pas la catégorie originale.

6. EDA univariée

6.1. Variables catégorielles

```
top_types.sort_values().plot(
    kind="barh",
    figsize=(8, 6),
    title="15 types de demandes les plus fréquents",
)
plt.xlabel("Nombre de demandes")
plt.tight_layout()
plt.savefig("../outputs/lab2_top_types.png", dpi=150)
```

Décrivez le volume, la concentration et les catégories rares. Une catégorie fréquente n'est pas nécessairement celle qui a le délai le plus long.

6.2. Variables numériques

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(
    data=df,
    x="processing_hours",
    bins=50,
    ax=axes[0],
)
sns.boxplot(
    data=df,
    x="processing_hours",
    ax=axes[1],
)
axes[0].set_xlim(0, df["processing_hours"].quantile(0.99))
axes[0].set_title("Délai : vue limitée au 99e percentile")
axes[1].set_title("Délai : valeurs extrêmes visibles")
plt.tight_layout()
plt.savefig("../outputs/lab2_processing_distribution.png", dpi=150)
```

La limitation de l'axe sert à lire la majorité des observations ; les calculs doivent rester faits sur la variable complète. Indiquez toujours cette différence dans le commentaire du graphique.

7. EDA bivariée et multivariée

7.1. Type de demande et délai

```
top_type_names = top_types.index
delay_by_type = (
    df[df["complaint_type"].isin(top_type_names)]
    .groupby("complaint_type")["processing_hours"]
    .agg(
        demandes="count",
        mediane="median",
    )
)
```

```

        moyenne="mean",
        q1=lambda s: s.quantile(0.25),
        q3=lambda s: s.quantile(0.75),
    )
    .sort_values("mediane", ascending=False)
)

delay_by_type.head(10)

```

Le nombre de demandes est indispensable pour interpréter une médiane. Un type avec 20 observations n'offre pas le même niveau de confiance descriptif qu'un type avec 20 000 observations.

7.2. Quartier et canal

```

pd.crosstab(
    df["borough"],
    df["open_data_channel_type"],
    normalize="index",
).round(3)

```

Cette table compare la composition des canaux dans chaque quartier. Elle ne mesure pas l'efficacité du canal : il faudrait une mesure de volume, de délai et une définition de la population comparable.

7.3. Volume dans le temps

```

monthly = (
    df.set_index("created_date")
    .resample("D")
    .size()
    .rename("requests")
)

monthly.plot(figsize=(10, 4), title="Demandes créées par jour")
plt.ylabel("Nombre de demandes")
plt.tight_layout()

```

Repérez les jours atypiques, mais ne les expliquez pas sans source complémentaire. Un pic peut venir d'un événement réel, d'une campagne de communication ou d'un changement de système.

8. Traiter les données avec une règle explicite

Pour créer un fichier de travail :

```

analysis_columns = [
    "unique_key",
    "created_date",
    "closed_date",
    "agency",
    "complaint_type",
    "borough_normalized",

```

```
"open_data_channel_type",
"status",
"processing_hours",
"created_weekday",
"created_hour",
]

analysis = df[analysis_columns].copy()

analysis = analysis[
    analysis["unique_key"].notna()
    & analysis["created_date"].notna()
]

analysis.to_csv(PROCESSED_PATH, index=False)
```

Écrivez ensuite dans `data/metadata/nyc311_cleaning_log.md` :

```
# Journal de préparation
- Fichier source :
- Période :
- Lignes initiales :
- Lignes finales :
- Colonnes supprimées :
- Dates converties :
- Valeurs manquantes conservées :
- Lignes exclues et raison :
- Variables dérivées :
- Limites :
```

Le fichier traité n'est pas « la vérité » : c'est une vue adaptée à une question. Une autre question peut nécessiter d'autres colonnes ou un autre filtre.

9. Données volumineuses

Lorsque le CSV devient trop grand pour la mémoire :

```
chunks = []

for chunk in pd.read_csv(
    "../data/raw/nyc311_2024.csv",
    usecols=[
        "created_date",
        "closed_date",
        "complaint_type",
        "borough",
    ],
    chunksize=100_000,
):
    chunk["created_date"] = pd.to_datetime(
        chunk["created_date"],
        errors="coerce",
    )
    chunk["closed_date"] = pd.to_datetime(
        chunk["closed_date"],
```

```

        errors="coerce",
    )
    chunk["processing_hours"] = (
        chunk["closed_date"] - chunk["created_date"]
    ).dt.total_seconds() / 3600
    chunks.append(chunk)

analysis_large = pd.concat(chunks, ignore_index=True)

```

Pour plusieurs millions de lignes, ne concaténez pas systématiquement toutes les données si vous avez seulement besoin d'un agrégat. Accumulez par exemple des comptes par mois et par catégorie, ou utilisez DuckDB/Parquet dans un Lab ultérieur.

10. Travail demandé

Partie A — Audit

1. Chargez le fichier brut et indiquez son nombre de lignes et de colonnes.
2. Produisez un tableau des types, valeurs manquantes et taux de valeurs manquantes.
3. Vérifiez l'unicité de `unique_key` et examinez les doublons éventuels.
4. Écrivez cinq règles de qualité adaptées à ce fichier.

Partie B — Prétraitement

5. Convertissez les dates et comptez les valeurs devenues `NaT`.
6. Normalisez les colonnes textuelles en conservant les valeurs originales.
7. Créez `processing_hours` et identifiez les délais négatifs.
8. Créez au moins trois variables temporelles.
9. Décidez comment traiter les demandes ouvertes pour l'analyse du délai. Justifiez.
10. Écrivez un journal de préparation sans supprimer silencieusement les erreurs.

Partie C — Exploration

11. Produisez le top 15 des types de demandes.
12. Comparez la moyenne, la médiane, Q_1 et Q_3 du délai pour cinq types fréquents.
13. Comparez les délais par arrondissement avec les effectifs.
14. Construisez une table croisée arrondissement/canal.
15. Produisez un graphique de volume quotidien ou hebdomadaire.
16. Ajoutez une boîte à moustaches par arrondissement.

Partie D — Interprétation

17. Donnez deux observations descriptives vérifiables par un chiffre.
18. Donnez une interprétation que les données ne permettent pas de soutenir.
19. Expliquez pourquoi une catégorie rare peut avoir une moyenne instable.
20. Proposez deux données complémentaires utiles à une analyse diagnostique.

Partie E — Préparer la suite

21. Sauvegardez une table propre et une table de synthèse mensuelle.
22. Rédigez le dictionnaire des colonnes destinées au Lab 3.
23. Identifiez trois KPI possibles pour le futur tableau de bord.
24. Expliquez quelles colonnes devront être des dimensions et lesquelles pourront être des mesures.

Lab 3 : Visualiser les données 311

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook** `.ipynb` pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 3 et téléchargez-le.
- **Fichier** `.qmd` pour travailler avec Quarto : [téléchargez le fichier source du Lab 3](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le `.qmd`, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Choisir** un type de graphique adapté à une question de distribution, de comparaison, de relation, d'évolution ou de composition ;
- **Construire** les graphiques essentiels avec `matplotlib` et `seaborn` à partir d'une table déjà nettoyée ;
- **Repérer et corriger** un graphique trompeur (axe tronqué, camembert surchargé, échelle mal choisie) ;
- **Produire** un premier graphique interactif avec Plotly ;
- **Concevoir** une figure de communication complète : titre-message, annotation, source, palette accessible ;
- **Organiser** un petit portfolio de graphiques reproductible, prêt à être présenté à un public non technique.

! Idée centrale

Le Lab 2 a produit une table propre. Ce Lab ne rajoute aucune nouvelle donnée : il transforme cette table en une série de graphiques qui **répondent chacun à une question précise**, en respectant les principes vus au chapitre sur la visualisation des données. Un graphique qui ne répond à aucune question n'a pas sa place dans ce Lab.

1. Le fichier de départ

Utilisez `data/processed/nyc311_janvier_clean.csv`, produit au Lab 2. Si vous ne l'avez pas, reprenez `data/raw/nyc311_janvier.csv` et appliquez au minimum la conversion des dates et le calcul de `processing_hours` (voir Lab 2, sections 4 et 5).

Rappel des colonnes disponibles :

Colonne	Rôle
<code>unique_key</code>	Identifiant de la demande
<code>created_date</code> , <code>closed_date</code>	Dates de création et de clôture
<code>agency</code>	Agence responsable
<code>complaint_type</code>	Type de problème déclaré

Colonne	Rôle
borough_normalized	Arrondissement, libellé normalisé
open_data_channel_type	Canal de transmission
status	Statut de la demande
processing_hours	Délai de traitement, en heures
created_weekday, created_hour	Jour de la semaine et heure de création

La question de communication de ce Lab est directement héritée du Lab 1 :

Quels types de demandes sont les plus fréquents, et quels facteurs semblent associés à un délai de traitement plus long ?

Le Lab 2 a permis d'explorer cette question avec des chiffres. Ce Lab la traduit en **graphiques présentables**.

2. Installer et préparer

```
uv add matplotlib seaborn plotly
```

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

PROCESSED_PATH = Path("../data/processed/nyc311_janvier_clean.csv")
OUTPUT_DIR = Path("../outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

df = pd.read_csv(PROCESSED_PATH, parse_dates=["created_date", "closed_date"])
df.shape
```

Les dates du fichier sont exprimées en heure locale de New York (voir le Lab 2). Pour les graphiques de délai, ne gardez que les demandes clôturées avec un délai valide : une demande encore ouverte n'a pas encore de durée complète observable.

```
df_delais = df.loc[
    df["processing_hours"].notna()
    & (df["processing_hours"] >= 0)
].copy()

print(f"Demandes dans le fichier : {df['unique_key'].nunique():,}")
print(f"Demandes avec délai valide : {df_delais['unique_key'].nunique():,}")
```

Adoptez dès maintenant une palette accessible pour l'ensemble du Lab, conformément aux bonnes pratiques du chapitre 3 :

```
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")
```

i Un dossier, un graphique, un nom explicite

Enregistrez chaque figure dans `outputs/` avec un nom qui décrit son contenu, par exemple `lab3_histogramme_delai.png`, plutôt que `figure1.png`. Vous devrez retrouver rapidement quel fichier correspond à quelle question.

3. Distribution : à quoi ressemble le délai de traitement ?

Reprenez la variable `processing_hours` déjà calculée au Lab 2 et construisez un histogramme et une boîte à moustaches, côte à côte, comme vu au chapitre 3 (§4.1 et §4.6) :

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(data=df_delais, x="processing_hours", bins=50, ax=axes[0])
axes[0].set_xlim(0, df_delais["processing_hours"].quantile(0.99))
axes[0].set_title("Distribution du délai de traitement (99e percentile)")
axes[0].set_xlabel("Délai (heures)")

sns.boxplot(data=df_delais, x="processing_hours", ax=axes[1])
axes[1].set_title("Délai de traitement - valeurs extrêmes visibles")
axes[1].set_xlabel("Délai (heures)")

plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_distribution_delai.png", dpi=150)
```

! Question de vérification

Pourquoi l'histogramme est-il limité au 99 percentile alors que la boîte à moustaches montre toute la variable ? Rédigez une phrase de légende qui explique cette différence à un lecteur non technique.

4. Comparaison : quels types de demandes dominant ?

Construisez un diagramme en barres **horizontales** et **triées**, conformément à la bonne pratique du §4.2 :

```
top_types = df["complaint_type"].value_counts().head(12).sort_values()

fig, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x=top_types.values, y=top_types.index, ax=ax, orient="h")
ax.set_title("Les 12 types de demandes les plus fréquents - janvier")
ax.set_xlabel("Nombre de demandes")
ax.set_ylabel("")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_top_types_barres.png", dpi=150)
```

4.1. Contre-exemple volontaire : le camembert

Construisez maintenant un camembert des huit types de demandes les plus fréquents, à comparer au diagramme en barres :

```
parts = df["complaint_type"].value_counts().head(8)

plt.figure(figsize=(6, 6))
plt.pie(parts.values, labels=parts.index, autopct="%1.0f%%")
plt.title("Répartition des 8 types de demandes les plus fréquents")
plt.savefig(OUTPUT_DIR / "lab3_camembert_a_critiquer.png", dpi=150)
```

⚠ À faire

Comparez ce camembert au diagramme en barres de la section 4. Lequel des deux permet de répondre le plus vite à la question « quel est le troisième type de demande le plus fréquent, et de combien dépasse-t-il le quatrième ? ». Notez votre réponse pour la partie C du travail demandé.

5. Relation : le délai dépend-il du moment de la journée ?

Construisez un nuage de points reliant l'heure de création et le délai de traitement, en limitant l'axe des délais pour rester lisible :

```
echantillon = df_delais.sample(n=min(5000, len(df_delais)), random_state=42)

fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(
    data=echantillon,
    x="created_hour",
    y="processing_hours",
    alpha=0.3,
    ax=ax,
)
ax.set_ylim(0, df_delais["processing_hours"].quantile(0.95))
ax.set_title("Délai de traitement selon l'heure de création de la demande")
ax.set_xlabel("Heure de création")
ax.set_ylabel("Délai (heures)")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_nuage_heure_delai.png", dpi=150)
```

i Pourquoi échantillonner ?

Un nuage de points de plusieurs centaines de milliers de points devient illisible (une bouillie de couleur) et lent à afficher. L'échantillonnage (`sample`) et la transparence (`alpha`) sont deux réponses complémentaires à ce problème — retenez les deux.

Complétez par une matrice de corrélation entre les variables numériques disponibles, sous forme de heatmap (§4.7) :

```
numeriques = df_delais[["processing_hours", "created_hour"]].corr()

plt.figure(figsize=(4, 3))
sns.heatmap(numeriques, annot=True, cmap="coolwarm", center=0, vmin=-1, vmax=1)
plt.title("Corrélation entre l'heure de création et le délai")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_heatmap_correlation.png", dpi=150)
```

6. Évolution : comment le volume de demandes varie-t-il dans le temps ?

```
volume_quotidien = (
    df.set_index("created_date")
    .resample("D")
    .size()
    .rename("nombre_demandes")
)

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(volume_quotidien.index, volume_quotidien.values, marker="o", markersize=3)
ax.set_title("Nombre de demandes créées par jour - janvier")
ax.set_xlabel("Date")
ax.set_ylabel("Nombre de demandes")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_evolution_quotidienne.png", dpi=150)
```

⚠ Rappel du chapitre 3

Une courbe suppose un ordre naturel entre les points : ici, le temps. Ne l'utilisez jamais pour relier des catégories comme les types de demandes ou les arrondissements.

7. Composition et multivarié : délai par arrondissement et par canal

Une boîte à moustaches par arrondissement, pour comparer des **distributions** plutôt que de simples moyennes :

```
fig, ax = plt.subplots(figsize=(9, 5))
sns.boxplot(
    data=df_delais,
    x="borough_normalized",
    y="processing_hours",
    ax=ax,
)
ax.set_ylim(0, df_delais["processing_hours"].quantile(0.95))
ax.set_title("Délai de traitement par arrondissement")
ax.set_xlabel("")
ax.set_ylabel("Délai (heures)")
plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_boxplot_arrondissement.png", dpi=150)
```

Puis une heatmap croisant arrondissement et canal de transmission, à partir d'un tableau croisé (vu au chapitre 2) :

```
tableau = pd.crosstab(df["borough_normalized"], df["open_data_channel_type"], normalize="index")

plt.figure(figsize=(7, 5))
sns.heatmap(tableau, annot=True, fmt=".0%", cmap="Blues")
plt.title("Répartition des canaux de transmission par arrondissement")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_heatmap_canal_arrondissement.png", dpi=150)
```

8. Un premier graphique interactif

Reproduisez le nuage de points de la section 5 avec Plotly, pour découvrir l'infobulle interactive présentée au chapitre 3 (§5.3) :

```
import plotly.express as px

fig = px.scatter(
    echantillon,
    x="created_hour",
    y="processing_hours",
    color="borough_normalized",
    hover_data=["complaint_type", "status"],
    title="Délai de traitement selon l'heure de création (interactif)",
    labels={"created_hour": "Heure de création", "processing_hours": "Délai (heures)"},
)
fig.write_html(str(OUTPUT_DIR / "lab3_scatter_interactif.html"))
```

Note

Ouvrez le fichier .html obtenu dans un navigateur : vous pouvez zoomer, filtrer par arrondissement en cliquant sur la légende, et lire le détail d'un point au survol. Aucune de ces actions n'est possible avec la version matplotlib.

9. Construire une figure de communication complète

Choisissez **un seul** graphique parmi ceux produits aux sections 3 à 7 — celui qui, selon vous, porte le message le plus utile — et transformez-le en véritable figure de communication, en appliquant l'anatomie d'un bon graphique (chapitre 3, §6.1) :

1. Un **titre-message** (pas un titre-thème) : reformulez le titre pour qu'il énonce une conclusion.
2. Une **annotation** directe sur le point ou la barre la plus importante (`ax.annotate(...)`).
3. Une **mention de la source et de la période** en petit texte sous le graphique.
4. Une **palette accessible** (déjà appliquée via `sns.set_palette("colorblind")`).

```
fig, ax = plt.subplots(figsize=(9, 5))
sns.barplot(x=top_types.values, y=top_types.index, ax=ax)
ax.set_title("Les demandes de bruit dominant largement le service 311 en janvier")
ax.set_xlabel("Nombre de demandes")
ax.set_ylabel("")
ax.annotate(
    "Type le plus fréquent",
    xy=(top_types.values[-1], len(top_types) - 1),
    xytext=(top_types.values[-1] * 0.6, len(top_types) - 3),
    arrowprops=dict(arrowstyle="->"),
)
fig.text(0.01, -0.02, "Source : NYC 311 Service Requests - janvier 2025", fontsize=8, color="gray")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "lab3_figure_communication.png", dpi=150, bbox_inches="tight")
```

Adaptez évidemment le titre, l'annotation et les valeurs à **votre** graphique et à **vos** résultats : cette figure doit être unique à votre analyse, pas un simple copier-coller.

10. Travail demandé

Partie A — Choisir avant de coder

1. Pour chacune des cinq familles de questions du chapitre 3 (distribution, comparaison, relation, évolution, composition), indiquez la variable ou la paire de variables de ce jeu de données que vous allez représenter, **avant** d'écrire le code.

Partie B — Graphiques essentiels

2. Produisez l'histogramme et la boîte à moustaches du délai de traitement (section 3).
3. Produisez le diagramme en barres des 12 types de demandes les plus fréquents (section 4).
4. Produisez le camembert des 8 mêmes catégories et rédigez, en trois phrases maximum, pourquoi le diagramme en barres est ici préférable.
5. Produisez le nuage de points heure de création / délai, avec échantillonnage et transparence (section 5).
6. Produisez la heatmap de corrélation (section 5).
7. Produisez la courbe d'évolution quotidienne du volume de demandes (section 6).
8. Produisez la boîte à moustaches par arrondissement et la heatmap arrondissement/canal (section 7).

Partie C — Critique et correction

9. Sur l'un de vos propres graphiques, tronquez volontairement l'axe des ordonnées d'un diagramme en barres et comparez les deux versions côte à côte. Expliquez en deux phrases l'effet produit.
10. Identifiez, parmi les huit graphiques produits, celui qui serait le plus à risque d'être mal interprété par un lecteur pressé, et proposez une correction (titre, annotation, ou axe).

Partie D — Interactivité

11. Produisez le graphique interactif `Plotly` de la section 8.
12. Citez une situation, dans votre futur projet de fin de module, où un graphique interactif apporterait une réelle valeur par rapport à une image statique — et une situation où il n'en apporterait aucune.

Partie E — Communication

13. Produisez la figure de communication complète de la section 9 (titre-message, annotation, source).
14. Rédigez, pour cette figure uniquement, une légende de 3 à 4 phrases destinée à un lecteur qui ne verra que le graphique et sa légende, sans lire le reste du rapport.
15. Constituez un court portfolio (un fichier Markdown ou un notebook) rassemblant vos huit graphiques, chacun accompagné d'un titre, d'une phrase de lecture et de la question à laquelle il répond.

Lab 4 : Entrepôt de données et dashboard avancé avec Power BI

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook** `.ipynb` pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 4 et téléchargez-le.
- **Fichier** `.qmd` pour travailler avec Quarto : [téléchargez le fichier source du Lab 4](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le `.qmd`, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

Le support contient les consignes ; la modélisation et le rapport se construisent dans **Power BI Desktop** et sont remis au format `.pbix`.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Construire** un modèle analytique en étoile à partir de données massives ;
- **Importer et transformer** des données avec Power Query ;
- **Définir** le grain d'une table de faits et les relations avec les dimensions ;
- **Créer** des mesures DAX fiables et contrôlées ;
- **Choisir** une visualisation selon la nature de la donnée et la question ;
- **Concevoir** un dashboard multi-pages, interactif et performant ;
- **Vérifier** les résultats à l'aide de totaux de contrôle.

i Question BI du projet

La ville veut piloter son service 311 : **où se concentrent les demandes, quels types de problèmes prennent le plus de temps à traiter et comment le volume évolue-t-il selon le quartier, le canal et la période ?**

Le résultat attendu n'est pas un mur de graphiques. C'est un rapport Power BI qui permet à une personne responsable de passer d'une vue générale à un diagnostic vérifiable.

1. Le jeu de données massif

Le projet utilise le jeu public **NYC 311 Service Requests from 2010 to Present** :

<https://data.cityofnewyork.us/resource/erm2-nwe9.json>

Le jeu contient plusieurs dizaines de millions de demandes historiques et continue d'être alimenté. Il comprend notamment :

Famille	Exemples
Identifiant	<code>unique_key</code>
Temps	<code>created_date</code> , <code>closed_date</code>

Famille	Exemples
Catégories	complaint_type, descriptor, agency
Localisation	borough, incident_zip, latitude, longitude
Canal et statut	open_data_channel_type, status

Pour un premier rendu, utilisez une période définie, par exemple l'année 2024 ou un trimestre comportant au moins plusieurs centaines de milliers de lignes. Indiquez exactement la période dans le rapport. Une donnée massive ne signifie pas qu'il faut charger toute l'histoire sans contrôle : il faut dimensionner le périmètre à la question et aux ressources de la machine.

2. Architecture cible

2.1. Les couches

Le projet comporte quatre couches :

```

API NYC 311
  ↓
staging / requête de préparation
  ↓
modèle en étoile Power BI
  ↓
mesures DAX + pages du rapport

```

La couche de staging conserve les champs utiles et prépare les types. Le modèle analytique sépare les événements (`fact_requests`) du contexte (`dim_date`, `dim_location`, `dim_complaint`, `dim_agency`).

2.2. Le grain

Le grain de `fact_requests` est :

une ligne représente une demande 311 identifiée par `unique_key`.

Ce choix permet de compter les demandes distinctes, de calculer un délai pour chaque demande clôturée et de regrouper par date, type, quartier ou canal.

2.3. Schéma en étoile

```

          dim_date
          |
dim_complaint ---- fact_requests ---- dim_agency
          |
          dim_location
          |
          dim_channel

```

Table de faits :

```
fact_requests
-----
request_key
created_date_key
closed_date_key
complaint_key
agency_key
location_key
channel_key
status
processing_hours
is_closed
```

Dimensions :

```
dim_date(date_key, date, year, quarter, month, month_name, weekday)
dim_complaint(complaint_key, complaint_type, descriptor)
dim_agency(agency_key, agency)
dim_location(location_key, borough, zip_code)
dim_channel(channel_key, channel)
```

Une dimension doit avoir une ligne par clé. Vérifiez cette unicité avant de la relier à la table de faits.

3. Acquérir les données dans Power Query

3.1. Connexion Web

Dans Power BI Desktop :

1. **Accueil** → **Obtenir les données** → **Web** ;
2. utilisez l'URL de l'API ou la requête M ci-dessous ;
3. ouvrez **Transformer les données** plutôt que de charger immédiatement ;
4. renommez la requête en **stg_requests**.

Pour une extraction légère de contrôle :

```
https://data.cityofnewyork.us/resource/erm2-nwe9.json?$limit=100
```

Pour un volume important, filtrez côté serveur. Ne demandez pas toutes les colonnes si la question n'en utilise que dix.

3.2. Pagination dans Power Query

L'API renvoie des pages. Dans l'Éditeur avancé, le principe suivant permet de lire plusieurs pages avec un filtre de période. Adaptez les dates et le nombre de pages à votre machine.

```
let
    BaseUrl = "https://data.cityofnewyork.us/resource/erm2-nwe9.json",
    PageSize = 50000,
    WhereClause =
        "created_date between '2024-01-01T00:00:00' " &
        "and '2024-12-31T23:59:59'",
    SelectClause =
```

```

"unique_key,created_date,closed_date,agency," &
"complaint_type,descriptor,borough,incident_zip," &
"open_data_channel_type,status",

GetPage = (Offset as number) as list =>
let
    Response = Json.Document(
        Web.Contents(
            BaseUrl,
            [
                Query = [
                    #"$select" = SelectClause,
                    #"$where" = WhereClause,
                    #"$order" = "created_date",
                    #"$limit" = Text.From(PageSize),
                    #"$offset" = Text.From(Offset)
                ],
                Timeout = #duration(0, 0, 10, 0)
            ]
        )
    )
in
    Response,

Pages = List.Generate(
    () => [Offset = 0, Rows = GetPage(0)],
    each List.Count([Rows]) > 0,
    each [
        Offset = [Offset] + PageSize,
        Rows = GetPage([Offset] + PageSize)
    ],
    each [Rows]
),

Combined = List.Combine(Pages),
AsTable = Table.FromRecords(Combined)
in
    AsTable

```

Contrôler avant un téléchargement massif

Commencez avec une journée ou une semaine, vérifiez les colonnes, la pagination et le nombre de lignes, puis élargissez à l'année. Un mauvais filtre peut déclencher un téléchargement très long ou produire un résultat incomplet. Conservez l'URL, la période et le nombre de lignes dans la documentation du rapport.

3.3. Transformer la staging table

Dans `stg_requests` :

1. donnez des noms lisibles aux colonnes ;
2. convertissez `created_date` et `closed_date` en Date/Heure ;
3. convertissez `incident_zip` en texte, car un code postal n'est pas une mesure ;
4. remplacez les chaînes vides par `null` ;
5. supprimez les colonnes non utilisées ;
6. ajoutez `processing_hours` :

```
Duration.TotalHours([closed_date] - [created_date])
```

7. ajoutez `is_closed`;
8. filtrez les dates invalides seulement après avoir compté et documenté les lignes concernées.

Ne faites pas dans Power Query une transformation qui devrait rester une mesure dynamique. Par exemple, le total des demandes est une mesure; le type d'une colonne et la normalisation d'un texte sont des transformations.

4. Créer les dimensions

4.1. Dimension Date

Créez une table Date avec DAX :

```
dim_date =  
ADDCOLUMNS(  
    CALENDAR(DATE(2024, 1, 1), DATE(2024, 12, 31)),  
    "date_key", YEAR([Date]) * 10000  
        + MONTH([Date]) * 100  
        + DAY([Date]),  
    "year", YEAR([Date]),  
    "month_number", MONTH([Date]),  
    "month_name", FORMAT([Date], "MMMM"),  
    "quarter", "T" & FORMAT([Date], "Q"),  
    "weekday_name", FORMAT([Date], "dddd")  
)
```

Triez `month_name` par `month_number`. Marquez la table comme table de dates si la version de Power BI le demande.

4.2. Dimensions par référence

Dans Power Query, dupliquez `stg_requests` pour créer :

- `dim_complaint` : `complaint_type`, `descriptor`, clé;
- `dim_agency` : `agency`, clé;
- `dim_location` : `borough`, `incident_zip`, clé;
- `dim_channel` : `open_data_channel_type`, clé.

Pour une première version, une clé composée lisible peut être utilisée. Dans un modèle professionnel, on peut créer des clés entières et gérer l'historique des dimensions; ces sujets sont hors du premier rendu.

4.3. Relations

Dans la vue Modèle :

- chaque dimension est du côté **1**;
- `fact_requests` est du côté *****;
- le filtrage va de la dimension vers la table de faits;
- une clé de dimension ne doit pas être dupliquée.

Pour `dim_date`, la relation principale utilise `created_date`. La date de clôture est une deuxième relation potentiellement inactive; n'utilisez-la qu'en connaissance du contexte.

5. Mesures DAX du modèle

Créez un dossier de mesures et ajoutez :

```

Demandes =
DISTINCTCOUNT(fact_requests[request_key])

Demandes clôturées =
CALCULATE(
    [Demandes],
    fact_requests[is_closed] = TRUE()
)

Taux de clôture =
DIVIDE([Demandes clôturées], [Demandes])

Délai médian (heures) =
MEDIAN(fact_requests[processing_hours])

Délai moyen (heures) =
AVERAGE(fact_requests[processing_hours])

Délai P90 (heures) =
PERCENTILEX.INC(
    FILTER(
        fact_requests,
        NOT ISBLANK(fact_requests[processing_hours])
    ),
    fact_requests[processing_hours],
    0.90
)

Demandes du mois précédent =
CALCULATE(
    [Demandes],
    DATEADD(dim_date[Date], -1, MONTH)
)

Variation mensuelle % =
DIVIDE(
    [Demandes] - [Demandes du mois précédent],
    [Demandes du mois précédent]
)

```

La médiane et le 90e percentile sont souvent plus parlants qu'une moyenne lorsque les délais sont asymétriques. La mesure P90 répond à une question comme : « en dessous de quel délai se trouvent 90 % des demandes avec délai connu ? »

5.1. Contrôles

Créez une page ou un tableau de contrôle avec :

- nombre de lignes importées ;
- nombre de clés distinctes ;
- demandes sans date de création ;
- demandes sans date de clôture ;
- délais négatifs ;

— demandes par mois.

Comparez au moins un mois avec un calcul pandas ou une requête API `count(*)`. Un beau rapport qui ne passe pas ces contrôles n'est pas prêt.

6. Choisir le bon visuel

Le choix dépend du type de donnée et de la question :

Question	Données	Visuel recommandé	Pourquoi
Combien de demandes ?	Un nombre	Carte KPI	Lecture immédiate
Comment le volume évolue-t-il ?	Date + mesure	Courbe	Tendance et ruptures
Quels types dominent ?	Catégorie + mesure	Barres horizontales triées	Comparaison de rangs
Quels quartiers ont le plus de demandes ?	Carte ou catégorie	Carte remplie ou barres	Dimension géographique
Comment les canaux se répartissent-ils ?	Deux catégories	Barres empilées à 100 %	Composition
Quel type a le plus long délai ?	Catégorie + distribution	Barres avec médiane ou boîte à moustaches	Comparer une distribution
Existe-t-il un lien volume/délai ?	Deux mesures	Nuage de points	Relation entre grandeurs
Comment détailler une sélection ?	Plusieurs champs	Matrice	Exploration hiérarchique
Qui contribue à un total ?	Catégories hiérarchisées	Arbre de décomposition	Diagnostic interactif

Évitez les camemberts pour une longue liste de catégories et les cartes lorsque la localisation est incomplète ou ambiguë. Un visuel doit répondre à une question avant de décorer la page.

7. Construire un dashboard sophistiqué mais lisible

Page 1 — Pilotage

En haut :

- demandes totales ;
- taux de clôture ;
- délai médian ;
- délai P90 ;
- date de dernière actualisation.

Au centre :

- courbe des demandes par mois ;
- barres des dix types les plus fréquents ;
- carte ou barres par borough.

À droite ou en haut :

- segments année/mois, borough, agence et canal ;
- bouton de remise à zéro des filtres.

Page 2 — Diagnostic des délais

- barres horizontales du délai médian par type ;
- comparaison moyenne/médiane ;
- histogramme ou distribution filtrée ;
- tableau avec demandes, taux de clôture, médiane, P90 ;
- mise en forme conditionnelle sur les types dépassant un seuil.

Page 3 — Canaux et géographie

- composition des canaux par borough ;
- évolution par canal ;
- carte uniquement si les coordonnées et la granularité sont fiables ;
- drillthrough vers un borough.

Page 4 — Exploration

- matrice type → agence → statut ;
- arbre de décomposition du volume ;
- page de tooltip avec les KPI et la période filtrée ;
- bouton vers la page de contrôle.

7.1. Interactions et navigation

Ajoutez :

- des interactions cohérentes entre les visuels ;
- des signets pour alterner vue volume et vue délai ;
- une navigation entre pages ;
- un drillthrough par borough ;
- une page tooltip courte ;
- des titres dynamiques indiquant les filtres actifs.

Ne rendez pas chaque visuel dépendant de chaque autre visuel sans réfléchir : une interaction inutile peut rendre le rapport difficile à comprendre.

8. Performance sur des données massives

Appliquez ces règles :

1. filtrer la période et les colonnes côté serveur ;
2. éviter de charger des textes longs non utilisés ;
3. utiliser des dimensions plutôt que de répéter des libellés ;
4. convertir correctement les types ;
5. éviter les colonnes calculées très lourdes lorsque Power Query peut faire la préparation ;
6. réduire la cardinalité des champs non nécessaires ;
7. créer une table d'agrégation mensuelle si la page générale n'a pas besoin du détail ;
8. mesurer le temps de rafraîchissement et la taille du modèle.

Import offre une exploration rapide après chargement ; **DirectQuery** interroge la source mais dépend davantage de la performance du serveur et des requêtes. Pour ce Lab, privilégiez un périmètre Import maîtrisé et documenté, puis expliquez comment une architecture de production pourrait évoluer.

9. Spécification du rapport attendu

Le fichier `lab4_nyc311_powerbi.pbix` doit contenir :

- les requêtes de staging ;
- les tables du modèle en étoile ;
- les relations ;
- les mesures DAX ;
- les pages de dashboard ;
- une page de contrôles ;
- une page « À propos des données » ;
- la période et la date de rafraîchissement.

La page « À propos des données » doit indiquer la source, l'URL, la date d'extraction, les paramètres et filtres utilisés, le nombre de lignes chargé, le périmètre, le fuseau horaire, le grain, les règles de traitement des dates manquantes et les limites.

10. Travail demandé

1. Définissez la question BI, la population et la période.
2. Importez un petit échantillon, puis validez la pagination avant de charger la période choisie.
3. Conservez au moins 300 000 lignes ou justifiez une limite technique.
4. Créez le modèle en étoile et vérifiez l'unicité des dimensions.
5. Créez les mesures de volume, clôture, médiane, moyenne, P90 et variation mensuelle.
6. Comparez au moins trois mesures Power BI avec des valeurs calculées hors de Power BI.
7. Réalisez les quatre pages proposées.
8. Justifiez le choix d'au moins huit visuels.
9. Ajoutez un drillthrough et une page tooltip.
10. Ajoutez une page de contrôle qualité.
11. Mesurez le temps de rafraîchissement et notez la taille du fichier.
12. Écrivez une conclusion de cinq à dix lignes répondant à la question BI sans causalité non prouvée.

Lab 5 : Entrepôt de données et dashboard avancé avec Taipy

! Avant de commencer — choisir votre support

Téléchargez un seul format avant de travailler :

- **Notebook** `.ipynb` pour Google Colab, JupyterLab, Jupyter Notebook ou VS Code : ouvrez le [dossier GitHub labsheets/](#), choisissez le notebook du Lab 5 et téléchargez-le.
- **Fichier** `.qmd` pour travailler avec Quarto : [téléchargez le fichier source du Lab 5](#).

Dans GitHub, ouvrez le fichier choisi puis cliquez sur **Download raw file** (ou **Raw**, puis enregistrez le fichier). Dans Colab, importez le notebook dans votre Drive ; dans Jupyter ou VS Code, ouvrez la copie téléchargée et choisissez un noyau Python. Pour le `.qmd`, utilisez Quarto. Travaillez dans votre propre copie, pas dans l'aperçu GitHub.

Colab peut servir à exécuter les analyses Python, mais le lancement local de l'application et le déploiement Docker nécessitent un environnement avec terminal ; utilisez de préférence VS Code en local pour ces étapes.

💡 Objectifs du Lab

À la fin de ce Lab, l'étudiant sera capable de :

- **Réutiliser** les mêmes définitions métier que dans le rapport Power BI ;
- **Préparer** des agrégats adaptés à une application web légère ;
- **Construire** une interface interactive avec Taipy ;
- **Ajouter** des filtres, KPI, tableaux et graphiques synchronisés ;
- **Comparer** une plateforme BI et une application Python sur des critères concrets ;
- **Déployer** une application Taipy sur une plateforme gratuite de type Hugging Face Spaces ;
- **Documenter** les limites, la sécurité et le coût réel d'un déploiement gratuit.

! Même projet, architecture différente

Le Lab 4 plaçait le modèle et les mesures dans Power BI. Ici, la source et la question restent les mêmes, mais la préparation et l'interface sont écrites en Python. Le chiffre d'affaires n'est pas remplacé par un autre calcul : les définitions doivent rester identiques entre les deux solutions.

1. Power BI ou Taipy ?

1.1. Comparaison par besoin

Besoin	Power BI	Taipy
Connecter rapidement de nombreuses sources	Très adapté	Nécessite du code ou des connecteurs Python
Transformer les données	Power Query	pandas, SQL, DuckDB, fonctions Python
Définir les KPI	DAX et modèle sémantique	Fonctions Python testables

Besoin	Power BI	Taipy
Créer rapidement des rapports métier	Très adapté	Possible, mais demande de construire l'interface
Intégrer l'analyse à une application	Limité selon l'architecture	Très adapté
Contrôler le code et l'interface	Moins central	Central
Déployer gratuitement	Dépend du compte et de l'organisation	Possible avec une plateforme gratuite, sous limites
Données massives	Modèle, agrégations, Import/DirectQuery	Pré-agrégation nécessaire sur une petite instance

Power BI est une plateforme de BI prête à l'emploi. Taipy est une brique de développement qui permet de construire une application de données. Comparer les deux ne revient pas à chercher un gagnant : il faut relier le choix à l'équipe, au public, aux données et au mode de maintenance.

1.2. Ce que l'on garde identique

Les deux Labs doivent utiliser :

- la même source NYC 311 ;
- la même période ;
- le même grain de la donnée source ;
- les mêmes définitions de Demandes, Taux de clôture, Délai médian et Délai P90 ;
- les mêmes règles pour les dates manquantes et les délais négatifs.

Si Power BI affiche 1 000 demandes et Taipy 980, il faut chercher la différence de filtre, de dédoublement ou de définition avant de comparer les graphiques.

2. Adapter une donnée massive à une application

Une application gratuite ne doit pas charger plusieurs millions de lignes à chaque ouverture. Le traitement lourd est exécuté **hors ligne** :

```
NYC 311 massif
  ↓ une seule fois
nettoyage et calcul des délais
  ↓
agrégats mensuels, borough, type et canal
  ↓
fichiers Parquet compacts
  ↓ à chaque interaction
filtre Taipy + graphique + tableau
```

Cette architecture sépare :

- le pipeline de préparation, qui peut utiliser une machine plus puissante ;
- l'application, qui sert des résultats déjà préparés.

2.1. Fichiers produits

```

taipy-nyc311/
  app.py
  prepare_data.py
  requirements.txt
  Dockerfile
  README.md
  data/
    processed/
      overall_summary.parquet
      monthly_summary.parquet
      borough_summary.parquet
      type_summary.parquet
      borough_type_summary.parquet
      channel_summary.parquet

```

Le pipeline produit aussi `monthly_borough_summary.parquet`, `monthly_type_summary.parquet` et `monthly_borough_type_summary.parquet`. Chaque combinaison de filtres possède ainsi son propre agrégat exact.

N'envoyez pas automatiquement le CSV brut massif dans un dépôt public. L'application finale doit contenir les agrégats strictement nécessaires à ses pages.

3. Préparer les agrégats avec Python

3.1. Installer les dépendances

```

uv init --python 3.11
uv add pandas pyarrow taipy
mkdir -p data/processed

```

Pour le développement, vous pouvez utiliser un fichier annuel ou mensuel déjà collecté au Lab 4 :

```

# prepare_data.py
from pathlib import Path
import pandas as pd

SOURCE = Path("data/raw/nyc311_2024.csv")
OUT = Path("data/processed")
OUT.mkdir(parents=True, exist_ok=True)

USECOLS = [
    "unique_key",
    "created_date",
    "closed_date",
    "complaint_type",
    "agency",
    "borough",
    "open_data_channel_type",
    "status",
]

df = pd.read_csv(SOURCE, usecols=USECOLS)

NYC_TZ = "America/New_York"

```

```

for column in ("created_date", "closed_date"):
    dates = pd.to_datetime(df[column], errors="coerce")
    if dates.dt.tz is None:
        dates = dates.dt.tz_localize(
            NYC_TZ,
            ambiguous="NaT",
            nonexistent="NaT",
        )
    else:
        dates = dates.dt.tz_convert(NYC_TZ)
    df[column] = dates

df = df.dropna(subset=["unique_key", "created_date"])
df = df.drop_duplicates(subset=["unique_key"])
df["is_closed"] = df["closed_date"].notna()
df["processing_hours"] = (
    df["closed_date"] - df["created_date"]
).dt.total_seconds() / 3600
df.loc[df["processing_hours"] < 0, "processing_hours"] = pd.NA
df["month"] = df["created_date"].dt.strftime("%Y-%m")

df["borough"] = df["borough"].fillna("Non renseigné")
df["complaint_type"] = df["complaint_type"].fillna("Non renseigné")
df["open_data_channel_type"] = df[
    "open_data_channel_type"
].fillna("Non renseigné")

def summarise(grouped):
    result = grouped.agg(
        Demandes=("unique_key", "nunique"),
        Demandes_cloturees=("is_closed", "sum"),
        Delai_median_heures=("processing_hours", "median"),
        Delai_moyen_heures=("processing_hours", "mean"),
        Delai_P90_heures=("processing_hours", lambda s: s.quantile(0.90)),
    ).reset_index()
    result["Taux_cloture"] = (
        result["Demandes_cloturees"] / result["Demandes"]
    )
    return result

overall = summarise(df.assign(_all="Toutes").groupby("_all")).drop(columns="_all")
monthly = summarise(df.groupby("month"))
borough = summarise(df.groupby("borough"))
types = summarise(df.groupby("complaint_type")).sort_values(
    "Demandes", ascending=False
)
borough_type = summarise(df.groupby(["borough", "complaint_type"]))
channels = summarise(df.groupby("open_data_channel_type"))
monthly_borough = summarise(df.groupby(["month", "borough"]))
monthly_type = summarise(df.groupby(["month", "complaint_type"]))
monthly_borough_type = summarise(
    df.groupby(["month", "borough", "complaint_type"])
)

overall.to_parquet(OUT / "overall_summary.parquet", index=False)
monthly.to_parquet(OUT / "monthly_summary.parquet", index=False)
borough.to_parquet(OUT / "borough_summary.parquet", index=False)

```

```
types.head(50).to_parquet(OUT / "type_summary.parquet", index=False)
borough_type.to_parquet(OUT / "borough_type_summary.parquet", index=False)
channels.to_parquet(OUT / "channel_summary.parquet", index=False)
monthly_borough.to_parquet(
    OUT / "monthly_borough_summary.parquet",
    index=False,
)
monthly_type.to_parquet(
    OUT / "monthly_type_summary.parquet",
    index=False,
)
monthly_borough_type.to_parquet(
    OUT / "monthly_borough_type_summary.parquet",
    index=False,
)
```

Cette version utilise une ligne par `unique_key`. Si le fichier source contient plusieurs versions d'une même demande, il faut documenter pourquoi l'on conserve la première, la dernière ou une ligne dédoublonnée.

3.2. Contrôler les agrégats

```
assert monthly["Demandes"].sum() == df["unique_key"].nunique()
assert borough["Demandes"].sum() == df["unique_key"].nunique()
assert monthly["Taux_cloture"].between(0, 1).all()
assert (monthly["Demandes"] >= 0).all()
```

Les contrôles ne prouvent pas que l'analyse est parfaite, mais ils détectent des erreurs évidentes avant publication.

4. Construire l'application Taipy

4.1. Charger les données

```
# app.py
from pathlib import Path
import pandas as pd
from taipy.gui import Gui

DATA_DIR = Path("data/processed")
monthly = pd.read_parquet(DATA_DIR / "monthly_summary.parquet")
borough = pd.read_parquet(DATA_DIR / "borough_summary.parquet")
types = pd.read_parquet(DATA_DIR / "type_summary.parquet")
overall = pd.read_parquet(DATA_DIR / "overall_summary.parquet")
borough_type = pd.read_parquet(DATA_DIR / "borough_type_summary.parquet")
monthly_borough = pd.read_parquet(
    DATA_DIR / "monthly_borough_summary.parquet"
)
monthly_type = pd.read_parquet(DATA_DIR / "monthly_type_summary.parquet")
monthly_borough_type = pd.read_parquet(
    DATA_DIR / "monthly_borough_type_summary.parquet"
)
```

4.2. Préparer l'état de l'interface

Taipy met à jour une interface à partir de variables Python. Nous commençons par une vue générale :

```
selected_borough = "Tous"
selected_type = "Tous"
selected_view = monthly.copy()
total_requests = int(overall.loc[0, "Demandes"])
total_closed = int(overall.loc[0, "Demandes_cloturees"])
closure_rate = f"{overall.loc[0, 'Taux_cloture']:.1%}"
median_delay = float(overall.loc[0, "Delai_median_heures"])
p90_delay = float(overall.loc[0, "Delai_P90_heures"])

borough_options = ["Tous"] + sorted(
    borough["borough"].dropna().unique().tolist()
)
type_options = ["Tous"] + types["complaint_type"].head(20).tolist()
```

La bonne granularité des agrégats est essentielle : une demande filtrée par mois, arrondissement ou type doit être calculée à partir des lignes appartenant exactement à cette sélection. Les tables mensuelles et les résumés globaux par combinaison sont préparés dans `prepare_data.py`, puis lus par l'application.

4.3. Fonction de filtrage

```
def make_view(borough_value, type_value):
    if borough_value != "Tous" and type_value != "Tous":
        monthly_data = monthly_borough_type
        summary_data = borough_type
        filters = (("borough", borough_value), ("complaint_type", type_value))
    elif borough_value != "Tous":
        monthly_data = monthly_borough
        summary_data = borough
        filters = (("borough", borough_value),)
    elif type_value != "Tous":
        monthly_data = monthly_type
        summary_data = types
        filters = (("complaint_type", type_value),)
    else:
        monthly_data = monthly
        summary_data = overall
        filters = ()

    for column, value in filters:
        monthly_data = monthly_data[monthly_data[column] == value]
        summary_data = summary_data[summary_data[column] == value]

    return monthly_data.sort_values("month"), summary_data.iloc[0]
```

Ne prétendez pas filtrer par type avec une table qui ne possède pas la dimension type. Cette vérification de granularité est une compétence essentielle du data analyst.

4.4. Construire la page

Taipy utilise une syntaxe Markdown enrichie. Le contrôle `selector` permet de choisir une valeur, `table` affiche un DataFrame et `chart` représente des colonnes :

```
page = """
# Suivi des demandes NYC 311

Période préparée : 2024

**Arrondissement**
<|{selected_borough}|selector|lov={borough_options}|dropdown|>

**Type de demande**
<|{selected_type}|selector|lov={type_options}|dropdown|>

**Demandes :** <|{total_requests}|text|><br />
**Taux de clôture :** <|{closure_rate}|text|><br />
**Délai médian (heures) :** <|{median_delay}|text|><br />
**Délai P90 (heures) :** <|{p90_delay}|text|>

## Vue mensuelle

<|{selected_view}|chart|x=month|y=Demandes|>

<|{selected_view}|table|>
"""
```

La syntaxe exacte de certains contrôles peut évoluer selon la version de Taipy. Vérifiez la documentation de la version installée si un contrôle ne s'affiche pas.

4.5. Réagir aux filtres

```
def on_change(state, var_name, value):
    if var_name in {"selected_borough", "selected_type"}:
        view, summary = make_view(
            state.selected_borough,
            state.selected_type,
        )
        state.selected_view = view
        state.total_requests = int(summary["Demandes"])
        state.total_closed = int(summary["Demandes_cloturees"])
        state.closure_rate = f"{summary['Taux_cloture']:.1%}"
        state.median_delay = float(summary["Delai_median_heures"])
        state.p90_delay = float(summary["Delai_P90_heures"])
```

Pour une application plus avancée, centralisez les calculs dans des fonctions et évitez de modifier des variables globales directement. L'état de l'interface doit rester compréhensible.

4.6. Lancer l'application

```
if __name__ == "__main__":  
    Gui(page=page).run(  
        title="NYC 311 BI avec Taipy",  
        host="0.0.0.0",  
        port=7860,  
        use_reloader=False,  
    )
```

En local :

```
uv run python app.py
```

Ouvrez l'adresse affichée par Taipy et testez plusieurs filtres. Le port 7860 est utilisé pour faciliter le déploiement sur Hugging Face Spaces.

5. Rendre l'application aussi riche que le rapport Power BI

L'application finale doit comporter au moins trois vues :

Vue 1 — Pilotage

- KPI demandes totales ;
- taux de clôture ;
- délai médian ;
- délai P90 ;
- courbe mensuelle ;
- filtre borough.

Vue 2 — Types de demandes

- top 20 des types ;
- barres du volume ;
- tableau volume, taux de clôture, médiane et P90 ;
- filtre de période ;
- message lorsqu'un type est sélectionné.

Vue 3 — Comparaison géographique

- classement des boroughs ;
- volume et délai médian ;
- graphique de composition des canaux ;
- tableau détaillé ;
- avertissement lorsque l'effectif est faible.

Taipy permet de construire une interface sur mesure, mais il faut coder la navigation et la logique que Power BI offre déjà en partie. Cette liberté est un avantage lorsque l'application doit s'intégrer à un workflow métier ; elle représente aussi un coût de maintenance.

6. Vérifier la cohérence avec Power BI

Choisissez trois filtres identiques dans les deux solutions :

1. toute la période ;
2. un borough ;
3. un type de demande fréquent.

Comparez :

Indicateur	Power BI	Taipy	Différence
Demandes			
Demandes clôturées			
Taux de clôture			
Médiane du délai			

Si une différence existe, examinez dans cet ordre :

- période et fuseau horaire ;
- doublons ;
- valeurs sans date ;
- définition d'une demande clôturée ;
- agrégat utilisé par Taipy ;
- format et arrondi.

Ne corrigez pas une différence en forçant un nombre dans l'interface. Corrigez la transformation ou expliquez la différence.

7. Déploiement gratuit avec Hugging Face Spaces

Hugging Face Spaces propose un environnement gratuit à ressources limitées, adapté à une démonstration pédagogique. Les règles de disponibilité et les limites peuvent évoluer ; vérifiez-les au moment du déploiement.

Si aucun hébergement gratuit adapté n'est disponible, réalisez et documentez une démonstration locale de l'application avec les mêmes fichiers d'agrégats.

7.1. Préparer les fichiers

requirements.txt :

```
taipy
pandas
pyarrow
```

Dockerfile :

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
```

```
COPY data/processed ./data/processed
```

```
EXPOSE 7860
```

```
CMD ["python", "app.py"]
```

Le dossier du dépôt doit contenir :

```
app.py
requirements.txt
Dockerfile
data/processed/*.parquet
```

N'incluez pas le CSV massif, les identifiants privés ou les fichiers temporaires.

7.2. Créer et lancer le Space

1. Créez un compte Hugging Face si nécessaire.
2. Choisissez **New Space**.
3. Sélectionnez le SDK **Docker** et une machine CPU gratuite.
4. Rendez le Space public uniquement avec des données et du code que vous pouvez publier.
5. Ajoutez les fichiers du projet.
6. Attendez le build de l'image.
7. Consultez les logs si l'application ne démarre pas.
8. Vérifiez que Taipy écoute bien sur 0.0.0.0:7860.

Le premier démarrage peut prendre du temps. Une instance gratuite peut s'endormir lorsqu'elle n'est pas utilisée ; le premier accès suivant peut donc être plus lent. La plateforme n'est pas un environnement de production garanti.

7.3. Autres options

Vous pouvez comparer avec une autre plateforme gratuite acceptant un conteneur Python, par exemple un service de déploiement disposant d'un plan gratuit. Vérifiez toujours :

- la commande de démarrage ;
- le port exposé ;
- la mise en veille ;
- la limite de mémoire ;
- la visibilité des fichiers ;
- les conditions du plan gratuit.

8. Sécurité et maintenance

Même pour une démonstration :

- ne mettez jamais de clé API dans `app.py` ou Git ;
- ne téléchargez pas les données massives à chaque requête utilisateur ;
- validez les valeurs des filtres ;
- affichez la date de préparation des agrégats ;
- documentez la source et la licence ;
- ne publiez pas de données personnelles ;
- prévoyez un message si un fichier Parquet est absent ou vide.

Une application publique doit aussi résister à une entrée inattendue. Les filtres doivent choisir parmi des valeurs connues, et les agrégats doivent être contrôlés avant mise en ligne.

9. Travail demandé

1. Reprenez la même question BI, période et source que le Lab 4.
2. Écrivez le pipeline Python qui produit au moins quatre agrégats Parquet.
3. Ajoutez les contrôles de qualité et conservez les résultats dans un journal.
4. Construisez une application Taipy avec trois vues.
5. Ajoutez au moins deux filtres synchronisés.
6. Affichez quatre KPI et au moins trois graphiques.
7. Ajoutez une table détaillée indiquant effectif, médiane et P90. Les médianes et percentiles des sous-groupes ne s'additionnent pas : conservez des agrégats calculés pour chaque combinaison de filtres.
8. Comparez cinq chiffres avec Power BI.
9. Expliquez une différence éventuelle entre les deux solutions.
10. Ajoutez un message sur la date de préparation et la période.
11. Conteneurisez l'application avec **Dockerfile**.
12. Déployez-la sur Hugging Face Spaces ou une plateforme gratuite équivalente.
13. Testez le lien public depuis un autre navigateur.
14. Rédigez les limites du déploiement gratuit.
15. En cinq à dix lignes, recommandez Power BI, Taipy ou une combinaison des deux pour ce projet.

References

Knafllic, Cole. 2015. *Storytelling with Data*. Wiley.

McKinney, Wes. 2022. *Python for Data Analysis*. O'Reilly Media.